

Advanced Algorithms

Companion Notes for Harvard COMPSCI 224

Kun Deng

Claude (Anthropic)

2026-05-03

Table of contents

Preface	1
I. Data Structures	3
1. The Word-RAM Model and Predecessor Data Structures	5
1.1. Why this lecture matters	5
1.2. The Word-RAM model	5
1.3. The static predecessor problem	6
1.4. Van Emde Boas trees	6
1.4.1. The recursive idea	6
1.4.2. Predecessor query	7
1.4.3. Insertion	7
1.5. From $O(u)$ space to $O(n)$ space	8
1.5.1. The space problem	8
1.5.2. Solution: replace arrays with hash tables	8
1.6. X-fast tries and Y-fast tries	8
1.6.1. X-fast trie	9
1.6.2. Y-fast trie (Willard, IPL 1983)	9
1.7. Optimality	9
1.8. Cheat sheet	9
1.9. FAQ — questions students actually asked	10
1.10. Further reading	10
1.11. Try it yourself	11
2. Fusion Trees	13
2.1. Why this lecture matters	13
2.2. Erratum: VEB space with hashing	13
2.3. From binary to k -ary search trees	14
2.3.1. The basic structure	14
2.3.2. Choosing k	14
2.4. Sketch compression	14
2.4.1. Branch bits	14
2.4.2. The sketch function	14
2.5. Handling arbitrary queries	15
2.5.1. The problem	15
2.5.2. The fix: find the divergence point	15
2.6. Sketch compression via multiplication	15
2.6.1. The problem	15
2.6.2. The multiplication trick	16
2.6.3. Proof sketch: choosing the m_i	16
2.7. Word-level parallelism: parallel comparison	16
2.7.1. The node sketch word	16
2.7.2. Query comparison	17
2.7.3. Counting the 1's via multiplication	17

2.8.	Most significant bit in $O(1)$	17
2.8.1.	Why we need it	17
2.8.2.	Algorithm	17
2.9.	Putting it all together	18
2.10.	Cheat sheet	19
2.11.	FAQ — questions students actually asked	19
2.12.	Further reading	20
2.13.	Try it yourself	20
3.	Hashing I: Load Balancing, Independence, and Linear Probing	23
3.1.	Why this lecture matters	23
3.2.	Hashing: the setup	23
3.3.	Load balancing and balls into bins	24
3.3.1.	The Chernoff bound	24
3.3.2.	Max load is $O(\log n / \log \log n)$	24
3.3.3.	A simpler proof via counting	25
3.4.	k -wise independence	25
3.4.1.	Polynomial hash families	26
3.4.2.	The mod-reduction issue	26
3.5.	The dictionary problem	26
3.5.1.	Hashing with chaining	27
3.6.	Linear probing	27
3.6.1.	How much independence does linear probing need?	27
3.6.2.	Full intervals and the analysis framework	28
3.6.3.	The moment method	28
3.7.	Cheat sheet	29
3.8.	FAQ — questions students actually asked	29
3.9.	Further reading	30
3.10.	Try it yourself	30
4.	Hashing II: Bloom Filters, Cuckoo Hashing, and Bloomier Filters	33
4.1.	Why this lecture matters	33
4.2.	Finishing linear probing	33
4.2.1.	Recap: the 7-wise argument	33
4.2.2.	Bounding the sixth moment via symmetrization	34
4.2.3.	The 5-wise trick: a binary tree argument	35
4.3.	Approximate membership: Bloom filters	35
4.3.1.	From a bit array to a Bloom filter	35
4.3.2.	Reducing the false positive rate	36
4.4.	Cuckoo hashing	36
4.4.1.	The cuckoo graph	37
4.5.	Bloomier filters and the retrieval problem	37
4.5.1.	The retrieval problem	37
4.5.2.	Construction via cuckoo hashing	38
4.6.	Cheat sheet	38
4.7.	FAQ — questions students actually asked	39
4.8.	Further reading	39
4.9.	Try it yourself	39
5.	Cuckoo Hashing Analysis and the Power of Two Choices	41
5.1.	Why this lecture matters	41
5.2.	Cuckoo hashing: finishing the insertion analysis	41
5.2.1.	The cuckoo graph	42
5.2.2.	Three insertion scenarios	42

5.2.3.	Setting up the analysis	42
5.2.4.	Bounding path probabilities	43
5.2.5.	Bounding single-cycle configurations	43
5.2.6.	Bounding double-cycle probability	44
5.2.7.	Wrapping up: choosing C	45
5.3.	The power of two choices	45
5.3.1.	Setup: balls into bins	45
5.3.2.	The algorithm	45
5.3.3.	Intuition via the layered argument	46
5.3.4.	Making it rigorous	47
5.3.5.	Power of d choices	47
5.4.	Preview: Fibonacci heaps	48
5.5.	Cheat sheet	48
5.6.	FAQ — questions students actually asked	49
5.7.	Further reading	50
5.8.	Try it yourself	50
6.	Amortized Analysis, Binomial Heaps, and Fibonacci Heaps	53
6.1.	Why this lecture matters	53
6.2.	Amortized analysis and the potential function method	53
6.2.1.	The idea	53
6.2.2.	The potential function method	54
6.2.3.	Designing a potential function	54
6.3.	Heaps: the landscape	55
6.4.	Binomial heaps	55
6.4.1.	Structure	55
6.4.2.	Operations	56
6.4.3.	Amortized analysis of insertion	56
6.5.	Fibonacci heaps	57
6.5.1.	Design philosophy: strategic laziness	57
6.5.2.	The problem with “ultra-lazy” decrease-key	57
6.5.3.	Cascading cuts: the fix	57
6.5.4.	Minimum tree sizes: Fibonacci numbers	57
6.5.5.	Operations in detail	58
6.5.6.	Amortized analysis	59
6.6.	Cheat sheet	60
6.7.	FAQ — questions students actually asked	61
6.8.	Further reading	62
6.9.	Try it yourself	62
7.	Splay Trees	65
7.1.	Why this lecture matters	65
7.2.	The dictionary problem in the BST model	65
7.2.1.	Rotations	66
7.3.	Splay trees: definition	66
7.4.	Why naive rotate-to-root fails	66
7.5.	The splay operation	67
7.5.1.	Splay on a path	67
7.6.	Properties of splay trees	67
7.6.1.	Static optimality (Sleator-Tarjan, 1985)	67
7.6.2.	Working set property	68
7.6.3.	Static finger property (Sleator-Tarjan, 1985)	68
7.6.4.	Dynamic finger property (Cole et al., 2000)	68
7.6.5.	Unified bound (Iacono, 2001)	68

7.7. The dynamic optimality conjecture	68
7.8. Amortized analysis of splay	69
7.8.1. The potential function	69
7.8.2. The splay lemma	69
7.8.3. Proof sketch for zig-zig (Case 2, left-left)	70
7.9. Corollary: $O(\log n)$ amortized	70
7.10. Static optimality via weights	71
7.11. Cheat sheet	71
7.12. FAQ — questions students actually asked	72
7.13. Further reading	72
7.14. Try it yourself	73
 II. Online Algorithms	 75
 8. Online Algorithms and Paging	 77
8.1. Why this lecture matters	77
8.2. Online algorithms and competitive analysis	77
8.3. Toy example 1: ski rental	78
8.4. Toy example 2: doubling search	78
8.5. The list update problem	79
8.5.1. Heuristics	79
8.5.2. Move-to-front is dynamically optimal	79
8.5.3. Proof via inversions	80
8.6. The paging problem	80
8.6.1. Connection to list update	80
8.6.2. Eviction policies	81
8.7. LRU is k -competitive	81
8.7.1. 1-bit LRU	81
8.7.2. Proof	81
8.8. Lower bound: no deterministic algorithm beats k	82
8.9. Circumventing the lower bound	82
8.9.1. 1. Cache associativity	82
8.9.2. 2. Resource augmentation (Sleator-Tarjan, 1985)	82
8.9.3. 3. Randomization	82
8.10. The marking algorithm	83
8.10.1. Proof sketch	83
8.11. Cheat sheet	84
8.12. FAQ — questions students actually asked	84
8.13. Further reading	85
8.14. Try it yourself	85
 9. Randomized Paging and the Online Primal-Dual Framework	 87
9.1. Why this lecture matters	87
9.2. Analysis of the Mark algorithm	87
9.2.1. Phase decomposition	88
9.2.2. Comparing Mark to OPT	88
9.2.3. Expected cost of Mark on stale requests	88
9.3. Lower bound: no randomised algorithm beats $\Omega(\log K)$	89
9.4. The K -server problem and open questions	89
9.5. Linear programming recap	90
9.5.1. LP duality	90
9.6. The online primal-dual framework	90
9.6.1. Approximate complementary slackness	90

9.7. Ski rental via primal-dual	91
9.7.1. LP formulation	91
9.7.2. The 2-competitive primal-dual algorithm	91
9.8. Cheat sheet	91
9.9. FAQ — questions students actually asked	92
9.10. Further reading	92
9.11. Try it yourself	93
10. Randomized Ski Rental, Online Set Cover, and Approximation Algorithms	95
10.1. Why this lecture matters	95
10.2. Randomised ski rental: beating the factor of 2	95
10.2.1. The fractional algorithm	96
10.2.2. Bounding the competitive ratio	96
10.2.3. Randomised rounding: from fractions to decisions	96
10.3. Online set cover	97
10.3.1. Problem definition	97
10.3.2. LP relaxation	97
10.3.3. The multiplicative-update algorithm	97
10.3.4. Analysis: the purely primal view	98
10.3.5. The primal-dual view	98
10.3.6. Randomised rounding for integrality	99
10.4. Transition to approximation algorithms	99
10.4.1. Weighted set cover and the greedy algorithm	99
10.4.2. Dual-fitting analysis (preview)	99
10.5. Cheat sheet	100
10.6. FAQ — questions students actually asked	100
10.7. Further reading	101
10.8. Try it yourself	101
III. Approximation Algorithms and LP	103
11. Integrality Gaps and Approximation Schemes	105
11.1. Why this lecture matters	105
11.2. Dual fitting for weighted set cover	105
11.2.1. The LP relaxation and its dual	105
11.2.2. The greedy algorithm and its dual interpretation	106
11.2.3. Proving y/H_n is dual-feasible	106
11.3. Vertex cover via primal-dual	107
11.3.1. The problem and its LP	107
11.3.2. The greedy algorithm	107
11.3.3. Primal-dual proof	107
11.4. Integrality gaps	108
11.4.1. What the gap measures	108
11.4.2. Vertex cover: gap of 2	108
11.4.3. Set cover: gap of $\Theta(\log n)$	108
11.5. Approximation schemes: PTAS, FPTAS, FPRAS	109
11.6. Knapsack	109
11.6.1. Problem definition	109
11.6.2. Exact solutions via dynamic programming	110
11.6.3. Fractional knapsack and the greedy approach	110
11.6.4. Modified greedy: a 2-approximation	110
11.6.5. PTAS for knapsack	111
11.7. Cheat sheet	112

11.8. FAQ — questions students actually asked	112
11.9. Further reading	113
11.10. Try it yourself	113
12. FPTAS, DNF Counting, and Semidefinite Programming	115
12.1. Why this lecture matters	115
12.2. FPTAS for knapsack	115
12.2.1. The rounding idea	115
12.2.2. The algorithm	116
12.2.3. Correctness proof	117
12.3. When FPTAS is impossible: strongly NP-hard problems	117
12.3.1. Bin packing	117
12.4. FPRAS for DNF counting	118
12.4.1. The problem	118
12.4.2. Why naive sampling fails	118
12.4.3. The Karp–Luby–Madras trick	118
12.4.4. The full algorithm	119
12.4.5. Analysis via Chernoff	120
12.5. Semidefinite programming	120
12.5.1. From LP to SDP	120
12.5.2. Positive semidefiniteness: three views	121
12.5.3. Vector programming	121
12.6. Max-Cut and the Goemans–Williamson algorithm	121
12.6.1. The problem	121
12.6.2. The 1/2 barrier	121
12.6.3. Quadratic program formulation	122
12.6.4. SDP relaxation	122
12.6.5. Rounding: random hyperplane	122
12.6.6. Analysis	122
12.7. Cheat sheet	123
12.8. FAQ — questions students actually asked	124
12.9. Further reading	124
12.10. Try it yourself	124
13. Learning Topic Models	127
13.1. Why this lecture matters	127
13.2. Supervised vs. unsupervised learning	127
13.3. Probabilistic models and learning	128
13.3.1. Warm-up: single Gaussian	128
13.3.2. Mixture of Gaussians	128
13.4. Maximum likelihood and NP-hardness	128
13.4.1. Ways around NP-hardness	129
13.5. Topic models: what and why	129
13.5.1. Assumption 1: bag of words	129
13.5.2. Assumption 2: a topic is a distribution over words	130
13.5.3. Assumption 3: a document is a distribution over topics	130
13.6. The topic matrix and document generation	130
13.6.1. The generative recipe	130
13.7. From topic models to matrix factorisation	131
13.8. The separability assumption	131
13.8.1. Translation to topic modelling	131
13.9. Geometric reduction: convex hull vertex-finding	132
13.9.1. Rescaling to convex combinations	132

13.10.The vertex-finding algorithm	132
13.10.1.The algorithm	133
13.11Practical considerations	134
13.11.1.Finite documents	134
13.11.2.Computational cost	134
13.12Cheat sheet	134
13.13FAQ — questions students actually asked	134
13.14Further reading	135
13.15Try it yourself	135
14.Solving Linear Programs: The Simplex Algorithm	137
14.1. Why this lecture matters	137
14.2. Standard form	137
14.2.1. How to get there	138
14.3. Geometry: the feasible polytope	138
14.3.1. Vertex definition	138
14.4. Finding a starting vertex	139
14.4.1. The auxiliary LP	139
14.5. Vertices are optimal	139
14.5.1. Proof sketch	139
14.5.2. Corollaries	140
14.6. Characterising vertices via linear algebra	140
14.6.1. Proof (both directions by contrapositive)	140
14.7. The Simplex algorithm	140
14.7.1. Bases and reduced costs	140
14.7.2. The algorithm	141
14.7.3. How pivoting works	141
14.8. Strong duality from Simplex	142
14.9. Worst-case complexity and beyond	142
14.9.1. Simplex: fast in practice, slow in theory	142
14.9.2. The ellipsoid method	142
14.10Cheat sheet	143
14.11FAQ — questions students actually asked	143
14.12Further reading	144
14.13Try it yourself	144
15.Strong Duality, Complementary Slackness, and the Ellipsoid Method	145
15.1. Why this lecture matters	145
15.2. Finishing Simplex: degeneracy and Bland's rule	145
15.2.1. Rewriting the LP in terms of the basis	146
15.2.2. The non-degenerate case	146
15.2.3. The degenerate case	146
15.3. Strong duality via Simplex	147
15.3.1. The shadow price vector	147
15.3.2. Dual feasibility at termination	147
15.4. Complementary slackness	147
15.4.1. Why complementary slackness matters	148
15.5. Simplex runtime and polytope diameter	148
15.5.1. Iteration count	148
15.5.2. The Hirsch conjecture	148
15.5.3. Smooth analysis	149
15.6. The ellipsoid method	149
15.6.1. From Simplex to polynomial time	149
15.6.2. Ellipsoids	150

15.6.3. The algorithm	150
15.6.4. Handling precision issues	150
15.6.5. From feasibility to optimisation	151
15.6.6. The oracle perspective	151
15.7. Interior point methods: a preview	152
15.7.1. The barrier approach	152
15.8. Cheat sheet	152
15.9. FAQ — questions students actually asked	153
15.10. Further reading	153
15.11. Try it yourself	154
16. Interior Point Methods: Path Following and Gradient Descent	157
16.1. Why this lecture matters	157
16.2. The interior point setup	157
16.2.1. The LP and its barrier reformulation	157
16.3. The central path	158
16.4. The cost gap: why large λ suffices	159
16.4.1. Gradient condition	159
16.4.2. Bounding the gap	159
16.5. How many iterations?	160
16.5.1. The lambda schedule	160
16.5.2. Recent improvements	160
16.6. The vague outline, made concrete	160
16.7. Digression: first-order optimisation	161
16.8. Gradient descent	161
16.8.1. The upper bound from Taylor's theorem	161
16.8.2. The update rule	162
16.8.3. The lower bound from strong convexity	162
16.8.4. Putting it together: convergence rate	162
16.9. Accelerated gradient descent	163
16.10. Looking ahead: Newton's method	164
16.11. Cheat sheet	164
16.12. FAQ — questions students actually asked	165
16.13. Further reading	165
16.14. Try it yourself	166
17. Newton's Method and Completing the Interior Point Method	169
17.1. Why this lecture matters	169
17.2. Newton's method: the second-order update	169
17.2.1. Setup and motivation	169
17.2.2. Convergence theorem	170
17.2.3. Proof of the convergence theorem	171
17.2.4. Comparison with gradient descent	172
17.3. Centrality and the path-following strategy	172
17.3.1. The barrier function (recap)	172
17.3.2. Centrality measure	172
17.3.3. Recovering fine centrality from coarse centrality	173
17.4. How much can we increment λ ?	173
17.4.1. The projection bound	173
17.4.2. The $O(\sqrt{m})$ bound	174
17.4.3. Putting it all together: the complete iteration count	174
17.5. Initialising the IPM	175
17.5.1. Phase 1: Augment the LP	175
17.5.2. Phase 2: Find the analytic centre	175

17.5.3. Phase 3: Walk forward with the real cost	175
17.6. Beyond $\sqrt{m}$: the Lee–Sidford improvement	175
17.6.1. Why the standard barrier is wasteful	176
17.6.2. The algorithmic challenge	176
17.7. Cheat sheet	176
17.8. FAQ — questions students actually asked	177
17.9. Further reading	177
17.10. Try it yourself	178
IV. Multiplicative Weights and Network Flows	179
18. Multiplicative Weights and the Expert Advice Framework	181
18.1. Why this lecture matters	181
18.2. Warm-up: the halving algorithm	181
18.2.1. Setup	181
18.2.2. Majority vote with elimination	182
18.2.3. Handling imperfect experts: the revival trick	182
18.3. The weighted majority algorithm	182
18.3.1. The key insight: penalise, don't kill	182
18.3.2. Analysis via potential function	183
18.4. The full multiplicative weights update	184
18.4.1. Generalising the model	184
18.4.2. The algorithm	184
18.4.3. The general theorem	185
18.4.4. Proof of the general theorem	185
18.5. Application: approximately solving LPs	186
18.5.1. The feasibility framework	186
18.5.2. The expert analogy	186
18.5.3. What property does P need?	186
18.5.4. Example: fractional set cover	187
18.5.5. The full MWU-LP pipeline	187
18.6. Cheat sheet	187
18.7. FAQ — questions students actually asked	188
18.8. Further reading	189
18.9. Try it yourself	189
19. Multiplicative Weights for LPs and Introduction to Network Flows	191
19.1. Why this lecture matters	191
19.2. Multiplicative weights for LP feasibility	191
19.2.1. The multiplicative weights guarantee (recap)	191
19.2.2. The LP feasibility problem and row-bounded oracles	192
19.2.3. The theorem: MW + oracle $\Rightarrow$ approximate LP solver	192
19.2.4. Proof sketch	193
19.3. Application: fractional set cover	193
19.3.1. Building the oracle	194
19.3.2. Bounding γ	194
19.4. Network flows: definitions	194
19.4.1. Setup	194
19.4.2. Flows	195
19.4.3. A worked example	195
19.4.4. Max flow as a linear program	195
19.5. The Ford–Fulkerson method	196
19.5.1. Residual graphs	196

19.5.2. The algorithm	196
19.5.3. Correctness: the max-flow min-cut theorem	197
19.6. Cheat sheet	197
19.7. FAQ — questions students actually asked	198
19.8. Further reading	198
19.9. Try it yourself	199
20. Faster Max Flow: Capacity Scaling and Blocking Flows	201
20.1. Why this lecture matters	201
20.2. Polynomial-time classifications	201
20.3. State of the art (as of 2014)	202
20.4. Capacity scaling	202
20.4.1. The idea	202
20.4.2. Pseudocode	203
20.4.3. Correctness	203
20.4.4. Running time	203
20.5. Blocking flows	204
20.5.1. Level graphs and admissible edges	204
20.5.2. Definition of a blocking flow	204
20.5.3. The key claim: distance strictly increases	204
20.5.4. Dinic's algorithm	205
20.6. Finding a blocking flow	205
20.6.1. Unit-capacity case: modified DFS	205
20.6.2. General-capacity case	206
20.7. Putting it all together	207
20.7.1. Dinic's algorithm: general capacities	207
20.7.2. Dinic's algorithm: unit capacities	207
20.7.3. Bipartite matching	207
20.7.4. With link-cut trees	207
20.7.5. Combined with scaling	207
20.8. Cheat sheet	208
20.9. FAQ — questions students actually asked	208
20.10. Further reading	209
20.11. Try it yourself	209
21. Link-Cut Trees	211
21.1. Why this lecture matters	211
21.2. Blocking flow revisited	211
21.2.1. Why DFS wastes work in general capacities	212
21.2.2. A concrete example	212
21.3. The link-cut tree interface	212
21.4. Blocking flow with link-cut trees	213
21.4.1. Counting operations	213
21.5. Preferred-path decomposition	214
21.5.1. Preferred children	214
21.5.2. Worked example	214
21.5.3. The tree of paths	214
21.6. Auxiliary trees: splay trees keyed by depth	215
21.6.1. Path-parent pointers	215
21.6.2. Concrete layout	215
21.7. The access operation	215
21.7.1. Step-by-step	215
21.7.2. Cost of access	216

21.8. Other operations via access	216
21.8.1. <code>find_root(v)</code>	216
21.8.2. <code>find_min(v)</code>	216
21.8.3. <code>cut(v)</code>	217
21.8.4. <code>link(v, w, x)</code>	217
21.9. Putting it all together	217
21.10. Cheat sheet	218
21.11. FAQ — questions students actually asked	218
21.12. Further reading	219
21.13. Try it yourself	219
22. Link-Cut Tree Analysis and Min-Cost Max Flow	221
22.1. Why this lecture matters	221
22.2. Amortised analysis of link-cut trees	221
22.2.1. Runtime in terms of preferred-child changes	221
22.3. Heavy-light decomposition	222
22.3.1. Definitions	222
22.3.2. Key property: few light edges on any root-to-leaf path	222
22.4. Bounding preferred-child changes	222
22.4.1. The decomposition	223
22.4.2. LPCC across the three operations	223
22.4.3. Totalling up	223
22.5. The $O(m \log^2 n)$ bound	224
22.6. Augmented BSTs for path operations	224
22.7. Min-cost max flow	224
22.7.1. Problem definition	224
22.7.2. Min-cost circulation	225
22.8. Negative-cycle cancelling	225
22.8.1. The algorithm	225
22.8.2. Correctness	225
22.8.3. Detecting negative cycles	225
22.8.4. Runtime: pseudo-polynomial	226
22.9. Price functions and reduced costs	226
22.9.1. Definition	226
22.9.2. Key property: cycle costs are invariant	226
22.9.3. Optimality characterisation	226
22.10. Faster min-cost flow for unit capacities	227
22.10.1. Setup	227
22.10.2. The algorithm	227
22.10.3. Why Dijkstra stays valid	227
22.10.4. Runtime	227
22.10.5. Handling negative costs or general capacities	228
22.11. Cheat sheet	228
22.12. FAQ — questions students actually asked	228
22.13. Further reading	229
22.14. Try it yourself	229
V. Streaming and Sketching	231
23. Inclusion-Exclusion, Möbius Inversion, and the Streaming Model	233
23.1. Why this lecture matters	233
23.2. Möbius inversion: undoing the Zeta transform	233
23.2.1. The two transforms	233

23.2.2. Worked example	234
23.2.3. The inversion theorem	234
23.3. Application: k -colourability	235
23.3.1. From colourings to independent sets	235
23.3.2. The key identity	235
23.4. Computing the transforms efficiently	236
23.4.1. Brute force: $O^*(3^n)$ time, polynomial space	236
23.4.2. Yates' algorithm: $O(n \cdot 2^n)$ time, $O(n \cdot 2^n)$ space	236
23.4.3. Summary of running times for k -colourability	237
23.5. The streaming model	237
23.5.1. Setup	237
23.5.2. The distinct-elements problem (F_0)	238
23.5.3. A linear lower bound for deterministic algorithms	238
23.5.4. The Flajolet–Martin algorithm (idealized)	239
23.5.5. What's possible and what isn't	240
23.6. Cheat sheet	240
23.7. FAQ — questions students actually asked	241
23.8. Further reading	241
23.9. Try it yourself	241
24. The Power of Random Signs	243
24.1. Why this lecture matters	243
24.2. ℓ_2 -norm estimation via linear sketching	243
24.2.1. The problem	243
24.2.2. Linear sketches	244
24.2.3. The AMS sketch	244
24.2.4. Analysis: expectation	244
24.2.5. Analysis: variance	245
24.2.6. Putting it together: Chebyshev	245
24.2.7. Amplification via the median trick	245
24.3. The Thorup–Zhang sparse sketch	246
24.3.1. Motivation: update time	246
24.3.2. Construction	246
24.4. Application: cosine similarity	246
24.5. Fast regression via subspace embeddings	247
24.5.1. The least-squares bottleneck	247
24.5.2. Sketched regression	247
24.5.3. Subspace embeddings	247
24.5.4. How many rows does Π need?	248
24.5.5. Time to compute ΠA	248
24.6. Johnson–Lindenstrauss dimensionality reduction	249
24.6.1. Statement	249
24.6.2. Proof idea	249
24.7. Point query via JL	249
24.7.1. The problem	249
24.7.2. Construction	250
24.7.3. Getting the right Π	250
24.8. Four-wise independence suffices	250
24.9. Cheat sheet	250
24.10FAQ — questions students actually asked	251
24.11Further reading	251
24.12Try it yourself	252

Preface

These are companion notes for Harvard COMPSCI 224 — Advanced Algorithms, taught by Jelani Nelson in Fall 2014. They are not transcripts. Each chapter reorganises a lecture for clarity, adds intuition and diagrams, and includes exercises — turning a 75-minute video into a self-contained study resource.

Who this is for. A graduate student (or ambitious undergrad) with CLRS-level algorithms, basic probability, and mathematical maturity. You should be comfortable with asymptotic notation, amortised analysis, and basic randomised algorithms.

What it covers. 24 lectures across five topics:

1. Data Structures — Word-RAM, van Emde Boas, fusion trees, hashing (universal, cuckoo, Bloom filters), amortised analysis, binomial and Fibonacci heaps, splay trees
2. Online Algorithms — Competitive analysis, paging, ski rental, primal-dual framework
3. Approximation and LP — Set cover, vertex cover, integrality gaps, PTAS/FPTAS, SDP rounding, simplex, ellipsoid, interior point methods
4. Multiplicative Weights and Flows — Expert advice, MW for LPs, max flow (Ford-Fulkerson, capacity scaling, blocking flows), link-cut trees, min-cost flow
5. Streaming and Sketching — Distinct elements, L2 estimation, Johnson-Lindenstrauss, fast regression

Each chapter includes a cheat sheet, FAQ from the actual lecture Q&A, primary-source references, and exercises.

Part I.

Data Structures

1. The Word-RAM Model and Predecessor Data Structures

COMPSCI 224 Lecture 1 — Fall 2014

The comparison-based $\Omega(n \log n)$ sorting lower bound is an artifact of a weak computational model. By moving to the Word-RAM — where keys are integers fitting in w -bit machine words and bitwise operations are free — predecessor queries can be answered in $O(\log w)$ time (van Emde Boas trees) or $O(\log_w n)$ time (fusion trees), beating binary search and implying $O(n\sqrt{\log n})$ sorting.

1.1. Why this lecture matters

Sorting in $O(n \log n)$ is optimal only if your algorithm is limited to pairwise comparisons. Real machines manipulate integers with shifts, masks, and arithmetic — operations that the comparison model ignores. This lecture formalises that power as the Word-RAM model and immediately exploits it to build a predecessor data structure (van Emde Boas trees) whose query time depends on the *word size*, not the *number of elements*.

The 30-second story

1. The $\Omega(n \log n)$ sorting bound assumes comparison-only access to keys.
2. Real CPUs perform bitwise operations, shifts, and arithmetic in $O(1)$ — the Word-RAM model captures this.
3. The static predecessor problem asks: given a stored set S , find the largest element $\leq z$.
4. Van Emde Boas trees solve predecessor in $O(\log w)$ time by recursively splitting the universe into $\sqrt{u}$ -sized clusters.
5. Hashing compresses space from $O(u)$ to $O(n)$; Y-fast tries achieve the same bounds via indirection.
6. Combined with fusion trees (next lecture), predecessor is solvable in $O(\sqrt{\log n})$, implying sub- $n \log n$ sorting.

1.2. The Word-RAM model

Binary search trees solve predecessor in $O(\log n)$ using only comparisons. To go faster, grant the algorithm more power.

Intuition. When you write C code, you routinely shift bits, mask with AND, and do integer arithmetic — all in one clock cycle. The Word-RAM model simply says: *count those operations*.

Model definition

Items are integers in $\{0, 1, \dots, 2^w - 1\}$, where w is the word size and $u = 2^w$ is the universe size. Given two words x, y , the following take $O(1)$ time:

- Integer arithmetic: $+$, $-$, $\times$, $\lfloor \cdot / \cdot \rfloor$
- Bitwise: NOT, AND, OR, XOR
- Bit shifts: $x \ll k$, $x \gg k$

We always assume $w \geq \log n$ (a pointer must address at least n memory cells).

Multiplication produces a result fitting in two words — the upper and lower halves are individually accessible. This is consistent with how hardware multipliers work.

1.3. The static predecessor problem

Problem. Store a set $S = \{x_1, \dots, x_n\}$ of w -bit integers. Support $\text{pred}(z) = \max\{x \in S : x < z\}$. Minimise space and query time.

A sorted array with binary search gives $O(\log n)$ query (static). A balanced BST (e.g., red-black tree) gives $O(\log n)$ query and $O(\log n)$ updates (dynamic).

Connection to sorting. A dynamic predecessor structure immediately yields a sorting algorithm: insert all n elements, then extract them in order by repeatedly taking the predecessor of the current maximum. If each operation costs T , sorting costs $O(nT)$.

! Results at a glance

Data structure	Query	Update	Space	Model
Balanced BST	$O(\log n)$	$O(\log n)$	$O(n)$	comparison
Van Emde Boas	$O(\log w)$	$O(\log w)$	$O(n)^*$	Word-RAM
Fusion trees	$O(\log_w n)$	— (static)	$O(n)$	Word-RAM
Best of both	$O(\sqrt{\log n})$	—	$O(n)$	Word-RAM

*With hashing.

Since $w \geq \log n$, the minimum of $\log w$ and $\log_w n$ is maximised when the two are equal: $\log w = \log n / \log w$, giving $\log n = (\log w)^2$, hence both terms equal $\sqrt{\log n}$.

Open problem. Can integers be sorted in $O(n)$ time in the Word-RAM? The best known deterministic bound is $O(n \log \log n)$ (Han, STOC 2002); the best randomised bound is $O(n\sqrt{\log \log n})$ (Han and Thorup, FOCS 2002).

1.4. Van Emde Boas trees

1.4.1. The recursive idea

Think of a universe of size u as a two-digit number in base $\sqrt{u}$. Every integer $x \in \{0, \dots, u-1\}$ decomposes as:

$$x = c \cdot \sqrt{u} + i, \quad c = \lfloor x / \sqrt{u} \rfloor, \quad i = x \bmod \sqrt{u} \quad (1.1)$$

Here c is the cluster index and i is the position within the cluster. Both fit in $\frac{w}{2}$ bits, extractable in $O(1)$ via a shift and a mask.

A VEB structure on universe size u stores:

- $v.\text{cluster}[0..\sqrt{u}-1]$ — an array of $\sqrt{u}$ sub-structures, each on universe $\sqrt{u}$.
- $v.\text{summary}$ — a single VEB structure on universe $\sqrt{u}$ tracking which clusters are non-empty.
- $v.\text{min}, v.\text{max}$ — the minimum and maximum elements (integers).

💡 Why store min separately?

The minimum is *not* stored recursively in its cluster — it lives only in the `min` field. This “lazy propagation” trick ensures that inserting into an empty cluster triggers at most one recursive call, keeping insertion $O(\log \log u)$ rather than $O(w)$.

1.4.2. Predecessor query

```
def predecessor(v, x):
    """Return largest element in v that is < x."""
    c, i = high(x), low(x)          # decompose x = c*sqrt(u) + i

    if x > v.max:
        return v.max                # x exceeds everything

    if v.cluster[c] is not None and v.cluster[c].min < x:
        # predecessor is in same cluster
        return join(c, predecessor(v.cluster[c], i))
    else:
        # predecessor is in an earlier cluster
        c_prime = predecessor(v.summary, c)
        return join(c_prime, v.cluster[c_prime].max)
```

At most one recursive call occurs in any branch, on a sub-problem of universe size $\sqrt{u}$.

$$T(u) = T(\sqrt{u}) + O(1) \implies T(u) = O(\log \log u) = O(\log w) \quad (1.2)$$

1.4.3. Insertion

```
def insert(v, x):
    if v.min is None:                # empty structure
        v.min = v.max = x
        return

    if x < v.min:
        x, v.min = v.min, x          # new min goes to buffer; old min gets inserted

    c, i = high(x), low(x)

    if v.cluster[c] is None or v.cluster[c].min is None:
        insert(v.summary, c)         # mark cluster as non-empty
        v.cluster[c] = new_veb()
        v.cluster[c].min = v.cluster[c].max = i
    else:
```

```

insert(v.cluster[c], i)

if x > v.max:
    v.max = x

```

Key insight: the two recursive calls (`insert(v.summary, c)` and `insert(v.cluster[c], i)`) never *both* recurse deeper. If the cluster was empty, inserting c into the summary recurses, but inserting i into the new cluster hits the empty-structure base case immediately. Therefore:

$$T(u) = T(\sqrt{u}) + O(1) \implies T(u) = O(\log \log u) \quad (1.3)$$

 Without the min trick, insertion is $O(w)$

If the minimum were stored recursively in its cluster, both the summary insertion and the cluster insertion could recurse, giving $T(u) = 2T(\sqrt{u}) + O(1)$, which solves to $T(u) = O(w)$.

1.5. From $O(u)$ space to $O(n)$ space

1.5.1. The space problem

The naive recurrence for space is:

$$S(u) = (\sqrt{u} + 1)S(\sqrt{u}) + O(1) \implies S(u) = \Theta(u)$$

Most clusters are empty — storing $\sqrt{u}$ pointers regardless of occupancy wastes space.

1.5.2. Solution: replace arrays with hash tables

Replace each cluster array with a dictionary (hash table). Keys are cluster indices c ; values are pointers to the corresponding non-empty VEB sub-structure.

 The dictionary problem

A dictionary stores key-value pairs (keys and values each fit in one word) with:

- `query(k)`: return associated value, or null — $O(1)$ worst-case
- `insert(k, v)` / `delete(k)`: $O(1)$ expected (or whp)

Linear space. Achieved by dynamic perfect hashing (Dietzfelbinger et al.).

Why $O(n)$ space? Each element is stored as the min of exactly one sub-structure at some recursion level. Charge the cost of the hash-table entry (cluster-id + pointer = 2 words) to that minimum element. Every element is charged exactly once.

1.6. X-fast tries and Y-fast tries

An alternative route to the same $O(\log \log u)$ bound uses a binary trie over the universe.

1.6.1. X-fast trie

1. Store all n elements in a doubly linked list (sorted order).
2. Build a perfect binary tree of height w over the u leaves. Each internal node stores the OR of its children (1 if any descendant is present).
3. For a query z : walk up from leaf z until finding a 1-node, then locate the predecessor or successor via the linked list.

The leaf-to-root path is monotone (0's then 1's), so binary search finds the transition in $O(\log w) = O(\log \log u)$ steps. To binary-search on ancestors, store the tree as an array (heap-order); the k -th ancestor is at index $\lfloor \text{pos}/2^k \rfloor$.

Space issue: storing all internal nodes takes $O(u)$. Fix: store only the 1-nodes in a hash table, one per tree level. Space becomes $O(nw)$ — better than u but not linear.

1.6.2. Y-fast trie (Willard, IPL 1983)

Apply indirection: partition the n elements into n/w groups of $\Theta(w)$ consecutive elements each. Build an X-fast trie on the n/w representatives (one per group). Store each group in a balanced BST.

	Space	Query	Insert (amortised)
X-fast trie on n/w items	$O(n)$	$O(\log \log u)$	$O(w)$
BST within a group ($\Theta(w)$ items)	$O(w)$	$O(\log w)$	$O(\log w)$
Y-fast trie total	$O(n)$	$O(\log \log u)$	$O(\log \log u)$ amortised

When a group overflows (size $> 2w$), split it and insert a new representative into the X-fast trie. This happens every $\Theta(w)$ insertions, amortising the $O(w)$ X-fast insertion cost.

💡 Indirection as a general technique

Whenever a data structure has cost C for n items but you want cost C for n/k items with $O(\log k)$ overhead per group, indirection lets you trade a factor of k in the global structure's size for a $\log k$ local search cost. This trick recurs throughout advanced data structures.

1.7. Optimality

A matching lower bound (Patrascu and Thorup, FOCS 2006) shows that for any linear-space data structure in the Word-RAM, predecessor query time is $\Omega(\min(\log w, \log_w n))$. Van Emde Boas trees and fusion trees are therefore optimal (up to constants) for the static predecessor problem.

1.8. Cheat sheet

! Cheat sheet — Lecture 1

Word-RAM model: integers in $\{0, \dots, 2^w - 1\}$; arithmetic, bitwise ops, shifts in $O(1)$; $w \geq \log n$.

Static predecessor: store n keys, answer $\text{pred}(z) = \max\{x \in S : x < z\}$.

Van Emde Boas trees (universe $u = 2^w$):

- Decompose $x = c\sqrt{u} + i$. Store $\sqrt{u}$ clusters + summary + min/max.

- Query/insert: $T(u) = T(\sqrt{u}) + O(1) = O(\log \log u) = O(\log w)$.
- Min stored in buffer (not recursively) — prevents double recursion.
- Space $O(u)$ with arrays; $O(n)$ with hash tables (charge to min elements).

X-fast trie: binary trie + hash tables per level. Query $O(\log w)$, space $O(nw)$.

Y-fast trie: indirection — n/w groups in X-fast trie, $\Theta(w)$ -item BSTs. Query $O(\log w)$, space $O(n)$, amortised insert $O(\log w)$.

Fusion trees (next lecture): query $O(\log_w n)$, space $O(n)$.

Combined: $O(\min(\log w, \log_w n)) = O(\sqrt{\log n})$. Optimal for linear space.

1.9. FAQ — questions students actually asked

Q: If multiplication overflows a word, what happens? A: The product of two w -bit integers fits in $2w$ bits. You can access the upper and lower halves separately. Alternatively, treat overflow as modular arithmetic (the low-order w bits). For fusion trees (next lecture), we will exploit full $2w$ -bit products.

Q: Why do we need dynamic predecessor for sorting — why not static? A: Building a static predecessor structure requires polynomial preprocessing (you already need the sorted order). A *dynamic* structure starts empty and accepts insertions one-by-one, which is exactly what a sorting algorithm does.

Q: In the VEB space argument, where exactly does each element get “charged”? A: Every element x is the min of exactly one sub-VEB at some recursion depth. The hash-table entry in x ’s parent node (storing the cluster-id and pointer) is charged to x . Since each element is a minimum in exactly one place, total charge is $O(n)$.

Q: Can the VEB hash-table solution be made fully deterministic (worst-case constant query)? A: Yes. Dynamic perfect hashing (e.g., cuckoo hashing with worst-case $O(1)$ lookup) suffices. The insertion remains expected or high-probability $O(1)$, so VEB updates are $O(\log w)$ in expectation. Fully worst-case $O(\log w)$ updates require deamortisation techniques.

Q: A student suggested storing a Git repo for scribe notes — is there context? A: A student proposed using version control instead of static PDFs on the course website. This is a logistics aside, not algorithmically relevant.

1.10. Further reading

- van Emde Boas, P. “Preserving order in a forest in less than logarithmic time and linear space.” *Information Processing Letters*, 6(3):80–82, 1977. — The original data structure; space $O(u)$.
- Willard, D. E. “Log-logarithmic worst-case range queries are possible in space $\Theta(N)$.” *Information Processing Letters*, 17(2):81–84, 1983. — X-fast and Y-fast tries achieving linear space.
- Fredman, M. L. and Willard, D. E. “Surpassing the information theoretic bound with fusion trees.” *Journal of Computer and System Sciences*, 47(3):424–436, 1993. — Fusion trees; $O(\log_w n)$ predecessor.
- Patrascu, M. and Thorup, M. “Time-space trade-offs for predecessor search.” *Proc. 38th STOC*, 2006. — Matching $\Omega(\min(\log w, \log_w n))$ lower bound.
- Han, Y. “Deterministic sorting in $O(n \log \log n)$ time and linear space.” *Proc. 34th STOC*, 2002. — Best known deterministic integer sorting.

1.11. Try it yourself

1. Warm-up. Show that deletion in a VEB tree also takes $O(\log \log u)$ time. Describe the pseudocode, paying attention to how min and max are maintained and when the summary must be updated.
2. Space accounting. The “charge to the minimum” argument assumes each element is the minimum of exactly one sub-structure. Construct a small example ($u = 16$, $n = 4$) and trace which element is charged at each recursion level. Verify total charge is $4 \cdot O(1) = O(n)$.
3. Y-fast split cost. In a Y-fast trie, when a group of size $2w$ splits, you insert a new representative into the X-fast trie (cost $O(w)$). Prove that the amortised insertion cost is $O(\log w)$ using the potential method. (Hint: let the potential of a group of size s be $\Phi = c \cdot |s - w|$ for an appropriate constant c .)
4. Implementation. Implement a simplified VEB tree in Python for universe size $u = 2^{16}$. Support insert, predecessor, and successor. Verify experimentally that query time is insensitive to n (for fixed u) and grows as $\log w$ when you vary u .

```
class VEB:
    def __init__(self, u):
        self.u = u
        self.min = None
        self.max = None
        if u > 2:
            self.sqrt_u = int(u ** 0.5)
            self.cluster = {} # hash table: cluster_id -> VEB(sqrt_u)
            self.summary = VEB(self.sqrt_u)

    def high(self, x):
        return x // self.sqrt_u

    def low(self, x):
        return x % self.sqrt_u

    def insert(self, x):
        # Implement following the pseudocode above
        ...
```

5. Lower bound intuition. The Patrascu-Thorup lower bound shows $\Omega(\min(\log w, \log_w n))$ for linear-space predecessor. Explain informally why making the word size larger doesn't help indefinitely: what happens to fusion trees as w grows, and what happens to VEB trees?

2. Fusion Trees

COMPSCI 224 Lecture 2 — Fall 2014

Fusion trees achieve $O(\log_w n)$ predecessor query time using $O(n)$ space by packing $w^{1/5}$ keys per node and searching each node in $O(1)$ via three Word-RAM techniques: sketch compression (reducing key width through multiplication), word-level parallelism (comparing against all keys simultaneously via subtraction), and constant-time most-significant-bit extraction.

2.1. Why this lecture matters

Van Emde Boas trees are fast when the word size w is small. Fusion trees are the complement: they shine when w is large relative to $\log n$. Together they yield $O(\sqrt{\log n})$ predecessor — optimal for linear space. The real payoff of this lecture is the *toolkit*: sketch compression, parallel comparison, and MSB extraction are bit-manipulation primitives that recur throughout algorithm design.

The 30-second story

1. A k -ary search tree has height $\log_k n$; set $k = w^{1/5}$ to get height $O(\log_w n)$.
2. A node holds k keys totalling $kw > w$ bits — it doesn't fit in a word. Sketch compression reduces each key to $O(k^4)$ bits via a carefully chosen multiplication.
3. A query key's sketch might land in the wrong position; an XOR + MSB computation identifies and corrects the error.
4. Word-level parallelism compares the query against all k sketches simultaneously using a single subtraction.
5. MSB in $O(1)$ is achieved by splitting into $\sqrt{w}$ chunks, summarising non-empty chunks, and applying parallel comparison at the chunk level.
6. Each node is processed in $O(1)$, giving total query time $O(\log_w n)$.

2.2. Erratum: VEB space with hashing

Subtlety in the CLRS exercise

The VEB-with-hash-tables space argument from Lecture 1 (and CLRS Exercise 20-1(f)) contains a subtle error. The “charge each hash entry to the minimum in that cluster” argument slightly over-counts. The correct bound is still $O(n)$ but requires a more careful argument — this was assigned as a problem set exercise. Y-fast tries, as presented, do achieve $O(n)$ space without this subtlety.

2.3. From binary to k -ary search trees

2.3.1. The basic structure

Replace the binary branching of a BST with k -ary branching. Each internal node stores $k - 1$ sorted keys $x_0 < x_1 < \dots < x_{k-2}$ and has k children.

With n total keys, the tree height is $\lceil \log_k n \rceil$.

2.3.2. Choosing k

Set $k = \Theta(w^{1/5})$. Then:

$$\text{height} = O(\log_{w^{1/5}} n) = O\left(\frac{\log n}{\frac{1}{5} \log w}\right) = O(\log_w n) \quad (2.1)$$

The catch: processing a node naively (linear scan: $O(k)$; binary search: $O(\log k)$) multiplied by the height never beats $O(\log n)$. To win, each node must be processed in $O(1)$ time.

! The core challenge

A single node has $k = \Theta(w^{1/5})$ keys, each w bits wide. Total node size: $\Theta(w^{6/5})$ bits — *more than a word*. How do you search it in $O(1)$?

2.4. Sketch compression

2.4.1. Branch bits

Consider the k keys $x_0 < x_1 < \dots < x_{k-1}$ in a single node. Write them as paths in a binary trie from the MSB to the LSB.

A branch bit is a bit position where at least two of the k paths diverge. There are at most $r \leq k - 1$ branch bits at positions $b_0 < b_1 < \dots < b_{r-1}$.

Claim: projecting every key to just its branch-bit positions preserves order. That is, if $x_i < x_j$, then $\text{sk}(x_i) < \text{sk}(x_j)$ where $\text{sk}(x)$ extracts bits $b_0, b_1, \dots, b_{r-1}$ from x .

This is immediate: the first bit where x_i and x_j differ is, by definition, a branch bit. All earlier branch bits are identical between them, so lexicographic order is preserved.

2.4.2. The sketch function

Define:

$$\text{sk}(x) = (x_{b_0}, x_{b_1}, \dots, x_{b_{r-1}}) \quad (2.2)$$

where x_{b_i} denotes the b_i -th bit of x . Each sketch is $r < k = w^{1/5}$ bits. Storing all k sketches takes $O(k \cdot r) = O(w^{2/5})$ bits — comfortably fits in one word.

🔥 Sketches can lie about query position

For stored keys x_i , sketches preserve order. But a *query* $q \notin \{x_0, \dots, x_{k-1}\}$ may sketch to a position that disagrees with its true rank among the x_i 's. We handle this next.

2.5. Handling arbitrary queries

2.5.1. The problem

Suppose $\text{sk}(q)$ lands between $\text{sk}(x_i)$ and $\text{sk}(x_{i+1})$. This does *not* guarantee $x_i < q < x_{i+1}$. The sketch lost information at some non-branch bit where q diverged from the stored keys.

2.5.2. The fix: find the divergence point

Let y be the node in the binary trie where q first departs from the paths of $\{x_0, \dots, x_{k-1}\}$. Compute this via:

$$y = \text{MSB}(x_i \oplus q) \quad \text{and} \quad y' = \text{MSB}(x_{i+1} \oplus q) \quad (2.3)$$

Take the more significant of the two — that is the first bit where q leaves the trie. This tells you:

- If q fell off to the right (bit was 1 where the path went 0): form $e = y \parallel 0 \parallel 1 \dots 1$. This e is larger than everything in the left subtree of y , so its rank among the x_i gives q 's true position.
- If q fell off to the left (bit was 0 where the path went 1): form $e = y \parallel 1 \parallel 0 \dots 0$. This e is smaller than everything in the right subtree of y .

Since e shares all branch bits with some x_i , we have $\text{sk}(e)$ correctly positioned among the stored sketches — and e 's rank equals q 's rank.

💡 Why XOR + MSB?

$x_i \oplus q$ has a 1 in every position where the two differ. The most significant such 1 is the *first* point of divergence (reading from MSB to LSB) — exactly the trie node where q 's path separates from x_i 's.

2.6. Sketch compression via multiplication

2.6.1. The problem

After masking out the r branch bits from x , you have a w -bit word with only r non-zero positions — potentially spread across the entire word. You need to *compress* them into a contiguous block of $O(r^4)$ bits so all k compressed sketches fit in a single word.

2.6.2. The multiplication trick

Let X denote x after masking (only branch-bit positions are non-zero):

$$X = \sum_{i=0}^{r-1} x_{b_i} \cdot 2^{b_i}$$

Choose a multiplier $M = \sum_{i=0}^{r-1} 2^{m_i}$ such that the product $X \cdot M$ places the branch bits in known, compact positions:

$$X \cdot M = \sum_{i=0}^{r-1} \sum_{j=0}^{r-1} x_{b_i} \cdot 2^{b_i+m_j} \quad (2.4)$$

The “diagonal” terms $b_i + m_j$ are where the i -th sketch bit lands. If we choose M so that:

1. All sums $b_i + m_j$ are distinct — no carries corrupt the result.
2. The diagonal sums are increasing: $b_0 + m_0 < b_1 + m_1 < \dots < b_{r-1} + m_{r-1}$.
3. The range is small: $(b_{r-1} + m_{r-1}) - (b_0 + m_0) = O(r^4)$.

Then after multiplying, mask out just the r diagonal positions and shift right by $b_0 + m_0$. The result is an $O(r^4) = O(w^{4/5})$ -bit compressed sketch.

i Computing a sketch at query time

The sketch operation is: mask $\rightarrow$ multiply by $M \rightarrow$ mask diagonal $\rightarrow$ shift. Four $O(1)$ Word-RAM operations. M and the masks are precomputed per node during construction.

2.6.3. Proof sketch: choosing the m_i

Lemma. There exist $m_0, \dots, m_{r-1}$ satisfying conditions (1)–(3) above.

Proof (by induction). Work modulo r^3 . At step t , having chosen $m'_0, \dots, m'_{t-1}$ such that all $b_i + m'_j$ are distinct mod r^3 , the new m'_t must avoid $r \cdot t < r^2$ forbidden residues. Since there are r^3 residues available and $r^2 < r^3$ are forbidden, a valid choice exists.

To get the final m_i : set $m_i = m'_i + i \cdot r^3 + (w - b_i)$ rounded to the nearest multiple of r^3 . This places $b_i + m_i$ in the i -th block of width r^3 , guaranteeing condition (2). The total span is $r \cdot r^3 = r^4 = O(w^{4/5})$, giving condition (3). Distinctness mod r^3 implies global distinctness.

2.7. Word-level parallelism: parallel comparison

2.7.1. The node sketch word

At each fusion tree node, precompute and store a single word:

$$\text{NODE} = 1 \parallel \text{sk}(x_0) \parallel 1 \parallel \text{sk}(x_1) \parallel \dots \parallel 1 \parallel \text{sk}(x_{k-1}) \quad (2.5)$$

Each sketch block is padded to a fixed width (say r^4 bits), with a leading 1-bit sentinel.

2.7.2. Query comparison

Given query q , form the replicated sketch:

$$\text{QUERY} = 0 \parallel \text{sk}(q) \parallel 0 \parallel \text{sk}(q) \parallel \dots \parallel 0 \parallel \text{sk}(q)$$

This is computed by sketching q once and multiplying by a word of the form $1 \underbrace{00 \dots 0}_{r^4} 1 \underbrace{00 \dots 0}_{r^4} \dots$.

Now compute:

$$D = \text{NODE} - \text{QUERY} \tag{2.6}$$

In each block, the sentinel 1 in NODE prevents borrows from propagating between blocks. The sentinel bit of block i in D is:

- 1 if $\text{sk}(x_i) \geq \text{sk}(q)$ (no borrow consumed the sentinel)
- 0 if $\text{sk}(x_i) < \text{sk}(q)$ (borrow consumed the sentinel)

Mask out all non-sentinel bits. The result is a monotone bitstring: some leading 0's (keys smaller than q) followed by 1's (keys $\geq q$). The number of 1's — equivalently, the position of the first 1 — tells you where q ranks.

2.7.3. Counting the 1's via multiplication

The sentinel bits are spaced at regular intervals. Multiply the masked result by a word with 1's at matching intervals — all sentinel bits accumulate into a single $\lceil \log k \rceil$ -bit counter at a known position. Extract with a final mask and shift.

! Total cost per node: $O(1)$

Sketch q (4 ops) $\boxtimes$ replicate (1 multiply) $\boxtimes$ subtract (1 op) $\boxtimes$ mask sentinels (1 op) $\boxtimes$ count via multiply (1 op) $\boxtimes$ extract (mask + shift). Constant operations total.

2.8. Most significant bit in $O(1)$

2.8.1. Why we need it

The query-correction step (Section 2.5) requires $\text{MSB}(x_i \oplus q)$. We cannot call ourselves recursively (that would be circular). Instead, we build MSB from the same primitives: splitting, parallel comparison, and multiplication.

2.8.2. Algorithm

Given a w -bit word x , find the index of its most significant 1-bit.

Step 1: Split into $\sqrt{w}$ chunks of $\sqrt{w}$ bits each.

$$x = [\text{chunk}_0 \mid \text{chunk}_1 \mid \dots \mid \text{chunk}_{\{\sqrt{w} - 1\}}]$$

Step 2: Determine which chunks are non-zero.

Let F be a word with a 1 at the leading position of each chunk and 0's elsewhere.

2. Fusion Trees

1. Compute $A = x \& F$ — this captures the leading bit of each chunk.
2. Clear those leading bits: $B = x \oplus A$.
3. Compute $C = F - B$ — within each chunk, the sentinel bit (from F) gets borrowed if and only if the chunk has any non-zero bit in positions below the leader.
4. Mask out $C \& F$ and XOR with F : get a 1 in the leading position of each chunk that was non-zero (after clearing leaders).
5. OR back in A : now you have a word where the leading bit of each chunk is 1 iff that chunk is non-zero.

Step 3: Compress chunk-summary to $\sqrt{w}$ consecutive bits.

Use the multiplication lemma for evenly-spaced bits (positions $\sqrt{w} - 1, 2\sqrt{w} - 1, \dots$). Multiply by a precomputed M to pack these $\sqrt{w}$ summary bits consecutively.

Step 4: Find MSB of the $\sqrt{w}$ -bit summary.

This is a word of only $\sqrt{w}$ bits. Apply parallel comparison against the set $\{1, 2, 4, \dots, 2^{\sqrt{w}-1}\}$ (or equivalently, against $\sqrt{w}$ threshold values). This uses the same subtraction trick from Section 2.7, but the “keys” and “query” together fit comfortably in $O(1)$ words because $\sqrt{w} \cdot \sqrt{w} = w$.

The result identifies the highest non-zero chunk (call it chunk j).

Step 5: Find MSB within chunk j .

Shift x to isolate chunk j (a $\sqrt{w}$ -bit number). Apply parallel comparison again — same technique, same size. Done.

Total: $O(1)$ Word-RAM operations (roughly 15–20 instructions; not recursive).

Aside (mine, not the lecturer’s)

On modern 64-bit hardware, compilers expose `__builtin_clz` (count leading zeros), which maps to a single BSR/LZCNT instruction. The algorithm above is theoretically important — it proves MSB is $O(1)$ in the Word-RAM *without* assuming a hardware CLZ — but in practice you would use the intrinsic.

2.9. Putting it all together

A predecessor query on a fusion tree proceeds as follows:

1. Start at the root.
2. At each node, sketch the query q using the node’s precomputed multiplier M and masks.
3. Parallel-compare $\text{sk}(q)$ against all stored sketches. Identify tentative rank.
4. Correct the rank: XOR q with the two neighbouring keys, extract MSB, form e , re-compare.
5. Follow the child pointer for the corrected rank.
6. Repeat until reaching a leaf (which stores the actual predecessor).

Each node: $O(1)$ operations. Height: $O(\log_w n)$. Total: $O(\log_w n)$.

Dynamic fusion trees

Fredman and Willard’s original paper handles only static sets. Anderson and Thorup later achieved $O(\log_w n + \log \log n)$ deterministic update time; Raman achieved $O(\log_w n)$ expected (randomised). The static version presented here captures the core ideas.

2.10. Cheat sheet

! Cheat sheet — Lecture 2

Fusion tree structure: $k = \Theta(w^{1/5})$ keys per node; height $O(\log_w n)$; linear space.

Sketch compression:

- Branch bits ($r \leq k - 1$): bit positions where stored keys diverge in the binary trie.
- Sketch = projection to branch bits. Preserves order for stored keys.
- Compression via multiplication: mask $\&$ multiply by M $\&$ mask diagonal $\&$ shift. Result fits in $O(r^4) = O(w^{4/5})$ bits.
- Choice of M : inductive argument mod r^3 ; guarantees no carries and $O(r^4)$ range.

Query correction (sketch can lie for non-stored keys):

- Find divergence point y via $\text{MSB}(x_i \oplus q)$.
- Form corrective element e (pad with 0's or 1's depending on fall-off direction).
- e 's true rank = q 's true rank.

Parallel comparison:

- Node word: $1 \parallel \text{sk}(x_0) \parallel 1 \parallel \text{sk}(x_1) \parallel \dots$
- Query word: $0 \parallel \text{sk}(q) \parallel 0 \parallel \text{sk}(q) \parallel \dots$
- Subtract $\&$ sentinel bits form monotone sequence $\&$ count 1's via multiply.

MSB in $O(1)$:

1. Split word into $\sqrt{w}$ chunks.
2. Detect non-empty chunks (subtraction borrow trick + OR leading bits).
3. Compress summary bits via multiplication.
4. Parallel-compare $\sqrt{w}$ -bit summary to find first non-zero chunk.
5. Parallel-compare within that chunk to find exact bit.

2.11. FAQ — questions students actually asked

Q: Do you store the branch bits of all keys, or only the ones relevant to a particular x_i ? A: The branch bits are a property of the *node* (the set of all x_i in that node), not of any individual key. Every key in the node is projected onto the same set of branch-bit positions, even if a particular x_i doesn't actually branch there.

Q: In preprocessing, why can't we just shift bits to form a perfect sketch? A: You can — for the stored keys x_i . But the query q arrives online. Whatever sketch mechanism you use must apply to q in $O(1)$ time. There is no single Word-RAM operation that extracts arbitrary bit positions and packs them contiguously. Multiplication with a carefully chosen M is the workaround.

Q: For the query correction, how do you know whether q fell off to the left or right? A: Compute $\text{MSB}(x_i \oplus q)$ for both neighbours x_i and x_{i+1} . The more significant of the two gives the divergence point. The bit value of q at that position tells you the direction: 1 means q went right, 0 means left.

Q: What if the sketch of e ties with a stored sketch? A: Compare e (or equivalently q) directly against the tied x_i . There are at most 2 candidates, so this is $O(1)$ comparisons.

Q: Is the $w^{1/5}$ exponent tight? A: It is chosen to make the three constraints fit: k keys $\times w$ bits/key spans $kw = w^{6/5}$ raw bits, the compressed sketch uses $r^4 \leq k^4 = w^{4/5}$ bits per key, and k sketches total $k \cdot r^4 = w$ bits — just fitting in one word. Other constant exponents work but $1/5$ gives the cleanest accounting.

Q: Can fusion trees avoid multiplication? A: Yes. A later result (Andersson, Hagerup, Nilsson, and Raman, 1998) gives multiplication-free fusion trees using AC^0 operations. The query time remains $O(\log_w n)$ but the constant factor is larger.

2.12. Further reading

- Fredman, M. L. and Willard, D. E. “Surpassing the information theoretic bound with fusion trees.” *Journal of Computer and System Sciences*, 47(3):424–436, 1993. — The original paper.
- Andersson, A. and Thorup, M. “Dynamic ordered sets with exponential search trees.” *Journal of the ACM*, 54(3):Article 13, 2007. — Dynamic fusion trees with $O(\log_w n + \log \log n)$ updates.
- Andersson, A., Hagerup, T., Nilsson, S., and Raman, R. “Sorting in linear time?” *Journal of Computer and System Sciences*, 57(1):74–93, 1998. — Multiplication-free fusion trees and connections to linear-time sorting.
- Patrascu, M. and Thorup, M. “Time-space trade-offs for predecessor search.” *Proc. 38th STOC*, 2006. — Lower bound: $\Omega(\min(\log w, \log_w n))$.
- Willard, D. E. “Log-logarithmic worst-case range queries are possible in space $\Theta(N)$.” *Information Processing Letters*, 17(2):81–84, 1983. — Y-fast tries (the $O(\log w)$ complement).

2.13. Try it yourself

1. Branch bits. Given the key set $\{0010, 0110, 1001, 1100\}$ (4-bit universe), draw the binary trie, identify the branch bits, and compute the sketch of each key. Verify that sketch order matches key order.
2. Sketch distortion. Using the same key set, construct a query q whose sketch lands in the wrong interval. Walk through the correction procedure: compute $\text{MSB}(x_i \oplus q)$, determine the fall-off direction, form e , and verify that e ’s rank is correct.
3. The multiplication lemma. Prove: when branch-bit positions are evenly spaced ($b_i = i\sqrt{w} + \sqrt{w} - 1$), there exists a multiplier M such that the diagonal positions $b_i + m_i$ are consecutive with no gaps. (Hint: set $m_i = (r - 1 - i)\sqrt{w}$ and verify.)
4. Parallel comparison by hand. Let $k = 4$ and sketch width = 3 bits. Suppose the node sketch word is $1|010|1|011|1|101|1|110$ and the query sketch is 100 . Write out the replicated query word, perform the subtraction, extract sentinels, and determine the query’s rank.
5. Implementation. Implement the MSB algorithm from Section 2.8 in Python for 64-bit integers. Split into 8 chunks of 8 bits, detect non-empty chunks, find the first non-empty, then find the MSB within it. Compare your instruction count against `int.bit_length()`.

```
def msb_fusion(x: int) -> int:
    """Return index of most significant set bit (0-indexed from LSB)."""
    assert 0 < x < (1 << 64)
    Sqrt_W = 8
    W = 64
    # F: leading bit of each chunk
    F = 0
    for i in range(Sqrt_W):
```

```

F |= (1 << (i * SQRT_W + SQRT_W - 1))

# Step 1: capture and clear leading bits
A = x & F
B = x ^ A

# Step 2: detect non-empty chunks via borrow
C = F - B
non_empty_from_body = (C & F) ^ F # 1 at leader pos if chunk body was non-zero
non_empty = non_empty_from_body | A # include chunks whose leader was 1

# Step 3: find highest non-empty chunk (simple loop for clarity)
for chunk_idx in range(SQRT_W - 1, -1, -1):
    leader_pos = chunk_idx * SQRT_W + SQRT_W - 1
    if non_empty & (1 << leader_pos):
        # Step 4: find MSB within this chunk
        base = chunk_idx * SQRT_W
        for bit in range(SQRT_W - 1, -1, -1):
            if x & (1 << (base + bit)):
                return base + bit
return -1 # unreachable for x > 0

```


3. Hashing I: Load Balancing, Independence, and Linear Probing

COMPSCI 224 Lecture 3 — Fall 2014

Hashing converts the unpredictability of random functions into practical guarantees about load balancing, collision avoidance, and dictionary operations. This chapter develops the balls-and-bins framework, proves that the maximum load is $O(\log n / \log \log n)$ via both the Chernoff bound and a direct counting argument, introduces k -wise independent hash families built from low-degree polynomials, and sets up the analysis of linear probing through full intervals and the moment method.

3.1. Why this lecture matters

Hashing is one of the most practically important ideas in computer science, yet its theoretical guarantees depend on subtle probabilistic reasoning. Understanding *how much* independence a hash function needs—and what breaks when you have less—is the key to designing efficient, space-conscious data structures. This chapter builds the machinery you will use for the rest of the hashing unit.

30-second story

1. A hash function maps items from a large universe into a small array; collisions are inevitable but controllable.
2. Throwing n balls into n bins, the maximum load is $\Theta(\log n / \log \log n)$ with high probability.
3. Full independence is expensive to store; k -wise independent families built from degree- $(k-1)$ polynomials need only $O(k \log u)$ bits.
4. Hash tables with chaining give expected $O(1)$ operations using only 2-wise independence.
5. Linear probing exploits cache prefetching for speed, but its analysis demands 5-wise independence (or more).
6. The *moment method*—raising a deviation to a high even power and applying Markov—is the main analytic tool.

3.2. Hashing: the setup

Fix a universe $U = \{0, 1, \dots, u-1\}$ and an array of size m . A *hash family* $\mathcal{H}$ is a collection of functions $h : U \rightarrow \{0, 1, \dots, m-1\}$. You draw one h at random from $\mathcal{H}$, and then every item i in some set $S \subseteq U$ ($|S| = n$) lands in cell $h(i)$.

The goal: choose $\mathcal{H}$ so that a random h “behaves nicely” on S —few collisions, balanced load, fast lookups—while keeping the description of h small.

Two items i, j collide when $h(i) = h(j)$. Since $|U|$ is typically much larger than m , collisions are inevitable by pigeonhole. The question is never “can we avoid collisions?” but “how bad do collisions get, and what does the hash family need to guarantee?”

Throughout this chapter, the computational model is the word RAM with word size $w \geq \log u$. Evaluating h takes $O(1)$ time once the coefficients are stored.

3.3. Load balancing and balls into bins

Suppose n jobs arrive online, each with a job ID in U , and must be assigned to one of m identical machines. If you know all jobs ahead of time, round-robin assignment gives each machine exactly n/m jobs. But if jobs arrive dynamically and you lack a global view of machine loads, you need a *local* rule.

The simplest local rule: hash. When job j arrives, send it to machine $h(j)$. Each machine processes whatever lands in its bin.

This is the *balls-and-bins* random process: n balls thrown independently and uniformly into m bins. The question is how overloaded the fullest bin gets. Ideally each bin receives n/m balls, but randomness guarantees some deviation.

3.3.1. The Chernoff bound

You need a tail bound. Let $X_1, \dots, X_n$ be independent $\{0, 1\}$ -valued random variables with $\mathbb{E}[X_i] = p_i$. Set $X = \sum_i X_i$.

$$\Pr[X \geq (1 + \delta) \mathbb{E}[X]] \leq \left(\frac{e^\delta}{(1 + \delta)^{1+\delta}} \right)^{\mathbb{E}[X]} \quad (3.1)$$

Proof sketch

The argument starts with Markov’s inequality: for a non-negative random variable Y and any $\lambda > 0$, $\Pr[Y \geq \lambda] \leq \mathbb{E}[Y]/\lambda$.

Apply Markov to the *moment-generating function*. For any $t > 0$,

$$\Pr[X \geq \lambda] = \Pr[e^{tX} \geq e^{t\lambda}] \leq e^{-t\lambda} \mathbb{E}[e^{tX}].$$

Independence lets you factor $\mathbb{E}[e^{tX}]$ into a product $\prod_i \mathbb{E}[e^{tX_i}]$. Each factor can be bounded using the constraint $X_i \in \{0, 1\}$. Optimising over t (the right choice turns out to be $t = \ln(1 + \delta)$) gives Equation 3.1.

This “exponentiate, apply Markov, optimise” pattern is the template for all exponential tail bounds.

3.3.2. Max load is $O(\log n / \log \log n)$

Focus on the case $m = n$ (one machine per job on average). Fix a machine t and define $X_i = 1[h(i) = t]$. Then $X = \sum_i X_i$ counts the jobs assigned to machine t , and $\mathbb{E}[X] = 1$.

Apply Equation 3.1 with $\delta = c \log n / \log \log n$ for a suitable constant c . Since $\mathbb{E}[X] = 1$, the bound becomes roughly $(e/\delta)^\delta$. You want $(e/k)^k \leq 1/\text{poly}(n)$, which forces $k = \Theta(\log n / \log \log n)$.

i Why $k^k = \text{poly}(n)$ iff $k = \Theta(\log n / \log \log n)$

Take logs: $k \ln k = c' \ln n$. If $k = \alpha \log n / \log \log n$, then $k \ln k \approx \alpha(\log n / \log \log n)(\ln \log n) \approx \alpha \ln n$. So $\alpha = c'$ works.

A union bound over all n machines contributes a factor of n , leaving the failure probability at $1/n^{c'-1}$:

$$\Pr[\exists \text{ machine with load} > c \frac{\log n}{\log \log n}] \leq \frac{1}{n^{c'-1}}. \quad (3.2)$$

3.3.3. A simpler proof via counting

The Chernoff bound is overkill here—it requires full independence and heavy machinery. A direct combinatorial argument needs nothing beyond a union bound and a factorial estimate, and it reveals exactly how much independence is required.

By symmetry, every machine has the same load distribution. By a union bound over n machines, $\Pr[\exists \text{ overloaded machine}] \leq n \cdot \Pr[\text{machine 1 is overloaded}]$. Focus on machine 1.

The probability that machine 1 receives $\geq k$ jobs equals the probability that *some* subset of k jobs all land there:

$$\Pr[\text{machine 1 gets} \geq k] \leq \binom{n}{k} \frac{1}{n^k} \leq \frac{1}{k!} \leq \frac{1}{(k/2)^{k/2}}. \quad (3.3)$$

The last step uses $k! \geq (k/2)^{k/2}$. Setting $(k/2)^{k/2}$ to a polynomial in n again forces $k = \Theta(\log n / \log \log n)$.

! Important

This proof uses only k -wise independence, not full independence—a crucial observation for the next section.

3.4. k -wise independence

The counting proof (Section 3.3.3) decomposed the probability into products of k indicator random variables. Full independence was never needed; all that mattered was that any k hash evaluations behave independently. This motivates a weaker—and cheaper—notation of independence.

Definition. Random variables $X_1, \dots, X_n$ are *k -wise independent* if for every set of k indices $i_1 < i_2 < \dots < i_k$, the variables $X_{i_1}, \dots, X_{i_k}$ are mutually independent.

A hash family $\mathcal{H}$ is *k -wise independent* if the random variables $h(0), h(1), \dots, h(u-1)$ —induced by drawing $h \sim \mathcal{H}$ uniformly—are k -wise independent.

At first glance, k -wise independence might seem like the same thing as full independence—after all, mutual independence means *every* subset is independent. The difference is quantitative: k -wise independence only constrains subsets of size *up to* k , not all subsets. A family can be 2-wise independent (pairwise) without being 3-wise independent.

3.4.1. Polynomial hash families

The payoff of k -wise independence is *space*. A fully random hash function requires $u \log m$ bits—one entry per universe element. A k -wise independent family built from polynomials needs only $k \log u$ bits.

Example 1. The family of *all* functions $U \rightarrow \{0, \dots, m-1\}$ is k -wise independent for every $k \leq u$. But storing a random member requires $u \log m$ bits—prohibitively expensive.

Example 2. Let $u = m = p$ (a prime). Define

$$\mathcal{H} = \left\{ h(x) = \sum_{i=0}^{k-1} a_i x^i \bmod p \mid a_0, \dots, a_{k-1} \in \{0, \dots, p-1\} \right\}. \quad (3.4)$$

This is a k -wise independent family. Storing h requires only k coefficients, i.e., $k \log p$ bits.

💡 Why polynomial interpolation implies k -wise independence

Fix any k distinct points $x_1, \dots, x_k$ and any target values $y_1, \dots, y_k$. By polynomial interpolation over $\mathbb{F}_p$, there is *exactly one* degree- $(k-1)$ polynomial mapping $x_j \mapsto y_j$ for each j . Since there are p^k polynomials and p^k target vectors, each target vector arises with probability exactly $1/p^k$ —the same as fully random evaluation.

⚠️ Why only k -wise, not fully independent?

Given any k evaluations of a degree- $(k-1)$ polynomial, the $(k+1)$ -th is determined by interpolation. Full independence would require degree $u-1$.

3.4.2. The mod-reduction issue

In practice $m \neq p$. You take $h(x) \bmod p \bmod m$. The first reduction $\bmod p$ is perfectly uniform; the second $\bmod m$ introduces a slight bias.

If $p = qm + r$ with $0 < r < m$, the first r residues each have probability $(q+1)/p$ instead of q/p . The worst case is $p = m + 1$: one residue gets twice the expected mass. In general,

$$\Pr[h(x_1) = y_1, \dots, h(x_k) = y_k] \leq (2/m)^k \quad (3.5)$$

for any $x_1, \dots, x_k$ and $y_1, \dots, y_k$. This factor-of-2 slack is harmless: plugging $(2/n)^k$ into the balls-and-bins analysis changes only the leading constant, not the $\Theta(\log n / \log \log n)$ max-load guarantee.

3.5. The dictionary problem

With the tools of hash families and k -wise independence in hand, you can tackle one of the most fundamental data structure problems.

A *dictionary* supports two operations on key-value pairs from U :

- **Update(k, v)**: associate key k with value v .
- **Query(k)**: return the value associated with k , or report that k is absent.

The dictionary problem comes in two flavours: *static* (all keys known in advance) and *dynamic* (keys inserted and deleted over time). Hashing gives excellent solutions to both.

3.5.1. Hashing with chaining

Maintain an array $A[0..m-1]$ of linked lists. Draw h from a 2-wise independent family.

- Update: compute $h(k)$; scan the list at $A[h(k)]$. If k is present, update its value; otherwise append (k, v) .
- Query: compute $h(k)$; scan the list at $A[h(k)]$ for k .

The running time of either operation equals the length of the list at $A[h(k)]$. With $m \geq n$, the expected list length is $O(1)$ under 2-wise independence—only pairwise collision probabilities matter.

To see why: the expected list length at cell $h(k)$ is $\sum_{j \neq k} \Pr[h(j) = h(k)] \leq n/m \leq 1$. This sum involves only pairwise collision probabilities, so 2-wise independence suffices.

i Static alternative: perfect hashing

If the key set is known in advance, *perfect hashing* (Fredman, Komlós, Szemerédi 1984) guarantees worst-case $O(1)$ query time with $O(n)$ space. The construction uses a two-level scheme: first hash into n buckets, then build a small collision-free table inside each bucket.

3.6. Linear probing

Hashing with chaining scatters linked-list nodes across memory. Each pointer dereference in a linked list is potentially a cache miss—the next node could be anywhere in the heap. *Linear probing* stores everything in one contiguous array, exploiting CPU cache prefetching for dramatically better real-world performance.

Prefetching. Modern CPUs predict that if you access memory address a , you will soon access $a+1, a+2, \dots$ (common in array loops). The hardware speculatively loads these adjacent cache lines before you request them. This means sequential memory accesses are far cheaper than random pointer chasing.

Scheme. To insert x : compute $h(x)$; if $A[h(x)]$ is occupied, walk forward through $A[h(x)+1], A[h(x)+2], \dots$ (wrapping around) until you find an empty cell, and store (x, v) there. Queries walk the same path: start at $h(x)$ and scan forward until you find x or an empty cell. Deletion requires care (typically “tombstone” markers or backward shifting).

i History

Linear probing was proposed by Samuel, Amdahl, and Boehme (1954) and first analysed by Knuth (1963), who showed that with $m = (1 + \epsilon)n$ and a fully random h , the expected time per operation is $O(1/\epsilon^2)$.

3.6.1. How much independence does linear probing need?

Table 3.1.: Independence thresholds for linear probing

Result	Reference	Independence
$O(1)$ expected time	Pagh, Pagh, Ružić (2007/2009)	5-wise
$\Omega(\log n)$ expected time exists	Pătraşcu & Thorup (2010)	4-wise
$O(1)$ for a specific 3-wise family	Pătraşcu & Thorup (2010)	simple tabulation

The sharp threshold at 5-wise is surprising. The intuition: conditioning on where the query element x lands consumes one “degree of freedom,” leaving four independent evaluations—exactly what the fourth-moment argument requires.

3.6.2. Full intervals and the analysis framework

The key insight for analysing linear probing is that long scan times correspond to dense clusters of occupied cells. You formalise “cluster density” through *full intervals*.

Definition. For an interval I of array cells, let $L(I) = |\{i \in S : h(i) \in I\}|$ be the number of items whose hash value falls in I . (This counts items by where they *hash to*, not where they end up after probing.) Call I *full* if $L(I) \geq |I|$.

A full interval of length k has at least k items hashing into it—so by pigeonhole, every cell in that interval is occupied (and possibly items spill beyond it).

Lemma. If querying x requires k probes, then $h(x)$ belongs to a full interval of length $\geq k$.

Proof sketch. The item at $A[h(x)]$ got there by hashing somewhere at or before $h(x)$ and walking forward. Every cell from that origin through x ’s final landing spot is occupied, and every occupant hashed into that window. Any prefix of length k starting from the earliest origin is full.

This reduces the analysis to bounding the expected number of full intervals containing $h(x)$:

$$\mathbb{E}[\text{probes for } x] \leq \sum_{k=1}^{\infty} \sum_{\substack{I: |I|=k, \\ h(x) \in I}} \Pr[I \text{ is full}]. \quad (3.6)$$

There are k intervals of length k that contain $h(x)$, so

$$\mathbb{E}[\text{probes}] \leq \sum_{k=1}^{\infty} k \cdot \max_{\substack{I: |I|=k, \\ h(x) \in I}} \Pr[I \text{ is full}]. \quad (3.7)$$

For this sum to converge, you need $\Pr[I \text{ full}] = O(1/k^3)$.

3.6.3. The moment method

With $m = 2n$, an interval of length k expects $k/2$ items. Being full means deviating from the mean by $k/2$.

Apply Markov’s inequality to the *sixth power* of the centered variable:

$$\Pr[L(I) - \mathbb{E}[L(I)] \geq k/2] \leq \frac{\mathbb{E}[(L(I) - \mathbb{E}[L(I)])^6]}{(k/2)^6}. \quad (3.8)$$

If the sixth central moment is $O(k^3)$ (which it is under 6-wise independence), the probability is $O(k^3/k^6) = O(1/k^3)$, making the sum in Equation 3.7 converge.

💡 Why raise to a high power?

Markov with $|X|$ gives a weak bound. Raising to the p -th power amplifies the tail: a concentrated random variable has a small p -th moment, so the bound tightens dramatically. This is the *moment method*, a workhorse throughout probability and algorithm analysis.

Including the hash evaluation of x itself, you need 7-wise independence in total (one for x , six for the moment).

i Chernoff vs. moments

The Chernoff bound uses the *moment-generating function* $\mathbb{E}[e^{tX}]$, which implicitly captures all moments simultaneously. The moment method uses a single finite moment $\mathbb{E}[X^p]$. Chernoff gives sharper tails but demands full independence; the moment method works under p -wise independence. This trade-off—precision vs. independence requirements—recurs throughout the hashing literature.

A more refined argument—covered in the next lecture—reduces the requirement from 7-wise to 5-wise by eliminating the factor-of- k union bound over intervals. The trick is to union-bound over $O(1)$ nodes in a binary tree rather than k overlapping intervals.

3.7. Cheat sheet

! Lecture 3 — key results

Balls and bins ($m = n$). Max load = $\Theta(\log n / \log \log n)$ w.h.p. Proof: Chernoff or direct counting with union bound.

k -wise independent hash families. Degree- $(k-1)$ polynomials over $\mathbb{F}_p$; storage $k \log p$ bits. k -wise $\Rightarrow$ any k evaluations are mutually independent; $(k+1)$ -th is determined.

Mod-reduction bias. $h(x) \bmod p \bmod m$ is k -wise independent but not perfectly uniform; collision probability $\leq (2/m)^k$.

Dictionary (chaining). Array of linked lists, 2-wise hash. Expected $O(1)$ per operation when $m \geq n$.

Linear probing. Cache-friendly, $O(1)$ expected time with 5-wise independence. 4-wise is insufficient (counterexample exists).

Moment method. $\Pr[|X - \mu| \geq t] \leq \mathbb{E}[(X - \mu)^p] / t^p$. Use even p ; higher p gives tighter tails for concentrated X .

3.8. FAQ — questions students actually asked

Q: Where does the factor of $2/m$ come from in the collision bound? When you reduce $\bmod p \bmod m$, the residues $0, \dots, r-1$ (where $p = qm + r$) each absorb one extra pre-image compared to residues $r, \dots, m-1$. The worst case is $p = m + 1$: residue 0 gets probability $2/p \approx 2/m$. This factor propagates to $(2/m)^k$ for k collisions.

Q: Could you use rejection sampling—repeatedly applying h until the output is less than m —instead of taking $\bmod m$? In principle, yes: if $h(x) \geq m$, feed the result back into h . But all n items share the *same* hash function (same coefficients a_i), so rejecting for one item doesn't help others. Worse, iterating h could create fixed points or short cycles. The $\bmod m$ approach is simpler and the factor-of-2 bias is benign.

Q: Why is the polynomial family only k -wise independent and not fully independent? A degree- $(k-1)$ polynomial over $\mathbb{F}_p$ is determined by k points (polynomial interpolation). Given any k evaluations, the $(k+1)$ -th is fixed. Full independence would require a polynomial of degree $u-1$ —equivalent to storing all u values explicitly.

Q: In the counting proof, you don't seem to use independence at all. What independence do you actually need? Look carefully at Equation 3.3. The step $\Pr[\text{all } k \text{ items hit bin } 1] = (1/n)^k$ factors the probability as a product—this requires exactly k -wise independence. The Chernoff-based proof needs full independence; the counting proof does not.

Q: For the load-balancing application, we need $\log n / \log \log n$ -wise independence. Is that cheap enough? With the polynomial family, storing h takes $k \log u = O((\log n / \log \log n) \log u)$ bits. For typical universe sizes ($u = \text{poly}(n)$), this is $O(\log^2 n / \log \log n)$ bits—far smaller than the $u \log m$ bits needed for a fully random function, but larger than the $O(\log u)$ bits for a 2-wise family. Whether this cost matters depends on the application. For linear probing, you only need 5-wise independence, which takes $5 \log u$ bits—essentially free.

Q: Why does linear probing need more independence than chaining? With chaining, the running time equals one linked-list length. Bounding the expected length uses only pairwise collision probabilities, so 2-wise independence suffices. With linear probing, the running time depends on *runs of contiguous occupied cells*—a global property that couples many hash values together. Proving that runs are short requires higher moments, which in turn need more independent hash evaluations.

3.9. Further reading

1. Fredman, Komlós, Szemerédi (1984). “Storing a sparse table with $O(1)$ worst-case access time.” *JACM* 31(3). The original perfect hashing scheme.
2. Knuth (1963/1998). *The Art of Computer Programming*, Vol. 3, §6.4. The first analysis of linear probing.
3. Pagh, Pagh, Ružić (2009). “Linear probing with constant independence.” *SIAM J. Comput.* 39(3). 5-wise independence suffices.
4. Pătraşcu & Thorup (2010). “On the k -independence required by linear probing and minwise independence.” *ICALP 2010*. 4-wise counterexample + simple tabulation analysis.
5. Mitzenmacher & Upfal (2017). *Probability and Computing*, Ch. 5. Textbook treatment of balls-and-bins and Chernoff bounds.

3.10. Try it yourself

1. Warm-up. Prove that $k! \geq (k/e)^k$ (Stirling’s lower bound) and use it to tighten the constant in Equation 3.3.
2. Mod-reduction. Let $p = 2m - 1$. Compute the exact collision probability $\Pr[h(x) \bmod p \bmod m = 0]$ for a uniformly random $h(x) \in \{0, \dots, p-1\}$. How does it compare to $1/m$?
3. Moment method practice. Let $X_1, \dots, X_n$ be 4-wise independent $\{0, 1\}$ random variables with $\mathbb{E}[X_i] = 1/2$. Show that $\Pr[|\sum X_i - n/2| \geq t] \leq O(n^2/t^4)$.
4. Implementation. Implement a hash table with chaining using the polynomial family Equation 3.4 with $k = 2$. Insert 10^5 random keys into a table of size $m = 2 \times 10^5$. Plot the distribution of chain lengths and compare with the Poisson distribution $\text{Poi}(1/2)$.

```
import numpy as np
from collections import Counter

def poly_hash(x, a0, a1, p, m):
    """2-wise independent hash:  $h(x) = (a0 + a1*x) \bmod p \bmod m$ ."""
    return ((a0 + a1 * x) % p) % m

# Parameters
n = 100_000
m = 200_000
p = 200_003 # a prime >= m

rng = np.random.default_rng(42)
a0, a1 = rng.integers(0, p, size=2)
keys = rng.integers(0, 10**9, size=n)

bins = Counter(poly_hash(int(k), int(a0), int(a1), p, m) for k in keys)
chain_lengths = list(bins.values())
```

```
print(f"Max chain length: {max(chain_lengths)}")
print(f"Mean chain length: {np.mean(chain_lengths):.3f}")
print(f"Fraction of empty cells: {1 - len(bins)/m:.3f}")
```

5. Open-ended. The counting proof shows that $\log n / \log \log n$ -wise independence suffices for the balls-and-bins max-load guarantee. Construct (or look up) an explicit 2-wise independent family for which the max load is $\Omega(\sqrt{n})$, demonstrating that 2-wise is not enough.

4. Hashing II: Bloom Filters, Cuckoo Hashing, and Bloomier Filters

COMPSCI 224 Lecture 4 — Fall 2014

This chapter completes the linear probing analysis—showing that 5-wise independence suffices via a clever binary-tree union-bound—then covers three data structures that push hashing beyond dictionaries: Bloom filters for approximate membership in $n \log(1/\epsilon)$ bits, cuckoo hashing for worst-case $O(1)$ lookups with two hash functions, and bloomier filters for the retrieval problem using cuckoo graphs and XOR queries.

4.1. Why this lecture matters

The previous lecture set up the machinery of k -wise independence and the moment method. This lecture *deploys* that machinery on linear probing, then pivots to three data structures—Bloom filters, cuckoo hashing, and bloomier filters—that show hashing is about far more than dictionaries. Each structure trades something (exactness, extra hash functions, static preprocessing) for a sharp gain in space or time.

30-second story

1. A binary-tree trick eliminates the factor-of- k union bound in the linear probing analysis, dropping the independence requirement from 7-wise to 5-wise.
2. Bloom filters solve approximate membership in $n \log(1/\epsilon)$ bits using $\log(1/\epsilon)$ independent hash functions.
3. Cuckoo hashing maintains a dictionary with worst-case $O(1)$ lookups and expected $O(1)$ insertions using two hash functions and a “kick out” strategy.
4. The *cuckoo graph*—vertices for array cells, edges for items—is the key analytic object: insertion works smoothly when the graph is acyclic.
5. Bloomier filters solve the *retrieval problem* (store an r -bit value per key in nr bits) by filling an acyclic cuckoo graph top-down with XOR constraints.

4.2. Finishing linear probing

4.2.1. Recap: the 7-wise argument

From last lecture, the expected number of probes to insert x satisfies

$$\mathbb{E}[\text{probes}] \leq \sum_{k=1}^{\infty} k \cdot \max_{\substack{I: |I|=k \\ h(x) \in I}} \Pr[I \text{ is full}], \quad (4.1)$$

where an interval I is *full* if $L(I) \geq |I|$ (at least $|I|$ items hash into I). With $m = 2n$, the expected number of items landing in a length- k interval is $k/2$, so being full means deviating from the mean by $k/2$.

Raising to the sixth power and applying Markov:

$$\Pr[I \text{ full}] \leq \frac{\mathbb{E}[(L(I) - \mathbb{E}[L(I)])^6]}{(k/2)^6}.$$

Under 6-wise independence (plus one for conditioning on $h(x)$, giving 7 total), the sixth central moment is $O(k^3)$, so $\Pr[I \text{ full}] = O(1/k^3)$. The factor of k in Equation 4.1 cancels one power, and $\sum 1/k^2$ converges.

4.2.2. Bounding the sixth moment via symmetrization

Define indicator variables $X_i = 1[h(i) \in I]$ so that $L(I) = \sum_{i \in S} X_i$. You need to bound

$$\mathbb{E}\left[\left(\sum_i X_i - \mathbb{E}\left[\sum_i X_i\right]\right)^6\right]. \quad (4.2)$$

Work with the p -norm of a random variable: $\|Z\|_p = (\mathbb{E}[|Z|^p])^{1/p}$. Two classical inequalities help.

Tools from analysis

Minkowski's inequality. $\|\cdot\|_p$ is a norm for $p \geq 1$: it satisfies the triangle inequality $\|Z_1 + Z_2\|_p \leq \|Z_1\|_p + \|Z_2\|_p$.
Jensen's inequality. If f is convex, $f(\mathbb{E}[Z]) \leq \mathbb{E}[f(Z)]$. In particular, $|\cdot|^p$ is convex for $p \geq 1$, so you can pull an expectation *outside* a p -th power at the cost of an inequality.

The symmetrization trick. Let X'_i be an independent copy of X_i (same distribution, independent of X_i). Then $\mathbb{E}[X'_i] = \mathbb{E}[X_i]$, so

$$\left\|\sum_i (X_i - \mathbb{E}[X_i])\right\|_6 = \left\|\sum_i (X_i - \mathbb{E}[X'_i])\right\|_6 = \left\|\mathbb{E}_{X'}\left[\sum_i (X_i - X'_i)\right]\right\|_6.$$

By Jensen (pulling $\mathbb{E}_{X'}$ outside the convex $|\cdot|^6$):

$$\leq \left\|\sum_i (X_i - X'_i)\right\|_6,$$

where the norm is now over both X and X' . Each $X_i - X'_i$ is a *symmetric* random variable (its distribution is unchanged by negation), so you may insert independent random signs $\sigma_i \in \{-1, +1\}$ without changing the distribution:

$$\left\|\sum_i (X_i - X'_i)\right\|_6 = \left\|\sum_i \sigma_i (X_i - X'_i)\right\|_6 \leq 2 \left\|\sum_i \sigma_i X_i\right\|_6, \quad (4.3)$$

where the last step uses the triangle inequality (Minkowski) to separate X_i and X'_i .

Why symmetrize?

Expand $(\sum_i \sigma_i X_i)^6 = \sum_{i_1, \dots, i_6} \sigma_{i_1} \cdots \sigma_{i_6} X_{i_1} \cdots X_{i_6}$. Taking expectations over the σ_i 's, any monomial where some index appears an *odd* number of times vanishes (since $\mathbb{E}[\sigma^{\text{odd}}] = 0$). Only monomials where every distinct index appears an *even* number of times survive. For the sixth power, this means at most three distinct indices, each appearing twice—giving a dominant term of $O(n^3 p^3)$.

Each $X_i = 1$ with probability $p = |I|/m = k/(2n)$, so $p^3 = k^3/(8n^3)$ and the dominant term is $O(n^3 \cdot k^3/n^3) = O(k^3)$.

Raising Equation 4.3 to the sixth power: the sixth central moment is $O(k^3)$, confirming that $\Pr[I \text{ full}] = O(1/k^3)$.

4.2.3. The 5-wise trick: a binary tree argument

The 7-wise proof pays a factor of k in Equation 4.1 by union-bounding over k intervals of length k containing $h(x)$. Pagh, Pagh, and Ružić eliminate this factor.

Key lemma (refined). If inserting x takes k probes, there are k full intervals containing $h(x)$, each of a *different* length.

This lets you write the expected probes as

$$\sum_{k=1}^{\infty} \Pr[\exists \text{ full interval of length } k \text{ containing } h(x)]. \quad (4.4)$$

Now argue that each such full interval forces a “dangerous” node in a perfect binary tree built over the array.

Construction. Build a complete binary tree over the m array cells. A node at height h represents an interval of 2^h cells. Call a node *dangerous* if at least $\frac{3}{4} \cdot 2^h$ items hash into its interval (expected: $\frac{1}{2} \cdot 2^h$ when $m = 2n$, so “dangerous” means 1.5× the expectation).

Spanning lemma. Any interval I of length k (where $2^h \leq k < 2^{h+1}$) is spanned by between 4 and 9 nodes at height $h - 2$. If I is full, at least one of these spanning nodes must be dangerous.

Proof sketch. The spanning nodes have length $2^{h-2} \geq k/8$. Together, their first four cover at least $3 \cdot 2^{h-2} + 1$ items that all hashed at or after the start of I . By pigeonhole, one of the four has load $\geq \frac{3}{4} \cdot 2^{h-2}$.

! Why this eliminates the factor of k

All length- k intervals containing $h(x)$ fit inside a window of length $2k - 2$, which is spanned by $O(1)$ nodes at height $h - 2$. So instead of union-bounding over k intervals, you union-bound over $O(1)$ tree nodes.

Now apply the *fourth* moment (not sixth):

$$\Pr[\text{node is dangerous}] \leq \frac{\mathbb{E}[(L(I') - \mathbb{E}[L(I')])^4]}{(|I'|/4)^4},$$

where I' is the node’s interval. By the same symmetrization argument, the fourth moment is $O(|I'|^2) = O(k^2)$, and the denominator is $\Theta(k^4)$, giving $O(1/k^2)$.

Summing $\sum_k O(1/k^2) = O(1)$. Including the conditioning on $h(x)$, the total independence needed is $4 + 1 = 5$.

i Aside (mine, not the lecturer’s)

Pătraşcu and Thorup (ICALP 2010) constructed an explicit 4-wise independent family where linear probing requires $\Omega(\log n)$ expected probes, confirming that 5 is tight. Interestingly, *simple tabulation hashing*—which is only 3-wise independent but has additional structural properties—also achieves $O(1)$ expected time.

4.3. Approximate membership: Bloom filters

4.3.1. From a bit array to a Bloom filter

Problem (membership). Maintain a set $S \subseteq U$ under insertions. Support queries “is $x \in S$?” with:

- *No false negatives*: $x \in S \Rightarrow$ query returns YES with probability 1.

- *Bounded false positives:* $x \notin S \Rightarrow$ query returns YES with probability $\leq \varepsilon$.

Baseline: single bit array. Choose a hash function $h : U \rightarrow \{0, \dots, m-1\}$ and maintain a bit array $A[0..m-1]$. Insert: set $A[h(x)] \leftarrow 1$. Query: return $A[h(x)]$.

If $x \in S$, we set $A[h(x)] = 1$, so no false negative. If $x \notin S$, a false positive occurs iff some $y \in S$ satisfies $h(y) = h(x)$. By a union bound:

$$\Pr[\text{false positive}] \leq \sum_{y \in S} \Pr[h(x) = h(y)] = \frac{n}{m}.$$

Setting $m = 2n$ gives a false positive rate of $1/2$.

Deterministic answers

The data structure is randomised (through h), but once built, queries are deterministic. Querying the same x twice gives the same answer. To reduce the false positive rate, you cannot simply repeat the query—you need independent copies.

4.3.2. Reducing the false positive rate

Bloom filter (Bloom, 1970). Use $d = \lceil \log_2(1/\varepsilon) \rceil$ independent hash functions $h_1, \dots, h_d$, each mapping into a separate bit array of length $m = 2n$.

- Insert x : for each j , set $A_j[h_j(x)] \leftarrow 1$.
- Query x : return YES iff $A_j[h_j(x)] = 1$ for *all* j .

A false positive requires a collision in *every* array independently, so

$$\Pr[\text{false positive}] \leq (1/2)^d = \varepsilon.$$

Space: d arrays of $2n$ bits each $= 2n \lceil \log_2(1/\varepsilon) \rceil$ bits $= O(n \log(1/\varepsilon))$ bits.

Time: $O(\log(1/\varepsilon))$ hash evaluations per operation.

Independence: Each hash function only needs 2-wise independence (the false-positive analysis uses only pairwise collision probabilities).

Practical note

More sophisticated variants (counting Bloom filters, cuckoo filters) support deletions and achieve tighter constants. The information-theoretic lower bound for the membership problem with false-positive rate ε is $n \log_2(1/\varepsilon)$ bits, so Bloom filters are optimal up to a constant factor.

4.4. Cuckoo hashing

Cuckoo hashing (Pagh and Rodler, *J. Algorithms*, 2004) achieves *worst-case* $O(1)$ lookup and *expected* $O(1)$ insertion.

Setup. Maintain an array $A[0..m-1]$ with $m = cn$ (e.g., $c = 4$) and two independent random hash functions $g, h : U \rightarrow \{0, \dots, m-1\}$. Each item x has two candidate cells: $A[g(x)]$ and $A[h(x)]$.

Lookup. Check $A[g(x)]$ and $A[h(x)]$. If x is stored in either, return its value. Worst-case time: $O(1)$.

Insertion. 1. Place x in $A[g(x)]$. 2. If some x' was already there, x' is “kicked out.” Move x' to its *other* candidate cell (whichever of $g(x')$, $h(x')$ it was *not* using). 3. This may kick out another item, which moves to *its* other cell, and so on. 4. If the chain of evictions exceeds $c' \log n$ steps, declare failure: pick fresh g, h and rebuild the entire table.

i Why ‘cuckoo’?

The cuckoo bird lays its eggs in other birds’ nests, evicting the original occupants—just as each new item may displace existing items from their cells.

4.4.1. The cuckoo graph

The analysis hinges on a random graph.

Definition. The *cuckoo graph* $G = (V, E)$ has m vertices (one per array cell) and n edges (one per item $x \in S$): item x contributes edge $(g(x), h(x))$. The graph may have parallel edges and self-loops.

Key observations:

- *Insertion succeeds quickly iff the connected component of x ’s edge is a tree or a unicyclic graph.* In a tree, the eviction chain ends at a leaf (empty cell). A single cycle means the chain loops but can still settle. Two or more cycles in a component make insertion fail.
- *If the cuckoo graph is a forest (no cycles), every item can be placed.* In a forest with n edges and m vertices, every component has more vertices than edges, guaranteeing enough empty cells.

Probability of cycles. For $m = cn$ with c a sufficiently large constant, the probability that the cuckoo graph contains *any* cycle is at most $1/2$.

! Important

When insertion fails (chain too long), rebuilding with fresh hash functions succeeds in expected $O(n)$ time. Because failure probability per insertion is $O(1/n)$, the *amortised* expected cost of rebuilds is $O(1)$ per insertion, so the total expected insertion time remains $O(1)$.

4.5. Bloomier filters and the retrieval problem

4.5.1. The retrieval problem

Problem. Given a static set S of n items, each associated with an r -bit string $v(x)$, build a data structure using $O(nr)$ bits such that:

- Query $x \in S$: return $v(x)$ correctly.
- Query $x \notin S$: return an arbitrary r -bit string (no guarantee).

This is weaker than a dictionary (which must also detect non-members) but requires far less space: nr bits instead of n words.

4.5.2. Construction via cuckoo hashing

Bloomier filters (Chazelle, Kilian, Rubinfeld, and Tal, 2004) use the cuckoo graph in a static, offline setting.

Step 1: Build an acyclic cuckoo graph. Choose random $g, h : U \rightarrow \{0, \dots, m-1\}$. Build the cuckoo graph. If it contains a cycle, pick new g, h and retry. Since $\Pr[\text{cycle}] \leq 1/2$, the expected number of attempts is 2.

Step 2: Root the forest and assign ownership. Root each tree arbitrarily. Each item x (an edge) *owns* its deeper endpoint. Since the graph is a forest, no two items own the same vertex.

Step 3: Fill in r -bit strings top-down. Allocate an array $A[0..m-1]$ of r -bit strings. Set $A[v] = 0^r$ for every root. Process non-root vertices from top to bottom. When you reach the vertex v owned by item x :

- x 's edge connects v to its parent u (which has already been filled in).
- Set $A[v] = A[u] \oplus v(x)$, where $\oplus$ is bitwise XOR.

Query. To retrieve the value for item x , compute $A[g(x)] \oplus A[h(x)]$. If $x \in S$, one of $g(x), h(x)$ is x 's owned vertex v and the other is its parent u . By construction, $A[u] \oplus A[v] = A[u] \oplus (A[u] \oplus v(x)) = v(x)$.

⚠ Non-members get garbage

If $x \notin S$, the query returns $A[g(x)] \oplus A[h(x)]$ —an arbitrary r -bit string with no meaningful guarantee. This is the retrieval problem's defining trade-off: you save space by not storing keys, at the cost of being unable to detect imposters.

i Aside (mine, not the lecturer's)

A student observed that you could instead store $v(x)$ directly in x 's owned vertex—but then queries would need to know *which* of $g(x), h(x)$ is the owned vertex. That requires solving a 1-bit retrieval problem (store one bit per item indicating the deeper endpoint), creating a circular dependency. The XOR trick avoids this elegantly: $A[g(x)] \oplus A[h(x)]$ is the same regardless of which endpoint is deeper.

4.6. Cheat sheet

! Lecture 4 — key results

Linear probing (5-wise). Pagh–Pagh–Ružić binary tree trick: union-bound over $O(1)$ tree nodes (not k intervals) $\Rightarrow$ fourth moment suffices $\Rightarrow$ 5-wise independence gives $O(1)$ expected time. 4-wise is insufficient (Pătraşcu–Thorup counterexample).

Symmetrization. Replace $X_i - \mu$ with $\sigma_i X_i$ (random signs). Only monomials with all-even exponents survive, simplifying moment computations. Sixth moment of $\sum X_i$: dominant term is $O(n^3 p^3)$ from three distinct indices appearing twice each.

Bloom filter. Approximate membership in $O(n \log(1/\epsilon))$ bits. $d = \log_2(1/\epsilon)$ independent hash functions into d bit arrays of size $2n$. Query: AND all lookups. 2-wise independence suffices per hash function.

Cuckoo hashing. Two hash functions g, h . Worst-case $O(1)$ lookup (check two cells). Expected $O(1)$ insertion (eviction chain + rare rebuilds). Cuckoo graph: m vertices, n edges; acyclicity $\Rightarrow$ all items fit.

Bloomier filter (retrieval). Static, $O(nr)$ bits. Build acyclic cuckoo graph, root forests, fill r -bit strings top-down via XOR. Query: $A[g(x)] \oplus A[h(x)]$. Non-members get arbitrary output.

4.7. FAQ — questions students actually asked

Q: Why do you introduce random signs in the symmetrization trick? Can't you just expand the sixth power directly? You can, but without symmetrization you must track cancellations between positive and negative monomials arising from the centered variable $X_i - \mathbb{E}[X_i]$. Symmetrization makes every monomial with an odd-powered index vanish automatically (since $\mathbb{E}[\sigma^{\text{odd}}] = 0$), leaving only the all-even monomials. This turns an algebraic cancellation problem into a combinatorial counting problem.

Q: For Bloom filters, if I query the same element twice, do I get the same answer? Yes. The hash functions are fixed once the data structure is built. The randomness is in the *choice* of hash functions, not in the query procedure. To reduce the false positive rate, you cannot repeat queries—you need independent data structures with different hash functions.

Q: In the cuckoo graph, can you have two edges between the same pair of vertices? Yes—the cuckoo graph is a multigraph. If two items x, y satisfy $g(x) = g(y)$ and $h(x) = h(y)$ (or $g(x) = h(y)$ and $h(x) = g(y)$), they produce parallel edges. You can also get self-loops if $g(x) = h(x)$ for some item x .

Q: Does cuckoo hashing break immediately if $g(x') = h(x')$ for some item? If $g(x') = h(x')$, then x' has only one candidate cell. If x' gets kicked out, it would be sent back to the same cell, creating an immediate cycle. This triggers a rebuild. But self-loops are rare: $\Pr[g(x) = h(x)] = 1/m$, and by a union bound over n items, the probability of *any* self-loop is $n/m < 1$ for $m > n$.

Q: For bloomier filters, why use XOR instead of just storing the value at the owned vertex? If you stored $v(x)$ at the owned vertex, the query would need to know *which* of $g(x), h(x)$ is the deeper node. You don't store the keys, so you can't look them up to determine depth. Answering “which endpoint is deeper?” is itself a 1-bit retrieval problem—a circular dependency. XOR sidesteps this: $A[g(x)] \oplus A[h(x)]$ is order-independent.

Q: What independence do Bloom filters need? Each of the d hash functions only needs 2-wise independence. The false-positive analysis for a single array uses a union bound over pairwise collisions $\Pr[h(x) = h(y)] = 1/m$, which requires only that each pair of hash values is independent.

4.8. Further reading

1. Pagh, Pagh, Ružić (2009). “Linear probing with constant independence.” *SIAM J. Comput.* 39(3). The 5-wise independence proof for linear probing.
2. Bloom (1970). “Space/time trade-offs in hash coding with allowable errors.” *Commun. ACM* 13(7). The original Bloom filter.
3. Pagh & Rodler (2004). “Cuckoo hashing.” *J. Algorithms* 51(2). The cuckoo hashing scheme and its analysis.
4. Chazelle, Kilian, Rubinfeld, Tal (2004). “The Bloomier filter: an efficient data structure for static support lookup tables.” *SODA 2004*. Bloomier filters for the retrieval problem.
5. Thorup (2013). “Simple tabulation, fast expanders, double tabulation, and high independence.” *FOCS 2013*. Fast k -wise independent families via tabulation.
6. Dietzfelbinger et al. (1997). “A reliable randomized algorithm for the closest-pair problem” (contains the multiply-shift hashing scheme). Pairwise independent hashing without modular arithmetic.

4.9. Try it yourself

1. Symmetrization warm-up. Let $X_1, \dots, X_n$ be independent $\{0, 1\}$ random variables with $\mathbb{E}[X_i] = p$. Expand $\mathbb{E}[(\sum \sigma_i X_i)^4]$ explicitly. Identify which monomials survive and show the result is $O(n^2 p^2)$.
2. Bloom filter experiment. Build a Bloom filter with $n = 10,000$, $m = 20,000$, and $d \in \{1, 3, 7, 10, 20\}$. For each d , measure the empirical false positive rate over 10^5 random non-member queries. Plot the measured rate against the theoretical $(1/2)^d$.

4. Hashing II: Bloom Filters, Cuckoo Hashing, and Bloomier Filters

```
import numpy as np

def bloom_filter_experiment(n, m, d, num_queries, seed=42):
    """Measure empirical false positive rate of a Bloom filter."""
    rng = np.random.default_rng(seed)
    p = 200_003 # prime >= m

    # Generate hash function coefficients (2-wise:  $h(x) = (a*x + b) \bmod p \bmod m$ )
    coeffs = rng.integers(0, p, size=(d, 2))

    def hash_val(x, a, b):
        return ((a * x + b) % p) % m

    # Insert n items
    items = rng.integers(0, 10**9, size=n)
    arrays = np.zeros((d, m), dtype=bool)
    for j in range(d):
        a, b = int(coeffs[j, 0]), int(coeffs[j, 1])
        for x in items:
            arrays[j, hash_val(int(x), a, b)] = True

    # Query non-members
    non_members = rng.integers(10**9, 2 * 10**9, size=num_queries)
    false_positives = 0
    for x in non_members:
        if all(arrays[j, hash_val(int(x), int(coeffs[j,0]), int(coeffs[j,1]))]
              for j in range(d)):
            false_positives += 1

    return false_positives / num_queries

for d in [1, 3, 7, 10, 20]:
    rate = bloom_filter_experiment(10_000, 20_000, d, 100_000)
    print(f"d={d:2d}  measured={rate:.6f}  theoretical={0.5*d:.6f}")
```

3. Cuckoo hashing. Implement cuckoo hashing with $m = 4n$ and two random hash functions. Insert $n = 10,000$ random keys. Count the average eviction chain length per insertion and the number of full rebuilds.
4. Binary tree trick. Draw the perfect binary tree for an array of size 16. Identify the $O(1)$ nodes at height 1 that span the interval $[3, 10]$. Verify that there are between 4 and 9 of them.
5. Open-ended. The bloomier filter construction requires the cuckoo graph to be acyclic. What if you allowed *one* cycle per component (unicyclic graphs)? Can you still fill in the XOR values consistently? Under what condition on the cycle?

5. Cuckoo Hashing Analysis and the Power of Two Choices

COMPSCI 224 Lecture 5 — Fall 2014

We complete the expected-constant-time insertion proof for cuckoo hashing by bounding path, single-cycle, and double-cycle configurations in the cuckoo graph. Then we turn to load balancing: by hashing each item to two random bins and placing it in the lighter one, the maximum load drops from $\Theta(\log n / \log \log n)$ to $\ln \ln n / \ln 2 + \Theta(1)$ — an exponential improvement from a single extra choice.

5.1. Why this lecture matters

The cuckoo hashing analysis demonstrates a clean union-bound technique that recurs throughout randomized algorithms: enumerate all “bad” configurations, bound each one’s probability, then show the sum is negligible. The power of two choices is one of the most surprising results in load balancing — it says that a *single* extra random probe, costing almost nothing at insertion time, buys you an exponential drop in the worst-case load.

30-second story

1. Cuckoo hashing inserts in expected $O(1)$ time because long paths, single-cycle tours, and double-cycle tours in the cuckoo graph each have geometrically decaying probability.
2. If a double cycle occurs, we rebuild the entire table; this happens with probability $O(1/n^2)$, so the amortized rebuild cost is negligible.
3. With one random hash function, the heaviest bin among n balls in n bins is $\Theta(\log n / \log \log n)$.
4. With *two* random hash functions and a “least-loaded” tie-break, the heaviest bin drops to $\ln \ln n / \ln 2 + \Theta(1)$.
5. Going from 2 to d choices only changes the base of the iterated logarithm — the big win is from 1 to 2.
6. A layered induction with Chernoff bounds makes the intuition rigorous.

5.2. Cuckoo hashing: finishing the insertion analysis

Recall from the previous lecture: we have a table A of size $m = 4n$ and two fully random hash functions $g, h : U \rightarrow [m]$. When we insert item x , we place it at $g(x)$; if that cell is occupied, we evict the current resident to its *other* hash location, and so on. If the chain of evictions exceeds $C \log n$ steps (for a constant C chosen later), we pick fresh g, h and rebuild the table from scratch.

The rebuild strategy. When rebuilding, we allocate a fresh array, choose new hash functions, and reinsert every item. If *any* insertion during rebuilding again exceeds $C \log n$ steps, we discard everything and start over. The key observation: each rebuild attempt succeeds with constant probability, so the expected number of attempts is $O(1)$.

5.2.1. The cuckoo graph

The cuckoo graph $G = (V, E)$ has:

- $|V| = m$ vertices, one per cell of the array A .
- $|E| = n$ edges, one per stored item. Item x contributes edge $(g(x), h(x))$.

This graph may have multi-edges (two items with the same pair of hash values) and self-loops ($g(x) = h(x)$ for some x). The structure of this graph completely determines the behavior of the insertion algorithm.

5.2.2. Three insertion scenarios

When we insert x , the eviction chain traces a walk in the cuckoo graph starting at $g(x)$. Three structural cases arise depending on the topology of the connected component we enter:

Case 1: Simple path. The chain of evictions follows a path $g(x) \rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$, where v_k is an empty cell. Each displaced item moves to its alternate location, and the last one finds a free spot. The insertion takes k steps and succeeds.

Case 2: Single cycle. The path hits a cycle. When we traverse the cycle once, every item in the cycle shifts one position. The element x then tries its alternate location $h(x)$, which may lead to another path that terminates at an empty cell. This resolves without rebuilding — a single cycle is not fatal.

Case 3: Double cycle. The path encounters two distinct cycles. After traversing the first cycle, the chain enters a second cycle, creating an infinite loop. This is the only fatal configuration — it triggers the $C \log n$ cutoff and forces a rebuild.

5.2.3. Setting up the analysis

Define indicator random variables for inserting a particular item x :

- P_k : the insertion produces a path of length $\geq k$,
- C_k : the insertion produces a single-cycle tour of length $\geq k$,
- D : the insertion encounters a double-cycle configuration.

Let T denote the insertion time. We can decompose the expected cost:

$$\mathbb{E}[T] = \sum_k \mathbb{E}[P_k] + \sum_k \mathbb{E}[C_k] + \Pr[D] \cdot n \mathbb{E}[T] + \Pr[\text{chain} > C \log n] \cdot n \mathbb{E}[T]. \quad (5.1)$$

The first two sums account for the normal insertion work. The last two terms account for rebuilds: a double cycle always triggers a rebuild (costing $n \mathbb{E}[T]$ to reinsert everything), and exceeding $C \log n$ steps (from a very long path or cycle) also triggers one.

The strategy: show $\Pr[D]$ and $\Pr[\text{chain} > C \log n]$ are $O(1/n^2)$, so after multiplying by n the rebuild contribution is $O(1/n) \cdot \mathbb{E}[T]$. Moving this to the left side and solving gives $\mathbb{E}[T] = O(1)$ provided the two sums are constants.

5.2.4. Bounding path probabilities

Fix a particular path configuration: choose k specific items $x_2, \dots, x_{k+1}$ and $k + 1$ specific vertices $v_0, v_1, \dots, v_k$ forming the path. The probability that g and h produce exactly this assignment has three components:

1. The starting vertex: $\Pr[g(x) = v_0] = 1/m$.
2. Each edge: Item x_i connects consecutive vertices. Either $g(x_i) = v_{i-1}$ and $h(x_i) = v_i$, or vice versa. Each specific assignment has probability $1/m^2$, and there are 2 valid orientations.
3. All k interior items are independent, giving $(2/m^2)^k$.

Combining:

$$\Pr[\text{this specific path}] = \frac{1}{m} \cdot \left(\frac{2}{m^2}\right)^k = \frac{2^k}{m^{2k+1}}. \quad (5.2)$$

💡 Where does 2^k come from?

Each interior item x_i connects two vertices via one of its hash values. Either $g(x_i)$ points left and $h(x_i)$ points right, or vice versa — two choices per item, hence 2^k total labelings. This is a symmetry factor, not a probability — the $1/m^{2k}$ already accounts for the hash values landing in specific positions.

Now union-bound over all configurations:

- At most m^{k+1} choices for the $k + 1$ vertices (with replacement, since vertices could theoretically repeat, though in a path they don't).
- At most $n(n-1) \cdots (n-k+1) \leq n^k$ choices for the k items.

Therefore:

$$\mathbb{E}[P_k] \leq n^k \cdot m^{k+1} \cdot \frac{2^k}{m^{2k+1}} = \frac{(2n)^k}{m^k} = \left(\frac{2n}{m}\right)^k. \quad (5.3)$$

With $m = 4n$, this becomes $(1/2)^k$. The geometric series sums to:

$$\sum_{k=1}^{\infty} \mathbb{E}[P_k] \leq \sum_{k=1}^{\infty} \frac{1}{2^k} = 1.$$

! Why $m = 4n$?

The table must be at least twice the number of items ($m > 2n$) for the geometric ratio $2n/m$ to be strictly less than 1. The choice $m = 4n$ gives ratio $1/2$, ensuring rapid convergence. Smaller constants (e.g., $m = 2.01n$) work but give a larger expected insertion time.

5.2.5. Bounding single-cycle configurations

A single-cycle tour of length k (counting edges) has three structural “segments”: a path leading into the cycle, the cycle itself, and a path exiting after resolution. By the pigeonhole principle, at least one of these three segments contains $\geq k/3$ edges.

Apply the path-probability analysis to just that longest segment. The same geometric decay applies, giving:

$$\mathbb{E}[C_k] \leq \frac{3}{2^{k/3}},$$

where the factor of 3 comes from the union bound over which of the three segments is longest. This again sums to a constant:

$$\sum_{k=1}^{\infty} \mathbb{E}[C_k] \leq 3 \sum_{k=1}^{\infty} \frac{1}{2^{k/3}} = \frac{3}{2^{1/3} - 1} \approx 11.6.$$

5.2.6. Bounding double-cycle probability

A double-cycle tour of length k (counting edges) involves $k - 2$ distinct vertices and k items. The structure is more complex: we must choose where the first cycle starts among the k steps, how long it is, and where the second cycle starts. This gives at most k^3 structural choices.

Fixing a specific configuration, the probability of seeing exactly those hash evaluations is:

$$\Pr[\text{specific double-cycle of length } k] = \frac{2^k}{m^{2k+2}}.$$

The extra factor of m^2 in the denominator (compared to a path) comes from the fact that x itself needs both $g(x)$ and $h(x)$ to land in specific positions (to enter the first and second cycles respectively).

Union-bounding:

- m^{k-2} vertex choices,
- n^k item choices,
- k^3 structural choices.

With $m = 4n$:

$$\Pr[D_k = 1] \leq m^{k-2} \cdot n^k \cdot k^3 \cdot \frac{2^k}{m^{2k+2}} = \frac{k^3}{2^k} \cdot \frac{1}{n^2} \quad (\text{up to off-by-one corrections}). \quad (5.4)$$

Summing over k : $\sum_{k=1}^{\infty} k^3/2^k = 26$ (a convergent series), so $\Pr[D] = O(1/n^2)$.

Off-by-one traps

The exact vertex count in a double-cycle tour is $k - 1$ or $k - 2$ depending on whether you count endpoints shared between segments. The lecturer noted this ambiguity live and deferred the fix to scribe notes. The analysis is robust: even if the exponent on n shifts by 1, we still get $\Pr[D] = O(1/\text{poly}(n))$ with the polynomial large enough to absorb the multiplication by n in the rebuild cost.

5.2.7. Wrapping up: choosing C

For the chain to exceed $C \log n$ steps without a double cycle, it must contain a path or single-cycle configuration of that length. By Equation 5.3:

$$\Pr[P_{C \log n}] \leq \frac{1}{2^{C \log n}} = \frac{1}{n^{C \log 2}} = \frac{1}{n^{0.69C}}.$$

Picking $C \geq 3/\log 2 \approx 4.33$ (say $C = 5$) makes this $O(1/n^3)$. After multiplication by n (the rebuild cost), the contribution is $O(1/n^2)$.

Combining all terms in Equation 5.1 and solving the resulting linear equation in $\mathbb{E}[T]$:

$$\mathbb{E}[T] = \frac{O(1)}{1 - O(1/n)} = O(1).$$

The expected insertion time for cuckoo hashing is constant.

5.3. The power of two choices

5.3.1. Setup: balls into bins

Consider the classical balls-into-bins model: throw n balls independently and uniformly at random into n bins. This models hashing with chaining when the hash function is perfectly random and the table size equals the number of items.

With a single random hash function, the maximum load (length of the longest chain) is:

$$\Theta\left(\frac{\log n}{\log \log n}\right)$$

with high probability. This is a classical result — the upper bound follows from a union bound over bins and a Stirling-type estimate of collision probabilities.

i Aside (mine, not the lecturer's)

The exact threshold is $(1 + o(1)) \log n / \log \log n$. The tightest analysis gives the leading constant as 1 — see Raab and Steger, “Balls into Bins — A Simple and Tight Analysis” (1998). The lower bound uses a second-moment argument on the number of bins with $\geq k$ balls.

5.3.2. The algorithm

Due to Azar, Broder, Karlin, and Upfal (STOC '94, journal version SICOMP '99):

Pick two independent random hash functions $g, h : U \rightarrow [n]$. When inserting item x , compare the current loads of bins $g(x)$ and $h(x)$, and place x in the *least loaded* one. Break ties arbitrarily.

Queries must check both $g(x)$ and $h(x)$, so the query time is at most twice the maximum load. The remarkable result:

Theorem. With high probability, the maximum load is:

$$\frac{\ln \ln n}{\ln 2} + \Theta(1). \quad (5.5)$$

This is an *exponential* improvement: $\log \log n$ vs. $\log n / \log \log n$. The ratio between the two grows like $(\log n)^{1-o(1)}$.

5.3.3. Intuition via the layered argument

Define $B(i)$ = number of bins with load at least i . Let the height of item x be its position within its bin: $H(x) = i$ means x is the i -th item placed in that bin.

For $H(x) \geq i + 1$ to happen, *both* bins $g(x)$ and $h(x)$ must have had load $\geq i$ at the time x was inserted. Since g and h are independent:

$$\Pr[H(x) \geq i + 1] \leq \Pr[\text{load}(g(x)) \geq i] \cdot \Pr[\text{load}(h(x)) \geq i] \leq \left(\frac{B(i)}{n}\right)^2. \quad (5.6)$$

Non-rigorous step

The inequality $\Pr[\text{load}(g(x)) \geq i] \leq B(i)/n$ treats $B(i)$ as fixed. In reality, $B(i)$ is a random variable correlated with the hash values. The rigorous proof (below) handles this via conditional thresholds.

Summing over all n items and pretending $B(i)$ is deterministic:

$$B(i + 1) \leq n \cdot \left(\frac{B(i)}{n}\right)^2 = \frac{B(i)^2}{n}. \quad (5.7)$$

Writing $\beta_i = B(i)/n$ (the fraction of bins at level $\geq i$):

$$\beta_{i+1} \leq \beta_i^2.$$

Once $\beta_i < 1/2$, iterated squaring drives it to zero extremely fast:

Step j	β_{i_0+j}
0	1/2
1	1/4
2	1/16
3	1/256
4	1/65536

After j steps: $\beta_{i_0+j} \leq 1/2^{2^j}$. We need $\beta_{i_0+j} < 1/n$ (meaning no bins at that level):

$$2^{2^j} > n \implies j > \log_2 \log_2 n.$$

Since $B(6)/n \leq 1/6 < 1/2$ trivially (at most $n/6$ bins can have 6+ items when there are only n items total), the maximum load is at most $6 + \lceil \log_2 \log_2 n \rceil + O(1)$.

! Key insight

The “squaring” recurrence $f(i+1) \leq f(i)^2$ converges to zero in $O(\log \log(1/f(i_0)))$ steps once $f(i_0) < 1$. This is why a single extra choice buys an exponential improvement: it turns a linear recurrence (where $B(i+1) \approx B(i)/e$ or similar) into a squaring one. The extra choice introduces a *product* of two independent probabilities, which translates to squaring.

5.3.4. Making it rigorous

The non-rigorous argument treated $B(i)$ as deterministic. To fix this, we use a layered induction:

1. Define thresholds. Set $\alpha_6 = n/(2e)$ and the recurrence:

$$\alpha_{i+1} = \frac{e \alpha_i^2}{n}.$$

The factor e provides slack above the “expected” value α_i^2/n .

2. Define good events. Let E_i be the event $\{B(i) \leq \alpha_i\}$. Observe $\Pr[E_6] = 1$ since $\alpha_6 = n/(2e) > n/6$.
3. Conditional concentration (Claim 1). Given that E_i holds:

$$\Pr[\overline{E_{i+1}} \mid E_i] \leq O(1) \cdot \Pr\left[\text{Bin}\left(n, \frac{\alpha_i^2}{n^2}\right) > \alpha_{i+1}\right].$$

The right side is bounded by a Chernoff bound — the mean of the binomial is $\alpha_i^2/n = \alpha_{i+1}/e$, and we’re asking for a deviation by factor e .

4. Chernoff gives exponential decay. The Chernoff bound yields $\Pr[\overline{E_{i+1}} \mid E_i] \leq \exp(-\Omega(\alpha_{i+1}))$, which decreases doubly-exponentially as i grows.
5. Union bound over levels. Since there are only $O(\log \log n)$ levels and each failure probability is doubly-exponentially small, the union bound over all levels gives a high-probability guarantee.

i Dependency subtlety

Conditioning on E_i introduces correlations: knowing that few bins are heavy constrains which hash assignments are possible. The fix (omitted in lecture) uses a stochastic dominance argument: even conditioned on E_i , each item’s height- $(i+1)$ indicator is dominated by an independent Bernoulli with parameter $(\alpha_i/n)^2$. This lets us apply Chernoff to the sum as if the indicators were independent.

5.3.5. Power of d choices

What if you hash into d random locations and place in the least loaded? The maximum load becomes:

$$\frac{\ln \ln n}{\ln d} + \Theta(1).$$

The recurrence generalizes to $\beta_{i+1} \leq \beta_i^d$, which converges in $\log_d \log n$ steps — merely changing the base of the outer logarithm from 2 to d . The dramatic improvement comes in the first step (1 to 2 choices); after that, diminishing returns set in.

Choices d	Max load (approx., $n = 10^6$)
1	≈ 5.3 (i.e., $\log n / \log \log n$)
2	≈ 4.3 (i.e., $\ln \ln n / \ln 2$)
3	≈ 3.7
10	≈ 2.6

i Aside (mine, not the lecturer's)

Vocking (STOC 2003) showed that by partitioning bins into d equal groups, probing one random bin per group, and breaking ties by choosing the *leftmost* group, the maximum load improves to $\ln \ln n / (d \ln \phi_d) + O(1)$ where ϕ_d is a constant growing with d . This effectively replaces $\ln d$ with d in the denominator — an exponential-in- d improvement. The asymmetric tie-breaking rule is essential; symmetric tie-breaking cannot achieve this.

5.4. Preview: Fibonacci heaps

The lecture closed with a preview of the next topic. A priority queue (heap) stores n items with comparable keys and supports insert, delete-min, and decrease-key. The running time of Dijkstra's shortest-path algorithm depends directly on these costs:

$$T_{\text{Dijkstra}} = O(n \cdot T_{\text{ins}} + n \cdot T_{\text{del-min}} + m \cdot T_{\text{dec-key}}).$$

Operation	Binary heap	Fibonacci heap
Insert	$O(\log n)$	$O(1)$ amortized
Decrease-key	$O(\log n)$	$O(1)$ amortized
Delete-min	$O(\log n)$	$O(\log n)$ amortized

With a binary heap, Dijkstra runs in $O(m \log n)$. With a Fibonacci heap, it runs in $O(m + n \log n)$ — a significant improvement for dense graphs where $m \gg n$.

5.5. Cheat sheet

! Cheat sheet — Lecture 5

Cuckoo hashing insertion analysis

- Table size $m = 4n$; two fully random hash functions g, h .
- Cuckoo graph: m vertices, n edges ($x \mapsto \{g(x), h(x)\}$).
- $\Pr[\text{path of length } \geq k] \leq (1/2)^k$ — geometric decay.
- Single-cycle tour: $\mathbb{E}[C_k] \leq 3/2^{k/3}$ — also geometric.
- $\Pr[\text{double cycle}] = O(1/n^2)$ — rebuild cost is amortized $O(1/n)$.
- Cutoff at $C \log n$ steps; choose $C \geq 5$ for $O(1/n^2)$ rebuild probability.
- Expected insertion time: $O(1)$.

Power of two choices

- 1 choice: max load = $\Theta(\log n / \log \log n)$.
- 2 choices (least loaded): max load = $\ln \ln n / \ln 2 + \Theta(1)$.

- d choices: max load = $\ln \ln n / \ln d + \Theta(1)$.
- Key recurrence: $B(i+1)/n \leq (B(i)/n)^2$ — iterated squaring.
- Rigorous proof: layer-by-layer induction + Chernoff bounds + stochastic dominance.
- Reference: Azar, Broder, Karlin, Upfal (SICOMP '99).

Fibonacci heaps (preview)

- Insert and decrease-key in $O(1)$ amortized; delete-min in $O(\log n)$ amortized.
- Dijkstra: $O(m + n \log n)$ instead of $O(m \log n)$.

5.6. FAQ — questions students actually asked

Q: When you say “fix the assignment of $k+1$ hash values to vertices,” what exactly are you fixing?

A: You’re choosing which $k+1$ cells of the table (vertices of the cuckoo graph) are involved in the path, *and* which k stored items form the edges. The probability calculation conditions on a specific assignment of hash function outputs to those vertices. Think of it as: “suppose this exact picture appears in the cuckoo graph — what’s the probability?” Then sum over all possible pictures.

Q: Where does the 2^k factor come from in the path probability?

A: Each interior item x_i sits at the meeting point of two consecutive edges. It could be there because $g(x_i)$ maps to the left vertex and $h(x_i)$ to the right, or vice versa. Two orientations per item, k items gives 2^k valid orientations for the same vertex sequence. The factor is multiplicative because each item’s hash functions are independent.

Q: For the double-cycle probability, you wrote $\Pr[D=1] \leq \sum_k \Pr[D_k=1]$ with a union bound. Shouldn’t each D_k be a *subset* of D , making the inequality go the other way?

A: The events $\{D_k=1\}$ are actually *disjoint* — D_k is the event that the k -th step is *exactly* where the second cycle closes. You can’t close the second cycle at both step 7 and step 9. So the inequality is in fact equality: $\Pr[D=1] = \sum_k \Pr[D_k=1]$. The “union bound” here is exact.

Q: For power of d choices, why doesn’t going from 2 to 3 choices help as much as going from 1 to 2?

A: The qualitative jump happens between $d=1$ and $d=2$ because going from $\beta_{i+1} \leq \beta_i \cdot (\text{something})$ to $\beta_{i+1} \leq \beta_i^2$ changes the convergence from polynomial-in- i to doubly-exponential-in- i . Going from β_i^2 to β_i^3 is merely a constant-factor speedup in the same doubly-exponential regime. Concretely: $\ln \ln n / \ln 2 \approx 4.3$ vs. $\ln \ln n / \ln 3 \approx 3.7$ for $n = 10^6$.

Q: Is the layered argument for two choices fully rigorous as stated?

A: No. Treating $B(i)$ as deterministic ignores correlations: conditioning on E_i (that $B(i)$ is small) affects which hash assignments are possible, introducing dependencies between different items’ height indicators. The rigorous version handles this via stochastic dominance (showing the conditional distribution is dominated by independent Bernoullis) before applying Chernoff.

Q: Could you just use d totally random locations instead of two hash functions?

A: Yes — d independent hash functions and “place in least loaded” gives the $\ln \ln n / \ln d$ bound. But there’s a subtlety: you need the d hash functions to be mutually independent. Two-wise independence between pairs is not sufficient for the analysis.

5.7. Further reading

- Pagh and Rodler (2004). “Cuckoo Hashing.” *Journal of Algorithms* 51(2):122–144. The original cuckoo hashing paper with the $O(1)$ worst-case lookup guarantee.
- Azar, Broder, Karlin, and Upfal (1999). “Balanced Allocations.” *SIAM Journal on Computing* 29(1):180–200. The foundational power-of-two-choices result.
- Mitzenmacher, Richa, and Sitaraman (2001). “The Power of Two Random Choices: A Survey of Techniques and Results.” Comprehensive survey covering extensions to weighted jobs, non-uniform distributions, and queueing.
- Vocking (2003). “How Asymmetry Helps Load Balancing.” *Journal of the ACM* 50(4):568–589. The grouped-bins variant with tie-breaking by position.
- Devroye and Morin (2003). “Cuckoo Hashing: Further Analysis.” *Information Processing Letters* 86(4):215–219. Tighter analysis of insertion time and the effect of the load factor n/m .

5.8. Try it yourself

1. Path probability generalization (warm-up). Verify that for $m = cn$ with $c > 2$, $\mathbb{E}[P_k] \leq (2/c)^k$, and compute the expected total path contribution $\sum_k \mathbb{E}[P_k]$ as a function of c . For what value of c does this sum first drop below 0.1?
2. Power of d choices simulation. Write a Python program that, for $n = 10^5$ and $d \in \{1, 2, 3, 5, 10\}$, simulates the load-balancing process and plots the maximum load vs. d . Overlay the theoretical bound $\ln \ln n / \ln d$. Run 50 trials per d and show error bars.

```
import numpy as np
import matplotlib.pyplot as plt

def max_load(n, d, trials=50):
    """Simulate d-choice load balancing and return average max load."""
    results = []
    for _ in range(trials):
        bins = np.zeros(n, dtype=int)
        for _ in range(n):
            choices = np.random.randint(0, n, size=d)
            best = choices[np.argmin(bins[choices])]
            bins[best] += 1
        results.append(bins.max())
    return np.mean(results), np.std(results)

n = 100_000
ds = [1, 2, 3, 5, 10]
means, stds = zip(*[max_load(n, d) for d in ds])

plt.figure(figsize=(7, 4))
plt.errorbar(ds, means, yerr=stds, fmt='o-', capsize=4, label='Simulated')
theoretical = [np.log(n) / np.log(np.log(n))]
theoretical += [np.log(np.log(n)) / np.log(d) + 1 for d in ds[1:]]
plt.plot(ds, theoretical, 's--', label='Theoretical bound')
plt.xlabel('Number of choices $d$')
plt.ylabel('Maximum load')
```

```
plt.legend()
plt.title(f'Power of  $d$  choices ( $n = \{n:, \}$ )')
plt.tight_layout()
plt.savefig('power_of_d_choices.png', dpi=150)
plt.show()
```

3. Cuckoo graph cycles. Prove that if the cuckoo graph on $m = (2 + \epsilon)n$ vertices is acyclic, then all insertions succeed without any eviction chain exceeding the diameter of the component. (*Hint*: in a forest, each connected component is a tree, and the eviction chain from any vertex traverses at most the tree's depth.)
4. Static optimality of two choices. Show that no *oblivious* scheme (one that doesn't inspect bin loads) can achieve maximum load $o(\log n / \log \log n)$ with n balls and n bins, even with d hash functions. This proves that the load *inspection* is essential, not just the extra randomness. (*Hint*: consider the first n/d balls — each sees d random bins with no load information.)
5. Breaking the symmetry (Vocking). Implement the grouped-bins variant: partition n bins into d equal groups, probe one random bin per group, place in the least loaded, break ties by choosing the *leftmost* group. Compare the maximum load against the standard d -choice scheme for $n = 10^5$ and $d = 4$ over 100 trials. How large is the improvement?

6. Amortized Analysis, Binomial Heaps, and Fibonacci Heaps

COMPSCI 224 Lecture 6 — Fall 2014

We develop the potential function method for proving amortized bounds, then apply it twice: first to binomial heaps (constant amortized insertion, $O(\log n)$ worst-case delete-min), then to Fibonacci heaps (constant amortized insertion *and* decrease-key, $O(\log n)$ amortized delete-min). The key insight behind Fibonacci heaps is strategic laziness — defer consolidation to delete-min, and use cascading cuts to prevent trees from becoming too shallow.

6.1. Why this lecture matters

Fibonacci heaps sit at the heart of the fastest known algorithms for shortest paths (Dijkstra), minimum spanning trees (Prim), and network flow. Understanding *how* they achieve their bounds — through the interplay of laziness and controlled degradation — teaches you a design pattern applicable far beyond heaps: let cheap operations accumulate structural debt, and repay it during the expensive operation that already has to examine the structure.

30-second story

1. Amortized analysis asks: what is the *total* cost of a sequence of operations, divided fairly among them?
2. The potential function method formalizes “structural debt” — cheap operations increase potential, expensive ones discharge it.
3. Binomial heaps achieve constant amortized insertion by treating the forest as a binary number and insertion as incrementing.
4. Fibonacci heaps go further: insertion just appends to a linked list (consolidation is deferred to delete-min), and decrease-key cuts the node out of its tree in $O(1)$ time.
5. Cascading cuts (cut a node when it loses a *second* child) prevent trees from becoming too sparse, ensuring rank- k trees have $\geq F_{k+2}$ nodes.
6. The potential $\Phi = t(H) + 2m(H)$ (trees + twice marked nodes) makes the bookkeeping work out exactly.

6.2. Amortized analysis and the potential function method

6.2.1. The idea

Worst-case analysis can be pessimistic. A data structure might have one expensive operation that only happens after many cheap ones — charging the full worst-case cost to every operation overstates the true cost of any realistic sequence.

Suppose a data structure supports operations A, B, C . Their amortized costs are $\hat{t}_A, \hat{t}_B, \hat{t}_C$ if *every* interleaving of n_A A -operations, n_B B -operations, and n_C C -operations has total actual cost at most:

$$n_A \hat{t}_A + n_B \hat{t}_B + n_C \hat{t}_C. \quad (6.1)$$

An individual operation may cost much more than its amortized bound (even $O(n)$), as long as the *aggregate* over any sequence respects the sum. Amortized bounds are deterministic — no probability is involved. They are strictly stronger than “expected” bounds (which could have rare bad sequences) but weaker than worst-case bounds per operation.

6.2.2. The potential function method

Define a potential function Φ that maps each state S of the data structure to a non-negative real number, with $\Phi(S_0) = 0$ for the initial (empty) state.

After k operations with actual costs $t_1, \dots, t_k$ producing states $S_0, S_1, \dots, S_k$, define the amortized cost of operation i as:

$$\hat{t}_i = t_i + \Phi(S_i) - \Phi(S_{i-1}). \quad (6.2)$$

Summing over all operations telescopes the potential terms:

$$\sum_{i=1}^k \hat{t}_i = \sum_{i=1}^k t_i + \Phi(S_k) - \underbrace{\Phi(S_0)}_{=0} \geq \sum_{i=1}^k t_i, \quad (6.3)$$

since $\Phi(S_k) \geq 0$. The sum of amortized costs *upper-bounds* the total actual cost. If we can show that each operation type has amortized cost at most some target $\hat{t}$, then any sequence of k operations costs at most $k\hat{t}$ in total.

Bank account intuition

Think of $\Phi(S)$ as the balance of a bank account attached to the data structure.

- Cheap operations (like insertion) may deposit money: they do little actual work but increase Φ , so their amortized cost = actual work + deposit is slightly more than what they truly cost.
- Expensive operations (like delete-min) withdraw money: they do a lot of actual work but decrease Φ , so their amortized cost = actual work – withdrawal is much less than what they truly cost.

As long as the bank balance never goes negative ($\Phi \geq 0$ always), the total deposits cover the total withdrawals, and the amortized costs are a valid upper bound.

6.2.3. Designing a potential function

There is no unique “correct” potential function. The art is engineering one that:

1. Is always non-negative and zero at the start.
2. Increases by a controlled amount during cheap operations.
3. Decreases by *at least* the excess actual cost during expensive operations.

In practice, you look at what “structural mess” the cheap operations create (more trees, more marked nodes, more items in a buffer) and define Φ to measure that mess. The expensive operation cleans up the mess — its actual cost is proportional to the mess size, but the potential drop cancels it.

6.3. Heaps: the landscape

A heap (priority queue) stores n comparable items and supports:

Operation	Meaning
<code>insert(x)</code>	Add item x to the collection
<code>delete-min()</code>	Return and remove the item with the smallest key
<code>decrease-key(p, k)</code>	Reduce the key at pointer p to new value $k \leq$ current key

These three operations are exactly what Dijkstra's single-source shortest path algorithm needs. On a graph with n vertices and m edges (non-negative weights), Dijkstra performs n inserts, n delete-mins, and up to m decrease-keys:

$$T_{\text{Dijkstra}} = O(n \cdot T_{\text{ins}} + n \cdot T_{\text{del-min}} + m \cdot T_{\text{dec-key}}). \quad (6.4)$$

Implementation	Insert	Decrease-key	Delete-min	Dijkstra
Binary heap (Williams '64)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(m \log n)$
Binomial heap (Vuillemin '78)	$O(1)$ amort.	$O(\log n)$	$O(\log n)$	$O(m \log n)$
Fibonacci heap (Fredman-Tarjan '87)	$O(1)$ amort.	$O(1)$ amort.	$O(\log n)$ amort.	$O(m + n \log n)$

i Why not all $O(1)$?

In a comparison-based data structure, at least one of these operations must cost $\Omega(\log n)$ amortized. Otherwise, you could sort n items in $O(n)$: insert all, then delete-min n times. This contradicts the $\Omega(n \log n)$ comparison-based sorting lower bound.

i Aside (mine, not the lecturer's)

Beyond comparison-based heaps: Brodal (SODA '96) achieves the Fibonacci heap bounds in *worst case* (not amortized). Brodal, Lagogiannis, and Tarjan (STOC 2012) do the same on a pure pointer machine. In the word RAM model, Van Emde Boas trees and dynamic fusion trees give all operations in $O(\sqrt{\log n})$, though these exploit the integer structure of keys.

6.4. Binomial heaps

6.4.1. Structure

A binomial heap maintains items in a forest of heap-ordered trees. Each tree has a rank equal to the degree of its root. Three invariants:

1. Heap order: each node's key $\leq$ all keys in its subtree.
2. Binomial tree structure: a rank- k tree has exactly 2^k nodes. Its root has k children, which are trees of ranks $0, 1, \dots, k-1$ (in that order, left to right).
3. Uniqueness: at most one tree of each rank exists in the forest.

A rank- k binomial tree B_k is constructed recursively: take two copies of B_{k-1} and make one the leftmost child of the other's root. Equivalently, the root of B_k has children that are $B_0, B_1, \dots, B_{k-1}$.

💡 Binary number analogy

A binomial heap with n items corresponds to the binary representation of n : there is a tree of rank k if and only if bit k of n is 1. For example, $n = 13 = 1101_2$ means the heap has trees of ranks 0, 2, and 3 (sizes 1, 4, 8). Since n has at most $\lfloor \log_2 n \rfloor + 1$ bits, the forest has at most $O(\log n)$ trees, and the maximum rank is at most $\lfloor \log_2 n \rfloor$.

6.4.2. Operations

Link (helper). Given two rank- k trees T_1, T_2 : compare their roots. The one with the larger key becomes the leftmost child of the other. Result: one rank- $(k+1)$ tree. Cost: $O(1)$.

Insert. Create a singleton (rank-0 tree) and add it to the forest. If there's already a rank-0 tree, link them to form a rank-1 tree. If there's already a rank-1 tree, link again. Continue carrying until there's no collision. This is exactly binary addition: incrementing the binary number by 1.

Delete-min. The minimum element is at one of the (at most $\log n$) roots. Find it, remove it. Its k children (which are $B_0, B_1, \dots, B_{k-1}$) become new root-level trees. Then merge trees of equal rank until uniqueness is restored — again, binary addition.

Decrease-key. Decrease the key at the given pointer. If heap order is now violated (new key $<$ parent's key), swap with parent. Repeat up the tree until heap order holds. Cost: $O(\text{depth}) = O(\log n)$ since a rank- k tree has depth $k \leq \log n$.

6.4.3. Amortized analysis of insertion

Define the potential $\Phi = \Phi$ number of trees in the forest.

The actual cost of an insertion equals $1 + (\text{number of links})$, since each link takes $O(1)$ work. If there are ℓ links (carries), then ℓ trees are merged and 1 new tree is created, so the number of trees changes by $1 - \ell$ (we lose ℓ trees to linking but gain 1 from the new item ... actually we gain a net of $1 - \ell$ since each link removes one tree).

Wait — let's be precise. Before insertion: t trees. After insertion: $t' = t - \ell + 1$ trees (we removed ℓ trees via linking, but the resulting tree plus any unmerged trees remain... actually $t' = t + 1 - 2\ell + \ell = t + 1 - \ell$... let me think again).

Each link merges two rank- k trees into one rank- $(k+1)$ tree, reducing the tree count by 1. Starting with t trees, adding the new singleton gives $t+1$ trees, then ℓ links reduce this to $t+1-\ell = t'$. So $\ell = t+1-t'$.

Actual cost: $O(1 + \ell) = O(t + 1 - t' + 1) = O(t - t' + 2)$.

! Amortized insertion cost

$$\hat{t}_{\text{ins}} = \underbrace{O(t - t' + 2)}_{\text{actual cost}} + \underbrace{(t' - t)}_{\Delta\Phi} = O(2).$$

The linking work (which could be $O(\log n)$ in the worst case) is exactly cancelled by the potential decrease. Every insertion has $O(1)$ amortized cost.

For delete-min: the actual cost is $O(\log n)$ (scan roots + link equal-rank trees). The potential change is bounded by $O(\log n)$ (we start with at most $\log n$ trees, remove the min root adding $\leq \log n$ new trees, then consolidate back down). The amortized bound is $O(\log n)$.

6.5. Fibonacci heaps

6.5.1. Design philosophy: strategic laziness

Fibonacci heaps start from two key observations about binomial heaps:

Observation 1: Insertion doesn't need to consolidate. If all we ever do is insert, why merge trees eagerly? Just append each new item to the root list. The only reason to consolidate is to keep the root list short for delete-min. So defer all consolidation to the delete-min operation — it has to scan the root list anyway.

Observation 2: Decrease-key shouldn't bubble up. Bubbling a node up through a tree of depth $\log n$ costs $O(\log n)$. A lazier approach: if decreasing a node's key violates heap order, simply *cut* the node out of its tree and make it a new root. This is $O(1)$.

6.5.2. The problem with “ultra-lazy” decrease-key

If we cut nodes out indiscriminately, a rank- k tree might lose most of its descendants. Consider a rank- k tree in a binomial heap — it has 2^k nodes. Now suppose we decrease-key every non-root node at depth 2 or below. After all these cuts, the tree still has rank k (same root degree), but it might have only $k + 1$ nodes remaining: just the root and its k immediate children.

This is a disaster for delete-min. Its cost is proportional to the maximum rank in the heap. In a binomial heap, the argument “rank- k tree has 2^k nodes, so maximum rank $\leq \log n$ ” guaranteed $O(\log n)$ delete-min. With sparse trees, the maximum rank could be $\Theta(n)$, making delete-min linear.

6.5.3. Cascading cuts: the fix

Each node carries a mark bit (initially 0). The rule is:

1. When a non-root node p loses its *first* child to a cut: set $\text{mark}(p) = 1$. No further action.
2. When p loses a *second* child: cut p itself out of its tree (making it a new root), clear its mark bit, and check p 's *parent* — if that parent is already marked, cut it too. This cascades upward.

The cascade stops when it hits either an unmarked node (which gets marked) or a root. Each cascading cut takes $O(1)$ time, and the total cascade length is what we'll need to account for in the amortized analysis.

Intuition for the rule

The mark bit is a communication channel: it tells the parent “you've already lost one child — if you lose another, you're too thin to remain attached.” This prevents any node from losing more than one child at each depth level, which is exactly what's needed to maintain the Fibonacci-number size bound.

6.5.4. Minimum tree sizes: Fibonacci numbers

Let s_k = minimum number of nodes in a rank- k tree (one that has suffered the maximum possible cuts). Build up from the base cases:

- $s_0 = 1$ (a single node).
- $s_1 = 2$ (root with one child).

6. Amortized Analysis, Binomial Heaps, and Fibonacci Heaps

For rank k : the root originally had children of ranks $0, 1, \dots, k-1$. Each child may have lost at most one of *its* children (otherwise it would have been cut itself). So the i -th child (originally rank i) now has rank $\geq i-1$. The smallest child has rank ≥ 0 , the next has rank ≥ 0 , then ≥ 1 , then ≥ 2 , ..., up to $\geq k-2$.

Therefore:

$$s_k \geq 1 + s_0 + s_0 + s_1 + s_2 + \dots + s_{k-2} = 2 + \sum_{j=0}^{k-2} s_j. \quad (6.5)$$

Compare with the Fibonacci sequence F_n (with $F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, \dots$). One can verify by induction that $s_k = F_{k+2}$:

Rank k	Min size s_k	F_{k+2}	Full binomial 2^k
0	1	1	1
1	2	2	2
2	3	3	4
3	5	5	8
4	8	8	16
5	13	13	32

Since Fibonacci numbers grow as $F_n = \Theta(\phi^n)$ where $\phi = (1 + \sqrt{5})/2 \approx 1.618$, a rank- k tree has at least ϕ^k nodes. Therefore:

$$\max \text{rank} \leq \log_{\phi} n \approx 1.44 \log_2 n = O(\log n).$$

This is the crucial bound: after consolidation in delete-min, we have at most one tree of each rank, and ranks are $O(\log n)$, so the root list has $O(\log n)$ trees.

6.5.5. Operations in detail

Insert(x). Create a singleton node, append it to the root list. Update the min-pointer if x is smaller than the current minimum. No consolidation. Actual cost: $O(1)$.

Delete-min. Remove the minimum root z . Splice z 's children into the root list (they become new roots, unmark them). Then consolidate:

1. Allocate an array $A[0 \dots \lceil \log_{\phi} n \rceil]$ of tree pointers (initially null).
2. For each root w in the root list: while $A[\text{rank}(w)]$ is non-null, link w with $A[\text{rank}(w)]$ (smaller root wins), clear that slot, and increment w 's rank. Set $A[\text{rank}(w)] = w$.
3. Rebuild the root list from the non-null entries of A . Update the min-pointer.

Actual cost: $O(t + \log n)$ where t is the root-list size before consolidation. After consolidation: at most $O(\log n)$ trees remain.

Decrease-key(p, k). Set p 's key to k . If $k < \text{parent's key}$:

1. Cut p from its parent, add p to the root list, unmark p .
2. If the parent was already marked: cut the parent too (cascading cut). Repeat.
3. If the parent was unmarked: mark it. Stop.

Let c = number of cascading cuts. Actual cost: $O(1 + c)$.

6.5.6. Amortized analysis

Define the potential:

$$\Phi(H) = t(H) + 2 m(H), \quad (6.6)$$

where $t(H)$ = number of trees in the root list and $m(H)$ = number of marked nodes in the entire heap.

This potential is always non-negative (both terms are non-negative) and starts at zero (empty heap has no trees and no marks).

6.5.6.1. Insertion

- Actual cost: $O(1)$.
- $\Delta t = +1$ (one new tree).
- $\Delta m = 0$ (no marks change).
- Amortized cost: $O(1) + 1 + 0 = O(1)$. ✓

6.5.6.2. Delete-min

Let t = number of trees before the operation.

- Actual cost: $O(t + \log n)$ — scan root list to find min ($O(t)$), splice children ($O(\log n)$), consolidate ($O(t + \log n)$ since we process each root).
- After consolidation: $t' \leq c_1 \log n$ trees for some constant c_1 .
- $\Delta t = t' - t \leq c_1 \log n - t$.
- $\Delta m \leq 0$ (we might unmark children that become roots, but we never add marks).
- Amortized cost: $O(t + \log n) + (c_1 \log n - t) + 0 = O(\log n)$. ✓

The t in the actual cost cancels with the $-t$ from the potential drop.

6.5.6.3. Decrease-key with c cascading cuts

- Actual cost: $O(1 + c)$.
- $\Delta t = +(c + 1)$: we create $c + 1$ new trees (the node itself plus c ancestors that get cut).

Wait — actually, only c cuts total (including the initial cut of x). Let me recount. The initial cut of x creates 1 new tree. Each of the $c - 1$ cascading cuts creates 1 more. Total: $\Delta t = +c$... Hmm, let me use the lecturer's notation: c = number of cascading cuts (not counting the initial cut). So we do $c + 1$ cuts total, creating $c + 1$ new trees? No — let me just be careful.

Let's say $c \geq 0$ cascading cuts happen after the initial cut of x . So total cuts = $c + 1$, each creating one new root-list tree. $\Delta t = c + 1$.

- Δm : The c cascading-cut nodes were marked (that's why they cascaded) and get unmarked when cut: $-c$. Node x itself may have been marked and gets unmarked: at most -1 more. But x 's parent (after the initial cut) might get newly marked: $+1$. Net: $\Delta m \leq -c + 1$.
- Amortized cost:

$$O(c + 1) + (c + 1) + 2(-c + 1) = O(c) + c + 1 - 2c + 2 = O(c) - c + 3 = O(1). \quad (6.7)$$

6. Amortized Analysis, Binomial Heaps, and Fibonacci Heaps

The key: each cascading cut contributes +1 to actual cost, +1 to Δt , and -2 to $2\Delta m$. Net per cascade: $1 + 1 - 2 = 0$. The cascading cuts are free in amortized terms — their cost is prepaid by the mark deposits from earlier decrease-key operations.

⚠ Why the coefficient 2?

Each cascading cut produces a new tree (+1 potential from the t term) but unmarks a node (-2 potential from the $2m$ term). Net change per cascading cut: -1 . This -1 exactly pays for the $O(1)$ actual work of the cut. If the coefficient were only 1, the net would be 0 — not enough to pay for the work. If it were 3, the analysis still works (with slightly looser bounds). The minimum coefficient that makes the proof go through is 2.

! The punchline

Fibonacci heaps achieve:

- Insert: $O(1)$ amortized.
- Decrease-key: $O(1)$ amortized.
- Delete-min: $O(\log n)$ amortized.

For Dijkstra's algorithm: $O(m + n \log n)$. This is optimal among comparison-based implementations (by the sorting lower bound), and it improves over binary heaps whenever $m = \omega(n)$.

6.6. Cheat sheet

! Cheat sheet — Lecture 6

Potential function method

- $\Phi : \text{states} \rightarrow \mathbb{R}_{\geq 0}$, with $\Phi(\text{empty}) = 0$.
- Amortized cost of operation i : $\hat{t}_i = t_i + \Phi(S_i) - \Phi(S_{i-1})$.
- Telescoping: $\sum \hat{t}_i = \sum t_i + \Phi(S_k) \geq \sum t_i$.
- Design Φ to measure “structural mess” that cheap ops create and expensive ops clean up.

Binomial heaps ($\$ \square = \$ \text{ number of trees}$)

- Forest of binomial trees; rank- k tree has 2^k nodes, at most one per rank.
- Corresponds to binary representation of n ; at most $\log n$ trees.
- Insert = add singleton + carry-propagate. Amortized $O(1)$.
- Delete-min: scan $O(\log n)$ roots, remove min, merge children. Worst case $O(\log n)$.
- Decrease-key: bubble up. Worst case $O(\log n)$.

Fibonacci heaps ($\Phi = t(H) + 2m(H)$)

- Insert: append to root list $\square O(1)$ amortized.
- Decrease-key: cut + cascading cuts $\square O(1)$ amortized.
- Delete-min: remove min root, splice children, consolidate $\square O(\log n)$ amortized.
- Cascading cut rule: mark on first child loss, cut on second.
- Tree size bound: rank- k tree $\geq F_{k+2} \geq \phi^k$ nodes $\square \text{ max rank} \leq \log_\phi n$.
- Delete-min worst case is $O(n)$ (after n lazy insertions), but amortized is $O(\log n)$.
- Dijkstra: $O(m + n \log n)$ — optimal for comparison-based heaps.

Key references

- Williams (1964) — binary heaps.
- Vuillemin (1978) — binomial heaps.
- Fredman & Tarjan (1987) — Fibonacci heaps.
- Brodal (1996) — worst-case version.
- Brodal, Lagogiannis & Tarjan (2012) — pointer-machine version.
- Kaplan, Tarjan & Zwick (2014) — single-tree simplification.

6.7. FAQ — questions students actually asked

Q: For Dijkstra’s runtime, shouldn’t it be m times decrease-key, not n times?

A: Yes. The relaxation step iterates over all edges leaving the extracted vertex, potentially performing one decrease-key per edge. Over the entire algorithm, that’s at most m decrease-keys. The correct formula is $O(n \cdot T_{\text{ins}} + n \cdot T_{\text{del-min}} + m \cdot T_{\text{dec-key}})$.

Q: Delete-min worst case in a Fibonacci heap — can it really be linear?

A: Yes. If you perform n insertions with no intervening delete-mins, the root list accumulates n singleton trees. The first delete-min must scan all n roots to find the minimum, then consolidate. Actual cost: $\Theta(n)$. But the amortized cost is $O(\log n)$ — those n insertions each deposited 1 unit of potential (one new tree), and the delete-min spends it all at once.

Q: When you do a cascading cut, do you also modify the key of the parent being cut?

A: No. Keys never change during cascading cuts. The parent p keeps its original key. The cut severs the edge between p and its parent, making p a new root-list tree. The subtree rooted at p (with whatever children remain) moves intact. Cascading cuts are purely structural — they rearrange the forest without touching keys.

Q: The coefficient 2 in front of the marked-node count seems arbitrary. Could you use a larger coefficient?

A: Any coefficient ≥ 2 gives a valid proof. The constraint is: each cascading cut creates +1 tree and removes -1 mark. If the mark coefficient is c , the net potential change per cascade is $+1 - c$. We need this to be ≤ -1 (to pay for the $O(1)$ actual work), so $c \geq 2$. Using $c = 2$ is the tightest choice; using $c = 3$ works but gives a looser (larger) amortized bound for decrease-key in the non-cascading case.

Q: Can’t you hope for all operations to be $O(1)$ amortized using word RAM tricks?

A: In the comparison model, no — sorting reduces to n inserts + n delete-mins, giving an $\Omega(n \log n)$ lower bound on total cost, hence $\Omega(\log n)$ amortized for at least one operation. In word RAM, yes: using Van Emde Boas or dynamic fusion trees, all priority queue operations can be done in $O(\sqrt{\log n})$ time (for polynomial-universe keys), beating the comparison lower bound.

Q: What’s the intuition for potential functions? They feel arbitrary.

A: Think of the potential as measuring “how much latent work has accumulated.” An insertion makes the structure messier — it deposits one unit of future work into the bank. A delete-min cleans up all the mess — it withdraws from the bank to pay for consolidation. The potential function quantifies this: $t(H)$ counts the “insertion debt” (trees that haven’t been consolidated yet), and $2m(H)$ counts the “cascading-cut debt” (nodes that are about to cascade if hit again). The numbers aren’t arbitrary — they emerge from asking “what cancels what in the amortized calculation?”

Q: If I use a Fibonacci heap, would I see a speedup in practice?

A: Probably not for typical graph sizes. Fibonacci heaps have large constant factors (pointer manipulations, mark-bit bookkeeping, complex consolidation logic). In practice, binary heaps with a d -ary variant (e.g., 4-heaps) often outperform Fibonacci heaps for $n < 10^6$ due to cache efficiency. Fibonacci heaps win only for very large, sparse graphs where $m \gg n \log n$.

6.8. Further reading

- Fredman and Tarjan (1987). “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms.” *Journal of the ACM* 34(3):596–615. The original paper. Worth reading for the motivation and the connection to shortest paths and MST.
- Brodal (1996). “Worst-Case Efficient Priority Queues.” *Proceedings of SODA* 1996:52–58. Same bounds as Fibonacci heaps, but worst-case. Remarkably clean construction.
- Brodal, Lagogiannis, and Tarjan (2012). “Strict Fibonacci Heaps.” *Proceedings of STOC* 2012:1177–1184. Pure pointer-machine implementation with no arrays.
- Kaplan, Tarjan, and Zwick (2014). “Fibonacci Heaps Revisited.” *arXiv preprint*. Simplifies the structure to maintain a single tree (no forest), aiming for practical simplicity.
- Vuillemin (1978). “A Data Structure for Manipulating Priority Queues.” *Communications of the ACM* 21(4):309–315. The original binomial heap paper.
- Cormen, Leiserson, Rivest, and Stein (2009). *Introduction to Algorithms* (CLRS), Chapter 19. Complete pseudocode and formal proofs for Fibonacci heaps.

6.9. Try it yourself

1. Dynamic array (potential function warm-up). A dynamic array starts empty and doubles its capacity (copying all elements) when full. Define a potential function Φ such that append has $O(1)$ amortized cost. What is Φ after exactly $n = 2^k$ appends? Verify that the total actual cost of n appends is $O(n)$ using your potential function.
2. Binomial heap merge. Show that merging two binomial heaps of sizes n_1 and n_2 takes $O(\log(n_1 + n_2))$ time. (*Hint*: adding two binary numbers of at most $\log n$ bits takes time proportional to the number of bits.)
3. Cascading cut depth. Construct an explicit sequence of $O(n)$ operations (insertions and decrease-keys) on an initially empty Fibonacci heap that produces a single decrease-key triggering $\Omega(\log n)$ cascading cuts. (*Hint*: first build a deep tree via insertions and delete-mins, then carefully decrease-key nodes at increasing depth to set up marks.)
4. Fibonacci tree construction. Write Python code that builds the minimum-size rank- k Fibonacci tree (the one with exactly F_{k+2} nodes) for $k = 0, 1, \dots, 10$. Print each tree’s structure and verify the node counts match Fibonacci numbers.

```
def min_fib_tree(k):
    """Build the minimum rank-k Fibonacci heap tree.

    Returns (rank, [children]) where each child is also a (rank, [children]) tuple.
    """
    if k == 0:
        return (0, [])
    if k == 1:
        return (1, [(0, [])])
    # Original children had ranks 0, 1, ..., k-1.
    # Each may have lost one child, reducing rank by at most 1.
    # Minimum: child i (originally rank i) now has rank max(0, i-1).
    children = [(0, [])] # child 0: still rank 0
    for i in range(1, k):
        children.append(min_fib_tree(i - 1)) # child i: rank i-1 (lost one)
```

```

    return (k, children)

def count_nodes(tree):
    """Count total nodes in a tree."""
    return 1 + sum(count_nodes(c) for c in tree[1])

def fib(n):
    """Return the n-th Fibonacci number (F_0=0, F_1=1, ...)."""
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

print(f"{'Rank k':<8} {'Nodes':<8} {'F(k+2)':<8} {'Match?'}")
print("-" * 35)
for k in range(11):
    t = min_fib_tree(k)
    n = count_nodes(t)
    f = fib(k + 2)
    print(f"{k:<8} {n:<8} {f:<8} {'yes' if n == f else 'NO'}")

```

5. Dijkstra comparison (empirical). Implement Dijkstra's algorithm with both a binary heap (Python's `heapq` with lazy deletion) and a simple Fibonacci heap simulation. Generate random sparse graphs ($m = 3n$, random edge weights in $[1, 1000]$). Plot wall-clock time vs. n for $n \in \{10^3, 10^4, 10^5\}$. Does the theoretical advantage of Fibonacci heaps materialize? Explain why or why not in terms of constant factors and cache behavior.

7. Splay Trees

COMPSCI 224 Lecture 7 — Fall 2014

Splay trees are self-adjusting binary search trees that achieve $O(\log n)$ amortized time per operation through a deceptively simple restructuring rule. Their real power lies not in matching balanced BSTs but in *adapting* to access patterns — the same data structure, with no parameter tuning, achieves static optimality, the working set property, and the static and dynamic finger properties. Whether splay trees are *dynamically optimal* — competitive with any BST algorithm, even one that knows the future — remains one of the longest-standing open problems in data structures.

7.1. Why this lecture matters

Splay trees won the 1999 Paris Kanellakis Theory and Practice Award for being “one of the most widely used data structures invented in the last 20 years.” The algorithm itself is trivial to implement — three cases, each a pair of rotations. The analysis is where all the difficulty lives, and the flexibility of the potential-function proof technique lets you derive a remarkable family of optimality properties by changing a single weight function.

The 30-second story

1. A splay tree is a binary search tree that moves every accessed node to the root via a specific sequence of double rotations.
2. Naive rotate-to-root doesn't work — it can leave the tree just as unbalanced as before.
3. The splay operation uses three cases (zig, zig-zig, zig-zag) that halve the depth of the path it traverses.
4. A potential-function argument with per-node *ranks* shows that each splay costs $O(\log(W/w(x)))$ amortized, where W is the total weight and $w(x)$ is the weight of the accessed node.
5. Setting all weights to 1 gives $O(\log n)$ amortized; setting weights to $1/3^{\{\text{depth in } T\}}$ gives static optimality against any fixed tree T .
6. The dynamic optimality conjecture — that splay trees are $O(1)$ -competitive against *any* BST algorithm — has been open since 1985.

7.2. The dictionary problem in the BST model

The dictionary problem asks you to support `insert(k, v)`, `find(k)`, and `delete(k)` on a dynamic set of key-value pairs. Hash tables solve this in expected $O(1)$ time, but they don't support ordered queries like predecessor. Comparison-based solutions — binary search trees — do.

A BST algorithm operates as follows. At the start of each operation you hold a pointer to the root. At each step you may:

- follow a child pointer (left or right),
- follow a parent pointer, or
- perform a rotation on the node you are currently pointing to.

The cost of an operation is the number of steps taken. This is the BST model.

Note

The BST model is restrictive: you cannot jump to an arbitrary node. Your pointer always starts at the root and must walk the tree one edge at a time. This is what makes the dynamic optimality conjecture hard — you have to show that splay trees, which also start at the root each time, cannot be beaten by much.

7.2.1. Rotations

A rotation at node x (whose parent is y) restructures the local neighbourhood: x becomes the parent of y , and the subtrees rearrange to preserve BST order. Concretely, if x is the left child of y with subtrees A (left) and B (right), and y has subtree C on its right:

- After `rotate(x)`: x becomes the root of this subtree, y becomes x 's right child, A stays as x 's left child, B becomes y 's left child, and C stays as y 's right child.

The operation is symmetric when x is a right child. Rotations are the *only* mechanism a BST algorithm has for restructuring the tree. A rotation is a local operation — it takes $O(1)$ time and preserves the BST ordering invariant.

7.3. Splay trees: definition

Splay trees are due to Sleator and Tarjan (1985). The data structure is a plain binary search tree. The twist: after every `find`, `insert`, or `delete`, the accessed node is *splayed* — brought to the root through a specific sequence of rotations. The original paper is titled “Self-Adjusting Binary Search Trees” — the tree adjusts itself after every operation without any external guidance.

`Find(x)`. Search for x as in any BST (walk from root to x), then splay x . After the splay, x is the new root.

`Insert(x)`. Insert x as in any BST (at a leaf), then splay x .

`Delete(x)`. Splay x (now it is the root). Remove x . If x had two children, splay the largest element in the left subtree and make it the new root — this element has no right child, so it cleanly becomes the root of both subtrees.

Warning

Splaying is *not* the same as rotating x to the root one step at a time. Naive rotate-to-root can leave the tree just as deep as it started. The magic is in the case structure of the splay operation.

Tip

The algorithms and data structure are simple — Sleator reportedly gives different answers when asked where the splay operation came from, and one answer was simply “the math works.” The implementation is easy; the proofs are where the difficulty lies. This is ideal for practitioners: you implement it once, and mathematicians prove the properties.

7.4. Why naive rotate-to-root fails

Consider a right-leaning path: nodes 1, 2, 3, 4, 5, 6 with 1 at the root and each node the right child of the one above. The tree has depth 6.

If you rotate 6 to the root one step at a time, the resulting tree has 6 at the root and 1 as its left child, with 2, 3, 4, 5 hanging off as a left-leaning chain. The depth is still n . No progress.

7.5. The splay operation

Let x be the node to splay, $y = \text{parent}(x)$, and $z = \text{parent}(y)$ (if it exists). Repeatedly apply one of three cases until x is the root.

Case 1 — Zig. y is the root. Rotate x . (This happens at most once, at the very end.)

Case 2 — Zig-zig. x and y are both left children (or both right children). Rotate y first, *then* rotate x .

Case 3 — Zig-zag. x and y go in opposite directions (one is a left child, the other a right child). Rotate x twice.

Why rotate y first in zig-zig?

In zig-zig, rotating y before x (rather than x twice) is the key difference from naive rotate-to-root. It compresses the path: the depth of every node on the access path roughly halves.

7.5.1. Splay on a path

Splay the deepest node of a right-leaning path of length n . After the splay, roughly half the original path hangs off the left spine and half off the right. The depth drops from n to about $n/2$. This is not a proof, but it is a strong hint that the operation is doing something structurally useful.

7.6. Properties of splay trees

Before proving anything, here is the remarkable catalogue of properties that splay trees satisfy. Each follows from the same potential-function framework with a different choice of weights.

7.6.1. Static optimality (Sleator-Tarjan, 1985)

For any sequence of find operations and any *fixed* binary search tree T , the time for the splay tree to serve the entire sequence is $O(\text{time for } T \text{ to serve it})$.

Think about what this means: you could use dynamic programming to compute the optimal fixed BST for a given access sequence (this is in CLRS as the optimal BST problem). Splay trees match that optimal tree's performance — *without knowing the sequence in advance*.

The optimal fixed tree for a given access distribution is related to Huffman coding; the resulting bound is proportional to the Shannon entropy of the access frequencies. Splay trees achieve this bound *without knowing the frequencies*.

7.6.2. Working set property

Let t_j be the number of distinct items accessed since the last access to the j -th item in the sequence. The total time for m operations is:

$$O\left(m + n \log n + \sum_{j=1}^m \log t_j\right) \quad (7.1)$$

If you repeatedly access the same 10 items, each access costs $O(\log 10)$ rather than $O(\log n)$. The “working set” stays near the root.

7.6.3. Static finger property (Sleator-Tarjan, 1985)

Fix any item f . The amortized cost of `find(i)` is $O(\log |f - i| + 1)$ — logarithmic in the *rank distance* to f .

7.6.4. Dynamic finger property (Cole et al., 2000)

The amortized cost of the i -th operation in a sequence $x_1, x_2, \dots, x_m$ is:

$$O(\log |x_i - x_{i-1}| + 1) \quad (7.2)$$

Your “finger” moves to the last-accessed item. The proof, due to Cole [–@cole2000dynamic] and Cole-Mishra-Schmidt-Siegel [–@cole2000dynamic2], is over 100 pages.

i Aside (mine, not the lecturer’s)

The dynamic finger property implies that splay trees are efficient for sequential or nearly-sequential access patterns — a common case in practice.

7.6.5. Unified bound (Iacono, 2001)

The *unified bound* tries to combine the working set and finger properties. The desired amortized cost of the j -th operation is:

$$O\left(\min_{i < j} \log(|x_i - x_j| + j - i)\right) \quad (7.3)$$

It is *not known* whether splay trees achieve this bound. Iacono gave a (non-BST) data structure that does. The closest BST result is by Derryberry and Sleator (2009), who achieved the unified bound plus an additive $O(m \log \log n)$ penalty.

7.7. The dynamic optimality conjecture

Let $\text{OPT}(\sigma)$ be the optimal cost of serving access sequence σ in the BST model, where the algorithm knows σ in advance and may rotate freely.

! Dynamic optimality conjecture (Sleator-Tarjan, 1985)

There exists a fixed constant C such that for all access sequences σ :

$$\text{cost}_{\text{splay}}(\sigma) \leq C \cdot \text{OPT}(\sigma) \quad (7.4)$$

The gap between static and dynamic optimality. Static optimality compares splay trees to a *fixed* tree — one that cannot rotate or adjust itself. Dynamic optimality compares splay trees to *any* BST algorithm, including one that rotates freely with full knowledge of the future. The static result is a theorem; the dynamic result is an open conjecture.

Known approximations. Any balanced BST gives $C = O(\log n)$ trivially (each operation costs at most $O(\log n)$, and OPT pays at least $\Omega(1)$). Tango trees (Demaine-Harmon-Iacono-Patrascu, 2004) achieved $C = O(\log \log n)$. Multi-splay trees (Wang-Derryberry-Sleator, 2006) also achieved $O(\log \log n)$ while simultaneously guaranteeing $O(\log n)$ amortized per operation. The conjecture asks for $C = O(1)$ and has been open for nearly 40 years.

7.8. Amortized analysis of splay

7.8.1. The potential function

Assign every item x a positive weight $w(x)$. Define:

- $s(x) = \sum_{z \in \text{subtree}(x)} w(z)$, the total weight in x 's subtree (including x),
- $r(x) = \log s(x)$, the rank of x ,
- $\Phi = \sum_{i=1}^n r(i)$, the potential of the entire tree.

i Note

The potential function looks arbitrary — “the math works,” as Sleator reportedly said when asked where it came from.

7.8.2. The splay lemma

The amortized cost of a single splay step satisfies:

Table 7.1.: Amortized cost of each splay case, where $r'(x)$ is the rank of x after the step.

Case	Amortized cost
Case 2 (zig-zig)	$\leq 3(r'(x) - r(x))$
Case 3 (zig-zag)	$\leq 3(r'(x) - r(x))$
Case 1 (zig)	$\leq 3(r'(x) - r(x)) + 1$

Summing over all steps in a full splay, the intermediate ranks telescope. The zig case contributes at most one extra unit (it happens at most once). The total amortized cost of $\text{splay}(x)$ is:

$$O\left(1 + \log \frac{W}{s(x)}\right) \quad (7.5)$$

where $W = \sum_i w(i)$ is the total weight.

7.8.3. Proof sketch for zig-zig (Case 2, left-left)

The tree has $z \rightarrow y \rightarrow x$ with x the left child of y and y the left child of z . Subtrees A, B hang off x ; C hangs off y ; D hangs off z .

Actual cost: 2 (two rotations).

Change in potential: Only x, y, z change rank (the subtrees A, B, C, D keep the same descendants). After the two rotations, x sits at the top and z is a descendant, so $r'(x) = r(z)$ (same total subtree). Also $r(y) \geq r(x)$ (parent has at least as large a subtree) and $r'(y) \leq r'(x)$.

The change in potential is:

$$\Delta\Phi = r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z)$$

Since $r'(x) = r(z)$, two terms cancel. Since $r'(y) \leq r'(x)$ and $r(y) \geq r(x)$, this simplifies to:

$$\Delta\Phi \leq r'(x) + r'(z) - 2r(x) \tag{7.6}$$

Now bound $r'(z)$. The key step uses the AM-GM inequality. After the rotations, $s(x) + s'(z)$ counts all nodes except y , so $s(x) + s'(z) \leq s'(x)$. Therefore:

$$\frac{r(x) + r'(z)}{2} = \log \sqrt{s(x) \cdot s'(z)} \leq \log \frac{s(x) + s'(z)}{2} \leq \log \frac{s'(x)}{2} = r'(x) - 1$$

Rearranging: $r'(z) \leq 2r'(x) - r(x) - 2$. Substituting into Equation 7.6:

$$\Delta\Phi \leq 3(r'(x) - r(x)) - 2$$

The amortized cost is $2 + \Delta\Phi \leq 3(r'(x) - r(x))$, as claimed.

7.9. Corollary: $O(\log n)$ amortized

Set $w(x) = 1$ for all x . Then $W = n$ and $s(x) \geq 1$, so the amortized cost of $\text{splay}(x)$ is $O(1 + \log n)$ by Equation 7.5.

! Important

Splay trees match the amortized performance of any balanced BST (red-black trees, AVL trees) with a simpler implementation and no balance metadata.

7.10. Static optimality via weights

This elegant proof is due to David Epstein (published on his LiveJournal, not in a formal paper).

Fix a tree T that places item i at depth ℓ_i . If $\text{find}(i)$ is called m_i times, the cost of serving the sequence with T is $O(m + \sum_i m_i \ell_i)$.

Set the weight of item i to $w(i) = 1/3^{\ell_i}$.

Total weight W . At depth ℓ in T , there are at most 2^ℓ nodes. So:

$$W = \sum_{\ell} \frac{|\text{nodes at depth } \ell|}{3^\ell} \leq \sum_{\ell=0}^{\infty} \left(\frac{2}{3}\right)^\ell = 3$$

W is a constant.

Amortized cost of $\text{find}(i)$. By Equation 7.5, it is $O(1 + \log(W/s(i)))$. Since $s(i) \geq w(i) = 1/3^{\ell_i}$ and $W \leq 3$:

$$\text{amortized cost of find}(i) = O(1 + \ell_i)$$

Potential bookkeeping. The potential can be negative (ranks involve $\log(1/3^{\ell_i})$, which is negative). The actual total cost is:

$$\text{actual} = \text{amortized total} + \Phi_{\text{initial}} - \Phi_{\text{final}}$$

Each rank lies in $[-O(n), 0]$, so $|\Phi_{\text{initial}} - \Phi_{\text{final}}| \leq O(n^2)$.

Conclusion. The splay tree serves the sequence in time:

$$O\left(n^2 + m + \sum_{i=1}^n m_i \ell_i\right) \quad (7.7)$$

The n^2 term is a startup cost; once m is large enough, the splay tree matches the fixed tree T up to constants.

Tip

The power of this framework is that you prove the splay lemma *once* and then derive different properties by plugging in different weight functions. The data structure itself never changes.

7.11. Cheat sheet

Splay trees — cheat sheet

Data structure. A binary search tree. After every operation, the accessed node is splayed to the root.

Splay operation. Repeatedly apply:

- Zig (parent is root): rotate x .
- Zig-zig (same direction): rotate parent, then rotate x .
- Zig-zag (opposite direction): rotate x twice.

Potential function. $\Phi = \sum_i \log s(i)$, where $s(i) = \sum_{z \in \text{subtree}(i)} w(z)$.

Amortized cost of splay. $O(1 + \log(W/s(x)))$.

Key results by weight choice:

Property	Weight $w(i)$	Amortized cost per op
$O(\log n)$	1	$O(\log n)$
Static optimality	$1/3^{\ell_i}$ (depth in fixed tree T)	$O(1 + \ell_i)$
Working set	(frequency-based)	$O(\log t_j)$

Dynamic optimality conjecture. $\text{cost}_{\text{splay}}(\sigma) \leq O(\text{OPT}(\sigma))$. Open since 1985. Best known: $O(\log \log n)$ -competitive (tango trees, 2004).

7.12. FAQ — questions students actually asked

Q: Can't you just bound $r'(z)$ by $r'(x)$ directly, since z is a descendant of x after the zig-zig?

Yes, $r'(z) \leq r'(x)$. That gives $\Delta\Phi \leq 2(r'(x) - r(x))$, and the amortized cost is $2 + 2(r'(x) - r(x))$. The problem: after telescoping, you cannot cancel the constant 2 per step. Each splay has $\Theta(\text{depth})$ steps, so you end up with $O(\text{depth})$ amortized — no better than the actual cost. The AM-GM trick produces the crucial -2 that absorbs the constant.

Q: In case 1 (zig), where does the extra +1 come from?

Case 1 involves a single rotation. You don't get the -2 from AM-GM because there is no grandparent z . So the constant from the actual cost isn't fully absorbed. Case 1 happens at most once per splay (only when the parent is the root), so the +1 contributes $O(1)$ total.

Q: The potential can be negative — doesn't that break the amortized argument?

The standard amortized argument requires $\Phi_{\text{final}} \geq \Phi_{\text{initial}}$ so that actual cost $\leq$ amortized cost. When Φ can be negative, you instead bound $\Phi_{\text{initial}} - \Phi_{\text{final}}$ explicitly (here, $O(n^2)$) and add that to the amortized total. The result is a one-time startup cost, not a per-operation cost.

Q: The static finger property says cost is $O(\log |f - i| + 1)$ for *any* fixed f . What if $f = i$?

Then the cost is $O(\log 1) = O(1)$. The +1 ensures the bound is always at least constant, since every access takes at least $\Omega(1)$ time.

Q: Is there a connection between the static optimality bound and compression?

Yes. The optimal fixed BST for an access sequence with frequencies $p_1, \dots, p_n$ achieves $\sum_i p_i \log(1/p_i)$, which is the Shannon entropy — exactly the information-theoretic lower bound for a prefix-free code. Splay trees achieving this bound means they adapt to the “compressibility” of the access sequence.

7.13. Further reading

1. Sleator and Tarjan (1985). “Self-Adjusting Binary Search Trees.” *JACM* 32(3). The original paper: splay trees, static optimality, working set, static finger.
2. Cole (2000). “On the Dynamic Finger Conjecture for Splay Trees. Part II: The Proof.” *STOC 2000*. The second half of the 100+ page proof of the dynamic finger property.
3. Demaine, Harmon, Iacono, Patrascu (2004). “Dynamic Optimality — Almost.” *FOCS 2004*. Tango trees: $O(\log \log n)$ -competitive BST.
4. Iacono (2001). “Alternatives to Splay Trees with $O(\log n)$ Worst-Case Access Times.” *SODA 2001*. Introduces the unified bound and a non-BST data structure achieving it.
5. Derryberry and Sleator (2009). A BST achieving the unified bound plus $O(m \log \log n)$.

7.14. Try it yourself

1. Zig-zag analysis. Prove that the amortized cost of a zig-zag step (Case 3) is at most $3(r'(x) - r(x))$. Follow the same template as the zig-zig proof in Section 7.8.3.
2. Working set property. Define $w(i)$ appropriately and use Equation 7.5 to prove Equation 7.1. (*Hint*: update weights dynamically — set the weight of the most recently accessed item to be larger.)
3. Splay simulation. Implement a splay tree in Python. Splay the deepest node of a right-leaning path of 15 nodes. Print the tree (in-order with depths) before and after the splay to verify that the depth roughly halves.

```
class Node:
    def __init__(self, key, left=None, right=None, parent=None):
        self.key = key
        self.left = left
        self.right = right
        self.parent = parent

def rotate(x):
    """Rotate x with its parent."""
    y = x.parent
    if y is None:
        return
    g = y.parent
    if y.left is x:
        y.left = x.right
        if x.right: x.right.parent = y
        x.right = y
    else:
        y.right = x.left
        if x.left: x.left.parent = y
        x.left = y
    x.parent = g
    y.parent = x
    if g:
        if g.left is y: g.left = x
        else: g.right = x

def splay(x):
    """Splay x to the root."""
    while x.parent is not None:
        y = x.parent
        z = y.parent
        if z is None:
            # zig
            rotate(x)
        elif (z.left is y) == (y.left is x): # zig-zig
            rotate(y); rotate(x)
        else:
            # zig-zag
            rotate(x); rotate(x)
    return x
```

4. Tango tree gap. Explain why balanced BSTs give $C = O(\log n)$ in Equation 7.4. Specifically, show that for any sequence σ of m accesses, a balanced BST costs $O(m \log n)$, and $\text{OPT}(\sigma) \geq m$ (each access costs at least 1).

7. Splay Trees

5. Lower bound on OPT. Prove that for any access sequence σ of m elements from a universe of size n , $\text{OPT}(\sigma) \geq m \cdot H(p_1, \dots, p_n) / \log 3$, where p_i is the fraction of accesses to item i and H is Shannon entropy. (*Hint*: use an information-theoretic argument — each comparison reveals at most 1 bit.)

Part II.

Online Algorithms

8. Online Algorithms and Paging

COMPSCI 224 Lecture 8 — Fall 2014

Online algorithms make irrevocable decisions without knowing future inputs. Competitive analysis — introduced by Sleator and Tarjan in the same year as splay trees — measures an online algorithm's cost against an omniscient adversary. This lecture develops the framework through two toy problems (ski rental, doubling search), then applies it to the list update problem (where move-to-front is dynamically optimal) and paging (where LRU is k -competitive and no deterministic algorithm can do better). Randomization against an oblivious adversary breaks the deterministic barrier, bringing the competitive ratio down to $O(\log k)$.

8.1. Why this lecture matters

Every caching layer, every memory hierarchy, and every load balancer faces the same dilemma: what to keep and what to discard when you don't know what's coming next. Competitive analysis gives you a way to reason about this rigorously — and the results are sharp enough to explain *why* LRU is the default eviction policy in practice and *when* you should expect it to fail.

The 30-second story

1. An online algorithm is r -competitive if its cost on *any* input is at most r times the cost of the best offline algorithm (plus an additive constant).
2. Ski rental (buy vs. rent) is 2-competitive; doubling search (find the pizza) is 9-competitive.
3. For the list update problem, move-to-front is 2-competitive against *any* algorithm — it is dynamically optimal.
4. For paging with cache size k , LRU and FIFO are k -competitive — and no deterministic algorithm can beat k .
5. Randomization helps: the marking algorithm is $2H_k \approx 2 \ln k$ competitive against oblivious adversaries, and this is tight up to constants.

8.2. Online algorithms and competitive analysis

An online algorithm receives a sequence of requests $\sigma = \sigma_1, \sigma_2, \dots, \sigma_m$ one at a time. Upon receiving σ_i , it must make an irrevocable decision before seeing σ_{i+1} .

Let OPT denote an algorithm that knows the entire sequence σ in advance and makes the best possible decisions.

Definition: competitive ratio

An online algorithm A is r -competitive if for all request sequences σ :

$$\text{cost}(A, \sigma) \leq r \cdot \text{cost}(\text{OPT}, \sigma) + c \quad (8.1)$$

for some constant c independent of σ .

This framework was introduced by Sleator and Tarjan (1985) — the same year as splay trees, in a different paper.

8.3. Toy example 1: ski rental

You're at a ski resort. Each day your friends decide whether to stay another day or go home. You must have skis: renting costs \$1/day; buying costs \$ B .

OPT knows the number of days m . If $m > B$, buy on day 1 (cost B). If $m \leq B$, rent every day (cost m).

Online strategy. Rent for the first B days. If there is a $(B + 1)$ -st day, buy.

Analysis.

- If $m \leq B$: you pay m , OPT pays m . Ratio = 1.
- If $m > B$: you pay B (rent) + B (buy) = $2B$. OPT pays B . Ratio = 2.

Tip

The ski rental problem distills the rent-or-buy dilemma that appears in many systems: should you cache this result, compile this query plan, or provision this server?

8.4. Toy example 2: doubling search

A hallway has rooms numbered $-n/2$ to $n/2$. You stand at room 0. One room has free pizza, but you don't know which. Walking one room costs 1 unit of time. OPT walks straight to the target (cost = $|t|$ for target room t).

Strategy. Zigzag with exponentially increasing reach: visit rooms 1, -2 , 4, -8 , 16, ...

Suppose the pizza is in room $t > 0$. Let 2^m be the first power of 2 that is at least t , so $2^m \leq 2t$. The total distance walked is at most:

$$t + 2(1 + 2 + 4 + \cdots + 2^m) \leq t + 4 \cdot 2^m \leq t + 8t = 9t \quad (8.2)$$

So the algorithm is 9-competitive.

Aside (mine, not the lecturer's)

This is an instance of the linear search problem (also called the cow-path problem). The competitive ratio can be improved to 9 with base-2 doubling. Using a different base b , the ratio is $(b + 1)^2 / (b - 1)$, minimized at $b = (\sqrt{5} + 1)/2 \approx 1.618$ (the golden ratio), giving a ratio of about $4 + 2\sqrt{5} \approx 8.47$.

8.5. The list update problem

There are n items in a linked list. Three operations:

- Access(x): traverse the list from the front to find x . Cost = position of x in the list.
- Insert(x): append x to the end.
- Delete(x): remove x after accessing it.

After accessing an item, you may move it *anywhere closer to the front* for free. These are called free exchanges. You may also swap any item with its predecessor at cost 1 (paid exchange).

8.5.1. Heuristics

Three classical approaches:

Heuristic	Rule
Move-to-front (MF)	Always move the accessed item to the front
Transpose	Swap the accessed item with its immediate predecessor
Frequency count	Keep items sorted by access frequency (most frequent first)

Bentley and McGeoch (1985) proved that MF costs at most twice what frequency count costs (under mild initial conditions) and found experimentally that transpose performed poorly.

Warning

Move-to-front's aggressive reordering looks wasteful, but it is the dynamically optimal strategy. Transpose — the “cautious” heuristic — is *not* competitive.

8.5.2. Move-to-front is dynamically optimal

Unlike splay trees (where dynamic optimality is a conjecture), move-to-front's dynamic optimality is a *theorem*.

Theorem (Sleator-Tarjan, 1985)

Let A be any algorithm (online or offline, with free and paid exchanges). If A and MF start with the same list ordering, then for every access sequence S of length m :

$$\text{cost}(\text{MF}, S) \leq 2 \cdot \text{cost}(A, S) + X_{\text{paid}}(A) - X_{\text{free}}(A) - m \quad (8.3)$$

where $X_{\text{paid}}(A)$ and $X_{\text{free}}(A)$ are the numbers of paid and free exchanges A makes.

8.5.3. Proof via inversions

Potential function. Φ = number of inversions between MF's list and A 's list. A pair (i, j) is an inversion if i precedes j in MF's list but j precedes i in A 's list.

Initial potential: $\Phi_0 = 0$ (both lists start in the same order). Final potential: $\Phi_{\text{final}} \geq 0$ (inversions are always non-negative). So actual cost $\leq$ amortized cost.

Amortized cost of $\text{Access}(x)$. Suppose x is the k -th item in MF's list and the i -th item in A 's list. Let t = number of items that precede x in MF but follow x in A (these are inversions involving x).

- Actual cost: k (walk to position k).
- Inversions undone by moving x to the front: t .
- Inversions created: at most $k - t - 1$ (the $k - 1$ items that were ahead of x in MF; t of them were already inverted).
- Change in potential: $\Delta\Phi = (k - t - 1) - t = k - 2t - 1$.
- Amortized cost: $k + k - 2t - 1 = 2(k - t) - 1$.

The items counted by $k - t - 1$ precede x in *both* lists. There are only $i - 1$ positions before x in A 's list, so $k - t - 1 \leq i - 1$, giving $k - t \leq i$.

$$\text{amortized cost} = 2(k - t) - 1 \leq 2i - 1 \quad (8.4)$$

Summing over all operations gives $\text{cost}(\text{MF}) \leq 2 \cdot \text{cost}(A) - m$.

Effect of exchanges. Each free exchange by A reduces Φ by at most 1 (potentially removes an inversion). Each paid exchange by A increases Φ by at most 1 (potentially creates an inversion). Adding these contributions gives Equation 8.3.

8.6. The paging problem

You have a cache of size k (fast memory) and an unbounded disk (slow memory). Pages are accessed one at a time:

- If the requested page is in cache: cache hit, cost 0.
- If the requested page is not in cache: page fault, cost 1. Bring the page into cache; if cache is full, evict one page first.

The goal is to minimise the number of page faults.

8.6.1. Connection to list update

Paging is a special case of list update with cost function $f(i) = 0$ if $i \leq k$ (page is in cache) and $f(i) = 1$ otherwise. After a page fault, the accessed page must be moved into the first k positions.

8.6.2. Eviction policies

Policy	Evict...
LRU (Least Recently Used)	The page accessed longest ago
FIFO (First In, First Out)	The page brought into cache longest ago
LFU (Least Frequently Used)	The page with fewest total accesses
Farthest in Future (Belady, 1966)	The page whose next access is farthest away

Farthest-in-future is optimal (Belady, 1966), but it requires knowing the future — it characterises OPT. LRU corresponds to move-to-front in the list update model.

Caution

LFU is *not* frequency count in disguise. In frequency count, you never need to bring a page into cache (all items are already in the list). In paging, bringing a cold page into cache forces an eviction that may displace a hot page. LFU can have an *unbounded* competitive ratio.

8.7. LRU is k -competitive

8.7.1. 1-bit LRU

A cleaner algorithm for the proof: 1-bit LRU.

1. Initially, all pages are unmarked.
2. On every access, mark the accessed page.
3. On a page fault, evict any unmarked page (choice is arbitrary). If all pages are marked, clear all marks, then evict an unmarked page.

LRU is a special case of 1-bit LRU: the least recently used page is always unmarked (it belongs to a previous phase), so LRU's eviction choice is always valid within the 1-bit LRU framework.

8.7.2. Proof

Break the access sequence into phases: a phase starts when all marks are cleared and ends just before the next clearing.

- Within a phase, 1-bit LRU has at most k page faults. Each fault brings in a new page and marks it. After k faults, all k pages in cache are marked. The phase ends when a $(k + 1)$ -st distinct page is requested.
- Within a phase, OPT has at least 1 fault. The phase involves $k + 1$ distinct pages, but OPT's cache holds only k . By the pigeonhole principle, OPT faults at least once.

Ratio per phase: at most k / at least $1 = k$.

Note

The 1-bit LRU proof works for FIFO too — FIFO also evicts a page from the previous phase. Both are k -competitive.

8.8. Lower bound: no deterministic algorithm beats k

! Theorem (Sleator-Tarjan, 1985)

No deterministic online paging algorithm has a competitive ratio less than k .

Proof. The adversary maintains a universe of $k + 1$ pages. At each step, request the one page not in the algorithm's cache. The algorithm faults on *every* request.

Meanwhile, OPT evicts the page whose next access is farthest in the future. Between consecutive OPT faults, at least k distinct pages must be requested (OPT's cache covers k pages, so it takes k requests before a page outside its cache is needed). OPT faults at most once every k steps.

Ratio: algorithm faults m times; OPT faults at least m/k times. Competitive ratio $\geq k$.

8.9. Circumventing the lower bound

Three escape routes:

8.9.1. 1. Cache associativity

Real caches are often set-associative: each page maps to a small set of cache lines. The competitive ratio depends on the associativity (e.g., 4-way), not the total cache size — a much smaller number.

8.9.2. 2. Resource augmentation (Sleator-Tarjan, 1985)

Give the online algorithm k cache slots but give OPT only $k' < k$ slots. Sleator and Tarjan showed that LRU and FIFO achieve a competitive ratio of:

$$\frac{k}{k - k' + 1} \quad (8.5)$$

If $k' = k/2$, the ratio is about 2. A modest resource advantage makes LRU nearly optimal.

💡 Tip

Resource augmentation is the most practically relevant escape. In real systems, your cache is usually larger than the “adversary's” working set, so LRU performs close to optimal.

8.9.3. 3. Randomization

A randomized eviction policy can defeat the deterministic lower bound — but the answer depends on the adversary's power.

Adversary types:

Adversary	Knows...	Deterministic LB applies?
Omniscient	All future random bits	Yes (randomness is useless)

Adversary	Knows...	Deterministic LB applies?
Adaptive	Past random bits, adapts future requests	Yes (knows cache state)
Oblivious	Nothing; fixes σ before any coins are flipped	No

The deterministic lower bound relies on the adversary seeing the cache state. An oblivious adversary cannot adapt, so randomization helps.

8.10. The marking algorithm

! Theorem (Fiat, Karp, Luby, McGeoch, Sleator, Young, 1991)

The marking algorithm — 1-bit LRU where the unmarked page to evict is chosen uniformly at random — is $2H_k$ -competitive against oblivious adversaries, where $H_k = 1 + 1/2 + \dots + 1/k = \ln k + O(1)$ is the k -th harmonic number.

No randomised algorithm can achieve a competitive ratio better than $\Omega(\log k)$ against oblivious adversaries.

Definition. A randomised algorithm A is r -competitive if for all sequences S :

$$\mathbb{E}[\text{cost}(A, S)] \leq r \cdot \text{cost}(\text{OPT}, S) + c \quad (8.6)$$

8.10.1. Proof sketch

Break the sequence into phases (as with 1-bit LRU). At a given point in time, a page is:

- Clean: not accessed in the previous phase *and* not yet accessed in this phase.
- Stale: accessed in the previous phase but not yet in this phase.

Claim. If a phase has ℓ clean page accesses:

1. OPT pays at least $\ell/2$.
2. The marking algorithm pays at most $\ell \cdot H_k$ in expectation.

The factor of 2 between these gives the $2H_k$ competitive ratio.

Why OPT pays at least $\ell/2$: clean pages were not in *anyone's* cache at the start of the phase (neither OPT's nor ours). Each clean access forces OPT to fault — but OPT might have preloaded some of them in the previous phase. A careful two-phase averaging argument shows OPT faults at least $\ell/2$ times across consecutive phases.

Why the marking algorithm pays at most $\ell \cdot H_k$: consider the j -th page fault in a phase. At that point, there are $k - (j - 1)$ unmarked pages in cache. The probability that the evicted page is the one the adversary will request next (causing another fault soon) is at most $1/(k - j + 1)$. Summing these probabilities over the phase gives the harmonic number.

8.11. Cheat sheet

! Online algorithms and paging — cheat sheet

Competitive analysis. A is r -competitive if $\text{cost}(A, \sigma) \leq r \cdot \text{OPT}(\sigma) + c$ for all σ .

Ski rental. Rent B days, then buy. Ratio: 2.

Doubling search. Zigzag with exponentially increasing reach. Ratio: 9.

List update. Move-to-front is 2-competitive against any algorithm (dynamically optimal). Proof: potential = inversions between MF and A .

Paging (deterministic):

Algorithm	Competitive ratio
LRU	k (tight)
FIFO	k
LFU	unbounded
Any deterministic	$\geq k$

Paging (randomised, oblivious adversary):

Algorithm	Competitive ratio
Marking algorithm	$2H_k \approx 2 \ln k$
Any randomised	$\geq \Omega(\log k)$

Escape routes: resource augmentation (give OPT less memory), randomisation (oblivious adversary), cache associativity.

OPT for paging: farthest-in-future (Belady, 1966).

8.12. FAQ — questions students actually asked

Q: Is least frequently used the same as frequency count from the list update problem?

No. In list update, all items are always in the list — you just reorder them. In paging, bringing a cold page into cache forces an eviction. LFU can be forced into an unbounded competitive ratio: imagine $k - 1$ hot pages and one cold page that keeps cycling in and out, evicting a hot page each time.

Q: You said 1-bit LRU has *exactly* k faults per phase. But what if a page was already in cache from the previous phase and gets accessed without faulting?

Good catch — it's *at most* k faults. If a page survives from the previous phase, accessing it marks it without a fault. The phase still ends after $k + 1$ distinct pages, but some of those may already be in cache.

Q: For the lower bound, why does OPT fault at most once every k steps?

OPT evicts the page whose next request is farthest in the future. After evicting, the remaining k pages in OPT's cache cover the next $k - 1$ requests (at worst, each request hits a different cached page). The k -th request might be the evicted page, causing the next fault.

Q: Why doesn't the deterministic lower bound apply to adaptive adversaries?

It *does*. An adaptive adversary knows your past random bits and therefore knows your current cache contents. It can always request the missing page, just like in the deterministic case. Randomisation only helps against oblivious adversaries who commit to the sequence before seeing any randomness.

Q: In the move-to-front proof, why is $k - t - 1 \leq i - 1$?

The $k - t - 1$ items precede x in *both* MF's list and A 's list. But A 's list has only $i - 1$ positions before x . So $k - t - 1 \leq i - 1$, which gives $k - t \leq i$. This is where the factor of 2 comes from.

8.13. Further reading

1. Sleator and Tarjan (1985). "Amortized Efficiency of List Update and Paging Rules." *Communications of the ACM* 28(2). The foundational paper: competitive analysis, list update, paging, k -competitiveness of LRU.
2. Fiat, Karp, Luby, McGeoch, Sleator, Young (1991). "Competitive Paging Algorithms." *Journal of Algorithms* 12(4). The marking algorithm and the $\Omega(\log k)$ lower bound for randomised algorithms.
3. Belady (1966). "A Study of Replacement Algorithms for a Virtual-Storage Computer." *IBM Systems Journal* 5(2). Proves that farthest-in-future is optimal for paging.
4. Borodin and El-Yaniv (1998). *Online Computation and Competitive Analysis*. Cambridge University Press. The standard textbook on competitive analysis.
5. Bentley and McGeoch (1985). "Amortized Analyses of Self-Organizing Sequential Search Heuristics." *Communications of the ACM*. The pre-competitive-analysis study of list update heuristics.

8.14. Try it yourself

1. Ski rental variant. Suppose renting costs \$1/day and buying costs \$B, but the skis wear out after W days of use (whether rented or owned). What is the optimal deterministic competitive ratio?
2. Transpose is not competitive. Construct an access sequence on 3 items where the transpose heuristic costs $\Omega(m)$ while OPT costs $O(1)$ per operation. Conclude that transpose has an unbounded competitive ratio.
3. LRU simulation. Implement LRU and farthest-in-future for a cache of size $k = 3$ with pages $\{1, 2, 3, 4\}$. Run both on the sequence 1, 2, 3, 4, 1, 2, 3, 4, ... (repeated 10 times). Count page faults for each.

```
from collections import OrderedDict

def lru_faults(sequence, k):
    """Count page faults for LRU with cache size k."""
    cache = OrderedDict()
    faults = 0
    for page in sequence:
        if page in cache:
            cache.move_to_end(page) # mark as recently used
        else:
            faults += 1
            if len(cache) >= k:
                cache.popitem(last=False) # evict least recently used
            cache[page] = True
    return faults

def fif_faults(sequence, k):
    """Count page faults for farthest-in-future with cache size k."""
    cache = set()
    faults = 0
    for idx, page in enumerate(sequence):
        if page not in cache:
```

```

    faults += 1
    if len(cache) >= k:
        # evict the page whose next use is farthest away
        def next_use(p):
            for j in range(idx + 1, len(sequence)):
                if sequence[j] == p:
                    return j
            return float('inf')
        victim = max(cache, key=next_use)
        cache.remove(victim)
        cache.add(page)
    return faults

seq = [1, 2, 3, 4] * 10
print(f"LRU faults: {lru_faults(seq, 3)}")
print(f"OPT faults: {fif_faults(seq, 3)}")

```

4. Resource augmentation. Prove that LRU with cache size k is $k/(k - k' + 1)$ -competitive against OPT with cache size $k' \leq k$. (*Hint*: modify the phase argument — OPT with a smaller cache faults more often.)
5. Marking algorithm analysis. Prove that in a single phase with ℓ clean accesses, the expected number of faults by the marking algorithm is at most $\ell \cdot H_k$. (*Hint*: for the j -th fault, there are $k - j + 1$ unmarked pages. What is the probability that the evicted page is requested before the phase ends?)

9. Randomized Paging and the Online Primal-Dual Framework

COMPSCI 224 Lecture 9 — Fall 2014

The Mark algorithm achieves a $2H_K$ -competitive ratio for randomized paging by exploiting the symmetry of random eviction within phases. No randomized algorithm can beat $\Omega(\log K)$, as a coupon-collector argument shows. The online primal-dual framework—built on LP duality and approximate complementary slackness—provides a systematic recipe for designing and analysing competitive online algorithms, demonstrated here on ski rental.

9.1. Why this lecture matters

Online algorithms must commit to irrevocable decisions without seeing the future. The Mark algorithm shows that even a single bit of randomness—choosing *which* page to evict uniformly at random—drops the competitive ratio from K (deterministic) to $O(\log K)$. The second half of the lecture introduces the online primal-dual framework, a general-purpose machine that converts LP relaxations into competitive online algorithms. Master this framework and you can design algorithms for ski rental, set cover, weighted paging, and the K -server problem from the same template.

30-second story

1. The Mark algorithm evicts a uniformly random unmarked page, achieving competitive ratio $2H_K \approx 2 \ln K$.
2. A coupon-collector lower bound proves no randomized algorithm beats $\Omega(\log K)$.
3. The K -server problem generalises paging; polylogarithmic competitive ratios remain an active research frontier.
4. LP duality gives a mechanical way to certify lower bounds on OPT.
5. Approximate complementary slackness converts feasible primal/dual pairs into competitive-ratio proofs.
6. Applied to ski rental, the primal-dual method recovers the 2-competitive deterministic algorithm—and sets the stage for $e/(e-1)$ via randomization.

9.2. Analysis of the Mark algorithm

The Mark algorithm (Fiat et al., *Journal of Algorithms*, 1991) is a randomised implementation of one-bit LRU: when a page must be brought into cache, evict a uniformly random *unmarked* page, then mark the newly loaded page. If all pages are marked, unmark everything and continue.

9.2.1. Phase decomposition

Divide time into phases. A phase starts when all pages are unmarked and ends just before the next unmark-all event. Within a single phase, exactly K distinct pages are requested (since each request marks a page, and the phase ends when all K slots are marked and a request falls outside the cache).

Key definitions

- Clean page: not requested in the previous phase *and* not yet requested in the current phase.
- Stale page: requested last phase but not yet this phase.

Let L_i = number of clean pages requested in phase i , and $S_i = K - L_i$ = number of stale pages requested. Every phase requests exactly K distinct pages, partitioned into L_i clean and S_i stale.

9.2.2. Comparing Mark to OPT

Define D_i = number of pages in OPT's cache at the start of phase i that are *not* in Mark's cache. Define D'_i = the same quantity at the end of phase i (so $D'_i = D_{i+1}$).

Claim 1. OPT pays at least $\max(L_i - D_i, D'_i)$ cache misses in phase i .

- *Why $\geq L_i - D_i$:* Mark doesn't hold any clean page at the start of the phase. OPT can hold at most D_i clean pages (the pages where it differs from Mark). The remaining $L_i - D_i$ clean requests force OPT to fault.
- *Why $\geq D'_i$:* At the end of a phase, Mark holds exactly the K pages requested in that phase. Each of OPT's D'_i differing pages corresponds to an eviction OPT made during the phase.

Taking the average of the two lower bounds:

$$\text{OPT cost in phase } i \geq \frac{L_i - D_i + D'_i}{2} \geq \frac{L_i}{2} + \frac{D'_i - D_i}{2} \quad (9.1)$$

Summing over all phases and noting that the telescoping D terms collapse (both algorithms start with the same cache):

$$\text{OPT total} \geq \frac{1}{2} \sum_i L_i \quad (9.2)$$

9.2.3. Expected cost of Mark on stale requests

Fix a point in a phase where s stale requests and c clean requests have already been served. Mark's cache looks like:

The remaining $K - s - c$ slots hold a *uniformly random* subset of the stale pages not yet requested. The next stale request faults if it is *not* among these survivors. Its fault probability is:

$$\Pr[\text{fault}] = \frac{c}{K - s} \leq \frac{L_i}{K - s} \quad (9.3)$$

Summing over all stale requests (s ranges from 0 to $K - L_i - 1$):

$$\mathbb{E}[\text{stale cost in phase } i] \leq L_i \sum_{s=0}^{K-L_i-1} \frac{1}{K-s} = L_i(H_K - H_{L_i}) \leq L_i(H_K - 1)$$

Adding the L_i deterministic faults from clean requests:

$$\mathbb{E}[\text{Mark cost in phase } i] = L_i + L_i(H_K - 1) = L_i \cdot H_K \quad (9.4)$$

Combining Equation 9.2 and Equation 9.4 gives a competitive ratio of $2H_K$.

! Important

The Mark algorithm is $2H_K$ -competitive, where $H_K = 1 + \frac{1}{2} + \dots + \frac{1}{K} = \ln K + O(1)$.

9.3. Lower bound: no randomised algorithm beats $\Omega(\log K)$

Theorem (Fiat et al.). Any randomised paging algorithm has competitive ratio $\Omega(\log K)$.

Proof sketch. Let the universe contain $K + 1$ pages. Generate a request sequence of length M uniformly at random.

- Any algorithm holds K of $K + 1$ pages, so its expected cost per request is $\frac{1}{K+1}$ —roughly $\frac{M}{K}$ total faults over the sequence.
- *OPT* uses the Bélády (furthest-in-future) policy. After each eviction, the evicted page won't reappear until every other page has been seen—a coupon-collector wait of $\Theta(K \log K)$ requests.
- *OPT* therefore faults only once per $\Theta(K \log K)$ requests, paying $\Theta\left(\frac{M}{K \log K}\right)$ total.

The ratio is $\frac{M/K}{M/(K \log K)} = \log K$.

💡 Intuition

OPT exploits the long tail of the coupon-collector distribution: the page it evicts is, in expectation, the last one to reappear. Any online algorithm—even randomised—cannot predict which page is farthest in the future.

9.4. The K -server problem and open questions

Paging is the special case of the K -server problem where all distances are 1. In the general problem, K servers live in a metric space, and serving a request means moving a server to the requested point at cost equal to the distance moved.

Result	Ratio	Reference
Work function algorithm (deterministic)	$2K - 1$	Koutsoupias & Papadimitriou, JACM 1995
Deterministic lower bound	K	—
Randomised (metric with n points)	$O(\log^2 K \cdot \log^3 n \cdot \log \log n)$	Bansal, Buchbinder, Madry, Naor, 2011
Conjectured randomised	$O(\log K)$	Open

i Aside (mine, not the lecturer's)

The randomised K -server conjecture—that $O(\log K)$ is achievable in any metric—remains one of the flagship open problems in online algorithms as of 2024.

Weighted paging (pages have non-uniform fetch costs) is a special case of K -server. Bansal, Buchbinder, and Naor (FOCS 2007) achieved $O(\log K)$ -competitive for weighted paging using the primal-dual framework.

9.5. Linear programming recap

A linear program (LP) in standard covering form:

$$\min c^T x \quad \text{s.t.} \quad Ax \geq b, \quad x \geq 0 \quad (9.5)$$

A covering LP requires all entries of A , b , c to be non-negative.

9.5.1. LP duality

The dual LP provides the tightest lower bound on the primal optimum obtainable by taking non-negative linear combinations of the primal constraints:

$$\max b^T y \quad \text{s.t.} \quad A^T y \leq c, \quad y \geq 0 \quad (9.6)$$

The dual of a covering LP is a packing LP (all entries non-negative, constraints are $\leq$).

! Weak duality

For any feasible primal x and feasible dual y :

$$b^T y \leq \text{OPT}_D \leq \text{OPT}_P \leq c^T x$$

Strong duality (not needed today) says $\text{OPT}_P = \text{OPT}_D$ when both are finite.

9.6. The online primal-dual framework

The framework (Buchbinder & Naor, 2005/2009) translates online covering problems into competitive algorithms:

1. Write the online problem as a covering LP whose constraints arrive one at a time.
2. Maintain feasible primal and dual solutions, with primal variables monotonically non-decreasing.
3. Use approximate complementary slackness to bound the competitive ratio.

9.6.1. Approximate complementary slackness

Lemma. Let x, y be feasible for the primal and dual respectively. Suppose there exist $\alpha, \beta \geq 1$ such that:

- If $x_j > 0$ then $\frac{c_j}{\alpha} \leq (A^T y)_j \leq c_j$ (the j -th dual constraint is nearly tight).
- If $y_i > 0$ then $b_i \leq (Ax)_i \leq \beta \cdot b_i$ (the i -th primal constraint is nearly tight).

Then $c^T x \leq \alpha\beta \cdot b^T y \leq \alpha\beta \cdot \text{OPT}$.

💡 Tip

The ratio $\alpha\beta$ is your competitive ratio. The art is constructing primal/dual pairs where both slackness parameters are small.

9.7. Ski rental via primal-dual

Problem. Each day you either rent skis (\$1) or buy them (\$ B). Once bought, you never pay again. The season lasts K days (unknown in advance).

9.7.1. LP formulation

Let $x \in \{0, 1\}$ indicate buying; $z_i \in \{0, 1\}$ indicate renting on day i .

$$\text{Primal: } \min Bx + \sum_{i=1}^K z_i \quad \text{s.t.} \quad x + z_i \geq 1 \quad \forall i, \quad x, z_i \geq 0 \quad (9.7)$$

$$\text{Dual: } \max \sum_{i=1}^K y_i \quad \text{s.t.} \quad \sum_{i=1}^K y_i \leq B, \quad 0 \leq y_i \leq 1 \quad \forall i \quad (9.8)$$

9.7.2. The 2-competitive primal-dual algorithm

When constraint i arrives (i.e., day i begins):

1. If $x = 1$ already, do nothing.
2. Otherwise, increase y_i until some dual constraint becomes tight:
 - If y_i hits 1 $\Rightarrow$ set $z_i = 1$ (rent today).
 - If $\sum y_i$ hits $B \Rightarrow$ set $x = 1$ (buy).

Analysis. Both primal and dual solutions are feasible by construction:

- If $x > 0$: the constraint $\sum y_i \leq B$ is exactly tight $\Rightarrow \alpha = 1$.
- If $z_i > 0$: the constraint $y_i \leq 1$ is exactly tight $\Rightarrow \alpha = 1$.
- If $y_i > 0$: the primal constraint $x + z_i \geq 1$ is satisfied, and might be as large as 2 (both $x = 1$ and $z_i = 1$) $\Rightarrow \beta = 2$.

By approximate complementary slackness: competitive ratio $\leq \alpha\beta = 1 \times 2 = 2$.

Warning

This is the same 2-competitive bound we obtained by the simpler “rent for $B - 1$ days, then buy” strategy. The primal-dual framework is overkill here—but it scales to harder problems (randomised ski rental, weighted paging, set cover) where ad-hoc arguments break down.

9.8. Cheat sheet

Cheat sheet — Lecture 9

Mark algorithm

- Evict uniformly random unmarked page; mark on access; unmark all when full.
- Competitive ratio: $2H_K$ (optimal up to factor 2; tight ratio is H_K).
- Analysis: phase decomposition $\Rightarrow$ stale faults bounded by harmonic sum.

Lower bound

- Universe of $K + 1$ pages, random requests $\Rightarrow$ any algorithm faults at rate $1/(K + 1)$.
- OPT faults at rate $1/\Theta(K \log K)$ (coupon collector).
- Gap: $\Omega(\log K)$.

Online primal-dual

- Write problem as covering LP with constraints arriving online.
- Maintain monotone primal + dual; compare objectives.
- Approximate complementary slackness: $\alpha\beta = \text{competitive ratio}$.

Ski rental LP

- Primal: $\min Bx + \sum z_i$ s.t. $x + z_i \geq 1$.
- Dual: $\max \sum y_i$ s.t. $\sum y_i \leq B, y_i \leq 1$.
- Deterministic primal-dual: 2-competitive.

9.9. FAQ — questions students actually asked

Q: In the Mark algorithm, does it matter what the initial cache contents are? A: No. The first phase simply unmarks everything. The analysis assumes both Mark and OPT start with identical cache contents; otherwise competitive analysis is vacuous (OPT could start with all future requests pre-loaded).

Q: When you sum the OPT lower bound over phases, why does the telescoping term vanish? A: The sum $\sum_i (D'_i - D_i) = D_{\text{final}} - D_{\text{initial}}$. Since both start with the same cache, $D_{\text{initial}} = 0$, and $D_{\text{final}} \geq 0$, so the sum is non-negative and can be dropped as a lower bound.

Q: The LP for ski rental doesn't capture *when* you buy. Isn't that a problem? A: The LP doesn't model time ordering—it just says “cover every day.” The online algorithm sees constraints sequentially and increases variables monotonically, which implicitly encodes irreversibility. The LP relaxation gives a valid lower bound on any feasible (including online) solution.

Q: Why does it matter that variables are monotonically non-decreasing? A: Without monotonicity, you could “undo” past decisions—set $z_i = 1$ then later reset it to 0 when you decide to buy. Monotonicity models the fact that rental money already spent cannot be reclaimed.

Q: Is the gap between $2H_K$ and H_K for paging closed? A: Yes. Mcgeoch and Sleator (Algorithmica, 1991) and later Achlioptas et al. (TCS, 2000) gave H_K -competitive algorithms. The Mark algorithm's factor-of-2 loss comes from averaging the two OPT lower bounds.

9.10. Further reading

- Fiat, A., Karp, R. M., Luby, M., McGeoch, L. A., Sleator, D. D., & Young, N. E. (1991). Competitive paging algorithms. *Journal of Algorithms*, 12(4), 685–699. — Original Mark algorithm and $\Omega(\log K)$ lower bound.
- McGeoch, L. A. & Sleator, D. D. (1991). A strongly competitive randomized paging algorithm. *Algorithmica*, 6(6), 816–825. — First H_K -competitive algorithm.
- Buchbinder, N. & Naor, J. (2009). The design of competitive online algorithms via a primal-dual approach. *Foundations and Trends in Theoretical Computer Science*, 3(2–3), 93–263. — The definitive reference for online primal-dual.
- Koutsoupias, E. & Papadimitriou, C. H. (1995). On the K -server conjecture. *Journal of the ACM*, 42(5), 971–983. — Work function algorithm, $2K - 1$ competitive.
- Bansal, N., Buchbinder, N., Madry, A., & Naor, J. (2011). A polylogarithmic-competitive algorithm for the K -server problem. *FOCS 2011*. — Breakthrough randomised bound.

9.11. Try it yourself

1. Harmonic warmup. Prove that $H_K = \ln K + \Theta(1)$ by bounding the harmonic sum with integrals from above and below.
2. Phase cost tightness. Construct a request sequence on $K + 1$ pages where the Mark algorithm's expected cost in a single phase is exactly $L \cdot H_K$ (not just an upper bound).
3. Dual construction. For the ski rental LP with $B = 5$ and $K = 8$, write out the primal-dual algorithm's execution step by step. Verify that the dual is feasible and the competitive ratio is 2.
4. Coupon collector concentration. Using Markov's inequality, show that for a random sequence of length $M = 2K \ln K$ over a universe of $K + 1$ pages, OPT's expected cost is $O(M/(K \log K))$.
5. Code it. Implement the Mark algorithm and the Bélády (MIN) algorithm in Python. Generate random request sequences over $K + 1$ pages and empirically measure the competitive ratio as a function of K for $K \in \{4, 8, 16, 32, 64\}$. Plot the ratio against $\ln K$.

```
import numpy as np

def mark_algorithm(requests, cache_size):
    """Simulate the Mark algorithm. Return total number of faults."""
    cache = set()
    marked = set()
    faults = 0

    for page in requests:
        if page in cache:
            marked.add(page)
        else:
            faults += 1
            if len(cache) == cache_size:
                # Evict random unmarked page
                unmarked = list(cache - marked)
                if not unmarked:
                    marked.clear()
                    unmarked = list(cache)
                victim = unmarked[np.random.randint(len(unmarked))]
                cache.remove(victim)
            cache.add(page)
            marked.add(page)

    return faults
```


10. Randomized Ski Rental, Online Set Cover, and Approximation Algorithms

COMPSCI 224 Lecture 10 — Fall 2014

Randomisation drops ski rental's competitive ratio from 2 to $e/(e-1) \approx 1.58$ by rounding a fractional primal-dual solution with a single uniform threshold. Online set cover, analysed through the same primal-dual lens, achieves $O(\log n \cdot \log m)$ -competitive via continuous multiplicative updates. The lecture closes by pivoting to offline approximation algorithms, previewing the dual-fitting analysis of greedy set cover.

10.1. Why this lecture matters

This lecture completes the online primal-dual toolkit by showing its two most important moves: (1) randomised rounding of fractional solutions to beat deterministic barriers, and (2) continuous multiplicative weight updates that naturally produce logarithmic competitive ratios. These techniques recur across algorithm design—from online learning to offline approximation—making them among the most versatile ideas in theoretical CS.

30-second story

1. A fractional primal-dual solution for ski rental achieves ratio $1 + 1/c$ where $c = (1 + 1/B)^B - 1 \rightarrow e - 1$.
2. A single random threshold converts the fractional solution to an integral one with the same expected cost.
3. Online set cover uses exponential growth ($dx/dt = x$) to satisfy constraints, giving $O(\log m)$ fractional competitive ratio.
4. Repeating $O(\log n)$ independent random thresholds ensures feasibility with high probability, yielding $O(\log n \cdot \log m)$ overall.
5. Offline approximation algorithms seek polynomial-time α -approximate solutions to NP-hard problems.
6. Dual fitting—violating dual feasibility then scaling down—proves the greedy set cover algorithm is $O(\log n)$ -approximate.

10.2. Randomised ski rental: beating the factor of 2

Recall the ski rental LP from Lecture 9:

$$\text{Primal: } \min Bx + \sum_{i=1}^K z_i \quad \text{s.t.} \quad x + z_i \geq 1, \quad x, z_i \geq 0 \quad (10.1)$$

$$\text{Dual: } \max \sum_{i=1}^K y_i \quad \text{s.t.} \quad \sum y_i \leq B, \quad 0 \leq y_i \leq 1 \quad (10.2)$$

10.2.1. The fractional algorithm

Initialise $x = 0$, all $z_i = 0$, all $y_i = 0$. When constraint i arrives:

- If $x \geq 1$: do nothing.
- Otherwise: set $z_i = 1 - x$, update $x \leftarrow (1 + 1/B)x + 1/(cB)$, set $y_i = 1$.

Here c is a constant chosen to ensure dual feasibility.

i Why this update rule?

The increments x_i to x form a geometric sequence with ratio $q = 1 + 1/B$. Starting from $x_1 = 1/(cB)$, after B days the sum reaches $\frac{1}{cB} \cdot \frac{q^B - 1}{q - 1} = \frac{(1+1/B)^B - 1}{c}$. Setting $c = (1 + 1/B)^B - 1$ makes $x = 1$ after exactly B days, ensuring dual feasibility ($\sum y_i \leq B$).

10.2.2. Bounding the competitive ratio

In each step where both objectives change:

- Dual objective increases by 1 (we set $y_i = 1$).
- Primal objective increases by $B \cdot \Delta x + z_i = B \cdot (x/B + 1/(cB)) + (1 - x) = x + 1/c + 1 - x = 1 + 1/c$.

Therefore:

$$\frac{\text{Primal cost}}{\text{Dual cost}} \leq 1 + \frac{1}{c} = 1 + \frac{1}{(1 + 1/B)^B - 1} \quad (10.3)$$

As $B \rightarrow \infty$, $(1 + 1/B)^B \rightarrow e$, so the ratio converges to $\frac{e}{e-1} \approx 1.582$.

! Important

The exact competitive ratio is $(1 + 1/B)^B / ((1 + 1/B)^B - 1)$. At $B = 1$ this equals 2 (matching deterministic); as $B \rightarrow \infty$ it approaches $e/(e - 1)$.

10.2.3. Randomised rounding: from fractions to decisions

The fractional solution isn't a valid ski-rental strategy (you can't rent 0.3 of a ski). The fix:

1. At the start, draw $\alpha \sim \text{Uniform}[0, 1]$.
2. Maintain the fractional solution x as before.
3. Buy on the first day i where $x \geq \alpha$. Until that day, rent.

Analysis of expected cost:

$$\mathbb{E}[\text{cost}] = \mathbb{E}[\text{buying cost}] + \mathbb{E}[\text{renting cost}]$$

- $\mathbb{E}[\text{buying}] = B \cdot \Pr[\text{buy}] = B \cdot \Pr[\alpha \leq x] = Bx$.
- $\mathbb{E}[\text{rent on day } i] = \Pr[\text{haven't bought by day } i] = \Pr[\alpha > x_1 + \dots + x_{i-1}] = 1 - (x_1 + \dots + x_{i-1}) = z_i$.

Therefore $\mathbb{E}[\text{cost}] = Bx + \sum z_i$ is the fractional primal objective.

 Tip

The randomised rounding preserves the fractional cost *exactly* in expectation. No approximation loss occurs in the rounding step—the entire competitive ratio comes from the fractional algorithm.

This result is originally due to Karlin et al. (*Algorithmica*, 1994); the primal-dual viewpoint is from Buchbinder and Naor.

10.3. Online set cover

10.3.1. Problem definition

Given a universe $\{1, \dots, n\}$ and m sets $S_1, \dots, S_m$ (each a subset of the universe), pick a minimum-size subcollection whose union covers the entire universe.

Online twist: The sets are known upfront, but elements arrive one at a time. When element i arrives, you learn which sets contain it. If i is not already covered, you must immediately select a set to cover it (irrevocably).

 Warning

Set cover is NP-hard to approximate within better than $\ln n$ (offline). Don't expect an online algorithm to do better than $O(\log n)$ -competitive, even with unlimited computation.

10.3.2. LP relaxation

$$\text{Primal: } \min \sum_{s \in \mathcal{C}} x_s \quad \text{s.t.} \quad \sum_{s \ni i} x_s \geq 1 \quad \forall i, \quad x_s \geq 0 \quad (10.4)$$

$$\text{Dual: } \max \sum_{i=1}^n y_i \quad \text{s.t.} \quad \sum_{i \in s} y_i \leq 1 \quad \forall s, \quad y_i \geq 0 \quad (10.5)$$

10.3.3. The multiplicative-update algorithm

Initialise: $x_s = 1/m$ for all sets s .

When element i arrives (uncovered): let $S_{j_1}, \dots, S_{j_r}$ be the sets containing i . Run a continuous process: simultaneously evolve all x_{j_k} according to

$$\frac{dx_s(t)}{dt} = x_s(t) \quad (10.6)$$

until $\sum_{s \ni i} x_s \geq 1$. This multiplies each relevant x_s by e^t for the duration t needed to satisfy the constraint.

10.3.4. Analysis: the purely primal view

Key observation: No set in OPT can evolve for more than $\ln m$ total seconds.

Proof: Every x_s starts at $1/m$. Whenever a constraint triggers evolution, $\sum_{s \ni i} x_s < 1$, so each individual $x_s < 1$. After $\ln m$ seconds of growth from $1/m$, a set reaches $(1/m) \cdot e^{\ln m} = 1$ —at which point its constraint is already satisfied and it never evolves again.

Since OPT has at most OPT sets, and each evolves for at most $\ln m$ seconds, the total evolution time is at most $\text{OPT} \cdot \ln m$.

Bounding the primal objective's growth rate: During evolution for constraint i :

$$\frac{d}{dt}(x_{j_1} + \cdots + x_{j_r}) = x_{j_1} + \cdots + x_{j_r} < 1$$

(since evolution runs only while the sum is below 1). The primal objective $\sum_s x_s$ therefore increases by less than 1 per second.

Conclusion: The primal objective starts at $m \cdot (1/m) = 1$ and grows by at most 1 per second for at most $\text{OPT} \cdot \ln m$ seconds:

$$\text{Primal cost} \leq 1 + \text{OPT} \cdot \ln m \leq O(\text{OPT} \cdot \ln m) \quad (10.7)$$

10.3.5. The primal-dual view

Augment the algorithm: alongside the x_s evolution, set $dy_i/dt = 1$ during the evolution triggered by element i .

- Primal cost vs. dual cost: In every second, the primal increases by < 1 and the dual increases by exactly 1. So $\text{Primal cost} \leq \text{Dual cost}$.
- Dual feasibility (after scaling): For any set s , whenever $\sum_{i \in s} y_i$ increases, x_s is also growing. Since x_s can grow for at most $\ln m$ seconds, $\sum_{i \in s} y_i \leq \ln m$. Therefore $y/\ln m$ is dual-feasible.

Chaining inequalities:

$$\frac{\text{Dual cost}}{\ln m} \leq \text{OPT}_D \leq \text{OPT}_P \leq \text{Primal cost} \leq \text{Dual cost}$$

So $\text{Primal cost} \leq \ln m \cdot \text{OPT}$.

i Aside (mine, not the lecturer's)

Buchbinder and Naor (2009) show this bound is tight up to $O(\log \log)$ factors for the online setting—you cannot achieve $o(\log m)$ -competitive online.

10.3.6. Randomised rounding for integrality

The fractional solution must be converted to actual set selections. Naive single-threshold rounding (as in ski rental) may fail to cover some elements.

Fix: For each set s , draw $r = O(\log n)$ independent thresholds $\alpha_{s,1}, \dots, \alpha_{s,r} \sim \text{Uniform}[0, 1]$. Take s into the solution as soon as $x_s \geq \alpha_{s,j}$ for any j .

- Expected cost increases by factor $r = O(\log n)$, giving $O(\log n \cdot \log m)$.
- Feasibility: For each element i , the probability of *not* being covered by a single threshold is $\leq 1/2$ (since $\sum_{s \ni i} x_s \geq 1$). With $r = O(\log n)$ independent trials, failure probability drops to $1/\text{poly}(n)$. Union bound over n elements gives high-probability feasibility.

! Important

Online set cover achieves $O(\log n \cdot \log m)$ -competitive ratio, nearly matching the $\Omega(\log m \cdot \log n / \log \log n)$ lower bound.

10.4. Transition to approximation algorithms

The remaining minutes pivot from online to offline:

Setting. Given an NP-hard optimisation problem, find a polynomial-time algorithm producing a solution within factor α of OPT.

The primal-dual method applies here too—in fact, it's *older* in the approximation setting (1970s) than in the online setting (2000s).

10.4.1. Weighted set cover and the greedy algorithm

Now sets have costs $c_s > 0$. Minimise $\sum_{s \text{ chosen}} c_s$ subject to covering the universe.

Greedy algorithm: While uncovered elements remain, pick the set s minimising $c_s / |\text{newly covered elements}|$ (cost per new element).

LP formulation:

$$\text{Primal: } \min \sum_s c_s x_s \quad \text{s.t.} \quad \sum_{s \ni i} x_s \geq 1 \quad \forall i, \quad x_s \geq 0 \quad (10.8)$$

$$\text{Dual: } \max \sum_{i=1}^n y_i \quad \text{s.t.} \quad \sum_{i \in s} y_i \leq c_s \quad \forall s, \quad y_i \geq 0 \quad (10.9)$$

10.4.2. Dual-fitting analysis (preview)

When the greedy algorithm takes set s :

- Set $x_s = 1$ (primal cost increases by c_s).
- For each newly covered $i \in s$, set $y_i = c_s / (\text{number newly covered})$.

The primal and dual costs are equal by construction. The dual may violate feasibility—some constraint $\sum_{i \in s} y_i \leq c_s$ may be exceeded—but by at most a factor of $H_n = O(\log n)$.

i Dual fitting

Scale y down by H_n to obtain a feasible dual solution. Then:

$$\frac{\text{Dual cost}}{H_n} \leq \text{OPT} \leq \text{Primal cost} = \text{Dual cost}$$

so $\text{Primal cost} \leq H_n \cdot \text{OPT}$. The greedy algorithm is an H_n -approximation.

The full proof (deferred to next lecture) shows that the per-set dual violation is at most $H_{|S|}$, which telescopes to H_n globally.

10.5. Cheat sheet**!** Cheat sheet — Lecture 10

Randomised ski rental

- Fractional primal-dual: geometric increments to x , ratio = $1 + 1/c$ where $c = (1 + 1/B)^B - 1$.
- Rounding: draw $\alpha \sim U[0, 1]$; buy when $x \geq \alpha$. Expected cost = fractional cost.
- Competitive ratio: $e/(e - 1) \approx 1.58$ as $B \rightarrow \infty$.

Online set cover

- Initialise $x_s = 1/m$. On new element: exponentially grow x_s for containing sets until covered.
- Fractional competitive ratio: $O(\log m)$.
- Integrality: $O(\log n)$ independent random thresholds $\Rightarrow O(\log n \cdot \log m)$ total.
- Primal-dual view: dual grows at rate 1, primal at rate ≤ 1 ; dual violates by $\ln m$ factor.

Approximation algorithms (intro)

- NP-hard problem + polynomial time $\Rightarrow$ accept factor- α loss.
- Dual fitting: build primal/dual with equal cost; scale infeasible dual down to get α .
- Greedy weighted set cover: H_n -approximate via dual fitting.

10.6. FAQ — questions students actually asked

Q: In the randomised ski rental, once we pick α , the buy-day is deterministic. Where's the randomness helping? A: The adversary doesn't know α . For any fixed season length K , some values of α buy early (bad if K is small) and some buy late (bad if K is large). The uniform distribution ensures no single K can be catastrophic in expectation—the adversary can't pick a worst case.

Q: For online set cover, why not just round each x_s to 1 once it exceeds $1/2$? A: That would give expected coverage $\geq 1/2$ per element—a coin flip for each. With n elements, some will fail. You need the expected coverage to be $\Omega(\log n)$ per element so that the failure probability drops below $1/n$ (for a union bound).

Q: Why does the differential equation $dx/dt = x$ give us what we want? A: The exponential $x(t) = x(0) \cdot e^t$ is the unique growth rate that makes the competitive-ratio analysis work. The primal objective's growth rate equals the current constraint slack (which is < 1), while the dual grows at rate exactly 1. Any faster growth overshoots; any slower growth doesn't satisfy constraints fast enough.

Q: Can you do set cover without knowing n in advance? A: The fractional algorithm doesn't need n —it only needs to know m (to initialise $x_s = 1/m$). The rounding step uses $\log n$ repetitions; you can remove this assumption by doubling the repetition count adaptively or using a standard “guess-and-double” technique.

Q: Is the $\log n \cdot \log m$ bound tight for online set cover? A: Nearly. There's a lower bound of $\Omega(\log m \cdot \log n / \log \log n)$ from Buchbinder and Naor. The gap is a $\log \log$ factor—closing it is a potential research question.

Q: For the greedy set cover analysis, why does the dual violation per set telescope to H_n ? A: When greedy picks set s covering k new elements, it assigns $y_i = c_s/k$ to each. For a fixed set s' , its elements get “charged” in rounds where progressively fewer elements remain uncovered. The charges form a sequence $c_{s'}/k_1 + c_{s'}/k_2 + \dots$ where $k_1 \geq k_2 \geq \dots$ —bounded by $c_{s'} \cdot H_{|s'|} \leq c_{s'} \cdot H_n$.

10.7. Further reading

- Karlin, A. R., Manasse, M. S., McGeoch, L. A., & Owicki, S. (1994). Competitive randomized algorithms for nonuniform problems. *Algorithmica*, 11(6), 542–571. — Original $e/(e-1)$ ski rental result.
- Buchbinder, N. & Naor, J. (2009). The design of competitive online algorithms via a primal-dual approach. *Foundations and Trends in TCS*, 3(2–3), 93–263. — Comprehensive treatment of online primal-dual including set cover.
- Alon, N., Awerbuch, B., Azar, Y., Buchbinder, N., & Naor, J. (2003). The online set cover problem. *STOC 2003*. — Original $O(\log m \cdot \log n)$ online set cover (purely primal view).
- Vazirani, V. V. (2001). *Approximation Algorithms*. Springer. — Standard reference for offline approximation, including greedy set cover and dual fitting.
- Chvátal, V. (1979). A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3), 233–235. — Classic H_n -approximation proof for greedy set cover.

10.8. Try it yourself

1. Compute the constant. For $B = 10$, compute the exact competitive ratio $(1 + 1/B)^B / ((1 + 1/B)^B - 1)$. How close is it to $e/(e-1)$?
2. Verify the rounding. In the randomised ski rental with $B = 4$, write out the increments x_1, x_2, x_3, x_4 . Draw the interval $[0, 1]$ partitioned by cumulative sums. For $\alpha = 0.6$, on which day do you buy?
3. Online set cover example. Consider $m = 4$ sets over universe $\{1, 2, 3, 4, 5\}$ with $S_1 = \{1, 2, 3\}$, $S_2 = \{2, 4\}$, $S_3 = \{3, 5\}$, $S_4 = \{1, 4, 5\}$. Elements arrive in order 3, 4, 1, 5, 2. Simulate the multiplicative-update algorithm (starting at $x_s = 1/4$) for the first two elements. What are the final x_s values?
4. Dual-fitting by hand. For the weighted set cover instance: $S_1 = \{a, b, c\}$ cost 6, $S_2 = \{b, c, d\}$ cost 3, $S_3 = \{a, d\}$ cost 4. Run greedy, construct the dual y values, and verify the maximum dual violation factor.
5. Code it. Implement the online set cover algorithm (fractional + randomised rounding) in Python. Test on random instances with $n = 100$, $m = 50$, sets of random size 5–15. Measure the empirical competitive ratio over 1000 trials and compare to $\ln n \cdot \ln m$.

```
import numpy as np

def online_set_cover_fractional(n, sets, element_order):
    """
    Simulate the fractional online set cover algorithm.

    Args:
        n: universe size
```

```

    sets: list of sets (each a set of element indices)
    element_order: order in which elements arrive

Returns:
    x: final fractional solution (array of length m)
    ....
m = len(sets)
x = np.full(m, 1.0 / m)

for elem in element_order:
    # Find sets containing this element
    containing = [j for j, s in enumerate(sets) if elem in s]

    # Check if already covered
    if sum(x[j] for j in containing) >= 1.0:
        continue

    # Evolve until covered (solve for t analytically)
    current_sum = sum(x[j] for j in containing)
    # After time t: sum becomes current_sum * e^t >= 1
    t = np.log(1.0 / current_sum)

    # Apply multiplicative update
    for j in containing:
        x[j] *= np.exp(t)

return x

```

Part III.

Approximation Algorithms and LP

11. Integrality Gaps and Approximation Schemes

COMPSCI 224 Lecture 11 — Fall 2014

LP relaxations are powerful but not omnipotent: the integrality gap puts a hard ceiling on what any LP-based analysis can achieve. This lecture finishes the dual-fitting proof that greedy gives an H_n approximation for weighted set cover, proves a primal-dual 2-approximation for vertex cover, exhibits tight integrality gaps for both problems (factor 2 for vertex cover, $\Theta(\log n)$ for set cover via a linear-algebraic construction over $\mathbb{F}_2^q$), then pivots to approximation schemes — PTAS, FPTAS, FPRAS — and builds a PTAS for knapsack by enumerating high-value items and feeding the rest to greedy.

11.1. Why this lecture matters

Every time you round an LP relaxation into an integer solution, you pay for the gap between the fractional optimum and the true optimum. Understanding where that gap is tight tells you when to stop trying to improve an LP-based algorithm and when to reach for a different formulation. The second half of the lecture introduces approximation schemes — algorithms that get arbitrarily close to optimal — and shows exactly when they can and cannot exist.

The 30-second story

1. Dual fitting shows greedy for weighted set cover achieves an $H_n \approx \ln n$ approximation by maintaining a dual solution whose cost equals the primal cost, then scaling down by H_n to restore feasibility.
2. Vertex cover admits a clean primal-dual 2-approximation: pick an uncovered edge, take both endpoints, set the dual variable to 1.
3. The integrality gap of the standard LP for vertex cover is 2 (exhibited by K_n), and for set cover it is $\Theta(\log n)$ (exhibited by a construction over $\mathbb{F}_2^q$).
4. A PTAS for knapsack enumerates the $O(1/\epsilon)$ high-value items and runs greedy on the rest, yielding $(1-\epsilon)$ -OPT in $n^{O(1/\epsilon)}$ time.
5. FPTAS and FPRAS refine the idea: polynomial in both n and $1/\epsilon$, with randomized variants allowed a $1/3$ failure probability.

11.2. Dual fitting for weighted set cover

Recall the setup. The universe is $\{1, \dots, n\}$. There are m sets, each set S has cost c_S . The goal is to cover the universe at minimum total cost.

11.2.1. The LP relaxation and its dual

The LP relaxation assigns a fractional value $x_S \in [0, 1]$ to each set:

$$\min \sum_S c_S x_S \quad \text{s.t.} \quad \sum_{S \ni i} x_S \geq 1 \quad \forall i, \quad x_S \geq 0 \quad \forall S. \quad (11.1)$$

The dual assigns a “price” y_i to each element:

$$\max \sum_{i=1}^n y_i \quad \text{s.t.} \quad \sum_{i \in S} y_i \leq c_S \quad \forall S, \quad y_i \geq 0 \quad \forall i. \quad (11.2)$$

 Tip

Think of y_i as how much element i is willing to “pay” to be covered. The dual constraint says no set collects more in payments than it costs. Weak duality guarantees $\sum y_i \leq \text{OPT}$.

11.2.2. The greedy algorithm and its dual interpretation

The weighted greedy algorithm repeatedly picks the set minimizing $c_S / |\text{new elements covered by } S|$. Dual fitting maintains a dual solution alongside the primal:

1. When greedy takes set S , set $x_S = 1$.
2. For each newly covered element e , set $y_e = c_S / |\text{newly covered elements}|$.

Two observations fall out immediately:

- Primal cost = Dual cost. The cost charged to the primal equals the total price assigned in the dual, by construction.
- Primal is feasible. Greedy runs until every element is covered, so every primal constraint is satisfied.

The dual y may *not* be feasible — some set S might collect more in prices than c_S . The key claim is that scaling down by H_n (the n -th harmonic number) fixes this.

11.2.3. Proving y/H_n is dual-feasible

Fix a set S with $|S| = k$. Order its elements $e_1, e_2, \dots, e_k$ by the time they are first covered (by any set, not necessarily S).

Right before e_i is covered, the set S still contains at least $k - i + 1$ uncovered elements. Greedy chose the set with the lowest price-per-element, so:

$$y_{e_i} \leq \frac{c_S}{k - i + 1}. \quad (11.3)$$

Summing over all elements of S :

$$\sum_{i=1}^k y_{e_i} \leq c_S \sum_{i=1}^k \frac{1}{k - i + 1} = c_S \cdot H_k \leq c_S \cdot H_n. \quad (11.4)$$

Dividing by H_n makes the dual constraint $\sum_{i \in S} y_i \leq c_S$ hold for every set S . Since primal cost equals dual cost, and the scaled dual is feasible:

$$\text{Greedy cost} = \sum y_i = H_n \cdot \sum \frac{y_i}{H_n} \leq H_n \cdot \text{OPT}_{\text{LP}} \leq H_n \cdot \text{OPT}.$$

! Important

If every set has size at most k , greedy gives an H_k approximation — potentially much better than H_n for sparse set systems.

11.3. Vertex cover via primal-dual

11.3.1. The problem and its LP

Given an undirected graph $G = (V, E)$ with $|V| = n$ and $|E| = m$, find the smallest set $S \subseteq V$ such that every edge has at least one endpoint in S .

Vertex cover is a special case of set cover: each vertex is a “set” containing its incident edges, and the universe is the edge set.

The LP relaxation and its dual:

$$\min \sum_v x_v \quad \text{s.t.} \quad x_u + x_v \geq 1 \quad \forall (u, v) \in E, \quad x_v \geq 0 \quad \forall v. \quad (11.5)$$

$$\max \sum_e y_e \quad \text{s.t.} \quad \sum_{e \ni v} y_e \leq 1 \quad \forall v, \quad y_e \geq 0 \quad \forall e. \quad (11.6)$$

11.3.2. The greedy algorithm

While there exists an uncovered edge (u, v) : add *both* u and v to the cover.

The edges chosen in each iteration form a matching (no two share an endpoint). The cover has $2|\text{matching}|$ vertices, and any vertex cover must include at least one endpoint of each matching edge, so:

$$|\text{cover}| = 2|\text{matching}| \leq 2 \cdot \text{OPT}.$$

11.3.3. Primal-dual proof

This is not dual fitting — it is pure primal-dual, because both solutions are automatically feasible.

When greedy covers edge $e = (u, v)$:

- Primal: set $x_u = 1$ and $x_v = 1$.
- Dual: set $y_e = 1$.

Feasibility. The primal is feasible because every edge has at least one endpoint set to 1. The dual is feasible because each vertex is added to the cover at most once, so each vertex is incident to at most one edge with $y_e = 1$.

Approximation ratio. Each dual increase of 1 causes a primal increase of at most 2 (both endpoints). So:

$$\text{Primal cost} \leq 2 \cdot \text{Dual cost} \leq 2 \cdot \text{OPT}.$$

i Note

On the homework, the problem extends to *weighted* vertex cover. Taking both endpoints is wasteful when one is much cheaper. The primal-dual perspective guides you to a better algorithm: increase y_e until some vertex's dual constraint goes tight, then add that vertex.

11.4. Integrality gaps

11.4.1. What the gap measures

An LP relaxation replaces integer constraints with linear ones. The integrality gap measures the worst-case ratio between the LP optimum and the true (integer) optimum over all instances:

$$\text{Integrality gap} = \sup_{\text{instances}} \frac{\text{OPT}_{\text{IP}}}{\text{OPT}_{\text{LP}}}. \quad (11.7)$$

If the gap is β , then no analysis that compares against OPT_{LP} can prove an approximation ratio better than β . This does *not* rule out better algorithms — only better *LP-based* analyses.

Warning

The integrality gap is a property of the *LP relaxation*, not of the problem. A different formulation of the same problem may have a smaller gap. The Sherali–Adams hierarchy systematically tightens LP relaxations, eventually matching the integer program — but at exponential LP size.

11.4.2. Vertex cover: gap of 2

Take the complete graph K_n . The LP optimum sets $x_v = 1/2$ for every vertex, achieving cost $n/2$. But any valid vertex cover must include at least $n - 1$ vertices (leaving out one vertex leaves no uncovered edges, but leaving out two leaves their shared edge uncovered).

$$\frac{\text{OPT}_{\text{IP}}}{\text{OPT}_{\text{LP}}} = \frac{n-1}{n/2} \rightarrow 2 \quad \text{as } n \rightarrow \infty.$$

So the standard LP for vertex cover cannot prove anything better than a 2-approximation.

11.4.3. Set cover: gap of $\Theta(\log n)$

This construction uses the vector space $\mathbb{F}_2^q$ (bit strings of length q with XOR arithmetic). Set $n = 2^q - 1$.

- Elements correspond to nonzero vectors in $\mathbb{F}_2^q$.
- Sets also correspond to vectors: for each $\alpha \in \mathbb{F}_2^q$, define $S_\alpha = \{e : \alpha \cdot e = 1 \pmod{2}\}$.

There are 2^q sets (including $S_0 = \emptyset$). Each nonzero element lies in exactly half the sets — if you pick α uniformly at random, $\Pr[\alpha \cdot e = 1] = 1/2$ for any fixed nonzero e .

Fractional solution. Set $x_\alpha = 2/m$ for every α (where $m = 2^q$ is the number of sets). Each element is in $m/2$ sets, so $\sum_{S \ni e} x_\alpha = (m/2)(2/m) = 1$. The total LP cost is $\sum x_\alpha = 2$.

Integer lower bound. Suppose $q - 1$ sets $S_{\alpha_1}, \dots, S_{\alpha_{q-1}}$ cover the universe. The set of elements *not* covered by any S_{α_i} is:

$$\{e : \alpha_i \cdot e = 0 \ \forall i = 1, \dots, q-1\}.$$

This is the null space of a $(q-1) \times q$ matrix over $\mathbb{F}_2$, which has dimension at least 1. So there exists a nonzero uncovered element — a contradiction. Any integer solution needs at least $q = \log_2(n+1)$ sets.

$$\frac{\text{OPT}_{\text{IP}}}{\text{OPT}_{\text{LP}}} \geq \frac{q}{2} = \frac{\log_2(n+1)}{2} = \Theta(\log n).$$

i Aside (mine, not the lecturer's)

The integrality gap gives a *barrier for LP-based proofs*, not for all algorithms. For set cover, the barrier turns out to be real: Feige (1998) showed that no polynomial-time algorithm can beat $\ln n$ unless SAT can be solved in $n^{O(\log \log n)}$ time. This result uses the PCP theorem — probabilistically checkable proofs — which is beyond this course's scope.

11.5. Approximation schemes: PTAS, FPTAS, FPRAS

For some problems, we want more than a constant-factor approximation — we want $(1 + \varepsilon)$ -OPT for any $\varepsilon > 0$.

! Definitions

For a minimization problem (swap $1 + \varepsilon$ with $1 - \varepsilon$ for maximization):

- PTAS (polynomial-time approximation scheme): for every fixed $\varepsilon > 0$, achieves $(1 + \varepsilon)$ -OPT in time $n^{f(1/\varepsilon)}$. The exponent may depend on ε .
- FPTAS (fully polynomial-time approximation scheme): achieves $(1 + \varepsilon)$ -OPT in time $\text{poly}(n, 1/\varepsilon)$. No ε in the exponent.
- FPRAS (fully polynomial-time randomized approximation scheme): like FPTAS, but the algorithm is randomized and may fail with probability at most $1/3$.

💡 Tip

Why probability $1/3$? Because you can amplify: run the algorithm $O(\log(1/\delta))$ times and take the median. If each run succeeds with probability $\geq 2/3$, the median is correct with probability $\geq 1 - \delta$. This amplification works because $1/3 < 1/2$ — any constant failure probability strictly below $1/2$ suffices.

11.6. Knapsack

11.6.1. Problem definition

You have a knapsack of capacity W and n items. Item i has weight w_i and value v_i . Pack a subset of items to maximize total value without exceeding the weight limit:

$$\max \sum_i v_i x_i \quad \text{s.t.} \quad \sum_i w_i x_i \leq W, \quad x_i \in \{0, 1\}. \quad (11.8)$$

11.6.2. Exact solutions via dynamic programming

Knapsack is NP-hard, but admits pseudo-polynomial time algorithms:

DP by weight. Define $f(i, b)$ = maximum value using items $\{1, \dots, i\}$ with remaining capacity b :

$$f(i, b) = \begin{cases} 0 & \text{if } i = 0, \\ \max(f(i-1, b), v_i + f(i-1, b - w_i)) & \text{if } b \geq w_i, \\ f(i-1, b) & \text{otherwise.} \end{cases} \quad (11.9)$$

Running time: $O(nW)$. This is *pseudo-polynomial* — polynomial in W , but W can be exponential in the input size $\log W$.

```
def knapsack_by_weight(weights, values, W):
    n = len(weights)
    # f[i][b] = max value using items 0..i-1 with capacity b
    f = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for b in range(W + 1):
            f[i][b] = f[i-1][b]
            if b >= weights[i-1]:
                f[i][b] = max(f[i][b], values[i-1] + f[i-1][b - weights[i-1]])
    return f[n][W]
```

DP by value. Define $g(i, p)$ = minimum weight to achieve value exactly p using items $\{1, \dots, i\}$. Running time: $O(n \cdot V)$ where $V = \sum v_i$. Also pseudo-polynomial.

Warning

Neither DP is truly polynomial. Writing W takes $\log W$ bits, so $O(nW)$ is exponential in the input size. This is why knapsack is NP-hard despite having a “fast” DP.

11.6.3. Fractional knapsack and the greedy approach

If items can be split, the optimal strategy is to sort by value-to-weight ratio v_i/w_i in decreasing order and fill the knapsack greedily. The last item may be taken fractionally.

A natural greedy algorithm for integral knapsack does the same but stops when the next item doesn't fit. This can perform arbitrarily badly:

Bad example. Item 1: $v_1 = 2, w_1 = 1$. Item 2: $v_2 = W, w_2 = W$. Greedy takes item 1 (better ratio) and can't fit item 2. OPT takes item 2 for value W .

11.6.4. Modified greedy: a 2-approximation

Take the *better* of: (a) the greedy output, or (b) the single most valuable item (that fits).

Lemma

Suppose greedy takes items in ratio order and fails to fit item k . Then $\sum_{i=1}^k v_i \geq \text{OPT}$.

Proof. The fractional optimum takes items $1, \dots, k-1$ fully and a fraction of item k , so $\text{OPT}_{\text{frac}} \leq \sum_{i=1}^k v_i$. Since $\text{OPT} \leq \text{OPT}_{\text{frac}}$, the lemma follows.

2-approximation proof. By the lemma, $\sum_{i=1}^{k-1} v_i + v_k \geq \text{OPT}$. One of $\sum_{i=1}^{k-1} v_i$ (greedy output) or v_k (most valuable item) is at least $\text{OPT}/2$.

11.6.5. PTAS for knapsack

Two observations drive the PTAS:

1. If no item has value $> \varepsilon \cdot \text{OPT}$, greedy achieves $\geq (1 - \varepsilon) \cdot \text{OPT}$. (By the lemma, greedy misses at most one item, worth $\leq \varepsilon \cdot \text{OPT}$.)
2. At most $1/\varepsilon$ items in OPT can have value $> \varepsilon \cdot \text{OPT}$.

The algorithm:

1. Enumerate all subsets S of size at most $1/\varepsilon$ from the n items. There are $O(n^{1/\varepsilon})$ such subsets.
2. For each S that fits in the knapsack: remove all items with value larger than any item in S , then run greedy on the remaining items.
3. Return the best solution found.

Correctness. In some iteration, S matches the high-value items in OPT. After removing items with higher value, greedy operates on “small” items (all $\leq \varepsilon \cdot \text{OPT}$) and by observation 1 loses at most $\varepsilon \cdot \text{OPT}$.

Running time. $O(n^{1/\varepsilon} \cdot \text{poly}(n))$ — polynomial for fixed ε , but the exponent depends on ε . This is a PTAS, not an FPTAS.

```
from itertools import combinations

def knapsack_ptas(weights, values, W, epsilon):
    n = len(weights)
    max_size = int(1 / epsilon)
    best_value = 0

    for size in range(max_size + 1):
        for S in combinations(range(n), size):
            s_weight = sum(weights[i] for i in S)
            if s_weight > W:
                continue
            s_value = sum(values[i] for i in S)
            max_val_in_S = max((values[i] for i in S), default=0)

            # Greedy on remaining small items
            remaining = [(values[j] / weights[j], j) for j in range(n)
                        if j not in S and values[j] <= max_val_in_S]
            remaining.sort(reverse=True)

            cap_left = W - s_weight
            greedy_val = s_value
            for _, j in remaining:
                if weights[j] <= cap_left:
                    greedy_val += values[j]
                    cap_left -= weights[j]
            best_value = max(best_value, greedy_val)
```

```
return best_value
```

🔥 Caution

The code above is correct but runs in $O(n^{1/\varepsilon+1})$ time. For large $1/\varepsilon$, the inner loop over combinations dominates. This is still polynomial for fixed ε , but not practical for very small ε — that is exactly why an FPTAS (next lecture) is worth pursuing.

11.7. Cheat sheet

! Cheat sheet — Lecture 11

Dual fitting (set cover). Greedy for weighted set cover is an H_n -approximation. Proof: assign dual prices $y_e = c_S/|new|$; the dual scaled by $1/H_n$ is feasible.

Primal-dual (vertex cover). Pick uncovered edge, take both endpoints, set $y_e = 1$. Primal cost $\leq 2 \times$ dual cost $\Rightarrow 2$ -approximation.

Integrality gap. $\sup_{\text{instances}} \text{OPT}_{\text{IP}}/\text{OPT}_{\text{LP}}$.

- Vertex cover LP: gap = 2 (tight, via K_n).
- Set cover LP: gap = $\Theta(\log n)$ (tight, via $\mathbb{F}_2^q$ construction).

Approximation schemes.

Scheme	Time	Error
PTAS	$n^{f(1/\varepsilon)}$	$(1 \pm \varepsilon) \cdot \text{OPT}$
FPTAS	$\text{poly}(n, 1/\varepsilon)$	$(1 \pm \varepsilon) \cdot \text{OPT}$
FPRAS	$\text{poly}(n, 1/\varepsilon)$	$(1 \pm \varepsilon) \cdot \text{OPT}$, fails w.p. $\leq 1/3$

Knapsack.

- Exact DP: $O(nW)$ or $O(nV)$ — pseudo-polynomial.
- Modified greedy: $\max(\text{greedy output}, \text{best single item})$ is a 2-approximation.
- PTAS: enumerate $\leq 1/\varepsilon$ high-value items, greedy on the rest. Time $O(n^{1/\varepsilon})$.

11.8. FAQ — questions students actually asked

Q: Is the LP relaxation of set cover unique? No. There are many ways to formulate set cover as an integer program, and each gives a different LP relaxation with potentially different integrality gaps. The Sherali–Adams hierarchy systematically tightens LP relaxations, eventually recovering the integer program — but the LP grows to exponential size.

Q: You said we “cannot hope to achieve” a ratio better than β — does that mean no algorithm can? No. It means no algorithm whose analysis *compares to the LP optimum* can beat β . A completely different approach (not LP-based) might still do better. For set cover, however, Feige’s result says even non-LP-based algorithms can’t beat $\ln n$ (assuming standard complexity assumptions).

Q: For the DP running time $O(nW)$ — why isn't that polynomial? Polynomial time means polynomial in the *bit length* of the input. The number W is written in $\log W$ bits, so $O(nW)$ is exponential in the input size. Algorithms with running time polynomial in the *value* of a number (rather than its bit length) are called *pseudo-polynomial*.

Q: Why is $1/3$ the magic failure probability for FPRAS? Any constant strictly less than $1/2$ works. Run the algorithm $O(\log(1/\delta))$ times and take the median — by a Chernoff argument, the median is correct with probability $\geq 1 - \delta$. The constant $1/3$ is conventional because it is the smallest reciprocal integer that is strictly less than $1/2$.

Q: The integrality gap for vertex cover — why doesn't K_n with $x_v = 1/2$ mean the gap is unbounded? The gap is a *ratio*. The LP gives $n/2$; the IP requires $n - 1$. The ratio $(n - 1)/(n/2) \rightarrow 2$, which is bounded.

Q: In the PTAS for knapsack, why remove items with value bigger than items in S ? Because S is supposed to be the set of *all* high-value items in OPT. Any item with higher value than everything in S would contradict the assumption that S captures the high-value items. Removing them ensures greedy operates only on items where missing one costs at most $\varepsilon \cdot \text{OPT}$.

11.9. Further reading

- Johnson (1974). “Approximation algorithms for combinatorial problems.” *Journal of Computer and System Sciences*. The original H_n analysis of greedy set cover (unweighted case).
- Chvátal (1979). “A greedy heuristic for the set-covering problem.” *Mathematics of Operations Research*. Extends the greedy analysis to weighted set cover.
- Feige (1998). “A threshold of $\ln n$ for approximating set cover.” *Journal of the ACM*. Proves that beating $\ln n$ is as hard as solving SAT in sub-exponential time, using the PCP theorem.
- Lawler (1979). “Fast approximation algorithms for knapsack problems.” *Mathematics of Operations Research*. The most efficient FPTAS for knapsack: running time $O(n \log(1/\varepsilon) + 1/\varepsilon^4)$.
- Arora, Lund, Motwani, Sudan, Szegedy (1998). “Proof verification and the hardness of approximation problems.” *Journal of the ACM*. The PCP theorem — the foundation for most inapproximability results.

11.10. Try it yourself

1. Integrality gap for a triangle. Write down the vertex cover LP for the triangle graph K_3 . Find the optimal fractional solution and the optimal integer solution. What is the integrality gap?
2. Dual fitting for vertex cover. We proved vertex cover's 2-approximation via pure primal-dual. Re-prove it using dual fitting: set $y_e = c/(k + 1)$ where c is some cost and k is some quantity. What scaling factor do you need to restore dual feasibility?
3. Set cover gap instance. For $q = 3$ (so $n = 7$), explicitly list all the sets S_α in the $\mathbb{F}_2^q$ integrality gap construction. Verify that each element is in exactly 4 of the 8 sets, and find a minimum set cover.
4. Greedy vs. modified greedy. Construct a knapsack instance with $n = 3$ items where greedy (sorted by v_i/w_i , take until full) achieves less than $0.6 \cdot \text{OPT}$, but modified greedy achieves at least $0.5 \cdot \text{OPT}$.
5. Implement the PTAS. Using the PTAS code above, solve the following instance with $\varepsilon = 0.25$: $W = 50$, items $= [(v = 60, w = 10), (v = 100, w = 20), (v = 120, w = 30), (v = 40, w = 15)]$. Verify that your answer is within $(1 - \varepsilon) \cdot \text{OPT}$.

12. FPTAS, DNF Counting, and Semidefinite Programming

COMPSCI 224 Lecture 12 — Fall 2014

Three distinct ideas, unified by the theme “relax and round.” An FPTAS for knapsack rounds item values to shrink the DP table, losing at most $\varepsilon \cdot \text{OPT}$. An FPRAS for DNF counting (Karp–Luby–Madras, 1989) sidesteps the difficulty of sampling rare satisfying assignments by inflating the sample space with a disjoint union trick that guarantees the hit probability is at least $1/m$. Semidefinite programming generalises LP by replacing non-negativity with positive-semidefiniteness; the Goemans–Williamson algorithm (1995) uses an SDP relaxation and random hyperplane rounding to cut at least $0.878 \cdot \text{OPT}$ edges in Max-Cut — shattering the $1/2$ barrier that had stood for decades.

12.1. Why this lecture matters

The FPTAS for knapsack is the canonical example of how rounding input data can make an exponential DP polynomial — a trick that shows up everywhere from streaming algorithms to compiler optimization. DNF counting introduces the idea of *importance sampling* against a disjoint union, a technique that recurs in randomized algorithms for counting and sampling. And the Goemans–Williamson Max-Cut algorithm opened the era of SDP-based approximation, inspiring dozens of results in combinatorial optimization.

The 30-second story

1. FPTAS for knapsack: scale and round item values so the DP table has $O(n^2/\varepsilon)$ entries. The rounding error per item is at most 1 (in the new scale), and there are at most n items, so total loss is $\leq \varepsilon \cdot \text{OPT}$.
2. Strongly NP-hard problems like bin packing *cannot* have an FPTAS (unless $P = NP$), because their optimal value is a small integer — any multiplicative $(1 + \varepsilon)$ approximation with ε small enough would be exact.
3. FPRAS for DNF counting (Karp–Luby–Madras): build a disjoint union B' over all clause-assignment pairs, guaranteeing hit probability $\geq 1/m$. Sample from B' , check if the assignment’s first satisfied clause matches — Chernoff gives $O(m/\varepsilon^2)$ samples suffice.
4. SDP generalises LP: variables are matrix entries, non-negativity becomes positive-semidefiniteness. Equivalently: “vector programming” where variables are dot products $v_i \cdot v_j$.
5. Goemans–Williamson for Max-Cut: solve the SDP relaxation, round by picking a random hyperplane. Each edge is cut with probability θ_{ij}/π , yielding a 0.878 -approximation.

12.2. FPTAS for knapsack

12.2.1. The rounding idea

Recall the knapsack problem: n items with weights w_i and values v_i , knapsack capacity W . Last lecture we saw two pseudo-polynomial DPs:

- $O(nW)$: tabulate by remaining capacity.
- $O(nV)$: tabulate by target value, where $V = \sum v_i$.

The $O(nV)$ DP is fast when values are small. The FPTAS *makes* values small by rescaling and rounding.

12.2.2. The algorithm

Let $v_{\max} = \max_i v_i$. Define new values:

$$v'_i = \left\lfloor \frac{n}{\varepsilon} \cdot \frac{v_i}{v_{\max}} \right\rfloor. \quad (12.1)$$

Run the exact $O(nV')$ DP on the instance with values v'_i (keeping original weights). Let A be the set of items returned.

Running time. Each $v'_i \leq n/\varepsilon$, so $V' = \sum v'_i \leq n^2/\varepsilon$. The DP runs in $O(n \cdot V') = O(n^3/\varepsilon)$.

```
import math

def knapsack_fptas(weights, values, W, epsilon):
    n = len(weights)
    v_max = max(values)
    alpha = n / (epsilon * v_max)

    # Round values
    v_prime = [math.floor(alpha * v) for v in values]
    V_prime = sum(v_prime)

    # DP by value: g[i][p] = min weight to achieve value exactly p
    # using items 0..i-1
    INF = float('inf')
    g = [[INF] * (V_prime + 1) for _ in range(n + 1)]
    g[0][0] = 0

    for i in range(1, n + 1):
        for p in range(V_prime + 1):
            g[i][p] = g[i-1][p] # don't take item i
            if p >= v_prime[i-1] and g[i-1][p - v_prime[i-1]] + weights[i-1] <= W:
                g[i][p] = min(g[i][p], g[i-1][p - v_prime[i-1]] + weights[i-1])

    # Find maximum achievable value
    best_p = max(p for p in range(V_prime + 1) if g[n][p] <= W)

    # Backtrack to find the set A
    A = []
    p = best_p
    for i in range(n, 0, -1):
        if g[i][p] != g[i-1][p]:
            A.append(i - 1)
            p -= v_prime[i-1]

    return sum(values[j] for j in A)
```

12.2.3. Correctness proof

The scaling factor is $\alpha = n/(\varepsilon \cdot v_{\max})$. For any set $S \subseteq \{1, \dots, n\}$, the rounded value relates to the original:

$$\alpha \cdot v(S) - |S| \leq v'(S) \leq \alpha \cdot v(S). \quad (12.2)$$

The left inequality holds because each floor loses at most 1. The right inequality is immediate.

Let A^* be the optimal set for the original problem. Then:

$$v(A) \geq \frac{1}{\alpha} \cdot v'(A) \geq \frac{1}{\alpha} \cdot v'(A^*) \geq \frac{1}{\alpha} (\alpha \cdot v(A^*) - |A^*|) \geq \text{OPT} - \frac{n}{\alpha} = \text{OPT} - \varepsilon \cdot v_{\max}.$$

Since $v_{\max} \leq \text{OPT}$:

$$v(A) \geq (1 - \varepsilon) \cdot \text{OPT}.$$

Tip

The core insight: multiplying all values by a constant doesn't change the optimal *set* — only rounding does. Rounding loses at most 1 per item (in the new scale), and n items $\times$ 1 unit each = n total loss, which is at most ε times the (rescaled) optimum.

12.3. When FPTAS is impossible: strongly NP-hard problems

Not every NP-hard problem has an FPTAS. A problem is strongly NP-hard if it remains NP-hard even when all numbers in the input are polynomial in the input size.

12.3.1. Bin packing

Input. n items with sizes $s_1, \dots, s_n \in (0, 1]$. Bins have capacity 1. Goal. Pack all items into as few bins as possible. No splitting.

The decision problem “can these items fit in 2 bins?” is already NP-complete (it reduces from partition). This kills any FPTAS:

No FPTAS for bin packing

An FPTAS with $\varepsilon \ll 1/n$ would give a packing using at most $(1 + \varepsilon) \cdot \text{OPT}$ bins. Since the number of bins is an integer, this would be *exactly* OPT bins — solving NP-complete problems in polynomial time.

Aside (mine, not the lecturer's)

Bin packing *does* admit an asymptotic PTAS: de la Vega and Lueker (1981) showed how to achieve $(1 + \varepsilon) \cdot \text{OPT} + O(1/\varepsilon^2)$ bins. The additive constant matters — it prevents the argument above from giving an exact algorithm.

12.4. FPRAS for DNF counting

12.4.1. The problem

A DNF formula is an OR of clauses, where each clause is an AND of literals:

$$\phi = C_1 \vee C_2 \vee \cdots \vee C_m, \quad C_i = (\ell_{i,1} \wedge \ell_{i,2} \wedge \cdots \wedge \ell_{i,r_i}).$$

DNF satisfiability is easy: check each clause independently. But DNF counting — how many of the 2^n assignments satisfy ϕ ? — is #P-complete. The proof: negate a CNF formula to get a DNF; the number of satisfying assignments of the DNF tells you the number of satisfying assignments of the original CNF (it's 2^n minus that count).

Warning

Exact counting is #P-hard, but approximate counting might still be tractable. The proof of hardness relies on distinguishing 0 from nonzero — any multiplicative $(1 \pm \varepsilon)$ approximation cannot make that distinction.

12.4.2. Why naive sampling fails

Let p be the fraction of 2^n assignments that satisfy ϕ . The obvious algorithm: sample random assignments, check each, estimate p by the empirical fraction. By Chernoff, $O(1/(p\varepsilon^2))$ samples suffice for $(1 \pm \varepsilon)$ accuracy.

The problem: p can be as small as 2^{-n} (e.g., $\phi = x_1 \wedge x_2 \wedge \cdots \wedge x_n$). Then you need $\Omega(2^n)$ samples.

12.4.3. The Karp–Luby–Madras trick

The idea: build a larger sample space where the “hit probability” is at least $1/m$.

Step 1: Define the inflated set B' .

Let S_i be the set of assignments satisfying clause C_i . The set we want to estimate is $B = \bigcup_{i=1}^m S_i$.

Define the disjoint union:

$$B' = \bigsqcup_{i=1}^m S_i = \{(i, x) : x \text{ satisfies } C_i\}. \quad (12.3)$$

Each satisfying assignment $x \in B$ appears in B' once for every clause it satisfies. So $|B'| \geq |B|$ and $|B'| \leq m \cdot |B|$.

Step 2: Compute $|B'|$ exactly. Each $|S_i| = 2^{n-r_i}$ where r_i is the number of literals in clause C_i . So $|B'| = \sum_i 2^{n-r_i}$ — computable in $O(m)$ time.

Step 3: Estimate $p' = |B|/|B'|$. Since each element of B appears at most m times in B' :

$$p' = \frac{|B|}{|B'|} \geq \frac{1}{m}. \quad (12.4)$$

This is the key: p' is not too small.

Step 4: Sample from B' . To sample a uniformly random element $(i, x) \in B'$:

1. Pick a clause C_i with probability $|S_i|/|B'|$ (proportional to its solution count).

2. Pick a random satisfying assignment for C_i : set the r_i constrained variables as required, set the remaining $n - r_i$ variables uniformly at random.

Step 5: Check membership in B . Given a sampled pair (i, x) , check whether i is the *first* clause that x satisfies. If yes, count it as a “hit.” This implements the bijection $B \leftrightarrow \{(i, x) \in B' : i = \min\{j : x \in S_j\}\}$.

12.4.4. The full algorithm

1. Sample T elements from B' .
2. Let $\tilde{p}'$ = fraction of samples that are “hits” (first-clause matches).
3. Output $\tilde{p}' \cdot |B'|$.

```
import random

def dnf_count_fpras(clauses, n, epsilon, delta=1/3):
    """
    Estimate number of satisfying assignments for a DNF.
    clauses: list of lists, each inner list contains signed variable indices
             (positive = true, negative = negated).
    n: number of variables.
    """
    m = len(clauses)

    # Compute |S_i| for each clause
    sizes = [2 ** (n - len(c)) for c in clauses]
    B_prime_size = sum(sizes)

    # Number of samples needed: O(m / (epsilon^2) * log(1/delta))
    import math
    T = int(3 * m * math.log(2 / delta) / epsilon ** 2) + 1

    hits = 0
    for _ in range(T):
        # Step 1: pick clause i proportional to |S_i|
        r = random.random() * B_prime_size
        cumulative = 0
        chosen_clause = 0
        for i, s in enumerate(sizes):
            cumulative += s
            if r < cumulative:
                chosen_clause = i
                break

        # Step 2: pick random satisfying assignment for clause i
        assignment = [random.choice([True, False]) for _ in range(n)]
        for lit in clauses[chosen_clause]:
            var = abs(lit) - 1
            assignment[var] = (lit > 0)

        # Step 3: check if chosen_clause is the FIRST satisfied clause
        first = None
        for j, clause in enumerate(clauses):
```

```

        if all(assignment[abs(lit)-1] == (lit > 0) for lit in clause):
            first = j
            break
    if first == chosen_clause:
        hits += 1

p_prime_estimate = hits / T
return p_prime_estimate * B_prime_size

```

12.4.5. Analysis via Chernoff

Let X_j be the indicator that sample j is a hit. Then $\mathbb{E}[X] = p'T$ where $X = \sum X_j$.

The algorithm fails when $|X - p'T| > \varepsilon \cdot p'T$. By the Chernoff bound:

$$\Pr[|X - \mathbb{E}[X]| > \varepsilon \cdot \mathbb{E}[X]] \leq 2 \exp\left(-\frac{\varepsilon^2 \cdot p'T}{3}\right).$$

This is at most δ when:

$$T \geq \frac{3 \ln(2/\delta)}{\varepsilon^2 \cdot p'} \leq \frac{3m \ln(2/\delta)}{\varepsilon^2}. \quad (12.5)$$

For $\delta = 1/3$: $T = O(m/\varepsilon^2)$. Each sample takes $O(nm)$ time to check, so the total is $O(nm^2/\varepsilon^2)$ — polynomial in n , m , and $1/\varepsilon$.

12.5. Semidefinite programming

12.5.1. From LP to SDP

In linear programming, the variable is a vector $x \in \mathbb{R}^n$ with a non-negativity constraint $x \geq 0$. In semidefinite programming, the variable is a symmetric matrix $X \in \mathbb{R}^{n \times n}$ with a positive-semidefiniteness constraint $X \geq 0$:

$$\min \text{tr}(C^\top X) \quad \text{s.t.} \quad \text{tr}(A_i^\top X) = b_i \quad \forall i, \quad X \geq 0. \quad (12.6)$$

The trace inner product $\text{tr}(A^\top B) = \sum_{i,j} A_{ij} B_{ij}$ is just the dot product of A and B viewed as n^2 -dimensional vectors. So an SDP is an LP over n^2 variables — plus the PSD constraint.

Tip

LP as a special case of SDP. Take C , A_i , and X to be diagonal matrices. Then $X \geq 0$ means all diagonal entries are non-negative (= the eigenvalues), and $\text{tr}(C^\top X) = c^\top x$. Ordinary LP falls out as a special case.

12.5.2. Positive semidefiniteness: three views

For a symmetric matrix $X \in \mathbb{R}^{n \times n}$, the following are equivalent:

1. Quadratic form. $y^\top X y \geq 0$ for all $y \in \mathbb{R}^n$.
2. Eigenvalues. All eigenvalues of X are non-negative (spectral theorem).
3. Gram matrix. $X = M^\top M$ for some real matrix M . Equivalently, $X_{ij} = v_i \cdot v_j$ where $v_1, \dots, v_n$ are the columns of M .

The third view is the most useful for approximation algorithms: it says optimizing over PSD matrices is the same as optimizing over *dot products of vectors*.

12.5.3. Vector programming

Rewrite the SDP using the Gram matrix decomposition $X_{ij} = v_i \cdot v_j$:

$$\min \sum_{i,j} C_{ij} v_i \cdot v_j \quad \text{s.t.} \quad \sum_{i,j} (A_k)_{ij} v_i \cdot v_j = b_k \quad \forall k. \quad (12.7)$$

No explicit PSD constraint — it is built into the structure. The variables are now vectors $v_1, \dots, v_n \in \mathbb{R}^n$, and every constraint is linear in the dot products.

Note

SDPs can be solved in polynomial time up to ℓ bits of precision in time $\text{poly}(n, m, \ell)$. The standard reference is Boyd and Vandenberghe, *Convex Optimization* (2004). For this lecture, treat the SDP solver as a black box.

12.6. Max-Cut and the Goemans–Williamson algorithm

12.6.1. The problem

Given an undirected graph $G = (V, E)$, find a partition $(S, V \setminus S)$ that maximizes the number of edges crossing the cut. Max-Cut is NP-hard.

12.6.2. The 1/2 barrier

Two simple algorithms both achieve a 1/2-approximation:

Random partition. For each vertex, flip a coin — left or right. Every edge crosses with probability 1/2, so the expected cut is $|E|/2 \geq \text{OPT}/2$.

Local search. Start with any partition. While some vertex has more neighbors on its own side than the other, swap it. This terminates (cut size strictly increases) and at termination, every vertex has $\geq$ half its neighbors across the cut, so the cut is $\geq |E|/2$.

For decades, no polynomial-time algorithm beat 1/2. Then Goemans and Williamson used SDP.

12.6.3. Quadratic program formulation

Assign $x_i \in \{-1, +1\}$ to each vertex ($-1 = \text{left}$, $+1 = \text{right}$). The number of crossing edges is:

$$\sum_{(i,j) \in E} \frac{1 - x_i x_j}{2}. \quad (12.8)$$

If $x_i = x_j$ (same side), the term is 0. If $x_i \neq x_j$ (opposite sides), the term is 1.

12.6.4. SDP relaxation

Replace each scalar $x_i \in \{-1, +1\}$ with a unit vector $v_i \in \mathbb{R}^n$. Replace multiplication $x_i x_j$ with dot product $v_i \cdot v_j$:

$$\max \sum_{(i,j) \in E} \frac{1 - v_i \cdot v_j}{2} \quad \text{s.t.} \quad v_i \cdot v_i = 1 \quad \forall i. \quad (12.9)$$

This is a vector program (= SDP) and can be solved in polynomial time. The constraint $v_i \cdot v_i = 1$ says each vector lies on the unit sphere.

12.6.5. Rounding: random hyperplane

The Goemans–Williamson algorithm:

1. Solve the SDP to get unit vectors $v_1, \dots, v_n$.
2. Pick a random vector $r \in \mathbb{R}^n$ (e.g., each coordinate i.i.d. standard Gaussian).
3. Put vertex i in S if $r \cdot v_i \geq 0$, otherwise in $V \setminus S$.

12.6.6. Analysis

For an edge (i, j) , the probability it is cut equals the probability that r separates v_i and v_j . Since v_i and v_j are unit vectors with angle θ_{ij} between them, the random hyperplane (normal to r) separates them with probability:

$$\Pr[\text{edge } (i, j) \text{ is cut}] = \frac{\theta_{ij}}{\pi}. \quad (12.10)$$

The SDP objective for this edge is $(1 - v_i \cdot v_j)/2 = (1 - \cos \theta_{ij})/2$. So the ratio of the algorithm's expected gain to the SDP's value on edge (i, j) is:

$$\frac{\theta_{ij}/\pi}{(1 - \cos \theta_{ij})/2} = \frac{2\theta_{ij}}{\pi(1 - \cos \theta_{ij})}. \quad (12.11)$$

! The Goemans–Williamson guarantee

The minimum of Equation 12.11 over $\theta \in (0, \pi]$ is:

$$\alpha_{\text{GW}} = \min_{0 < \theta \leq \pi} \frac{2\theta}{\pi(1 - \cos \theta)} \approx 0.878.$$

Therefore $\mathbb{E}[\text{cut}] \geq \alpha_{\text{GW}} \cdot \text{SDP-OPT} \geq 0.878 \cdot \text{OPT}$.

```
import numpy as np

def goemans_williamson_round(vectors):
    """
    Given unit vectors v_1, ..., v_n (rows of 'vectors'),
    perform random hyperplane rounding.
    Returns: set S (list of vertex indices on one side of the cut).
    """
    n, d = vectors.shape
    r = np.random.randn(d)          # random hyperplane normal
    projections = vectors @ r        # dot products r · v_i
    S = [i for i in range(n) if projections[i] >= 0]
    return S
```

i Aside (mine, not the lecturer's)

Khot, Kindler, Mossel, and O'Donnell (2007) showed that under the Unique Games Conjecture, $\alpha_{\text{GW}} \approx 0.878$ is the best possible approximation ratio for Max-Cut. So the GW algorithm may be *optimal* — not just among SDP-based methods, but among all polynomial-time algorithms.

12.7. Cheat sheet

! Cheat sheet — Lecture 12

FPTAS for knapsack. Scale values: $v'_i = \lfloor (n/\varepsilon) \cdot v_i / v_{\max} \rfloor$. Run $O(nV')$ DP. Rounding loses ≤ 1 per item $\Rightarrow \leq n$ total $\leq \varepsilon \cdot \text{OPT}'$. Time: $O(n^3/\varepsilon)$.

Strongly NP-hard. No FPTAS unless $P = NP$. Test: if OPT is a small integer, any $(1 + \varepsilon)$ -approximation with $\varepsilon < 1/\text{OPT}$ would be exact. Example: bin packing (even deciding if 2 bins suffice is NP-complete).

FPRAS for DNF counting (KLM).

- $B = \bigcup S_i$ (satisfying assignments), $B' = \bigsqcup S_i$ (disjoint union, each x appears once per satisfied clause).
- $|B'|$ is easy to compute: $\sum 2^{n-r_i}$.
- Sample from B' : pick clause $i \propto |S_i|$, random satisfying assignment.
- Check: is i the *first* clause x satisfies? Hit probability $p' \geq 1/m$.
- Samples needed: $T = O(m \log(1/\delta)/\varepsilon^2)$ by Chernoff.

SDP = Vector programming.

LP	SDP
Variables: $x_i \in \mathbb{R}$	Variables: X_{ij} (or vectors v_i)
Constraint: $x \geq 0$	Constraint: $X \succeq 0$
Objective: $c^\top x$	Objective: $\text{tr}(C^\top X)$

Max-Cut (Goemans–Williamson). Relax $x_i \in \{-1, +1\}$ to $v_i \in \mathbb{R}^n$, $\|v_i\| = 1$. Solve SDP. Round: random hyperplane r , put i in S iff $r \cdot v_i \geq 0$. Cut probability per edge: θ_{ij}/π . Ratio: $\alpha_{\text{GW}} = \min_{\theta} \frac{2\theta}{\pi(1-\cos \theta)} \approx 0.878$.

12.8. FAQ — questions students actually asked

Q: Can knapsack be solved exactly in polynomial time using the FPTAS with very small ε ? No. If all values are integers, OPT is an integer, so setting $\varepsilon < 1/(n \cdot v_{\max})$ would give an exact answer. But then $1/\varepsilon > n \cdot v_{\max}$, and the running time $\text{poly}(n, 1/\varepsilon)$ becomes $\text{poly}(n, n \cdot v_{\max})$ — pseudo-polynomial, not truly polynomial. Writing $v_{\max}$ takes only $\log v_{\max}$ bits.

Q: Is there a deterministic FPTAS for DNF counting? This is an open question. The best known deterministic algorithms for approximate counting problems related to #P are significantly more complex and often require additional structural assumptions.

Q: For the KLM algorithm, why can't you just sample a random assignment and check all clauses? You can, but if the fraction p of satisfying assignments is exponentially small (e.g., $\phi = x_1 \wedge \cdots \wedge x_n$), you need exponentially many samples before you ever see a satisfying one. The disjoint-union trick ensures the hit probability is at least $1/m$, so $O(m/\varepsilon^2)$ samples suffice.

Q: Why is p' at least $1/m$? Every satisfying assignment $x \in B$ appears in B' at least once (for some clause) and at most m times (once per clause). So $|B'| \leq m \cdot |B|$, hence $p' = |B|/|B'| \geq 1/m$.

Q: In the SDP relaxation of Max-Cut, why are the vectors n -dimensional? The Gram matrix X is $n \times n$, so $X = M^T M$ where M has n columns — each column is a vector $v_i \in \mathbb{R}^n$. In practice, the SDP optimum often has much lower rank, so fewer dimensions suffice.

Q: What is the probability that r separates v_i and v_j ? The vectors v_i and v_j span a 2D plane. In that plane, they make angle θ_{ij} . The random hyperplane's normal r is rotationally symmetric, so the hyperplane is equally likely to point in any direction. The fraction of directions that separate v_i from v_j is exactly θ_{ij}/π (the angle subtended, divided by the half-circle).

12.9. Further reading

- Lawler (1979). “Fast approximation algorithms for knapsack problems.” *Mathematics of Operations Research*. Gives an FPTAS with running time $O(n \log(1/\varepsilon) + 1/\varepsilon^4)$.
- Karp, Luby, and Madras (1989). “Monte-Carlo approximation algorithms for enumeration problems.” *Journal of Algorithms*. The FPRAS for DNF counting presented in this lecture.
- Goemans and Williamson (1995). “Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming.” *Journal of the ACM*. The 0.878-approximation for Max-Cut that launched the SDP approximation era.
- Boyd and Vandenberghe (2004). *Convex Optimization*. Cambridge University Press. The standard reference for SDP and its algorithms (interior-point methods).
- Khot, Kindler, Mossel, and O'Donnell (2007). “Optimal inapproximability results for MAX-CUT and other 2-variable CSPs?” *SIAM Journal on Computing*. Shows that α_{GW} is optimal under the Unique Games Conjecture.

12.10. Try it yourself

1. FPTAS by hand. Consider knapsack with $W = 10$, items: $(v = 6, w = 2)$, $(v = 10, w = 4)$, $(v = 12, w = 6)$, $(v = 4, w = 3)$. Apply the FPTAS with $\varepsilon = 0.5$. Compute the rounded values v'_i , solve the DP on rounded values, and verify the result is within $(1 - \varepsilon) \cdot \text{OPT}$.
2. DNF counting. Consider the DNF $\phi = (x_1 \wedge x_2) \vee (x_2 \wedge x_3) \vee (x_1 \wedge x_3)$ over 3 variables. Enumerate all $2^3 = 8$ assignments to find the exact count. Then compute $|B'|$ and p' for the KLM algorithm. Verify that $p' \geq 1/m$.

3. Max-Cut lower bound. For the cycle graph C_5 (5-vertex cycle), find the maximum cut by brute force. Then write the SDP relaxation and argue that the SDP optimum is at least as large. What vectors would the SDP choose?
4. SDP as LP. Write out the SDP for Max-Cut on the triangle K_3 explicitly as a 3×3 matrix optimization problem. What are the constraints? Solve it by symmetry.
5. Implement GW rounding. Given vectors $v_1 = (1, 0)$, $v_2 = (-\cos 60^\circ, \sin 60^\circ)$, $v_3 = (-\cos 60^\circ, -\sin 60^\circ)$ on the unit circle, and edges forming K_3 : write code to perform 10,000 random hyperplane roundings (in 2D) and estimate the expected cut size. Compare with $\alpha_{\text{GW}} \cdot \text{SDP-OPT}$.

13. Learning Topic Models

COMPSCI 224 Lecture 13 — Fall 2014

Topic models discover latent semantic structure in text corpora by treating documents as mixtures of probability distributions over words. Learning such models is equivalent to non-negative matrix factorization, which is NP-hard in general. Under a natural *separability* assumption — every topic has at least one “anchor word” unique to it — the problem reduces to finding vertices of a convex hull, solvable in polynomial time via linear programming. This lecture, a guest talk by Rong Ge, traces that path from unsupervised learning to geometry.

13.1. Why this lecture matters

Every time you use a recommendation engine, a document tagger, or a search filter that groups articles by theme, a topic model (or its descendant) is doing the work underneath. The conceptual arc here — probabilistic model → NMF → geometry — is a template you will see again whenever an NP-hard learning problem is tamed by a structural assumption. The algorithm itself is clean enough to implement and fast enough to run on real corpora.

The 30-second story

1. Unsupervised learning seeks latent structure (clusters, topics) without labels; the structure is specified by a probabilistic model with unknown parameters.
2. A topic model represents each topic as a distribution over words, and each document as a distribution over topics; the key parameter is the $n \times k$ topic matrix A .
3. Learning the model from observed documents reduces to non-negative matrix factorization (NMF): $M = AW$ with $A, W \geq 0$.
4. NMF is NP-hard in general, but under the separability assumption — each topic has an anchor word — the problem becomes geometric: find the vertices of a convex hull.
5. Vertex-finding is solvable in polynomial time by checking convex hull membership with a linear program.

13.2. Supervised vs. unsupervised learning

In supervised learning, you receive data points *and* labels (e.g., emails marked spam / not-spam) and train a function to predict labels on new data. A spam filter is the canonical example: you are given thousands of emails, each tagged spam or not-spam, and the goal is to learn a classifier that generalises to unseen emails. That setting has decades of beautiful theory — computational learning theory, VC dimension, PAC learning — but it is not today’s topic.

In unsupervised learning, you receive data points x alone — no labels, no teacher, no ground truth. The goal is to discover *structure* hiding in the data: clusters, low-rank representations, latent factors. Think of a pile of unlabelled documents: you cannot ask a human to tag each one, but you hope the data itself reveals patterns — groups of related articles, recurring themes, shared vocabulary.

The simplest unsupervised task is clustering: given a cloud of points, partition them into groups so that points within a group are similar and points across groups are different. But clustering is just one kind of structure. The broader question is: given data without labels, what can you learn at all?

 Tip

The core question of unsupervised learning: “I see data with no labels — what structure is there?” The answer depends on the *model* you assume generated the data.

13.3. Probabilistic models and learning

A probabilistic model is a function that maps a parameter θ to a data distribution $\mathcal{D}(\theta)$. Given θ , you can *generate* data by sampling from $\mathcal{D}(\theta)$. The *learning* problem is the inverse: given samples from $\mathcal{D}(\theta)$, recover (an approximation to) θ .

13.3.1. Warm-up: single Gaussian

The simplest example: $\theta = (\mu, \sigma^2)$ specifies a one-dimensional Gaussian. Given the parameters, generating data is trivial — sample from $\mathcal{N}(\mu, \sigma^2)$. The learning problem is the inverse: given n samples, estimate μ by the sample mean and σ^2 by the sample variance. The estimates converge to the true parameters as $n \rightarrow \infty$, and the rate of convergence is well understood.

This is straightforward — which is exactly why it motivates asking: what happens with a harder model?

13.3.2. Mixture of Gaussians

A mixture of Gaussians assumes each data point is drawn from one of k component Gaussians. The parameters are:

- Means $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^d$ (the cluster centres).
- Weights $w_1, w_2, \dots, w_k \geq 0$ with $\sum_i w_i = 1$ (the mixing proportions).
- For simplicity, assume all components share a known, spherical covariance matrix.

To generate a point: pick component i with probability w_i , then sample from $\mathcal{N}(\mu_i, \sigma^2 I)$. The learner sees only the samples — not which component each came from.

 Note

The learner has oracle access to the distribution: each query returns a fresh sample from $\mathcal{D}(\theta)$. The complexity question is how many samples (and how much time) are needed to approximate θ . Even d^k time is non-trivial; $\text{poly}(d, k)$ time is the gold standard.

13.4. Maximum likelihood and NP-hardness

The standard approach: find the parameter θ that maximises the probability of the observed data.

$$\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(v_i) \quad (13.1)$$

where p_θ is the density of $\mathcal{D}(\theta)$.

For a single Gaussian, Equation 13.1 yields the sample mean and variance — easy. For a mixture of Gaussians, the problem is NP-hard (it is at least as hard as k -means, which is itself NP-hard). The intuition for why: the log-likelihood for a mixture involves a $\log \sum$ (log of a sum of Gaussian densities), which does not decompose cleanly. Optimising it requires simultaneously figuring out which component generated each point *and* estimating the component parameters — a chicken-and-egg problem.

Warning

NP-hardness here refers to the worst case — the input data points can be adversarially chosen. When the data genuinely come from the model ($v_i \sim \mathcal{D}(\theta)$), the problem may be easier. This distinction between worst-case and average-case is central to the rest of the lecture.

13.4.1. Ways around NP-hardness

Strategy	Idea
Approximation	Accept a constant-factor approximation. But a constant factor in log-probability space does not correspond to a constant factor in probability space — so the recovered θ can be wildly wrong.
Average-case analysis	The data really do come from $\mathcal{D}(\theta)$, so worst-case hardness may not apply.
Natural assumptions	Exploit structural properties of the domain (e.g., topic-word relationships) to sidestep hardness.
Heuristics	Use EM or gradient methods and hope they work. Even better: analyse the heuristic under natural assumptions.

Important

These strategies are not mutually exclusive. The ideal outcome: a practical algorithm that also has provable guarantees under realistic assumptions. Today's lecture gives exactly that for topic models.

13.5. Topic models: what and why

Goal. Learn semantic structure from a large text corpus automatically — which topics exist, what words characterise each topic, and how each document mixes topics.

A topic model is a probabilistic model for document generation. Two foundational simplifying assumptions make the problem tractable.

13.5.1. Assumption 1: bag of words

The *order* of words in a document does not matter — only the word frequencies. Permute the words of a headline and you can still guess the topic:

13. Learning Topic Models

preparations, DU, hotels, rise, Ebola ? clearly a health/travel article.

This is unrealistic for syntax but reasonable for semantics, and it removes the need to model grammar.

13.5.2. Assumption 2: a topic is a distribution over words

A topic is a probability distribution over the vocabulary of n words. If a word comes from the “food” topic, it is “banana” with probability 0.01, “recipe” with probability 0.008, etc. Conditioned on the topic, words are drawn independently.

13.5.3. Assumption 3: a document is a distribution over topics

Each document has a topic proportion vector $\mathbf{w} = (w_1, \dots, w_k)$ with $w_i \geq 0$ and $\sum w_i = 1$. A document about 80% politics and 20% finance will draw words from the politics distribution 80% of the time.

Note

Early models like pLSI (Hofmann, 1999) assigned each document a *single* topic. Latent Dirichlet Allocation (Blei, Ng, Jordan, 2003) introduced a Dirichlet prior over topic proportions, allowing documents to mix multiple topics. The Dirichlet prior encourages *sparse* mixtures — most documents discuss only a few topics — which agrees with intuition.

13.6. The topic matrix and document generation

All topic-word distributions are collected into a single topic matrix $A \in \mathbb{R}^{n \times k}$:

- Rows index words (n = vocabulary size, often tens of thousands).
- Columns index topics (k = number of topics, often 100–200).
- Entry $A_{v,j}$ = probability that word v appears, given topic j .
- Each column is a probability distribution: $A_{v,j} \geq 0$ and $\sum_v A_{v,j} = 1$.

13.6.1. The generative recipe

To generate a document:

1. Pick a length ℓ (treat as fixed and known).
2. Pick a topic proportion vector $\mathbf{w} \in \mathbb{R}^k$ (this step differs across models — pLSI, LDA, etc.).
3. For each word position $i = 1, \dots, \ell$:
 - a. Draw a topic $t_i \in \{1, \dots, k\}$ according to $\mathbf{w}$.
 - b. Draw a word $z_i \in \{1, \dots, n\}$ according to column $A_{:,t_i}$.

Warning

This lecture treats the topic proportion vectors $\mathbf{w}$ as arbitrary parameters (not drawn from a Dirichlet). The algorithm works regardless of how $\mathbf{w}$ is chosen, as long as a separability condition holds on A .

13.7. From topic models to matrix factorisation

Assume documents are long enough that word frequencies converge to their true probabilities. For document d with topic proportions $w^{(d)}$:

$$\Pr[\text{word } v \mid \text{document } d] = \sum_{j=1}^k \Pr[\text{topic } j \mid d] \cdot \Pr[\text{word } v \mid \text{topic } j] = \sum_{j=1}^k w_j^{(d)} \cdot A_{v,j} \quad (13.2)$$

Stack these probabilities into an $n \times m$ document-word matrix M (columns = documents, rows = words). Then Equation 13.2 in matrix form is:

$$M = A W \quad (13.3)$$

where $A \in \mathbb{R}_{\geq 0}^{n \times k}$ and $W \in \mathbb{R}_{\geq 0}^{k \times m}$. This is precisely the non-negative matrix factorisation (NMF) problem: given M , find A and W with non-negative entries such that $M = AW$.

! Why non-negativity matters

Without the non-negativity constraint, factoring M into two smaller matrices is just rank decomposition — solvable via SVD in polynomial time. But SVD factors can have negative entries, which cannot be interpreted as probabilities. Non-negativity is the constraint that makes NMF NP-hard (Vavasis, 2009) — and the one we must keep.

13.8. The separability assumption

To sidestep NP-hardness, impose a structural assumption on the topic matrix A .

! Definition: separability (Donoho & Stodden, 2003)

An $n \times k$ non-negative matrix A is separable if, for every column $i \in \{1, \dots, k\}$, there exists a row a_i such that:

1. $A_{a_i,i} > 0$ (positive entry in column i), and
2. $A_{a_i,j} = 0$ for all $j \neq i$ (zero in every other column).

Equivalently, after permuting rows, A has a $k \times k$ diagonal submatrix on top (with positive diagonal), followed by an arbitrary non-negative block below.

13.8.1. Translation to topic modelling

- A column = a topic.
- A row = a word.
- Separability means: for each topic, there exists at least one anchor word that appears *only* in that topic.

For example, “polar vortex” might be an anchor for the weather topic — it has near-zero probability under politics, food, or sports. The word “snow” is less ideal: it appears in weather articles but also in sports coverage (snow games in football) and lifestyle pieces. The assumption asks for at least one word per topic that is truly exclusive.

 Tip

Separability does not require most words to be anchors. It requires exactly *one* anchor per topic. The vast majority of the vocabulary can — and does — appear in multiple topics.

Why is this assumption helpful? If you see an anchor word in a document, you know with certainty that the document has non-zero weight on the corresponding topic. Anchor words act as signatures: they reveal which topics are present. The algorithm exploits this by first finding the anchor words (the vertices), then using them to decompose every other word's distribution.

13.9. Geometric reduction: convex hull vertex-finding

Separability transforms the algebraic NMF problem into a geometric one.

13.9.1. Rescaling to convex combinations

Rescale each row of M so that it sums to 1. Apply the corresponding normalisation to A and W . Now the rows of W also sum to 1, and for each anchor row a_i :

$$M_{a_i} = W_i \quad (13.4)$$

(the scaling constant becomes 1). For any *non-anchor* row j :

$$M_j = \sum_{i=1}^k A_{j,i} W_i \quad (13.5)$$

Since $A_{j,i} \geq 0$ and $\sum_i A_{j,i} = 1$ after normalisation, every row of M is a convex combination of the rows of W . But the anchor rows *are* the rows of W . So:

 Important

The rows of M are points in $\mathbb{R}^m$. The anchor-word rows are the vertices of their convex hull. Learning the topic model reduces to finding the vertices of a set of points.

13.10. The vertex-finding algorithm

Problem. Given n points $v_1, \dots, v_n$ in $\mathbb{R}^m$ that lie in the convex hull of k unknown vertices $v_{a_1}, \dots, v_{a_k}$ (which are among the n points), find the vertices.

Assumption. No vertex lies *inside* the convex hull of the others. (If it did, we could remove it and get a factorisation with smaller k — a degenerate case analogous to using rank $k + 1$ for a rank- k matrix.)

13.10.1. The algorithm

For each point v_i , test whether v_i is in the convex hull of the remaining points $\{v_j : j \neq i\}$.

Test: solve the linear program

$$\text{find } \alpha_j \geq 0 \quad \text{s.t.} \quad v_i = \sum_{j \neq i} \alpha_j v_j, \quad \sum_{j \neq i} \alpha_j = 1 \quad (13.6)$$

- If Equation 13.6 is feasible: v_i is *not* a vertex (it is an interior point).
- If Equation 13.6 is infeasible: v_i is a vertex.

After removing duplicate points, this correctly identifies all k vertices.

```
import numpy as np
from scipy.optimize import linprog

def find_vertices(V):
    """
    V: (n, m) array – rows are points.
    Returns indices of vertices of the convex hull.
    """
    n, m = V.shape
    vertices = []
    for i in range(n):
        # Build LP: find alpha >= 0, sum(alpha) = 1, V_others^T @ alpha = V[i]
        others = np.delete(V, i, axis=0)          # (n-1, m)
        n_others = others.shape[0]

        # Equality constraint: others^T @ alpha = V[i] and sum(alpha) = 1
        A_eq = np.vstack([others.T, np.ones((1, n_others))])
        b_eq = np.append(V[i], 1.0)

        c = np.zeros(n_others)                    # feasibility only
        bounds = [(0, None)] * n_others

        res = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')
        if not res.success:                        # infeasible → vertex
            vertices.append(i)
    return vertices
```

Warning

This presentation assumes exact arithmetic. Real data introduces noise: word frequencies are estimated from finite documents, so points may not lie exactly in a lower-dimensional subspace. The practical algorithm checks whether v_i is *close to* the convex hull (within a tolerance), which is still a linear program but requires a slightly different formulation.

13.11. Practical considerations

13.11.1. Finite documents

The reduction to NMF assumes documents are long enough that word frequencies approximate their true probabilities. For short documents, the probability estimates are noisy. The improved version of this algorithm does not estimate the full document-word matrix M directly. Instead, it estimates word co-occurrence probabilities — the probability that two words both appear in the same document — which can be estimated accurately with enough documents even when individual documents are short.

13.11.2. Computational cost

The naive vertex-finding algorithm solves n linear programs, each of size $O(n)$ variables and $O(m)$ constraints. For a vocabulary of $n = 50,000$ words, this is expensive but feasible. Improved algorithms (Arora, Ge, Moitra, 2012) reduce the cost significantly and scale to real corpora like the New York Times archive, producing results comparable to heuristic algorithms like variational inference for LDA — and running faster.

i Aside (mine, not the lecturer's)

Since this 2014 lecture, neural topic models (e.g., using variational autoencoders) have become popular. They drop the bag-of-words assumption and use word embeddings. Nevertheless, the separability/anchor-word framework remains theoretically important: it is one of the few settings where provably correct topic recovery is possible in polynomial time.

13.12. Cheat sheet

! Cheat sheet — Learning topic models

Setup. A topic model defines topics as distributions over n words, documents as distributions over k topics. The topic matrix $A \in \mathbb{R}_{\geq 0}^{n \times k}$ has columns summing to 1.

Key equation. The document-word matrix factors as $M = AW$, where $W \in \mathbb{R}_{\geq 0}^{k \times m}$ holds topic proportions for m documents. This is non-negative matrix factorisation (NMF).

Hardness. NMF is NP-hard in general (Vavasis, 2009). Maximum likelihood for mixtures of Gaussians is at least as hard as k -means (also NP-hard).

Separability. A is separable if every topic has an anchor word — a word appearing in that topic only. Under separability, after rescaling, the rows of M lie in the convex hull of the rows of W , and anchor-word rows are the vertices.

Algorithm. For each point, test convex hull membership via an LP. Infeasible $\Rightarrow$ vertex (anchor word). Feasible $\Rightarrow$ interior point. Recovers W , then A from $M = AW$.

Practical. Use word co-occurrence instead of direct frequency estimation. Improved algorithms (Arora, Ge, Moitra, 2012) run on large corpora, matching LDA quality with faster runtime.

13.13. FAQ — questions students actually asked

Q: When you say “polynomial number of samples,” polynomial in what?

You have oracle access to the distribution: each query returns a sample. If your algorithm runs in polynomial time, it automatically takes at most polynomially many samples. The complexity can be polynomial in d (dimension) and k (number of components). Even d^k time is non-trivial; the goal is $\text{poly}(d, k)$.

Q: Does the bag-of-words assumption mean each word is independently drawn?

Yes — conditioned on the topic assignment. Given a word's topic, its identity is drawn independently of all other words. Words within the same document are *not* marginally independent (they share the same topic proportion vector), but they are conditionally independent given their topic assignments.

Q: In the pLSI model, each document has only one topic?

Yes. The topic proportion vector w is a basis vector (one entry is 1, the rest 0), drawn with some prior probability π_i . Later models like LDA let w be a general distribution, allowing documents to mix topics. This better fits real data — most documents touch multiple themes.

Q: Can't we just use SVD instead of NMF?

SVD gives you a low-rank factorisation, but the factors can have negative entries. Negative entries cannot be interpreted as probabilities. Non-negativity is the essential constraint that makes the problem NP-hard — and meaningful.

Q: What if a topic doesn't have an anchor word?

Then the separability assumption fails for that topic, and this algorithm cannot identify it. In practice, most topics of reasonable granularity do have words that are highly specific to them. The assumption is empirically reasonable for corpora like news articles with 100–200 topics.

13.14. Further reading

- Arora, Ge, Halpern, Mimno, Moitra, Sontag, Wu, Zhu (2013). “A Practical Algorithm for Topic Modeling with Provable Guarantees.” *ICML 2013*. The fast, practical version of the anchor-word algorithm applied to real corpora.
- Arora, Ge, Moitra (2012). “Learning Topic Models — Going Beyond SVD.” *FOCS 2012*. The theoretical foundation: polynomial-time NMF under separability.
- Blei, Ng, Jordan (2003). “Latent Dirichlet Allocation.” *JMLR 3*: 993–1022. The foundational LDA paper that popularised mixed-membership topic models.
- Donoho, Stodden (2003). “When Does Non-Negative Matrix Factorization Give a Correct Decomposition into Parts?” *NeurIPS 2003*. Introduced the separability condition.
- Vavasis (2009). “On the Complexity of Nonnegative Matrix Factorization.” *SIAM J. Optim.* 20(3): 1364–1377. NP-hardness proof for NMF.

13.15. Try it yourself

1. Warm-up. Suppose $k = 2$ and the topic matrix is

$$A = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.3 \\ 0.5 & 0.7 \end{pmatrix}.$$

Verify that A is separable. Identify the anchor words and the non-anchor word. If a document has topic proportions $w = (0.6, 0.4)$, compute the word-frequency vector for that document.

13. Learning Topic Models

2. Convex hull membership. Write down the LP that tests whether the point $(0.3, 0.3)$ is in the convex hull of $\{(1, 0), (0, 1), (0, 0)\}$. Solve it by hand and verify the answer.
3. SVD vs. NMF. Generate a 6×4 matrix $M = AW$ where A is a 6×2 separable non-negative matrix and W is 2×4 non-negative. Compute the rank-2 SVD of M . Verify that the SVD factors have negative entries. Then use the vertex-finding LP to recover the rows of W from M .
4. Anchor words on real data (code). Using scikit-learn's `fetch_20newsgroups`, build a term-document frequency matrix for two newsgroups (e.g., `rec.sport.baseball` and `sci.space`). Normalise rows to sum to 1. Apply the vertex-finding algorithm from Section 13.10.1 to identify candidate anchor words. Do they make semantic sense?
5. Robustness (open-ended). Add Gaussian noise with standard deviation σ to each entry of a synthetic $M = AW$ (clip negatives to zero, renormalise). At what noise level does the vertex-finding algorithm start returning incorrect anchors? How would you modify the LP to tolerate noise?

14. Solving Linear Programs: The Simplex Algorithm

COMPSCI 224 Lecture 15 — Fall 2014

Linear programs have been a black box so far — “there is a magic machine that solves them.” This lecture opens the box. The Simplex algorithm (Dantzig, 1947) walks greedily along vertices of the feasible polytope, improving the objective at every step. Along the way, we prove that bounded LPs always have optimal vertices, characterise vertices as points with linearly independent active constraints, and show how to read off strong duality from Simplex’s halting condition. Simplex has no known polynomial-time pivoting rule, yet dominates practice; the ellipsoid method provides the polynomial-time guarantee but not the speed.

14.1. Why this lecture matters

You have used LPs throughout this course as a modelling tool — for approximation algorithms, for integrality gaps, for relaxations. Every time, you trusted that the LP could be solved. Now you see *how*. Simplex is one of the most important algorithms ever implemented: it runs inside every operations-research solver, every logistics optimizer, every portfolio allocation engine. Understanding its mechanics — especially the vertex-hopping intuition and the duality connection — will sharpen your ability to reason about LP-based algorithms.

The 30-second story

1. Every LP can be written in standard form: minimise $c^T x$ subject to $Ax = b, x \geq 0$.
2. If a standard-form LP is bounded and feasible, it has an optimal vertex — a point x where the active constraint columns of A are linearly independent.
3. Simplex starts at a vertex (found by solving an auxiliary LP), then greedily moves to adjacent vertices with lower cost by pivoting: swap one variable into the basis and one out.
4. The halting condition — all reduced costs non-negative — simultaneously certifies optimality and yields a dual feasible solution proving strong duality.
5. No known pivoting rule makes Simplex run in polynomial worst-case time, but it is fast in practice. The ellipsoid method (Khachiyan, 1979) gives a polynomial-time guarantee.

14.2. Standard form

All LP formulations — $\leq, \geq, =, \max, \min$ — can be converted to a single canonical shape.

Standard form

$$\min c^T x \quad \text{subject to} \quad Ax = b, \quad x \geq 0 \tag{14.1}$$

where $x \in \mathbb{R}^n, A \in \mathbb{R}^{m \times n}$, and $n \geq m$.

14.2.1. How to get there

Three transformations suffice:

1. Maximisation $\leftrightarrow$ minimisation. Replace $\max c^\top x$ with $\min (-c)^\top x$.
2. Inequality $\leftrightarrow$ equality via slack variables. Replace $a_i^\top x \geq b_i$ with

$$a_i^\top x - s_i = b_i, \quad s_i \geq 0. \quad (14.2)$$

The slack s_i absorbs the gap between the left- and right-hand sides. Similarly, $a_i^\top x \leq b_i$ becomes $-a_i^\top x \geq -b_i$, then add a slack.

3. Free variables $\leftrightarrow$ non-negative pairs. Replace any unconstrained variable x_i with $x_i = x_i^+ - x_i^-$, where $x_i^+, x_i^- \geq 0$.

After these steps, every variable (original, slack, and split) is non-negative, and every constraint is an equality. The number of variables n is at least m (the number of constraints) because each constraint contributes at least one slack.



Tip

Standard form is not a restriction — it is a notational convenience. The Simplex algorithm operates on this form; every LP you have seen can be rewritten this way without loss.

14.3. Geometry: the feasible polytope

The set of feasible solutions is a polytope:

$$P = \{x \in \mathbb{R}^n : Ax = b, x \geq 0\}$$

Simplex's geometric picture: start at a vertex, walk along edges to adjacent vertices that improve the objective, stop at the optimal vertex.

14.3.1. Vertex definition

! Definition: vertex

A point $x \in P$ is a vertex of P if there does *not* exist a non-zero vector y such that both $x + y \in P$ and $x - y \in P$.

A vertex is a point that cannot be expressed as the midpoint of two distinct feasible points. It is an “extreme point” — you cannot wiggle in any direction while staying feasible. In two dimensions, vertices are the corners of the polygon; in higher dimensions, they are the analogous sharp points of the polytope where many constraint hyperplanes meet.



Tip

Think of the vertex definition operationally: try to “jiggle” x by adding and subtracting a direction y . If *both* $x + y$ and $x - y$ stay feasible, you are in the interior (or on a face, but not at a corner). If no such y exists, you are stuck at a vertex — every direction of motion violates at least one constraint.

14.4. Finding a starting vertex

Simplex needs a vertex to start. But finding *any* feasible point is, in general, as hard as solving an LP (you can binary-search on the objective and add it as a constraint). The trick: construct an auxiliary LP with a known starting vertex, solve it with Simplex, and extract a starting vertex for the original problem.

14.4.1. The auxiliary LP

Given the original LP with constraints $Ax = b, x \geq 0$, fix any feasible point v (or set $v = 0$ if that works). Define:

$$\min t \quad \text{subject to} \quad Ax = (1 - t)b, \quad x \geq 0, \quad 0 \leq t \leq 1 \quad (14.3)$$

To put this in standard form, replace $t \leq 1$ with $s_t + t = 1, s_t \geq 0$.

Starting point. Set $x = 0, t = 1, s_t = 0$. This satisfies all constraints: $A \cdot 0 = (1 - 1) \cdot b = 0$. It is a vertex because every variable is at a boundary — any perturbation y would push some variable negative.

Goal. Minimise t . If the optimal $t = 0$, then $Ax = b$ and $x \geq 0$ — so x is a feasible point for the original LP. In fact, the point returned is a vertex of the original polytope.

Note

Run Simplex on the auxiliary LP (starting from $x = 0, t = 1$). If the minimum $t > 0$, the original LP is infeasible. If $t = 0$, read off x — it is a starting vertex for the original LP. Then run Simplex a second time on the original LP.

14.5. Vertices are optimal

The central structural result: you never need to look beyond vertices.

Claim

If an LP is bounded (optimal value $> -\infty$), then for every feasible x there exists a vertex x' with $c^\top x' \leq c^\top x$.

14.5.1. Proof sketch

Start from any feasible x . If x is not a vertex, there exists a non-zero y with $x + y \in P$ and $x - y \in P$. This implies $Ay = 0$ (subtract the two equality constraints). Two cases:

Case 1: there exists j with $y_j < 0$. Without loss of generality, $c^\top y \leq 0$ (otherwise replace y with $-y$). If $c^\top y = 0$, we can arrange for some $y_j < 0$. Now increase t from 0: consider $x + ty$ for $t > 0$. As t grows, some coordinate j with $y_j < 0$ will hit zero:

$$t^* = \min_{j: y_j < 0} \frac{-x_j}{y_j} > 0 \quad (14.4)$$

($t^* > 0$ because $x_j \geq 0$ whenever $y_j \neq 0$, by feasibility). Replace x with $x + t^*y$. The new point is feasible, has $c^\top(x + t^*y) \leq c^\top x$, and has at least one more zero coordinate. This case can happen at most n times before we reach a vertex.

14. Solving Linear Programs: The Simplex Algorithm

Case 2: $y_j \geq 0$ for all j (with at least one strict inequality). Then $x + ty$ is feasible for all $t > 0$, and $c^\top(x + ty) = c^\top x + t \cdot c^\top y$. Since we arranged $c^\top y < 0$ (case 1 handles equality), sending $t \rightarrow \infty$ gives $c^\top(x + ty) \rightarrow -\infty$ — contradicting boundedness. So case 2 cannot happen for bounded LPs.

Each application of case 1 produces a new feasible point with at least one more zero coordinate, and the objective does not increase ($c^\top y \leq 0$, so $c^\top(x + t^*y) \leq c^\top x$). There are only n coordinates, so after at most n repetitions, every negative component of y has been exhausted and x is a vertex. ■

14.5.2. Corollaries

1. If the LP is bounded, there exists a vertex achieving the optimum.
2. If P is non-empty, it has at least one vertex. (Apply the claim with *any* objective function, e.g., $\min 0^\top x$.)

14.6. Characterising vertices via linear algebra

Define the support of x as $B_x = \{j : x_j > 0\}$ and the submatrix A_{B_x} as the columns of A indexed by B_x .

! Vertex characterisation

$x \in P$ is a vertex $\iff$ the columns of A_{B_x} are linearly independent.

14.6.1. Proof (both directions by contrapositive)

Not a vertex $\Rightarrow$ not linearly independent. If x is not a vertex, there exists non-zero y with $Ay = 0$ and $y_j = 0$ whenever $x_j = 0$ (since perturbing a zero coordinate in either direction would make it negative). So y has support contained in B_x , meaning A_{B_x} has a non-trivial null vector — its columns are dependent.

Not linearly independent $\Rightarrow$ not a vertex. If the columns of A_{B_x} are dependent, there is a non-zero vector y supported on B_x with $Ay = 0$. Since y only has mass where x has mass (and x is strictly positive there), for small enough $t > 0$ both $x + ty$ and $x - ty$ remain non-negative and satisfy $A(x \pm ty) = b$. So x is not a vertex. ■

Since A has m rows, the column rank is at most m . Linear independence of A_{B_x} forces $|B_x| \leq m$: a vertex has at most m non-zero coordinates. There are at most $\binom{n}{m}$ vertices — one for each choice of m columns from the n total. This is a finite (though potentially exponential) bound, which is what guarantees that Simplex terminates.

i Note

Assume A has full row rank m (drop redundant constraints if not). Then every vertex has $|B_x| \leq m$. If $|B_x| < m$, we can extend B_x to a set B of size m with linearly independent columns — this is the basis for the vertex.

14.7. The Simplex algorithm

14.7.1. Bases and reduced costs

Fix a vertex x with basis B (size m , columns of A_B linearly independent, A_B invertible). Let $N = \{1, \dots, n\} \setminus B$ be the non-basic indices. Partition the LP:

$$\min c_B^\top x_B + c_N^\top x_N \quad \text{s.t.} \quad A_B x_B + A_N x_N = b, \quad x_B, x_N \geq 0$$

From the equality constraint: $x_B = A_B^{-1}b - A_B^{-1}A_Nx_N$. Substitute into the objective:

$$c^T x = c_B^T A_B^{-1} b + \tilde{c}_N^T x_N \quad (14.5)$$

where the reduced cost vector is:

$$\tilde{c}_N = c_N - A_N^T (A_B^{-1})^T c_B \quad (14.6)$$

The first term in Equation 14.5 is a constant (the cost at the current vertex, where $x_N = 0$). The objective improves if and only if we can increase some x_j ($j \in N$) with $\tilde{c}_j < 0$.

14.7.2. The algorithm

! Simplex algorithm (Dantzig, 1947)

1. Initialise. Start with a vertex x and its basis B .
2. Compute reduced costs. Find $\tilde{c}_N$ via Equation 22.6.
3. Optimality check. If $\tilde{c}_N \geq 0$ (all reduced costs non-negative), halt — x is optimal.
4. Pivot. Choose an index $j \in N$ with $\tilde{c}_j < 0$. Increase x_j as much as possible while keeping $x_B \geq 0$. Some basic variable x_k hits zero — swap j into B and k out. The new x is an adjacent vertex with strictly lower cost.
5. Repeat from step 2.

14.7.3. How pivoting works

When we increase x_j (a non-basic variable with $\tilde{c}_j < 0$), the basic variables change via:

$$x_B = A_B^{-1}b - A_B^{-1}A_Nx_N$$

Increasing x_j decreases some coordinates of x_B . The rate of decrease depends on $A_B^{-1}a_j$ — the column of A corresponding to j , expressed in the basis. We increase x_j until the first basic variable hits zero — that variable leaves the basis, and j enters. This is one pivot step: the new basis $B' = (B \setminus \{k\}) \cup \{j\}$ defines a new adjacent vertex.

Geometrically, each pivot moves along an *edge* of the polytope to an adjacent vertex. The edge direction is determined by the entering variable, and the edge length is determined by which basic variable hits zero first. Because $\tilde{c}_j < 0$, the objective strictly decreases along this edge (assuming non-degeneracy), so Simplex never revisits a vertex.

! Degeneracy and cycling

If the current vertex has a basic variable that is already zero ($x_k = 0$ for some $k \in B$), the pivot may not actually move — x_j cannot increase at all. Worse, careless pivoting can cycle: swap j in and k out, then swap k back in on the next iteration, forever.

Bland's rule (Bland, 1977): among all eligible entering variables ($\tilde{c}_j < 0$), choose the one with the smallest index; among all eligible leaving variables, choose the one with the smallest index. This provably prevents cycling.

14.8. Strong duality from Simplex

When Simplex halts, the reduced cost vector satisfies $\tilde{c}_N \geq 0$. This halting condition directly yields a dual certificate.

The dual of Equation 14.1 is:

$$\max b^\top y \quad \text{subject to} \quad A^\top y \leq c \quad (14.7)$$

At the Simplex halting condition, set $y = (A_B^{-1})^\top c_B$. Then:

- For $j \in B$: $a_j^\top y = a_j^\top (A_B^{-1})^\top c_B = c_j$ (the j -th column of A_B times $A_B^{-1} c_B$ extracts c_j). So $a_j^\top y \leq c_j$. $\square$
- For $j \in N$: $a_j^\top y = a_j^\top (A_B^{-1})^\top c_B = c_j - \tilde{c}_j \leq c_j$ (since $\tilde{c}_j \geq 0$). $\square$

So y is dual feasible. Its objective value is $b^\top y = b^\top (A_B^{-1})^\top c_B = (A_B^{-1} b)^\top c_B = x_B^\top c_B = c^\top x$ (since $x_N = 0$). The primal and dual objectives match — strong duality holds.

This also gives complementary slackness for free: for each j , either $x_j = 0$ or the dual constraint $a_j^\top y = c_j$ is tight (i.e., no slack). For $j \in B$, $x_j > 0$ and $a_j^\top y = c_j$ (tight). For $j \in N$, $x_j = 0$ and $a_j^\top y \leq c_j$ (possibly with slack). This pairing between primal variables and dual constraints is the algebraic backbone of LP duality.

! Important

Simplex does not merely solve the primal LP. At termination, it hands you a dual feasible solution achieving the same objective — a certificate of optimality. Strong duality falls out as a corollary of Simplex's correctness.

14.9. Worst-case complexity and beyond

14.9.1. Simplex: fast in practice, slow in theory

Each pivot moves to an adjacent vertex with strictly lower cost (assuming non-degeneracy or Bland's rule). There are at most $\binom{n}{m}$ vertices, so Simplex terminates. But $\binom{n}{m}$ can be exponential.

No known pivoting rule achieves polynomial worst-case time. Klee and Minty (1972) constructed polytopes where the standard pivoting rule visits *every* vertex — exponentially many. Despite this, Simplex is fast in practice. Spielman and Teng (2004) gave a partial theoretical explanation via smoothed analysis: under tiny random perturbations of the data, the expected number of Simplex pivots is polynomial.

14.9.2. The ellipsoid method

Khachiyan (1979) resolved a major open question: LP is in P.

The ellipsoid method maintains an ellipsoid guaranteed to contain the optimal solution. At each step, it queries whether the centre is feasible. If not, a violated constraint defines a half-space; the algorithm shrinks the ellipsoid to the half contained in that half-space. The volume shrinks by a constant factor each step, guaranteeing convergence in polynomially many iterations.

i Aside (mine, not the lecturer's)

The ellipsoid method is polynomial but slow in practice — much slower than Simplex on real problems. Interior-point methods (Karmarkar, 1984) achieve the best of both worlds: polynomial worst-case time *and*

practical speed. Modern LP solvers (CPLEX, Gurobi, HiGHS) typically offer both Simplex and interior-point as options, choosing automatically based on problem structure.

14.10. Cheat sheet

! Cheat sheet — Solving LPs

Standard form. $\min c^\top x$ s.t. $Ax = b, x \geq 0$. Convert via: $\max \rightarrow \min$ (negate c), inequalities $\rightarrow$ slacks, free vars $\rightarrow$ non-negative pairs.

Vertex. $x \in P$ where you cannot perturb in any direction and stay feasible. Equivalently: columns of A_{B_x} (active columns) are linearly independent. At most m non-zero coordinates; $\leq \binom{n}{m}$ vertices total.

Key theorem. Bounded LP $\Rightarrow$ some vertex is optimal.

Simplex.

- Start at a vertex (via auxiliary LP).
- Compute reduced costs: $\tilde{c}_j = c_j - a_j^\top (A_B^{-1})^\top c_B$.
- If $\tilde{c}_N \geq 0$: halt, x is optimal.
- Else: pivot — increase x_j with $\tilde{c}_j < 0$, swap basis elements.
- Bland's rule prevents cycling under degeneracy.

Strong duality. At halt, $y = (A_B^{-1})^\top c_B$ is dual feasible with $b^\top y = c^\top x$. Primal and dual objectives match.

Complexity. Simplex: exponential worst case, fast in practice (smoothed analysis: polynomial). Ellipsoid: polynomial worst case, slow in practice. Interior-point: polynomial worst case, fast in practice.

14.11. FAQ — questions students actually asked

Q: Why would the binary search terminate when reducing optimization to feasibility?

The optimal value is a rational number whose precision (number of bits) is bounded by the input size — it is the solution to a linear system on a submatrix of A . You only need that many iterations of binary search to pin down the exact optimum.

Q: Why does the auxiliary LP have a starting vertex?

At $x = 0, t = 1, s_t = 0$, every variable is at its boundary. Any perturbation y would push some variable negative (since they are all zero or at their upper bound). By the vertex characterisation, this is a vertex.

Q: If $|B_x| < m$, can we still run Simplex?

Yes. Extend B_x to a set B of size exactly m by adding columns that maintain linear independence. The extra columns correspond to variables that are zero — they are in the basis “for free.” This is a technical convenience that keeps A_B square and invertible.

Q: Can Simplex cycle forever?

Without a careful pivoting rule, yes. If a basis has a degenerate variable (zero-valued basic variable), a pivot might not change the vertex at all, and repeated pivots can cycle. Bland's rule (1977) prevents this: always pick the smallest-index eligible entering and leaving variables. Other anti-cycling rules exist, but in practice, many implementations use rules that can theoretically cycle and simply don't worry about it — degenerate cycling is rare on real problems.

Q: If no implementation of Simplex is polynomial, why does everyone use it?

Worst-case exponential does not mean typical-case exponential. Spielman and Teng's smoothed analysis (2004) shows that under tiny random perturbations, the expected number of pivots is polynomial. Real-world LPs are never exactly worst-case instances. Simplex's practical speed, warm-start capability, and ability to produce dual solutions make it the workhorse of optimisation.

14.12. Further reading

- Dantzig, G. B. (1963). *Linear Programming and Extensions*. Princeton University Press. The original treatment of the Simplex method by its inventor.
- Bland, R. G. (1977). "New Finite Pivoting Rules for the Simplex Method." *Mathematics of Operations Research*, 2(2): 103–107. The anti-cycling pivoting rule.
- Khachiyan, L. G. (1979). "A Polynomial Algorithm in Linear Programming." *Doklady Akademii Nauk SSSR*, 244: 1093–1096. The ellipsoid method — first polynomial-time LP algorithm.
- Spielman, D. A. & Teng, S.-H. (2004). "Smoothed Analysis of Algorithms: Why the Simplex Algorithm Usually Takes Polynomial Time." *Journal of the ACM*, 51(3): 385–463. Theoretical explanation for Simplex's practical speed.
- Klee, V. & Minty, G. J. (1972). "How Good Is the Simplex Algorithm?" In *Inequalities III*, pp. 159–175. Academic Press. Exponential lower bound for a specific pivoting rule.

14.13. Try it yourself

1. Standard form practice. Convert the following LP to standard form:

$$\max 3x_1 + 2x_2 \quad \text{s.t.} \quad x_1 + x_2 \leq 4, \quad 2x_1 - x_2 \geq 1, \quad x_1 \geq 0, \quad x_2 \text{ unrestricted.}$$

How many variables and constraints does the standard-form LP have?

2. Vertex counting. A standard-form LP has $m = 3$ equality constraints and $n = 6$ variables. What is the maximum number of vertices? Give an example polytope achieving this bound.
3. One Simplex iteration by hand. Consider:

$$\min -2x_1 - 3x_2 \quad \text{s.t.} \quad x_1 + x_2 + s_1 = 4, \quad x_1 + 2x_2 + s_2 = 6, \quad x_1, x_2, s_1, s_2 \geq 0.$$

Start at the vertex $x_1 = x_2 = 0, s_1 = 4, s_2 = 6$ with basis $B = \{s_1, s_2\}$. Compute the reduced costs. Identify the entering variable. Perform one pivot: find the leaving variable and the new vertex.

4. Reading off the dual. After solving the LP in exercise 3 to optimality via Simplex, construct the dual feasible solution from $y = (A_B^{-1})^T c_B$. Verify that $b^T y$ equals the primal optimum.
5. Klee–Minty in code. Implement the Simplex algorithm in Python (or use `scipy.optimize.linprog` with `method='revised simplex'`). Construct the Klee–Minty cube in d dimensions and count the number of pivots for $d = 3, 4, 5, 6$. Plot pivots vs. d — does it look exponential?

15. Strong Duality, Complementary Slackness, and the Ellipsoid Method

COMPSCI 224 Lecture 16 — Fall 2014

Simplex termination via Bland's rule leads directly to a proof of strong duality: the shadow price vector is dual-feasible at halt, collapsing the duality gap to zero. Complementary slackness sharpens this into a coordinate-by-coordinate certificate of optimality. The ellipsoid method—the first polynomial-time LP algorithm—needs only a *separation oracle*, enabling it to solve exponentially large programs. A brief preview of interior point methods sets the stage for Lecture 17.

15.1. Why this lecture matters

This lecture ties three threads together. Simplex terminates (via Bland's rule) and, as a corollary, proves strong duality—arguably the most important theorem in linear programming. Complementary slackness then gives a pointwise certificate that lets you verify optimality without re-solving the LP. The ellipsoid method shifts the computational model: you no longer need the constraint matrix written down, just an oracle that finds a violated constraint. That oracle-based view is what makes LP a tractable subroutine inside combinatorial optimisation.

30-second story

1. Simplex moves vertex-to-vertex; Bland's rule prevents cycling in degenerate cases.
2. The shadow price vector $y = (A_B^{-1})^T c_B$ is dual-feasible exactly when Simplex halts, proving strong duality.
3. Complementary slackness: for each coordinate, either the primal variable or the dual slack is zero.
4. Simplex takes at most $\binom{n - \lfloor m/2 \rfloor}{\lfloor m/2 \rfloor}$ iterations (McMullen's upper bound theorem), but no polynomial pivoting rule is known.
5. Santos (2012) disproved the Hirsch conjecture: polytope diameter can exceed $m - d$.
6. The ellipsoid method solves LP in polynomial time using only a separation oracle.

15.2. Finishing Simplex: degeneracy and Bland's rule

Recall the setup. The LP is

$$\min c^T x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0, \quad (15.1)$$

where A is $m \times n$, $b \in \mathbb{R}^m$, and $c \in \mathbb{R}^n$. The polytope $P = \{x : Ax = b, x \geq 0\}$. A vertex $x \in P$ has the property that $A_{\mathcal{B}'(x)}$ —the submatrix of columns indexed by $\mathcal{B}'(x) = \{j : x_j > 0\}$ —has full column rank.

At each step Simplex maintains a basis $\mathcal{B} \subseteq \{1, \dots, n\}$ with $|\mathcal{B}| = m$ and $A_{\mathcal{B}}$ invertible. Writing $\mathcal{N} = \{1, \dots, n\} \setminus \mathcal{B}$ for the non-basic indices, we substitute $x_{\mathcal{B}} = A_{\mathcal{B}}^{-1}(b - A_{\mathcal{N}}x_{\mathcal{N}})$ into the objective and obtain the reduced cost

$$\tilde{c}_{\mathcal{N}} = c_{\mathcal{N}} - A_{\mathcal{N}}^{\top} (A_{\mathcal{B}}^{-1})^{\top} c_{\mathcal{B}}. \quad (15.2)$$

15.2.1. Rewriting the LP in terms of the basis

The substitution $x_{\mathcal{B}} = A_{\mathcal{B}}^{-1}(b - A_{\mathcal{N}}x_{\mathcal{N}})$ lets you rewrite the objective entirely in terms of the non-basic variables. The constant term $c_{\mathcal{B}}^{\top} A_{\mathcal{B}}^{-1} b$ is the objective value at the current vertex (where $x_{\mathcal{N}} = 0$). The reduced cost $\tilde{c}_{\mathcal{N}}$ tells you the rate at which the objective changes if you start increasing any non-basic variable away from zero.

Think of it this way: you are sitting at a vertex. Each non-basic variable is a door you could open. The reduced cost of that variable tells you whether walking through that door improves or worsens the objective.

15.2.2. The non-degenerate case

If $v_{\mathcal{B}} > 0$ (every basic variable is strictly positive), the rule is clean:

- Negative reduced cost exists ($\tilde{c}_j < 0$ for some $j \in \mathcal{N}$): pick such a j and increment v_j . As v_j increases from zero, the basic variables $v_{\mathcal{B}}$ adjust according to $v_{\mathcal{B}} = A_{\mathcal{B}}^{-1}b - A_{\mathcal{B}}^{-1}A_{\cdot j}v_j$. Some of them decrease. Keep going until the first basic variable hits zero—that's your new vertex. The entering variable j joins the basis; the variable that hit zero leaves.
- All reduced costs non-negative ($\tilde{c}_{\mathcal{N}} \geq 0$): halt and declare optimality. At the current vertex $x_{\mathcal{N}} = 0$, and since $x_{\mathcal{N}} \geq 0$ throughout the polytope, any feasible movement can only increase the objective (the reduced costs are non-negative). The current vertex is optimal.

The increment is guaranteed to be positive because all basic variables started strictly positive and move continuously.

15.2.3. The degenerate case

When some $v_j = 0$ for $j \in \mathcal{B}$, incrementing a non-basic variable with negative reduced cost may produce zero progress: a basic variable is already at zero, so the step length is zero. Naïve pivoting rules can cycle forever.

Cycling is real

Without a careful pivoting rule, Simplex can revisit the same sequence of bases indefinitely. The Beale cycling example (1955) is a classic 6-variable, 3-constraint LP that cycles under the largest-coefficient rule.

Bland's rule resolves this. Given multiple choices:

1. Entering variable j : among all $j \in \mathcal{N}$ with $\tilde{c}_j < 0$, pick the one with the smallest index.
2. Leaving variable k : compute $b' = A_{\mathcal{B}}^{-1}b$, $a' = A_{\mathcal{B}}^{-1}A_{\cdot j}$, and $q_i = b'_i/a'_{ij}$ for all i with $a'_{ij} > 0$. Among those minimising q_i , pick the smallest index.

Bland's rule is provably anti-cycling—Simplex terminates in finitely many steps.

15.3. Strong duality via Simplex

15.3.1. The shadow price vector

Define the shadow price vector

$$y = (A_{\mathcal{B}}^{-1})^T c_{\mathcal{B}}. \quad (15.3)$$

This vector changes as the basis changes during Simplex. At the current vertex v :

Claim. $c^T v = b^T y$.

Proof. Since $A_{\mathcal{B}} v_{\mathcal{B}} = b$, we have $v_{\mathcal{B}} = A_{\mathcal{B}}^{-1} b$. Then $c^T v = c_{\mathcal{B}}^T v_{\mathcal{B}} = c_{\mathcal{B}}^T A_{\mathcal{B}}^{-1} b = (A_{\mathcal{B}}^{-1})^T c_{\mathcal{B}})^T b = y^T b = b^T y$. $\square$

This already looks like strong duality—primal cost equals dual cost—except we need y to be dual-feasible. The claim says that at *every* intermediate vertex, the primal objective equals $b^T y$. But y is not necessarily dual-feasible during the algorithm—it becomes feasible only at termination.

15.3.2. Dual feasibility at termination

The dual of Equation 15.1 is

$$\max b^T y \quad \text{s.t.} \quad A^T y \leq c \quad (y \text{ unrestricted}). \quad (15.4)$$

Define the dual slack $r = c - A^T y$. By the definition of y :

- $r_{\mathcal{B}} = 0$ (direct computation).
- $r_{\mathcal{N}} = \tilde{c}_{\mathcal{N}}$ (the reduced cost).

Simplex halts when $\tilde{c}_{\mathcal{N}} \geq 0$, which is exactly $r \geq 0$ —dual feasibility. At that moment both primal and dual are feasible with equal objective values, so strong duality holds.

! Strong duality (LP)

If the primal LP has an optimal solution, so does the dual, and $\min c^T x = \max b^T y$.

15.4. Complementary slackness

The complementarity gap is

$$x^T r = x^T (c - A^T y) = c^T x - (Ax)^T y = c^T x - b^T y. \quad (15.5)$$

At optimality strong duality gives $c^T x^* = b^T y^*$, so $(x^*)^T r^* = 0$. Each term $x_i^* r_i^*$ is non-negative (both x^* and r^* are componentwise non-negative). For non-negative terms to sum to zero, every term must be zero.

! Complementary slackness

If x^* and y^* are optimal for the primal and dual respectively, then for every coordinate i :

$$x_i^* = 0 \quad \text{or} \quad (c - A^T y^*)_i = 0.$$

15.4.1. Why complementary slackness matters

Complementary slackness is the workhorse behind certifying optimality. Given a candidate primal solution x and dual solution y , you don't need to re-solve the LP to check whether they're optimal—just verify that $x_i(c - A^T y)_i = 0$ for every i . If it holds and both are feasible, they're optimal. Period.

Earlier in the course, when analysing online algorithms, the lecturer used approximate complementary slackness: if each product $x_i r_i$ is bounded by some factor of the optimum rather than being exactly zero, you can extract a competitive-ratio bound. The online primal-dual framework systematises this into a recipe for designing competitive online algorithms.

Complementary slackness as economic intuition

Think of x_i as the quantity of resource i used and r_i as the “surplus price” (how much cheaper the resource is than its shadow price). If a resource has positive surplus price ($r_i > 0$), no rational agent uses it ($x_i = 0$). If a resource is actively used ($x_i > 0$), it must be priced at exactly its shadow value ($r_i = 0$). At optimality, there is no waste: every resource is either unused or priced at cost.

15.5. Simplex runtime and polytope diameter

15.5.1. Iteration count

Each Simplex pivot takes polynomial time. The number of pivots is at most the number of vertices. A vertex corresponds to a choice of m columns from n , so the crude bound is $\binom{n}{m}$. Not every choice yields a feasible vertex (the solution $A_B^{-1}b$ may have negative entries), so the true bound is tighter.

McMullen's upper bound theorem (1970)

The maximum number of vertices of a polytope defined by n inequalities in m dimensions is

$$\binom{n - \lfloor m/2 \rfloor}{\lfloor m/2 \rfloor}.$$

This is still exponential, but strictly better than $\binom{n}{m}$.

15.5.2. The Hirsch conjecture

A necessary condition for any polynomial Simplex implementation is a polynomial bound on the graph diameter of the polytope (vertices connected when their bases differ by one constraint).

The Hirsch conjecture (1957): every polytope in d dimensions defined by m hyperplanes has graph diameter at most $m - d$.

In 2012, Santos exhibited a counterexample with $d = 43$, $m = 86$, and diameter at least 44. He further showed that for fixed d and any $\epsilon > 0$, there exist infinite families of polytopes with diameter at least $(1 + \epsilon)m$.

These lower bounds are far from ruling out polynomial diameter ($m^{10}d^5$ would suffice), so a polynomial Simplex remains possible—just unproven.

i Open problem landscape

Three questions remain open, in decreasing order of ambition:

1. Is there a polynomial pivoting rule for Simplex? (No candidate is known.)
2. Is the diameter of every polytope polynomial in m and d ? (The polynomial Hirsch conjecture—still wide open.)
3. Is there *any* strongly polynomial algorithm for LP? (One of Smale’s “problems for the 21st century.”)

The disproof of the Hirsch conjecture rules out the sharpest possible diameter bound but leaves plenty of room for weaker polynomial bounds.

15.5.3. Smooth analysis

Spielman and Teng (2004) offered a theoretical explanation for Simplex’s practical speed via smoothed analysis.

The model: an adversary picks a worst-case LP instance. Each entry is then perturbed by independent Gaussian noise with mean zero and variance σ^2 . The expected running time over the noise is polynomial in n , m , and $1/\sigma$ under the shadow-vertex pivot rule.

Formally: there exists a pivot rule such that for all inputs x ,

$$\mathbb{E}_{\tilde{x}}[\text{runtime}(A, \tilde{x})] = \text{poly}(n, m, 1/\sigma),$$

where $\tilde{x}$ is a randomly perturbed version of x . The quantifiers matter: the adversary picks the worst-case input *first*, then the noise is applied. If the expected running time is polynomial for *every* adversarial choice, the algorithm has polynomial smoothed complexity.

The intuition is that pathological LP instances—like the Klee–Minty cube, which forces Simplex through 2^n vertices—are *fragile*. They rely on precise coordinate alignments that vanish under the slightest perturbation. Smoothed analysis captures this: the probability that a random perturbation preserves the pathological structure is negligible.

💡 Smooth analysis in one sentence

Smoothed analysis interpolates between worst-case and average-case: perturb every worst-case input by tiny random noise and bound the expected runtime. It explains why pathological inputs are fragile—small perturbations destroy them.

15.6. The ellipsoid method

15.6.1. From Simplex to polynomial time

Simplex (Dantzig, 1947) has no known polynomial pivot rule. The ellipsoid method (Khachiyan, 1979) was the first provably polynomial algorithm for LP. It is rarely used in practice (it is slow compared to Simplex and interior point), but its oracle-based structure makes it uniquely powerful in theory.

15.6.2. Ellipsoids

An ellipsoid is

$$E(a, \mathcal{A}) = \{x : (x - a)^\top \mathcal{A}^{-1}(x - a) \leq 1\}, \quad (15.6)$$

where $a \in \mathbb{R}^n$ is the centre and $\mathcal{A} \succ 0$ is a positive-definite shape matrix. A sphere is the special case $\mathcal{A} = r^2 I$.

15.6.3. The algorithm

The ellipsoid method tests feasibility of $P = \{x : Ax \geq b\}$:

1. Initialise with an ellipsoid E_0 guaranteed to contain P (radius derived from bit complexity L).
2. While the centre a of the current ellipsoid is not in P :
 - a. A separation oracle returns a hyperplane H separating a from P .
 - b. Replace the current ellipsoid with a smaller ellipsoid E' that still contains the half of E on P 's side of H .
3. Terminate when either $a \in P$ (feasible point found) or the volume of E has shrunk below a threshold (declare P empty).

The key volume-shrinkage guarantee:

$$\frac{\text{vol}(E')}{\text{vol}(E)} \leq e^{-1/(2n)}. \quad (15.7)$$

After k iterations, $\text{vol}(E_k) \leq \text{vol}(E_0) \cdot e^{-k/(2n)}$. Comparing with the minimum volume of a non-empty polytope (exponentially small in L) gives a polynomial iteration bound of $O(n^2 L)$.

i Aside (mine, not the lecturer's)

The ellipsoid method also works for general convex programming: any closed convex set with a polynomial-time separation oracle can be optimised over. This follows from the supporting hyperplane theorem (Hahn–Banach).

15.6.4. Handling precision issues

If P has equality constraints, its volume in the ambient space is zero—the algorithm would declare it empty immediately. The standard fix: inflate P slightly to

$$P' = \{x : Ax \leq b + 2^{-L} \mathbf{1}, -2^L \leq x_j \leq 2^L \forall j\},$$

where L is the bit complexity. Then P is infeasible if and only if P' is infeasible, and P' has volume bounded away from zero.

15.6.5. From feasibility to optimisation

The ellipsoid method solves feasibility, not optimisation directly. But optimisation reduces to feasibility in three steps:

Step 1. Check that $P = \{x : Ax = b, x \geq 0\}$ is feasible by running the ellipsoid method on P . If infeasible, stop.

Step 2. Check that the dual $\{y : A^T y \leq c\}$ is feasible. If not, the primal is unbounded—there is no finite minimum.

Step 3. After steps 1 and 2, strong duality guarantees that the set

$$\{(x, y) : Ax = b, x \geq 0, A^T y \leq c, c^T x = b^T y\}$$

is non-empty. Run the ellipsoid method on this joint system. Any feasible (x, y) in this set is simultaneously primal-optimal and dual-optimal.

The key insight: the constraint $c^T x = b^T y$ (a single equation) encodes optimality via strong duality. You don't need to optimise—you just need to find a point that is primal-feasible, dual-feasible, and has zero duality gap.

15.6.6. The oracle perspective

The ellipsoid method's real power: it never explicitly reads the constraint matrix. It only calls the separation oracle. For problems with exponentially many constraints but a polynomial-time separation oracle, the ellipsoid method solves the LP in polynomial time.

```
# Conceptual sketch: ellipsoid method with a separation oracle
import numpy as np

def ellipsoid_feasibility(oracle, n, max_iter):
    """
    oracle(x) -> (feasible: bool, hyperplane: (a, b))
        If x is infeasible, returns a violated constraint a^T x >= b.
    """
    # Start with a large sphere
    centre = np.zeros(n)
    shape = np.eye(n) * (2 ** 20) # large initial ellipsoid

    for k in range(max_iter):
        feasible, constraint = oracle(centre)
        if feasible:
            return centre # found a feasible point

        a_vec, _ = constraint
        # Update centre and shape (minimum-volume ellipsoid containing the half)
        denom = np.sqrt(a_vec @ shape @ a_vec)
        centre = centre - (1 / (n + 1)) * (shape @ a_vec) / denom
        shape = (n**2 / (n**2 - 1)) * (
            shape - (2 / (n + 1)) * np.outer(shape @ a_vec, shape @ a_vec)
            / (a_vec @ shape @ a_vec)
        )

    return None # likely infeasible
```

 Not for production

The ellipsoid method is $O(n^4L)$ per iteration (matrix updates). In practice, Simplex and interior point methods are orders of magnitude faster. The ellipsoid method's value is theoretical: it proves LP is in P and handles exponentially many constraints.

15.7. Interior point methods: a preview

The interior point method (Karmarkar, *Combinatorica*, 1984) is the practical polynomial-time alternative to ellipsoid. Where Simplex walks along the polytope's edges, interior point stays *inside* the polytope and converges to an optimal vertex from the interior.

15.7.1. The barrier approach

Convert the constrained LP $\min c^\top x$ subject to $Ax \geq b$ into an unconstrained problem:

$$\min_{x \in \mathbb{R}^n} \lambda c^\top x - \sum_{i=1}^m \ln(a_i^\top x - b_i). \quad (15.8)$$

The log barrier $p(s) = -\sum_i \ln s_i$ goes to $+\infty$ as any slack $s_i \rightarrow 0$, keeping the iterates in the interior.

- $\lambda = 0$: the minimiser is the analytic centre—the point farthest from all constraints.
- $\lambda \rightarrow \infty$: the minimiser approaches the optimal vertex.

The path-following strategy: start with λ_0 very small (exponentially small in L), solve approximately, then incrementally increase λ . At each increment, use the previous approximate solution as a warm start for Newton's method.

The iteration count is $O(\sqrt{m}L)$, each involving a constant number of $n \times n$ linear system solves. Details in Lecture 17.

15.8. Cheat sheet

Lecture 16 — key results

Simplex termination

- Bland's rule (smallest-index pivot) guarantees finite termination, even with degenerate vertices.
- Vertex bound: $\binom{n-\lfloor m/2 \rfloor}{\lfloor m/2 \rfloor}$ (McMullen).
- No polynomial pivoting rule is known. Hirsch conjecture (diameter $\leq m - d$) is false (Santos, 2012).
- Smoothed analysis (Spielman–Teng): polynomial expected time under small random perturbations.

Strong duality

- Shadow price: $y = (A_{\mathcal{B}}^{-1})^\top c_{\mathcal{B}}$.
- At any vertex: $c^\top v = b^\top y$.
- At termination: reduced cost $\tilde{c}_{\mathcal{N}} \geq 0 \Rightarrow$ dual feasibility $\Rightarrow$ strong duality.

Complementary slackness

- $x_i^* = 0$ or $(c - A^\top y^*)_i = 0$ for every i .

- Proof: $\sum x_i r_i = 0$ with all terms non-negative.

Ellipsoid method

- First poly-time LP algorithm (Khachiyan, 1979).
- Needs only a separation oracle, not the explicit constraint matrix.
- Volume shrinks by factor $e^{-1/(2n)}$ per step; $O(n^2 L)$ iterations.
- Handles exponentially many constraints (if oracle is efficient).

Interior point (preview)

- Log barrier converts constrained LP to unconstrained optimisation.
- Path-following: track the central path from analytic centre to optimal vertex.
- $O(\sqrt{m} L)$ iterations, each a linear system solve.

15.9. FAQ — questions students actually asked

Q: How do you check that A has full row rank in practice? You can add constraints one by one: if adding a row doesn't increase the rank, drop it (it's redundant). Computing rank can be done in input-sparsity time (proportional to the number of non-zero entries times polylog factors). Theoretically, you could also compute $\det(A^\top A)$ and check it is non-zero.

Q: In the polytope graph, when are two vertices adjacent? Two vertices share an edge when their bases differ by exactly one constraint—swap one column in, one column out.

Q: Is the number of iterations for interior point at least L ? Yes. Even the best known interior point methods need $\Omega(L)$ iterations. The best bound before 2014 was $O(\sqrt{m} L)$ per Renegar (1988). Lee and Sidford (2014) improved the $\sqrt{m}$ to $\sqrt{\text{rank}(A)}$.

Q: Are there strongly polynomial LP algorithms? No. Whether LP can be solved in time polynomial in m and n alone (independent of L) is a major open problem.

Q: Why is the Hirsch conjecture called “the” upper bound theorem? McMullen's upper bound theorem (on the number of vertices) and the Hirsch conjecture (on diameter) are distinct results. The “upper bound theorem” specifically refers to McMullen's tight bound on the number of faces of a convex polytope. The Hirsch conjecture, now disproved, was about graph diameter.

15.10. Further reading

1. Santos, F. “A counterexample to the Hirsch conjecture.” *Annals of Mathematics* 176(1), 2012. The disproof, with an explicit $d = 43$, $m = 86$ polytope.
2. Spielman, D. and Teng, S.-H. “Smoothed analysis of algorithms: why the simplex algorithm usually takes polynomial time.” *Journal of the ACM* 51(3), 2004. The foundational smoothed-analysis paper.
3. Khachiyan, L. “A polynomial algorithm in linear programming.” *Doklady Akademii Nauk SSSR* 244(5), 1979. The original ellipsoid method for LP.
4. Grötschel, M., Lovász, L., and Schrijver, A. *Geometric Algorithms and Combinatorial Optimization*. Springer, 1988. Definitive treatment of the ellipsoid method and its applications to combinatorial optimisation.
5. Karmarkar, N. “A new polynomial-time algorithm for linear programming.” *Combinatorica* 4(4), 1984. The interior point method that launched a field.

15.11. Try it yourself

1. Complementary slackness by hand. Solve the LP $\min 3x_1 + 2x_2$ subject to $x_1 + x_2 \geq 4$, $x_1 + 3x_2 \geq 6$, $x_1, x_2 \geq 0$. Write the dual, find optimal primal and dual solutions, and verify complementary slackness coordinate by coordinate.
2. Bland's rule trace. Consider the LP with $A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$, $b = (1, 1)^T$, $c = (-1, -1, 0)^T$. Starting from basis $\{3\}$ extended to $\{1, 3\}$, trace two Simplex pivots using Bland's rule. At each step, write the reduced cost, the entering and leaving variables.
3. Ellipsoid volume decay. How many iterations does the ellipsoid method need to shrink its volume by a factor of 10^6 in $n = 100$ dimensions? Use the bound $\text{vol}(E_k)/\text{vol}(E_0) \leq e^{-k/(2n)}$.
4. Smooth analysis intuition. Explain informally why adding Gaussian noise $\mathcal{N}(0, \sigma^2)$ to every entry of an LP prevents the pathological behaviour that makes Simplex exponential (e.g., Klee–Minty cubes).
5. Implement the ellipsoid method. Write a Python function that takes a list of linear constraints $a_i^T x \geq b_i$ and uses the ellipsoid method to find a feasible point or declare infeasibility. Test it on a simple 2D polygon.

```
import numpy as np

def ellipsoid_2d(constraints, max_iter=500):
    """
    constraints: list of (a, b) where a is 2D vector, b is scalar
                  representing a^T x >= b
    Returns feasible point or None.
    """
    n = 2
    centre = np.zeros(n)
    P = np.eye(n) * 100.0 # initial ellipsoid shape

    for _ in range(max_iter):
        # Check feasibility
        violated = None
        for a, b in constraints:
            if a @ centre < b:
                violated = (a, b)
                break

        if violated is None:
            return centre # feasible

        a_vec, _ = violated
        # Ellipsoid update
        Pa = P @ a_vec
        denom = np.sqrt(a_vec @ Pa)
        centre = centre - (1 / (n + 1)) * Pa / denom
        P = (n**2 / (n**2 - 1)) * (
            P - (2 / (n + 1)) * np.outer(Pa, Pa) / (a_vec @ Pa)
        )

    return None # likely infeasible

# Test: triangle x1 >= 0, x2 >= 0, x1 + x2 <= 3
```

```
constraints = [  
    (np.array([1, 0]), 0.0),    #  $x_1 \geq 0$   
    (np.array([0, 1]), 0.0),    #  $x_2 \geq 0$   
    (np.array([-1, -1]), -3.0) #  $-x_1 - x_2 \geq -3$ , i.e.,  $x_1 + x_2 \leq 3$   
]  
point = ellipsoid_2d(constraints)  
print(f"Feasible point: {point}")
```


16. Interior Point Methods: Path Following and Gradient Descent

COMPSCI 224 Lecture 17 — Fall 2014

The path-following interior point method converts an LP into an unconstrained optimisation problem via a log barrier, then traces the central path from the analytic centre to the optimal vertex by incrementally raising a parameter λ . The cost gap is m/λ , so $O(\sqrt{m}L)$ multiplicative increments of size $1 + 1/\sqrt{m}$ suffice. Gradient descent—the prototypical first-order method—converges at rate $1 - \mu/L$ per step, where μ and L bound the Hessian’s eigenvalues. Newton’s method (Lecture 18) will replace gradient descent inside the inner loop.

16.1. Why this lecture matters

Interior point methods are how modern solvers actually solve large LPs. This lecture lays out the entire architecture: a log barrier converts constraints into a penalty, the central path traces optimal solutions as the penalty weight grows, and an iterative method (gradient descent today, Newton’s method next time) refines each approximate solution. The gradient-descent detour is valuable beyond LP—it is the foundation of continuous optimisation and machine learning.

30-second story

1. The log barrier $-\sum \ln s_i$ keeps iterates in the polytope’s interior; λ controls the trade-off between the barrier and the true objective.
2. The minimiser $x(\lambda)$ traces the central path from the analytic centre ($\lambda = 0$) to the optimal vertex ($\lambda \rightarrow \infty$).
3. The cost gap satisfies $c^\top x(\lambda) - c^\top x^* \leq m/\lambda$, so large λ means near-optimality.
4. Incrementing λ by factor $1 + 1/\sqrt{m}$ per step yields $O(\sqrt{m}L)$ total iterations.
5. Gradient descent converges at rate $\alpha = 1 - \mu/L$ per step, where μ and L are eigenvalue bounds on the Hessian (the condition number $\kappa = L/\mu$ governs speed).
6. Newton’s method (Lecture 18) will replace gradient descent for the inner loop, using second-order information.

16.2. The interior point setup

16.2.1. The LP and its barrier reformulation

Start with the LP

$$\min c^\top x \quad \text{s.t.} \quad Ax \geq b, \tag{16.1}$$

where A is $m \times n$, $b \in \mathbb{R}^m$, $x \in \mathbb{R}^n$. Define the slack vector $s(x) \in \mathbb{R}^m$ with entries $s_i(x) = a_i^\top x - b_i$. For any feasible x , every slack is non-negative.

The interior point method replaces Equation 16.1 with the unconstrained problem

$$\min_{x \in \mathbb{R}^n} f_\lambda(x) = \lambda c^\top x - \sum_{i=1}^m \ln(s_i(x)). \quad (16.2)$$

The log barrier $p(s) = -\sum_{i=1}^m \ln(s_i)$ has two critical properties:

- It goes to $+\infty$ as any slack approaches zero (keeps iterates away from the boundary).
- It is strictly convex on the interior $\{x : s(x) > 0\}$, so the minimiser is unique.

Barrier intuition

Think of the polytope as a room. The barrier function is like a repulsive force from every wall. With $\lambda = 0$, you stand at the point farthest from all walls (the analytic centre). As λ increases, the cost function pulls you toward the optimal corner, but the barrier keeps you from crashing into the walls. You slide along a smooth curve—the central path—from the centre to the corner.

16.3. The central path

For each $\lambda \geq 0$, define $x(\lambda) = \arg \min_x f_\lambda(x)$. The curve

$$\{(\lambda, x(\lambda)) : \lambda \geq 0\}$$

is the central path.

- At $\lambda = 0$: $x(0)$ is the analytic centre—the point maximising $\sum_i \ln(s_i(x))$.
- As $\lambda \rightarrow \infty$: $x(\lambda) \rightarrow x^*$, the optimal vertex.

The path-following algorithm doesn't compute $x(\lambda)$ exactly. It maintains an approximate minimiser $\tilde{x}(\lambda_k)$ that stays near the central path, then uses it as a warm start for the next λ_{k+1} .

The name “path-following” comes from this picture: you start near one end of the central path (small λ , near the analytic centre) and walk along the curve, never straying too far from it, until you reach the other end (large λ , near the optimal vertex). The iterates $\tilde{x}(\lambda_0), \tilde{x}(\lambda_1), \dots$ are not on the curve itself—they orbit nearby. Each Newton step pulls them closer; each λ increment nudges them slightly off. The algorithm's art is balancing these two forces.

Why not just set λ large immediately?

If you jump directly to a very large λ , the barrier function f_λ has a narrow, steep valley near the optimal vertex. Starting from a random interior point, no iterative method would converge quickly—the initial error is enormous. By starting with small λ (where f_λ is smooth and easy to minimise) and incrementing gradually, each successive problem is only slightly harder than the last. The warm start ensures you're always close to the answer.

16.4. The cost gap: why large λ suffices

16.4.1. Gradient condition

Since f_λ is strictly convex, its minimiser $x(\lambda)$ satisfies

$$\nabla f_\lambda(x(\lambda)) = 0. \quad (16.3)$$

The gradient of Equation 16.2 is

$$\nabla f_\lambda(x) = \lambda c - A^\top S(x)^{-1} \mathbf{1}, \quad (16.4)$$

where $S(x) = \text{diag}(s_1(x), \dots, s_m(x))$ is the diagonal slack matrix and $\mathbf{1}$ is the all-ones vector.

16.4.2. Bounding the gap

Take the inner product of $\nabla f_\lambda(x(\lambda)) = 0$ with $x(\lambda) - x^*$:

$$0 = \langle \nabla f_\lambda(x(\lambda)), x(\lambda) - x^* \rangle.$$

Expanding:

$$\lambda c^\top (x(\lambda) - x^*) = \mathbf{1}^\top S(x(\lambda))^{-1} (s(x(\lambda)) - s(x^*)).$$

The right-hand side simplifies because $A(x(\lambda) - x^*) = s(x(\lambda)) - s(x^*)$ (the b 's cancel):

$$\lambda (c^\top x(\lambda) - c^\top x^*) = \sum_{i=1}^m \frac{s_i(x(\lambda)) - s_i(x^*)}{s_i(x(\lambda))}.$$

Since $s_i(x^*) \geq 0$ (it's a feasible point), each numerator satisfies $s_i(x(\lambda)) - s_i(x^*) \leq s_i(x(\lambda))$, so each fraction is at most 1. This is the only inequality in the entire argument—everything else is algebra and calculus. Summing over m terms:

$$c^\top x(\lambda) - c^\top x^* \leq \frac{m}{\lambda}. \quad (16.5)$$

! Cost gap bound

The gap between the central-path solution and the true optimum is at most m/λ . To achieve error ε , set $\lambda \geq m/\varepsilon$.

16.5. How many iterations?

16.5.1. The lambda schedule

The algorithm starts at λ_0 (exponentially small in L , the bit precision) and must reach $\lambda_{\text{final}} = \Theta(m \cdot 2^{O(L)})$ so that m/λ is small enough to round to the optimal vertex.

At each step:

$$\lambda_{k+1} = \left(1 + \frac{1}{\sqrt{m}}\right) \lambda_k. \quad (16.6)$$

💡 Counting the steps

A useful heuristic: $(1 + 1/x)^x \approx e$. So after $\sqrt{m}$ iterations, λ grows by a constant factor. To go from λ_0 to λ_{final} , the number of iterations is

$$\sqrt{m} \cdot \log\left(\frac{\lambda_{\text{final}}}{\lambda_0}\right) = O(\sqrt{m} \cdot L).$$

This $O(\sqrt{m}L)$ bound is due to Renegar (1988). Karmarkar's original 1984 algorithm needed $O(mL)$ iterations.

16.5.2. Recent improvements

Lee and Sidford (FOCS 2014—presented the week before this lecture) improved the bound to $\tilde{O}(\sqrt{\text{rank}(A)} \cdot L)$, where $\tilde{O}$ hides log factors. Since $\text{rank}(A) \leq \min(m, n)$, this is never worse and can be significantly better when the constraint matrix is low-rank.

i Aside (mine, not the lecturer's)

As of 2024, the state of the art for LP is $\tilde{O}(\sqrt{\text{rank}} \cdot L)$ iterations, each requiring roughly one linear-system solve. Whether LP admits a strongly polynomial algorithm—polynomial in m and n alone, independent of L —remains one of Smale's “problems for the 21st century.”

16.6. The vague outline, made concrete

Here is the full algorithm structure. Steps 1–4 are initialisation; step 5 is the main loop.

1. Modify the LP to LP' that has a trivial interior point (add a slack variable with a large penalty in the objective).
2. Show that if the original LP is feasible and bounded, the optimal solution to LP' encodes the optimal solution to the original LP.
3. Obtain an initial interior point x' for LP' .
4. From x' , compute $\tilde{x}(\lambda_0)$ —an approximate minimiser of f_{λ_0} with λ_0 very small.
5. Main loop: while $k < K$:
 - a. Use $\tilde{x}(\lambda_k)$ as a warm start for $f_{\lambda_{k+1}}$.
 - b. Run an iterative method (Newton) to refine $\tilde{x}(\lambda_k)$ into $\tilde{x}(\lambda_{k+1})$.
 - c. Increment k .
6. Round $\tilde{x}(\lambda_K)$ to the nearest vertex.

Each iteration of step 5 involves solving a constant number of $n \times n$ linear systems (the dominant cost). The total work is $O(\sqrt{m}L)$ system solves.

i Path-following vs. other IPMs

Not all interior point methods follow the central path. Potential-reduction methods, for example, decrease a potential function directly. But path-following methods are the most common in practice and the easiest to analyse, which is why this lecture focuses on them.

16.7. Digression: first-order optimisation

Before tackling Newton's method (Lecture 18), the lecturer builds up from the simpler case: gradient descent. This detour serves two purposes: it introduces the convergence-analysis template (upper-bound the function, choose the step that minimises the bound, show geometric error decay) and it clarifies what "second-order" methods buy you over "first-order" ones.

The goal is to minimise $f : \mathbb{R}^n \rightarrow \mathbb{R}$ given two ingredients:

1. A first-order oracle: given x , returns $f(x)$ and $\nabla f(x)$.
2. Hessian bounds: there exist $0 < \mu \leq L$ such that for all x ,

$$\mu I \leq \nabla^2 f(x) \leq LI, \quad (16.7)$$

where $\leq$ is the Löwner (positive semidefinite) ordering. In words: every eigenvalue of the Hessian lies in $[\mu, L]$.

i Löwner ordering

$A \leq B$ means $B - A$ is positive semidefinite: $v^\top (B - A)v \geq 0$ for all v . Equivalently, every eigenvalue of $B - A$ is non-negative. The upper bound $\nabla^2 f \leq LI$ says the function doesn't curve too sharply; the lower bound $\mu I \leq \nabla^2 f$ says it doesn't flatten out. Together, they say f is μ -strongly convex with L -Lipschitz gradient.

16.8. Gradient descent

16.8.1. The upper bound from Taylor's theorem

For any two points $x_0, x_1 \in \mathbb{R}^n$, Taylor's theorem with integral remainder gives

$$f(x_1) = f(x_0) + \nabla f(x_0)^\top (x_1 - x_0) + \int_0^1 \int_0^t (x_1 - x_0)^\top \nabla^2 f(x_\alpha) (x_1 - x_0) d\alpha dt, \quad (16.8)$$

where $x_\alpha = x_0 + \alpha(x_1 - x_0)$.

Using the upper bound $\nabla^2 f(x_\alpha) \leq LI$:

$$f(x_1) \leq f(x_0) + \nabla f(x_0)^\top (x_1 - x_0) + \frac{L}{2} \|x_1 - x_0\|^2. \quad (16.9)$$

16.8.2. The update rule

Gradient descent chooses x_1 to minimise the right-hand side of Equation 16.9. The right-hand side is a quadratic in x_1 with a known minimum. Differentiate with respect to x_1 and set to zero: $\nabla f(x_0) + L(x_1 - x_0) = 0$, giving

$$x_{k+1} = x_k - \frac{1}{L} \nabla f(x_k). \quad (16.10)$$

The step size $1/L$ is not a tuning parameter—it is determined entirely by the curvature bound. If L is large (the function curves sharply), you take small steps. If L is small (the function is nearly linear), you take large steps. This is why knowing the Hessian bound matters: it gives you the right step size for free.

Substituting back into Equation 16.9:

$$f(x_{k+1}) \leq f(x_k) - \frac{1}{2L} \|\nabla f(x_k)\|^2. \quad (16.11)$$

Gradient descent in one sentence

At each step, approximate f by a quadratic upper bound, jump to its minimum, and repeat. The step size $1/L$ ensures you never overshoot.

16.8.3. The lower bound from strong convexity

Using $\mu I \preceq \nabla^2 f$ in Equation 16.8 (with the inequality reversed):

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2} \|y - x\|^2 \quad \forall x, y. \quad (16.12)$$

Minimise the right-hand side over y :

$$f(x^*) \geq f(x) - \frac{1}{2\mu} \|\nabla f(x)\|^2,$$

which rearranges to

$$\|\nabla f(x)\|^2 \geq 2\mu(f(x) - f(x^*)). \quad (16.13)$$

16.8.4. Putting it together: convergence rate

The upper bound (Equation 16.11) says each step makes progress proportional to the squared gradient norm. The lower bound (Equation 16.13) says the squared gradient norm is at least 2μ times the current error. Chain these together:

$$f(x_{k+1}) - f(x^*) \leq f(x_k) - f(x^*) - \frac{1}{2L} \|\nabla f(x_k)\|^2 \leq \left(1 - \frac{\mu}{L}\right) (f(x_k) - f(x^*)). \quad (16.14)$$

! Gradient descent convergence

Each iteration of gradient descent multiplies the error by at most $\alpha = 1 - \mu/L$. After k iterations:

$$f(x_k) - f(x^*) \leq \left(1 - \frac{\mu}{L}\right)^k (f(x_0) - f(x^*)).$$

The condition number $\kappa = L/\mu$ governs the rate: to halve the error, you need $O(\kappa)$ iterations; to reduce by factor ε , $O(\kappa \log(1/\varepsilon))$ iterations.

```
import numpy as np

def gradient_descent(f, grad_f, x0, L_smooth, n_iter=100):
    """
    Minimise f using gradient descent with step size 1/L.

    Parameters
    -----
    f : callable          - objective function
    grad_f : callable     - gradient of f
    x0 : ndarray          - initial point
    L_smooth : float      - Lipschitz constant of the gradient (upper Hessian bound)
    n_iter : int          - number of iterations

    Returns
    -----
    x : ndarray - approximate minimiser
    history : list of f(x_k) values
    """
    x = x0.copy()
    step = 1.0 / L_smooth
    history = [f(x)]

    for _ in range(n_iter):
        x = x - step * grad_f(x)
        history.append(f(x))

    return x, history

# Example: minimise f(x) = 5*x1^2 + x2^2 (mu=2, L=10, kappa=5)
f = lambda x: 5 * x[0]**2 + x[1]**2
grad_f = lambda x: np.array([10 * x[0], 2 * x[1]])
x0 = np.array([3.0, 4.0])

x_opt, hist = gradient_descent(f, grad_f, x0, L_smooth=10.0, n_iter=50)
print(f"After 50 steps: x = {x_opt}, f(x) = {hist[-1]:.6e}")
```

16.9. Accelerated gradient descent

Gradient descent's $O(\kappa \log(1/\varepsilon))$ iteration count is not optimal in the first-order oracle model. Nesterov (*Soviet Mathematics Doklady*, 1983) showed that accelerated gradient descent achieves

$$O\left(\sqrt{\kappa} \log \frac{1}{\varepsilon}\right) \text{ iterations,}$$

a quadratic improvement in the condition number. The method adds a “momentum” term that uses the previous iterate’s direction.

i Aside (mine, not the lecturer’s)

For the interior point application in this course, the condition numbers involved will be $O(1)$, so the distinction between κ and $\sqrt{\kappa}$ doesn’t matter. But in machine learning and large-scale optimisation, acceleration is essential—it’s the difference between practical and impractical for ill-conditioned problems.

16.10. Looking ahead: Newton’s method

Gradient descent is a first-order method: the update depends only on f and ∇f . For the interior point inner loop, the algorithm uses Newton’s method, a second-order method that also uses the Hessian $\nabla^2 f$.

The key differences:

	Gradient descent	Newton’s method
Oracle	$f(x), \nabla f(x)$	$f(x), \nabla f(x), \nabla^2 f(x)$
Update	$x - \frac{1}{L} \nabla f(x)$	$x - [\nabla^2 f(x)]^{-1} \nabla f(x)$
Per-step cost	$O(n)$	$O(n^3)$ or one linear-system solve
Convergence	Linear: error $\times (1 - \mu/L)$	Quadratic (near optimum): error $\rightarrow$ error ²

Newton’s method replaces the global quadratic upper bound (gradient descent) with a local quadratic model built from the Hessian at the current point. Where gradient descent uses the same step size $1/L$ everywhere (based on a worst-case curvature estimate), Newton’s method adapts to the local curvature at each iterate. The payoff: once you’re close enough, the error squares at each step—so-called quadratic convergence.

Why does this matter for interior point? In the inner loop, each increment of λ changes f_λ only slightly. The warm start from the previous λ is already close to the new minimiser, so Newton’s method is already in its quadratic-convergence regime. A constant number of Newton steps—typically just one or two—suffice to bring the error down to the required tolerance. This is what makes the total iteration count $O(\sqrt{m}L)$ rather than much larger.

The cost per Newton step is one $n \times n$ linear system solve: $\nabla^2 f(x_k) \Delta x = -\nabla f(x_k)$. For dense matrices this is $O(n^3)$; for sparse constraint matrices, fast solvers can do much better. The next lecture will make all of this precise.

16.11. Cheat sheet

! Lecture 17 — key results

Interior point setup

- Barrier objective: $f_\lambda(x) = \lambda c^\top x - \sum_i \ln(s_i(x))$.
- Gradient: $\nabla f_\lambda(x) = \lambda c - A^\top S(x)^{-1} \mathbf{1}$.
- Analytic centre: minimiser when $\lambda = 0$.

Cost gap

- $c^\top x(\lambda) - c^\top x^* \leq m/\lambda$.
- To achieve error ε : need $\lambda \geq m/\varepsilon$.

Path-following algorithm

- Start: $\lambda_0 = e^{-O(L)}$.
- Update: $\lambda_{k+1} = (1 + 1/\sqrt{m}) \lambda_k$.
- Stop: $\lambda_K = \Theta(m \cdot e^{O(L)})$.
- Iterations: $O(\sqrt{m} L)$.
- Per iteration: constant number of $n \times n$ linear system solves.

Gradient descent

- Update: $x_{k+1} = x_k - \frac{1}{L} \nabla f(x_k)$.
- Progress: $f(x_{k+1}) \leq f(x_k) - \frac{1}{2L} \|\nabla f(x_k)\|^2$.
- Convergence: error $\times (1 - \frac{\mu}{L})$ per step.
- To halve error: $O(L/\mu) = O(\kappa)$ steps.

Key constants

- L : upper bound on Hessian eigenvalues (smoothness).
- μ : lower bound on Hessian eigenvalues (strong convexity).
- $\kappa = L/\mu$: condition number.
- Nesterov acceleration: $O(\sqrt{\kappa} \log(1/\varepsilon))$ iterations (optimal for first-order methods).

16.12. FAQ — questions students actually asked

Q: How many steps does it take to actually minimise f_λ inside each iteration? The algorithm doesn't compute exact minimisers—it computes *approximate* minimisers. In the path-following framework, the warm start from the previous λ is close enough that a constant number of Newton steps suffice. Each Newton step involves solving one $n \times n$ linear system.

Q: Do you always have a bound on one higher order of derivative than you use? Not always. For gradient descent (first-order), you need bounds on the Hessian (second-order). For Newton's method (second-order), the interior point analysis uses a different kind of control—self-concordance—rather than bounds on third derivatives.

Q: Newton's method normally just uses the gradient but achieves second-order convergence—right? Not quite. Standard Newton's method computes and inverts the Hessian at each step: $x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$. It achieves quadratic convergence near the optimum. “Quasi-Newton” methods (like BFGS) approximate the Hessian using only gradient information.

Q: What's the bit precision L in terms of the input? L captures the number of bits needed to represent the LP's data: $L \sim \log \|b\|_\infty + \log(\max \text{submatrix determinant of } A) + \log \|c\|_\infty + \log m + \log n$. Different papers define it slightly differently—some fold $\log m$ and $\log n$ into L , which is why some write the iteration count as $O(\sqrt{m} L)$ and others as $O(\sqrt{m}(L + \log m))$.

Q: How do you ensure the optimal vertex is unique? Perturb the cost vector c at very low-order bits (beyond the L -th bit of precision). The perturbation is too small to change which vertex is optimal (vertices are separated by at least $2^{-O(L)}$ in cost), but it breaks ties generically, making the optimum unique with probability 1.

16.13. Further reading

1. Karmarkar, N. “A new polynomial-time algorithm for linear programming.” *Combinatorica* 4(4), 1984. The original interior point method for LP.
2. Renegar, J. “A polynomial-time algorithm, based on Newton’s method, for linear programming.” *Mathematical Programming* 40(1), 1988. Improved the iteration count to $O(\sqrt{m}L)$.
3. Lee, Y. T. and Sidford, A. “Path finding methods for linear programming: solving linear programs in $\tilde{O}(\sqrt{\text{rank}})$ iterations and faster algorithms for maximum flow.” *FOCS*, 2014. State-of-the-art rank-dependent bound.
4. Nesterov, Y. “A method of solving a convex programming problem with convergence rate $O(1/k^2)$.” *Soviet Mathematics Doklady* 27(2), 1983. Accelerated gradient descent.
5. Boyd, S. and Vandenberghe, L. *Convex Optimization*. Cambridge University Press, 2004, Chapters 9–11. The standard textbook treatment of barrier methods and Newton’s method.
6. Nesterov, Y. and Nemirovski, A. *Interior-Point Polynomial Algorithms in Convex Programming*. SIAM, 1994. Comprehensive monograph on self-concordance and interior point theory.

16.14. Try it yourself

1. Cost gap calculation. An LP has $m = 100$ constraints. You want the cost gap $c^\top x(\lambda) - c^\top x^*$ to be at most 10^{-6} . What value of λ suffices? Starting from $\lambda_0 = 10^{-10}$ with the update $\lambda_{k+1} = (1 + 1/\sqrt{m})\lambda_k$, how many iterations are needed?
2. Gradient descent convergence. Consider $f(x) = \frac{1}{2}x^\top Qx$ where $Q = \text{diag}(1, 100)$. What are μ , L , and κ ? Starting from $x_0 = (1, 1)^\top$, run 200 steps of gradient descent and plot $f(x_k) - f(x^*)$ on a log scale. Verify that the slope matches $\log(1 - \mu/L)$.

```
import numpy as np
import matplotlib.pyplot as plt

Q = np.diag([1.0, 100.0])
mu, L_smooth = 1.0, 100.0

f = lambda x: 0.5 * x @ Q @ x
grad_f = lambda x: Q @ x

x = np.array([1.0, 1.0])
errors = []
for _ in range(200):
    errors.append(f(x))
    x = x - (1.0 / L_smooth) * grad_f(x)

plt.semilogy(errors)
plt.xlabel("Iteration k")
plt.ylabel("f(x_k) - f(x*)")
plt.title(f"Gradient descent: kappa = {L_smooth/mu:.0f}")
plt.show()
```

3. Central path visualisation. For the 2D LP $\min -x_1 - x_2$ subject to $x_1 \leq 3$, $x_2 \leq 2$, $x_1 + x_2 \leq 4$, $x_1, x_2 \geq 0$: write the barrier function $f_\lambda(x)$ and, for $\lambda \in \{0.1, 1, 10, 100\}$, numerically compute the minimiser using `scipy.optimize.minimize`. Plot the four points inside the feasible region and observe how they trace the central path toward the optimal vertex $(2, 2)$.

4. Condition number experiment. Generate random quadratic functions $f(x) = \frac{1}{2}x^T Qx$ with Q having condition number $\kappa \in \{10, 100, 1000\}$. For each, run gradient descent and measure how many iterations are needed to reduce the error by a factor of 10^{-4} . Verify the linear relationship between iteration count and κ .
5. Implementing the lambda schedule. Write a function that computes the sequence $\lambda_0, \lambda_1, \dots, \lambda_K$ for given m, L (bit precision), λ_0 , and stopping threshold. Return the number of iterations K .

```
import math

def lambda_schedule(m, L_bits, lam0=None, target_gap=1e-10):
    """
    Compute the path-following lambda schedule.

    Parameters
    -----
    m : int - number of constraints
    L_bits : int - bit precision
    lam0 : float - initial lambda (default: 2^{-L_bits})
    target_gap : float - desired cost gap m/lambda

    Returns
    -----
    K : int - number of iterations
    lam_final : float - final lambda value
    """
    if lam0 is None:
        lam0 = 2.0 ** (-L_bits)
    lam_target = m / target_gap
    factor = 1.0 + 1.0 / math.sqrt(m)

    K = math.ceil(math.log(lam_target / lam0) / math.log(factor))
    lam_final = lam0 * factor ** K

    return K, lam_final

K, lam_f = lambda_schedule(m=100, L_bits=32)
print(f"Iterations: {K}, final lambda: {lam_f:.2e}")
# Compare with O(sqrt(m) * L) estimate:
print(f"sqrt(m)*L estimate: {math.sqrt(100) * 32:.0f}")
```


17. Newton's Method and Completing the Interior Point Method

COMPSCI 224 Lecture 18 — Fall 2014

Newton's method exploits second-order information to converge quadratically when the Hessian is spectrally stable along the optimization path. Applied inside the interior point framework, it restores “fine centrality” in a constant number of steps after each barrier-parameter increment, yielding the $O(\sqrt{m}L)$ iteration bound for linear programming. The lecture closes with the initialization trick that bootstraps the algorithm from an arbitrary interior point.

17.1. Why this lecture matters

Gradient descent sees the landscape through a first-order lens: a slope and a step size. Newton's method adds curvature, treating the objective as locally quadratic and jumping directly to the minimum of that quadratic. The payoff is dramatic—under mild spectral-stability conditions, one Newton step squares the error. This lecture wires Newton's method into the interior point framework from the previous lecture, proving the $O(\sqrt{m}L)$ iteration complexity of path-following IPMs. The initialization trick at the end closes the last gap: how to find a starting point without already having solved the LP.

30-second story

1. Newton's method sets $x_1 = x_0 - H(x_0)^{-1} \nabla f(x_0)$, minimising the second-order Taylor expansion.
2. If the Hessian is spectrally stable along the Newton step, one step multiplies the error by $\approx \epsilon/(1 - \epsilon)$.
3. “Centrality” measures how close a point is to the minimiser of the barrier function f_λ in the Hessian-inverse norm.
4. A finely central point ($\delta \leq 1/100$) for λ_k remains coarsely central ($\delta \leq 1/3$) for $\lambda_{k+1} = (1 + \alpha)\lambda_k$ when $\alpha = O(1/\sqrt{m})$.
5. Constant Newton steps restore fine centrality, so the IPM needs $O(\sqrt{m}L)$ outer iterations total.
6. Initialisation: find an obvious interior point for an augmented LP, walk *backwards* along the central path to the analytic centre, swap in the real objective, then walk forward.

17.2. Newton's method: the second-order update

17.2.1. Setup and motivation

You want to minimise a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$. You have oracle access to three quantities at any query point x :

- $f(x)$ — the function value,
- $\nabla f(x)$ — the gradient (first-order),
- $\nabla^2 f(x)$ — the Hessian (second-order).

17. Newton's Method and Completing the Interior Point Method

Gradient descent approximates f by its *first-order* Taylor expansion plus a quadratic penalty, then minimises that surrogate. Newton's method goes one step further: it approximates f by its full *second-order* Taylor expansion and minimises that directly.

By Taylor's theorem, for any y near x_0 :

$$f(y) \approx f(x_0) + \nabla f(x_0)^\top (y - x_0) + \frac{1}{2}(y - x_0)^\top \nabla^2 f(x_0) (y - x_0) \quad (17.1)$$

Setting the gradient of the right-hand side to zero and solving gives the Newton update:

$$x_1 = x_0 - [\nabla^2 f(x_0)]^{-1} \nabla f(x_0) \quad (17.2)$$

💡 Intuition

Gradient descent asks “which direction is downhill?” and takes a cautious step. Newton's method asks “where is the bottom of the bowl I'm sitting in?” and jumps there in one shot. When the objective is nearly quadratic, that jump is nearly perfect.

17.2.2. Convergence theorem

The key assumption is spectral stability of the Hessian along the line segment from x_0 to x_1 .

Define $x_t = x_0 + t(x_1 - x_0)$ for $t \in [0, 1]$. Suppose there exists $\varepsilon > 0$ such that for all $t \in [0, 1]$:

$$(1 - \varepsilon) \nabla^2 f(x_0) \leq \nabla^2 f(x_t) \leq (1 + \varepsilon) \nabla^2 f(x_0) \quad (17.3)$$

Here $A \leq B$ means $B - A$ is positive semidefinite.

Theorem (Newton's method convergence). Under Equation 17.3:

$$\|\nabla f(x_1)\|_{\nabla^2 f(x_1)^{-1}} \leq \frac{\varepsilon}{1 - \varepsilon} \|\nabla f(x_0)\|_{\nabla^2 f(x_0)^{-1}} \quad (17.4)$$

where $\|v\|_A = \sqrt{v^\top A v}$ is the norm induced by a PSD matrix A .

i Interpreting the progress guarantee

At the minimiser, $\nabla f = 0$. The quantity $\|\nabla f(x)\|_{H^{-1}}$ measures distance to optimality in a geometry adapted to the curvature. Each Newton step multiplies this distance by $\varepsilon/(1 - \varepsilon)$. When $\varepsilon < 1/2$, this is a contraction—the error shrinks geometrically.

17.2.3. Proof of the convergence theorem

The proof has three moves: (1) express the new gradient as an integral, (2) exploit the Hessian stability to bound that integral, (3) switch norms carefully.

Step 1: Fundamental theorem of calculus. The gradient at the new iterate can be written as:

$$\nabla f(x_1) = \nabla f(x_0) + \int_0^1 \nabla^2 f(x_t)(x_1 - x_0) dt \quad (17.5)$$

Substitute the Newton step $x_1 - x_0 = -[\nabla^2 f(x_0)]^{-1} \nabla f(x_0)$ from Equation 17.2. The $\nabla f(x_0)$ and the $-\nabla^2 f(x_0)[\nabla^2 f(x_0)]^{-1} \nabla f(x_0)$ cancel, leaving:

$$\nabla f(x_1) = \int_0^1 [\nabla^2 f(x_0) - \nabla^2 f(x_t)] [\nabla^2 f(x_0)]^{-1} \nabla f(x_0) dt \quad (17.6)$$

Step 2: A spectral lemma. The condition $(1 - \varepsilon)A \leq B \leq (1 + \varepsilon)A$ is equivalent to $-\varepsilon I \leq A^{-1/2}(B - A)A^{-1/2} \leq \varepsilon I$. Define:

$$M_t = [\nabla^2 f(x_0)]^{-1/2} [\nabla^2 f(x_t) - \nabla^2 f(x_0)] [\nabla^2 f(x_0)]^{-1/2}$$

By Equation 17.3, $-\varepsilon I \leq M_t \leq \varepsilon I$ for all t . This implies $M_t^2 \leq \varepsilon^2 I$ —squaring a matrix sandwiched between $\pm \varepsilon I$ gives one bounded by $\varepsilon^2 I$.

Step 3: Norm computation. Take the $\|\cdot\|_{\nabla^2 f(x_0)^{-1}}$ norm of Equation 17.6. By the triangle inequality (bringing the norm inside the integral):

$$\|\nabla f(x_1)\|_{\nabla^2 f(x_0)^{-1}} \leq \int_0^1 \left\| [\nabla^2 f(x_0)]^{-1/2} M_t [\nabla^2 f(x_0)]^{-1/2} \nabla f(x_0) \right\|_2 dt$$

Writing $z = [\nabla^2 f(x_0)]^{-1/2} \nabla f(x_0)$, the integrand becomes $\|M_t z\|_2 \leq \varepsilon \|z\|_2$ (since $M_t^2 \leq \varepsilon^2 I$). But $\|z\|_2 = \|\nabla f(x_0)\|_{\nabla^2 f(x_0)^{-1}}$. So:

$$\|\nabla f(x_1)\|_{\nabla^2 f(x_0)^{-1}} \leq \varepsilon \|\nabla f(x_0)\|_{\nabla^2 f(x_0)^{-1}} \quad (17.7)$$

Step 4: Switch norms. The theorem states the bound in the x_1 -norm, not the x_0 -norm. Since the Hessians are $(1 \pm \varepsilon)$ -close, their inverses satisfy $\frac{1}{1+\varepsilon} H_0^{-1} \leq H_1^{-1} \leq \frac{1}{1-\varepsilon} H_0^{-1}$. Converting:

$$\|\nabla f(x_1)\|_{H_1^{-1}} \leq \frac{1}{\sqrt{1-\varepsilon}} \|\nabla f(x_1)\|_{H_0^{-1}} \leq \frac{\varepsilon}{\sqrt{1-\varepsilon}} \|\nabla f(x_0)\|_{H_0^{-1}}$$

A similar conversion on the right-hand side (expressing in the x_0 -norm) gives the final bound $\varepsilon/(1 - \varepsilon)$. $\square$

The spectral-switch step

Switching the norm from $\nabla^2 f(x_1)^{-1}$ to $\nabla^2 f(x_0)^{-1}$ costs a factor $1/(1 - \varepsilon)$ because the inverse Hessians are sandwiched within that factor. This is where the denominator in $\varepsilon/(1 - \varepsilon)$ comes from. The proof is tight: you cannot avoid paying this factor.

17.2.4. Comparison with gradient descent

	Gradient descent	Newton's method
Oracle	$f, \nabla f$	$f, \nabla f, \nabla^2 f$
Assumption	Hessian bounded ($\mu I \leq H \leq LI$)	Hessian <i>stable</i> along step
Convergence	Linear: error $\times (1 - \mu/L)$	Quadratic-like: error $\times \varepsilon$
Step cost	$O(n)$	$O(n^\omega)$ (linear system)
Step size	Requires tuning ($\eta = 1/L$)	No step size—determined by curvature

💡 When Newton wins

Newton's method pays more per step (matrix inversion vs. gradient evaluation) but needs far fewer steps. For the IPM, where we need high precision and the Hessian is structured (enabling fast solves), Newton is the right choice.

17.3. Centrality and the path-following strategy

17.3.1. The barrier function (recap)

The IPM solves $\min c^\top x$ subject to $Ax \geq b$ by repeatedly minimising the log-barrier:

$$f_\lambda(x) = \lambda c^\top x - \sum_{i=1}^m \ln(s_i(x)), \quad s(x) = Ax - b \quad (17.8)$$

As $\lambda \rightarrow \infty$, minimisers of f_λ converge to the LP optimum. The gradient and Hessian are:

$$\nabla f_\lambda(x) = \lambda c - A^\top S(x)^{-1} \mathbf{1}, \quad \nabla^2 f_\lambda(x) = A^\top S(x)^{-2} A \quad (17.9)$$

where $S(x) = \text{diag}(s(x))$.

17.3.2. Centrality measure

Define the centrality of a point x for parameter λ_k as:

$$\delta_k(x) = \left\| \nabla f_{\lambda_k}(x) \right\|_{[\nabla^2 f_{\lambda_k}(x)]^{-1}} \quad (17.10)$$

This is exactly the quantity that Newton's method contracts. When $\delta_k(x) = 0$, the point x is the exact minimiser of f_{λ_k} .

Definitions.

- x is finely central for f_{λ_k} if $\delta_k(x) \leq 1/100$.
- x is coarsely central for f_{λ_k} if $\delta_k(x) \leq 1/3$.

! The path-following invariant

The IPM maintains a finely central point $\tilde{x}_{\lambda_k}$ at all times. After incrementing λ , the point may degrade to coarsely central. A constant number of Newton steps restore fine centrality.

17.3.3. Recovering fine centrality from coarse centrality

If x is coarsely central, $\delta_k(x) \leq 1/3$. To apply the Newton convergence theorem, we must verify that the Hessian is spectrally stable along the Newton step. The Hessian $A^\top S(x)^{-2}A$ depends entirely on the slack vector $s(x) = Ax - b$. So we need: how much do slacks change along the Newton direction?

For a point x_t along the Newton step (i.e., $x_t = x + t(x_1 - x)$ where x_1 is the Newton iterate), the relative slack change is:

$$\|S(x)^{-1}(s(x_t) - s(x))\|_\infty \leq \|S(x)^{-1}A(x_t - x)\|_2$$

The right-hand side equals t times the centrality $\delta_k(x)$. Why? Because $x_t - x = -t H^{-1} \nabla f_\lambda(x)$, and composing with $S^{-1}A$ and taking the ℓ_2 -norm recovers exactly the centrality definition (after algebraic simplification using the Hessian structure $H = A^\top S^{-2}A$).

Since centrality is at most $1/3$, we get $\|S^{-1}(s(x_t) - s(x))\|_\infty \leq t/3$. For the Hessian to be $(1 \pm \varepsilon)$ -stable, we need this to be at most ε . Taking $\varepsilon = 1/3$: the condition holds for all $t \in [0, 1]$.

The Newton progress theorem then gives contraction factor $\varepsilon/(1 - \varepsilon) = (1/3)/(2/3) = 1/2$. Starting from centrality $1/3$, after k Newton steps: $\delta \leq (1/2)^k/3$. Reaching $1/100$ requires $k = \lceil \log_2(100/3) \rceil = 6$ steps.

💡 The self-referential structure

The proof is almost circular: we need small centrality to guarantee Hessian stability, and we need Hessian stability to guarantee that Newton's method reduces centrality. The trick is that coarse centrality ($\leq 1/3$) is *just barely* enough to guarantee stability with $\varepsilon = 1/3$, which is *just barely* a contraction ($1/2 < 1$). The constants $1/100$ and $1/3$ are chosen to make this self-consistency work.

17.4. How much can we increment λ ?

The critical question: given a finely central point for λ_k , how large can λ_{k+1} be while the point remains coarsely central?

Set $\lambda_{k+1} = (1 + \alpha)\lambda_k$. By the triangle inequality on the Hessian-inverse norm:

$$\delta_{k+1}(\tilde{x}_{\lambda_k}) \leq (1 + \alpha) \delta_k(\tilde{x}_{\lambda_k}) + \alpha \|A^\top S(\tilde{x})^{-1}1\|_{[V^2 f(\tilde{x})]^{-1}} \quad (17.11)$$

17.4.1. The projection bound

The key algebraic fact: the quantity $\|A^\top S^{-1}1\|_{[A^\top S^{-2}A]^{-1}}$ has a clean geometric interpretation.

Expand the norm definition. Let $H = A^\top S^{-2}A$. Then:

$$\|A^\top S^{-1}1\|_{H^{-1}}^2 = 1^\top S^{-1}A (A^\top S^{-2}A)^{-1} A^\top S^{-1}1$$

The matrix $S^{-1}A(A^\top S^{-2}A)^{-1}A^\top S^{-1}$ is an orthogonal projection Π onto the column space of $S^{-1}A$. This follows from the general formula: for any full-rank B , the matrix $B(B^\top B)^{-1}B^\top$ projects onto $\text{col}(B)$.

So the norm equals $\|\Pi 1\|_2$. Since projection cannot increase norm:

$$\|\Pi 1\|_2 \leq \|1\|_2 = \sqrt{m} \quad (17.12)$$

i Why a projection?

The projection matrix $\Pi = S^{-1}A(A^\top S^{-2}A)^{-1}A^\top S^{-1}$ is the “hat matrix” of weighted least squares. It projects 1 onto the subspace of vectors expressible as $S^{-1}Ay$ for some y . When A has full row rank ($m \leq n$), this subspace is all of $\mathbb{R}^m$ and $\Pi 1 = 1$ exactly, making the bound tight.

17.4.2. The $O(\sqrt{m})$ bound

Substituting Equation 17.12 into Equation 17.11:

$$\delta_{k+1}(\tilde{x}_{\lambda_k}) \leq (1 + \alpha) \cdot \frac{1}{100} + \alpha\sqrt{m}$$

For this to stay below $1/3$ (maintaining coarse centrality), we need:

$$\frac{1 + \alpha}{100} + \alpha\sqrt{m} \leq \frac{1}{3}$$

The dominant term is $\alpha\sqrt{m}$. Choosing $\alpha = 1/(10\sqrt{m})$ gives:

$$\frac{1}{100} + \frac{1}{1000\sqrt{m}} + \frac{1}{10} \approx 0.11 < \frac{1}{3} \quad \checkmark$$

17.4.3. Putting it all together: the complete iteration count

The IPM algorithm:

1. Start with a finely central point for $\lambda_0 \approx e^{-O(L)}$.
2. Set $\lambda_{k+1} = (1 + 1/(10\sqrt{m}))\lambda_k$.
3. Run $O(1)$ Newton steps to restore fine centrality.
4. Repeat until λ reaches $\approx m e^{O(L)}$.

The number of outer iterations is $\log_{1+1/\sqrt{m}}(m e^{2L}) \approx \sqrt{m} \cdot (2L + \ln m) = O(\sqrt{m} L)$.

! Iteration complexity

Each outer step increments λ by factor $(1 + 1/\sqrt{m})$. Going from $\lambda_{\text{init}} \approx e^{-L}$ to $\lambda_{\text{final}} \approx m e^L$ requires $O(\sqrt{m} L)$ increments, each costing $O(1)$ Newton steps. Total: $O(\sqrt{m} L)$ Newton steps, each dominated by solving an $n \times n$ linear system. The overall runtime is $O(\sqrt{m} L \cdot n^\omega)$ where ω is the matrix multiplication exponent.

17.5. Initialising the IPM

The path-following algorithm needs a finely central starting point. But we can't just pick an arbitrary feasible point—it won't have the right centrality. The trick is a three-phase bootstrap.

17.5.1. Phase 1: Augment the LP

Introduce a slack variable z and form:

$$\text{LP}' : \quad \min \quad c^\top x + Nz \quad \text{s.t.} \quad Ax + 1z \geq b, \quad 0 \leq z \leq 2^{L+1}, \quad -2^{L+1}1 \leq x \leq 2^{L+1}1$$

where N is exponentially large (e.g., $N = 100^L$). If the original LP is bounded and feasible, the optimum of LP' must have $z = 0$ —the penalty Nz forces z down.

LP' has an obvious interior point: set $x = 0$ and $z = \|b\|_\infty$. The constraint $Ax + 1z \geq b$ becomes $1\|b\|_\infty \geq b$, which holds entry-wise.

17.5.2. Phase 2: Find the analytic centre

The obvious interior point is not yet central for any useful λ . The trick: reverse-engineer a cost function for which it *is* perfectly central.

Set $\lambda = 1$. The gradient of f_λ is zero when $c = A^\top S(x')^{-1}1$. So define a synthetic cost $c' = A^\top S(x')^{-1}1$ —then the obvious point x' is *perfectly* central (centrality zero) for f_1 with cost c' .

Now walk backwards: decrement λ by factor $(1 - 1/\sqrt{m})$ repeatedly, running Newton steps to maintain fine centrality. When λ is exponentially small, the cost term is negligible and the point is near the analytic centre of the feasible region. At this point, fine centrality holds for *any* cost function.

17.5.3. Phase 3: Walk forward with the real cost

Switch the cost from c' to the real c . The point remains finely central because λ is tiny. Now run the forward path-following algorithm, incrementing λ by $(1 + 1/\sqrt{m})$ each step until convergence.

i Aside (mine, not the lecturer's)

The backwards-then-forwards initialisation is elegant but adds $O(\sqrt{m}L)$ steps to the runtime. In practice, most IPM implementations use heuristic warm-starts or the “big- M ” method, which avoids the backward walk at the cost of weaker worst-case guarantees.

17.6. Beyond $\sqrt{m}$: the Lee–Sidford improvement

The $\sqrt{m}$ in the iteration count comes from the projection bound Equation 17.12: projecting $1 \in \mathbb{R}^m$ onto the column space of $S^{-1}A$ can yield a vector as large as $\sqrt{m}$. But this is tight only when A has full row rank. What if $m \gg \text{rank}(A)$ —many redundant or nearly-redundant constraints?

Lee and Sidford (2014) improved the iteration count to $O(\sqrt{\text{rank}(A)}L)$ by using a weighted barrier:

$$\phi_w(x) = - \sum_{i=1}^m w_i \ln(s_i(x)) \quad (17.13)$$

17.6.1. Why the standard barrier is wasteful

Consider a simple example: the LP $\min c^\top x$ subject to $a^\top x \geq b$, but with the constraint written k times. The standard barrier sums $-\ln(a^\top x - b)$ a total of k times—effectively treating the same constraint as k independent constraints. The resulting centrality bound scales with $\sqrt{k}$ even though the feasible region is identical to the single-constraint version.

The weighted barrier assigns a weight w_i to each constraint. If you give the repeated constraint total weight 1 (split evenly as $1/k$ each), the effective “barrier parameter” depends on $\sum_i w_i$ rather than m . Lee and Sidford show that appropriate weights can reduce the effective dimension from m to $\text{rank}(A)$.

17.6.2. The algorithmic challenge

Finding the optimal weight vector requires solving a geometric problem: the weights must be “Lewis weights”—the leverage scores of the constraint matrix in the current slack geometry. These depend on x (through the slack), so they change as the algorithm progresses. Lee and Sidford show how to maintain an approximate weight vector efficiently, updating it alongside the central path traversal.

The analysis must also account for how centrality changes when both λ and w are updated simultaneously—a more complex version of the single-parameter analysis we did above.

i Aside (mine, not the lecturer's)

The weighted-barrier idea traces to Nesterov and Nemirovski's theory of self-concordant barriers, which proved *existence* of good weights non-constructively. Lee and Sidford's contribution was making the weight maintenance algorithmic and efficient. Their 2014 paper “Path Finding Methods for Linear Programming: Solving Linear Programs in $\tilde{O}(\sqrt{\text{rank}})$ Iterations” appeared at FOCS 2014. As of 2022, further improvements by Cohen, Lee, and Song achieve $O(n^\omega \log(n/\epsilon))$ time using a completely different approach based on inverse maintenance.

17.7. Cheat sheet

! Cheat sheet — Newton's Method and IPM Completion

Newton's method

- Update: $x_1 = x_0 - [\nabla^2 f(x_0)]^{-1} \nabla f(x_0)$
- Progress: $\|\nabla f(x_1)\|_{H(x_1)^{-1}} \leq \frac{\epsilon}{1-\epsilon} \|\nabla f(x_0)\|_{H(x_0)^{-1}}$
- Requires: Hessian stable within $(1 \pm \epsilon)$ along the step
- Converges when $\epsilon < 1/2$ (geometric contraction)

IPM centrality

- Centrality: $\delta_k(x) = \|\nabla f_{\lambda_k}(x)\|_{H(x)^{-1}}$
- Fine: $\delta \leq 1/100$. Coarse: $\delta \leq 1/3$.
- At coarse centrality, Hessian stability holds with $\epsilon = 1/3$
- $O(1)$ Newton steps: coarse $\rightarrow$ fine

Lambda schedule

- Increment: $\lambda_{k+1} = (1 + 1/(10\sqrt{m})) \lambda_k$
- Centrality jump: at most $(1 + \alpha)/100 + \alpha\sqrt{m}$
- Total outer iterations: $O(\sqrt{m}L)$
- Each iteration: one $n \times n$ linear system solve

Initialisation

1. Augment LP with slack variable z , large penalty Nz
2. Reverse-engineer cost for which obvious point is perfectly central
3. Walk backwards to analytic centre ($\lambda \rightarrow 0$)
4. Swap to real cost, walk forward ($\lambda \rightarrow \infty$)

Lee–Sidford (2014): weighted barriers reduce $\sqrt{m}$ to $\sqrt{\text{rank}(A)}$

17.8. FAQ — questions students actually asked

Q: Does the Hessian need to be invertible for Newton’s method to work?

For the IPM barrier function f_λ , the Hessian $A^\top S(x)^{-2}A$ is invertible whenever A has full column rank and x is in the strict interior (all slacks positive). In general, if the Hessian is singular, you restrict to its column space—but for our application, invertibility holds throughout.

Q: The epsilon in Newton’s theorem depends on the function. Does it change as we change λ ?

Yes, but that’s fine. We apply Newton’s method independently for each λ_k . As long as the ε for each instantiation is bounded by a fixed constant (here $1/3$), the constant number of Newton steps per outer iteration is guaranteed.

Q: Why does the projection bound give $\sqrt{m}$ and not something smaller?

The projection of $1 \in \mathbb{R}^m$ onto any subspace has ℓ_2 -norm at most $\|1\|_2 = \sqrt{m}$. This bound is tight when A has full row rank ($m \leq n$)—the projection is all of 1. When $\text{rank}(A) < m$, the projection is smaller, which is why Lee–Sidford achieves $\sqrt{\text{rank}}$.

Q: What’s the best known iteration complexity for LP?

As of this lecture (2014), Lee and Sidford achieved $O(\sqrt{\text{rank}(A)}L)$ iterations. The per-iteration cost is dominated by solving a linear system, which can be done in $O(n^\omega)$ time (where ω is the matrix multiplication exponent).

Q: Why walk backwards to the analytic centre instead of just starting at a small λ ?

You need a point that is *finely central* for a small λ . An arbitrary interior point has no centrality guarantee. Walking backwards from a perfectly central point (for the synthetic cost) maintains fine centrality at every step, delivering a certified starting point.

17.9. Further reading

- Nesterov and Nemirovski, *Interior-Point Polynomial Algorithms in Convex Optimization* (SIAM, 1994). The foundational monograph on self-concordant barriers and polynomial IPMs.
- Renegar, *A Mathematical View of Interior-Point Methods in Convex Optimization* (MPS-SIAM, 2001). A concise, readable treatment of the path-following framework.

- Lee and Sidford, “Path Finding Methods for Linear Programming: Solving Linear Programs in $\tilde{O}(\sqrt{\text{rank}})$ Iterations” (FOCS 2014). The improvement discussed at the end of lecture.
- Wright, *Primal-Dual Interior-Point Methods* (SIAM, 1997). Practical algorithms and implementation details.
- Nocedal and Wright, *Numerical Optimization*, Chapter 19 (Springer, 2006). Newton's method in the context of general nonlinear programming.

17.10. Try it yourself

1. Verify the Taylor step. Show that the minimiser of the second-order Taylor expansion $f(x_0) + \nabla f(x_0)^\top (y - x_0) + \frac{1}{2}(y - x_0)^\top H(y - x_0)$ is indeed $x_0 - H^{-1}\nabla f(x_0)$, by taking the gradient with respect to y and setting it to zero.
2. Spectral sandwiching of inverses. If $(1 - \varepsilon)A \leq B \leq (1 + \varepsilon)A$ for PSD invertible matrices A, B and $\varepsilon < 1$, show that $\frac{1}{1+\varepsilon}A^{-1} \leq B^{-1} \leq \frac{1}{1-\varepsilon}A^{-1}$.
3. Centrality after increment. Suppose $\delta_k(\tilde{x}) = 0$ (the point is the exact minimiser). Show that after setting $\lambda_{k+1} = (1 + \alpha)\lambda_k$, the centrality $\delta_{k+1}(\tilde{x}) \leq \alpha\sqrt{m}$. (Hint: the $(1 + \alpha)\delta_k$ term vanishes.)
4. Implement one Newton step. Given the LP $\min c^\top x$ s.t. $Ax \geq b$ with $A \in \mathbb{R}^{5 \times 3}$, implement a function that computes one Newton step for the barrier f_λ at a given interior point x . Use `numpy.linalg.solve` for the linear system. Verify that centrality decreases.

```
import numpy as np

def newton_step(A, b, c, lam, x):
    """One Newton step for the log-barrier f_lambda."""
    s = A @ x - b # slack vector
    S_inv = np.diag(1.0 / s)
    S_inv2 = np.diag(1.0 / s**2)
    grad = lam * c - A.T @ S_inv @ np.ones(len(b))
    H = A.T @ S_inv2 @ A
    dx = np.linalg.solve(H, -grad)
    return x + dx

# Example: 5 constraints, 3 variables
np.random.seed(42)
A = np.random.randn(5, 3)
x_feas = np.ones(3)
b = A @ x_feas - 1.0 # ensures slack >= 1
c = np.array([1.0, 2.0, 3.0])
lam = 1.0

x_new = newton_step(A, b, c, lam, x_feas)
print("New point:", x_new)
```

5. The rank gap. Construct an LP where $m = 100$ constraints but $\text{rank}(A) = 5$. Experimentally verify that the centrality jump $\delta_{k+1} - \delta_k$ grows with $\sqrt{\text{rank}(A)}$ rather than $\sqrt{m}$ when using the appropriate projection.

Part IV.

Multiplicative Weights and Network Flows

18. Multiplicative Weights and the Expert Advice Framework

COMPSCI 224 Lecture 19 — Fall 2014

The multiplicative weights update (MWU) method provides a near-optimal strategy for online prediction with expert advice: achieve total loss within $(1 + \epsilon)$ of the best expert plus an $O(\ln n/\epsilon)$ additive term. Beyond prediction, MWU yields fast approximate solvers for packing and covering LPs, reducing feasibility to a sequence of simple “oracle” calls on a convex set. The framework has been independently discovered across economics, game theory, machine learning, and algorithms.

18.1. Why this lecture matters

Most algorithms assume all information is available upfront. Online algorithms must act under uncertainty, making irrevocable decisions as data arrives. The multiplicative weights method is perhaps the single most versatile tool for this setting: it solves expert prediction, zero-sum games, flow problems, and LP relaxations through one unified update rule. This lecture builds the algorithm from first principles—starting with the trivial “kill wrong experts” strategy, progressing to weighted majority, and culminating in the full MWU guarantee. The LP application at the end connects online learning back to the optimisation thread running through the course.

30-second story

1. With a perfect expert, majority vote + killing wrong experts makes at most $\log_2 n$ mistakes.
2. With a best expert making e errors, a revival strategy achieves $(e + 1) \log_2 n$ mistakes.
3. Multiplicative weights does better: total mistakes $\leq 2(1 + \epsilon) \cdot e + O(\ln n/\epsilon)$.
4. The analysis uses a potential function (sum of weights) bounded above by induction and below by the best expert's weight.
5. The method generalises to losses in $[-1, 1]$ and probability distributions over experts.
6. For LP feasibility, each constraint becomes an “expert” and MWU focuses weight on unsatisfied constraints, yielding approximate solutions in $O(\ln m/\epsilon^2)$ rounds.

18.2. Warm-up: the halving algorithm

18.2.1. Setup

There are n experts and T days. Each day, every expert predicts “yes” or “no.” You must make a prediction, then reality is revealed. Your goal: minimise total mistakes.

Without a benchmark, this is hopeless—reality could be adversarial. The standard benchmark: be competitive with the best expert in hindsight.

18.2.2. Majority vote with elimination

Algorithm. Take a majority vote among surviving experts. At the end of each day, eliminate every expert who was wrong.

Why this works

Every time *you* are wrong, the majority was wrong—so at least half the surviving experts are eliminated. The pool halves on each mistake.

Claim. If there exists a perfect expert (one who is never wrong), you make at most $\lceil \log_2 n \rceil$ mistakes.

Proof. Each mistake kills at least half the experts. The perfect expert survives forever. After k mistakes, at most $n/2^k$ experts remain. Since at least one remains, $n/2^k \geq 1$, giving $k \leq \log_2 n$. $\square$

Limitation

If every expert errs on day 1, they're all dead. The algorithm has no recovery mechanism.

18.2.3. Handling imperfect experts: the revival trick

The halving algorithm assumes a perfect expert exists. In practice, even the best expert makes mistakes. If on day 1 every expert is wrong, you kill everyone—now what?

Modified algorithm. Run majority-vote-with-elimination as before. If all experts are dead, revive everyone and restart a new “phase.”

Claim. If the best expert makes e errors over all T days, you make at most $(e + 1) \log_2 n$ mistakes.

Proof. A “phase” begins at a revival event and ends at the next revival. The phase ends only when all experts—including the best one—are dead. So the best expert is killed exactly once per phase, meaning it made a mistake in that phase. Since the best expert makes e total mistakes, there are at most $e + 1$ phases (counting the initial one).

Within each phase, the best expert is alive and acting as a “perfect expert” among the survivors. By the halving argument, we make at most $\log_2 n$ mistakes before killing everyone. Summing across phases: total mistakes $\leq (e + 1) \log_2 n$. $\square$

This is a $\log_2 n$ competitive ratio: your mistakes $\leq \log_2 n \cdot (\text{best expert's mistakes}) + \log_2 n$. The factor $\log_2 n$ is large. Can we reduce the multiplicative factor to a constant?

The competitive analysis connection

This setup mirrors the competitive analysis framework from earlier in the course. OPT corresponds to the best expert in hindsight. The competitive ratio measures how much worse you perform compared to this benchmark that has the luxury of knowing the future.

18.3. The weighted majority algorithm

18.3.1. The key insight: penalise, don't kill

Instead of eliminating wrong experts, reduce their weight. Maintain a weight vector $w \in \mathbb{R}_{\geq 0}^n$, initialised to $w_i = 1$ for all i . On each day:

1. Take a weighted majority vote: predict the answer backed by the larger total weight.

2. After observing reality, for each wrong expert i : set $w_i \leftarrow (1 - \varepsilon) w_i$.

i Why not reward correct experts?

Multiplying correct experts by $(1 + \varepsilon)$ while leaving wrong experts unchanged is equivalent up to rescaling. All that matters is the *ratio* between weights. The convention of penalising wrong experts simplifies the algebra.

18.3.2. Analysis via potential function

The proof uses a classic technique: sandwich a potential function between an upper bound (from algorithm behaviour) and a lower bound (from any single expert), then compare.

Define the potential $\Phi(t) = \sum_{i=1}^n w_i(t)$. Initially $\Phi(1) = n$.

Upper bound on Φ . Consider a day when you (the algorithm) make a mistake. The weighted majority was wrong, meaning at least half the total weight supported the wrong answer. Those experts get penalised:

- At least $\Phi(t)/2$ weight was on wrong experts $\rightarrow$ those weights shrink by factor $(1 - \varepsilon)$
- At most $\Phi(t)/2$ weight was on correct experts $\rightarrow$ those weights stay the same

So:

$$\Phi(t+1) \leq \frac{\Phi(t)}{2}(1 - \varepsilon) + \frac{\Phi(t)}{2} = \left(1 - \frac{\varepsilon}{2}\right)\Phi(t) \quad (18.1)$$

On days when you're correct, some experts may still be wrong (and get penalised), so Φ can only decrease. The worst case for the upper bound is that the potential stays flat on correct days. By induction over the $M(T)$ mistake days:

$$\Phi(T+1) \leq n \left(1 - \frac{\varepsilon}{2}\right)^{M(T)} \leq n e^{-\varepsilon M(T)/2} \quad (18.2)$$

where the second inequality uses $1 - x \leq e^{-x}$.

Lower bound on Φ . The potential is a sum of non-negative weights, so it's at least any single weight. Pick expert i :

$$\Phi(T+1) \geq w_i(T+1) = (1 - \varepsilon)^{m_i(T)} \quad (18.3)$$

where $m_i(T)$ is expert i 's total mistakes by time T . Each mistake multiplies the weight by $(1 - \varepsilon)$; correct days leave it unchanged.

Combining the bounds. Since the lower bound $\leq \Phi(T+1) \leq$ upper bound:

$$(1 - \varepsilon)^{m_i(T)} \leq n e^{-\varepsilon M(T)/2}$$

Take logarithms:

$$m_i(T) \ln(1 - \varepsilon) \leq \ln n - \frac{\varepsilon}{2} M(T)$$

Rearrange to isolate $M(T)$:

$$M(T) \leq \frac{2 \ln n}{\varepsilon} - \frac{2}{\varepsilon} m_i(T) \ln(1 - \varepsilon)$$

18. Multiplicative Weights and the Expert Advice Framework

Now use the calculus bound: for $\varepsilon \in (0, 1/2]$, Taylor expansion gives $\ln(1-\varepsilon) \geq -\varepsilon - \varepsilon^2$, so $-\ln(1-\varepsilon) \leq \varepsilon + \varepsilon^2 = \varepsilon(1+\varepsilon)$. Therefore:

$$M(T) \leq 2(1 + \varepsilon) m_i(T) + \frac{2 \ln n}{\varepsilon} \quad (18.4)$$

Since this holds for any i , take the minimum:

! The multiplicative weights guarantee

For any $\varepsilon \in (0, 1/2]$, the weighted majority algorithm's total mistakes satisfies:

$$M(T) \leq 2(1 + \varepsilon) \cdot \min_i m_i(T) + \frac{2 \ln n}{\varepsilon}$$

The multiplicative factor is $2(1 + \varepsilon)$ (constant!) and the additive term is $O(\ln n/\varepsilon)$. Compared to the halving/revival algorithm's $O(\log n)$ multiplicative factor, this is a huge improvement—the price is paid only once as an additive term.

18.4. The full multiplicative weights update

18.4.1. Generalising the model

Two extensions make MWU applicable far beyond binary prediction:

1. Randomised predictions. Instead of committing to one answer, output a probability distribution $p(t)$ over experts. Your expected loss on day t is $m(t) \cdot p(t)$.
2. Continuous losses. Expert i 's loss $m_i(t) \in [-1, 1]$ (not just $\{0, 1\}$). Negative values represent gains.

18.4.2. The algorithm

Maintain weights $w_i(t)$, starting at $w_i(1) = 1$. The output distribution and update rule:

$$p_i(t) = \frac{w_i(t)}{\sum_j w_j(t)}, \quad w_i(t+1) = w_i(t) \cdot (1 - \varepsilon m_i(t)) \quad (18.5)$$

When $m_i(t) = 1$ (mistake), the weight shrinks by $(1 - \varepsilon)$ —exactly as before. When $m_i(t) = -1$ (gain), the weight grows by $(1 + \varepsilon)$.

💡 Intuition

MWU implements a “trust but verify” philosophy. Every expert starts equally trusted. After each round, the algorithm adjusts trust multiplicatively: wrong experts lose a fixed *fraction* of their influence. This ensures no expert is ever fully silenced (unlike elimination), but chronically wrong experts become negligible.

18.4.3. The general theorem

Theorem (MWU regret bound). For any expert i and any $\varepsilon \in (0, 1/2]$:

$$\sum_{t=1}^T m(t) \cdot p(t) \leq \sum_{t=1}^T m_i(t) + \varepsilon \sum_{t=1}^T |m_i(t)| + \frac{\ln n}{\varepsilon} \quad (18.6)$$

18.4.4. Proof of the general theorem

The proof follows the same potential-function template, adapted for continuous losses.

Upper bound. Write out how Φ changes in one step:

$$\Phi(t+1) = \sum_i w_i(t)(1 - \varepsilon m_i(t)) = \Phi(t) - \varepsilon \sum_i w_i(t) m_i(t)$$

Since $p_i(t) = w_i(t)/\Phi(t)$, the sum $\sum_i w_i(t) m_i(t) = \Phi(t) \cdot m(t) \cdot p(t)$. So:

$$\Phi(t+1) = \Phi(t)(1 - \varepsilon m(t) \cdot p(t)) \leq \Phi(t) e^{-\varepsilon m(t) \cdot p(t)}$$

Telescoping over all T steps:

$$\Phi(T+1) \leq n \exp\left(-\varepsilon \sum_{t=1}^T m(t) \cdot p(t)\right) \quad (18.7)$$

Lower bound. Isolate any expert i :

$$\Phi(T+1) \geq w_i(T+1) = \prod_{t=1}^T (1 - \varepsilon m_i(t)) \quad (18.8)$$

Bounding the product. Split into days with $m_i(t) \geq 0$ and $m_i(t) < 0$:

- When $m_i(t) \in [0, 1]$: the function $f(x) = (1 - \varepsilon)^x$ is convex, so $(1 - \varepsilon x) \geq (1 - \varepsilon)^x$. This gives a lower bound of $(1 - \varepsilon)^{m_i(t)}$.
- When $m_i(t) \in [-1, 0]$: similarly, $(1 - \varepsilon x) = (1 + \varepsilon|x|) \geq (1 + \varepsilon)^{|x|}$, giving $(1 + \varepsilon)^{|m_i(t)|}$.

Combining (and using $\ln(1 - \varepsilon) \geq -\varepsilon - \varepsilon^2$ and $\ln(1 + \varepsilon) \geq \varepsilon - \varepsilon^2$):

$$\ln w_i(T+1) \geq -(\varepsilon + \varepsilon^2) \sum_{t: m_i(t) \geq 0} m_i(t) + (\varepsilon - \varepsilon^2) \sum_{t: m_i(t) < 0} |m_i(t)|$$

Combining and rearranging. Set $\ln(\text{lower bound}) \leq \ln(\text{upper bound})$, isolate $\sum_t m(t) \cdot p(t)$, and use that $\varepsilon^2 \leq \varepsilon$ (since $\varepsilon \leq 1$) to absorb second-order terms into the $\varepsilon \sum_t |m_i(t)|$ term. The result is Equation 18.6.

i The many names of MWU

MWU has been rediscovered independently as: the Hedge algorithm (Freund & Schapire, 1997), the weighted majority algorithm (Littlestone & Warmuth, 1994), exponentiated gradient descent (Kivinen & Warmuth, 1997), and multiplicative weights (Arora, Hazan, Kale, 2012). In game theory, it appears as fictitious play and regret matching. In economics, it underpins no-regret learning in repeated games. The unifying survey is the Arora–Hazan–Kale monograph (2012).

18.5. Application: approximately solving LPs

18.5.1. The feasibility framework

Consider the feasibility problem: given $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, and a convex set $P \subseteq \mathbb{R}^n$, either:

- Certify infeasibility: no $x \in P$ satisfies $Ax \geq b$, or
- Find approximately feasible $x \in P$: $Ax \geq b - \varepsilon \mathbf{1}$.

18.5.2. The expert analogy

Each constraint i (row of A) is an “expert.” On each day t , the algorithm picks some $x(t) \in P$ and defines the loss vector:

$$m_i(t) = a_i^\top x(t) - b_i \quad (18.9)$$

(After appropriate normalisation to $[-1, 1]$.)

Positive $m_i(t)$ means constraint i is satisfied—the “expert” claiming it’s hard was wrong. Negative means the constraint is violated. MWU increases weight on violated constraints and decreases weight on well-satisfied ones, focusing future iterations on the hardest constraints.

18.5.3. What property does P need?

The algorithm requires an oracle: given a weight distribution p over constraints, find $x \in P$ that minimises (or approximately minimises) the weighted violation:

$$\text{Oracle}(p) : \min_{x \in P} \sum_{i=1}^m p_i (b_i - a_i^\top x)$$

This asks: “which $x \in P$ best satisfies the constraints that currently carry the most weight?” If the oracle returns an x that maximises weighted *satisfaction*, then the MWU framework guarantees convergence.

For packing/covering LPs, this oracle often decomposes beautifully. In a covering LP where $P = \{x \geq 0 : \mathbf{1}^\top x \leq \alpha\}$, the oracle reduces to: allocate budget α to the variable that covers the most weighted elements—a simple greedy or sorting step.

! MWU for LP feasibility

If the oracle can be evaluated in time T_{oracle} , MWU finds an ε -approximate feasible solution (or certifies infeasibility) in $O(\ln m/\varepsilon^2)$ oracle calls. Total time: $O(T_{\text{oracle}} \cdot \ln m/\varepsilon^2)$. For packing/covering LPs where the oracle is $O(n)$, the total is $O(n \ln m/\varepsilon^2)$ —often much faster than general LP solvers.

💡 Why the constraints-as-experts analogy works

Think of it this way: each constraint “complains” proportionally to how violated it is. MWU allocates attention (weight) to the loudest complainers. Over time, either all constraints become approximately satisfied, or the weight concentrates on a subset that *cannot* be satisfied from within P —a certificate of infeasibility.

18.5.4. Example: fractional set cover

Problem. Given a universe of elements and a collection of sets, find the minimum-weight sub-collection covering every element (LP relaxation: each $x_S \in [0, 1]$).

Suppose you know (or binary-search for) an upper bound $\alpha \geq \text{OPT}$. Set:

$$P = \{x \geq 0 : \sum_S x_S \leq \alpha\}$$

The constraints are: for each element e , $\sum_{S \ni e} x_S \geq 1$.

The MWU oracle with weights p on elements asks: which sets should you “buy” (allocate weight to) to maximally cover high-weight elements, subject to total budget α ? This decomposes into a simple greedy step.

After $O(\ln m/\varepsilon^2)$ rounds, MWU returns x with $\sum_{S \ni e} x_S \geq 1 - \varepsilon$ for all e . Rescaling by $1/(1 - \varepsilon)$ gives a feasible solution of cost at most $\alpha/(1 - \varepsilon) = (1 + O(\varepsilon))\alpha$.

! MWU vs. interior point for LPs

MWU gives $(1 + \varepsilon)$ -approximate solutions in time polynomial in $1/\varepsilon$. Interior point gives exact solutions (up to bit precision L) in $O(\sqrt{m}L)$ iterations. For applications needing only constant-factor approximations (like feeding into randomised rounding), MWU is often faster. For exact LP solving, interior point wins because it needs $O(\log(1/\varepsilon))$ iterations to reach precision ε , while MWU needs $O(1/\varepsilon^2)$.

18.5.5. The full MWU-LP pipeline

Here is the complete procedure for using MWU to solve a covering LP:

1. Binary search on the objective value α (reducing optimisation to feasibility).
2. Define $P = \{x \geq 0 : c^\top x \leq \alpha\}$ and the constraints $Ax \geq b$.
3. Initialise weights $w_i = 1$ for each constraint i .
4. Repeat for $T = O(\ln m/\varepsilon^2)$ rounds:
 - a. Normalise: $p_i = w_i / \sum_j w_j$.
 - b. Call the oracle: find $x(t) \in P$ minimising $\sum_i p_i(b_i - a_i^\top x)$.
 - c. Compute the loss: $m_i(t) = (a_i^\top x(t) - b_i)$ (normalised to $[-1, 1]$).
 - d. Update weights: $w_i \leftarrow w_i(1 - \varepsilon m_i(t))$.
5. Output the average $\bar{x} = \frac{1}{T} \sum_{t=1}^T x(t)$.

The averaging step is crucial: individual iterates $x(t)$ may violate some constraints badly, but the *average* inherits the MWU regret bound—ensuring approximate feasibility across all constraints simultaneously.

18.6. Cheat sheet

! Cheat sheet — Multiplicative Weights

Halving algorithm (perfect expert exists)

- Majority vote + kill wrong experts
- At most $\log_2 n$ mistakes
- Fails when all experts err simultaneously

Weighted majority (binary losses)

- $w_i \leftarrow (1 - \varepsilon) w_i$ for wrong experts
- Guarantee: $M(T) \leq 2(1 + \varepsilon) \min_i m_i(T) + 2 \ln n / \varepsilon$
- Multiplicative factor is constant (not $\log n!$)

Full MWU (losses in $[-1, 1]$, randomised output)

- $p_i(t) = w_i(t) / \sum_j w_j(t)$
- Update: $w_i(t+1) = w_i(t)(1 - \varepsilon m_i(t))$
- Regret: $\sum_t m(t) \cdot p(t) \leq \sum_t m_i(t) + \varepsilon \sum_t |m_i(t)| + \ln n / \varepsilon$

Potential function technique

- $\Phi(t) = \sum_i w_i(t)$
- Upper: $\Phi(T+1) \leq n e^{-\varepsilon \sum_t m(t) \cdot p(t)}$
- Lower: $\Phi(T+1) \geq w_i(T+1) = \prod_t (1 - \varepsilon m_i(t))$
- Take logs, apply calculus bounds, rearrange

LP application

- Each constraint = one expert
- Loss = how much constraint is satisfied (positive) or violated (negative)
- Oracle: minimise weighted violation over P
- Rounds: $O(\ln m / \varepsilon^2)$ for ε -feasibility

18.7. FAQ — questions students actually asked

Q: Can you go back to a killed expert in the revival algorithm?

Yes. “Killing” means ignoring, not permanent deletion. When everyone is dead, you revive all experts—including those killed earlier. The best expert gets revived each time it’s incorrectly eliminated.

Q: Why not increase weights for correct experts instead of decreasing for wrong ones?

They’re equivalent up to rescaling. If you multiply correct experts by $(1 + \varepsilon)$ and leave wrong ones unchanged, the *ratios* between weights evolve identically to the penalty-only version. The convention of only penalising simplifies notation because the total potential never increases on correct days.

Q: In the LP application, what happens if the problem is actually infeasible?

If no $x \in P$ satisfies $Ax \geq b$, then for any $x(t)$ the oracle returns, there’s always significant violation. The MWU guarantee implies that the weighted average loss $\sum_t m(t) \cdot p(t)$ is bounded. If after $O(\ln m / \varepsilon^2)$ rounds no approximately feasible point emerges, you can certify infeasibility (the dual certificate comes from the weight vector p itself).

Q: The MWU guarantee has a factor of 2. Can you get $1 + \varepsilon$ instead?

The factor of 2 comes from the “at least half the weight was wrong” argument for the deterministic weighted majority. In the randomised version (output a distribution p), the expected loss is $m(t) \cdot p(t)$ —no factor of 2. The general theorem Equation 18.6 achieves $(1 + \varepsilon)$ -competitiveness.

Q: Why is MWU’s dependence on $1/\varepsilon$ polynomial while interior point’s is logarithmic?

MWU makes “soft” progress—adjusting weights multiplicatively converges slowly because it can never fully zero out a constraint. Interior point’s Newton steps converge quadratically because they exploit second-order information. For exact LP solving, interior point’s $O(\sqrt{m} \log(1/\epsilon))$ beats MWU’s $O(1/\epsilon^2)$. MWU shines when you only need constant-factor approximations.

18.8. Further reading

- Arora, Hazan, and Kale, “The Multiplicative Weights Update Method: A Meta-Algorithm and Applications” (*Theory of Computing*, 2012). The definitive survey—covers history, variants, and applications to games, LP solving, and boosting.
- Freund and Schapire, “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting” (*JCSS*, 1997). Introduces the Hedge algorithm and connects MWU to AdaBoost.
- Littlestone and Warmuth, “The Weighted Majority Algorithm” (*Information and Computation*, 1994). The original weighted majority paper with the $2(1 + \epsilon)$ bound.
- Plotkin, Shmoys, and Tardos, “Fast Approximation Algorithms for Fractional Packing and Covering Problems” (*Mathematics of Operations Research*, 1995). First application of MWU-style methods to LP approximation.
- Cesa-Bianchi and Lugosi, *Prediction, Learning, and Games* (Cambridge, 2006). Textbook treatment of online learning and expert advice.

18.9. Try it yourself

1. Tight example for halving. Construct an instance with $n = 2^k$ experts and one perfect expert where the majority-vote-with-elimination algorithm makes exactly $k = \log_2 n$ mistakes.
2. Tuning ϵ . The bound Equation 18.4 has two terms: $2(1 + \epsilon)e$ and $2 \ln n/\epsilon$. If you know in advance that the best expert makes e errors, what ϵ minimises the bound? What total mistake count does this give?
3. Potential function for MWU. Carry out the full calculation: starting from $\Phi(T + 1) \geq \prod_t (1 - \epsilon m_i(t))$ and $\Phi(T + 1) \leq n e^{-\epsilon \sum_t m(t) \cdot p(t)}$, derive Equation 18.6. You will need: $\ln(1 - x) \geq -x - x^2$ for $x \in [0, 1/2]$, and the convexity bound $(1 - \epsilon)^x \leq 1 - \epsilon x$ for $x \in [0, 1]$.
4. Implement MWU for set cover. Given a universe $U = \{1, \dots, 20\}$ and random sets, implement MWU to find an approximately feasible fractional set cover. Track how the constraint weights evolve over iterations.

```
import numpy as np

def mwu_set_cover(A, alpha, eps=0.1, max_iter=500):
    """
    A: m x n binary matrix (A[i,j] = 1 if element i in set j)
    alpha: budget upper bound on sum of x
    eps: approximation parameter
    Returns: approximately feasible x
    """
    m, n = A.shape
    w = np.ones(m) # weights on elements (constraints)
    x_avg = np.zeros(n)

    for t in range(max_iter):
        p = w / w.sum() # distribution over constraints

        # Oracle: greedily allocate budget to maximise weighted coverage
```

```

# weighted benefit of set j = sum of p[i] for elements i in set j
benefit = A.T @ p # shape (n,)
x_t = np.zeros(n)
# allocate all budget to highest-benefit set (simple oracle)
x_t[np.argmax(benefit)] = alpha

x_avg += x_t

# Compute losses: positive = satisfied, negative = violated
coverage = A @ x_t # how much each element is covered
loss = (coverage - 1.0) # want coverage >= 1
loss = np.clip(loss, -1, 1) # normalise to [-1,1]

# Update weights
w *= (1 - eps * loss)

x_avg /= max_iter
return x_avg

# Example
np.random.seed(0)
n_elements, n_sets = 20, 30
A = (np.random.rand(n_elements, n_sets) > 0.7).astype(float)
# Make sure every element is in at least one set
for i in range(n_elements):
    if A[i].sum() == 0:
        A[i, np.random.randint(n_sets)] = 1.0

x_sol = mwu_set_cover(A, alpha=5.0, eps=0.1)
coverage = A @ x_sol
print(f"Min coverage: {coverage.min():.3f} (want >= 1)")
print(f"Total cost: {x_sol.sum():.3f}")

```

5. MWU vs. interior point. For random LP instances of increasing size ($m = 10, 50, 200, 1000$ constraints), compare the number of iterations MWU needs for $\varepsilon = 0.01$ feasibility versus the $O(\sqrt{m}L)$ iteration count of a path-following IPM. At what problem size does one dominate the other?

19. Multiplicative Weights for LPs and Introduction to Network Flows

COMPSCI 224 Lecture 20 — Fall 2014

Multiplicative weights, applied through a row-bounded oracle, yields an approximate LP feasibility solver whose iteration count depends only on the oracle's error and the number of constraints. The set cover LP illustrates the technique: a greedy oracle gives $O(m^2 \log m/\epsilon^2)$ iterations for a $(1 + \epsilon)$ -approximate fractional solution. The lecture then pivots to network flows, defining feasible flows, residual graphs, and the Ford–Fulkerson augmenting-path framework, setting the stage for faster combinatorial flow algorithms.

19.1. Why this lecture matters

Multiplicative weights is one of the most versatile algorithmic paradigms in theoretical CS, but its power is only realised once you can *plug it in* to concrete problems. This lecture shows the plug-in mechanism: reduce LP feasibility to a sequence of simpler “one-constraint” sub-problems, each solved by a *row-bounded oracle*. The oracle abstraction cleanly separates the general framework from the problem-specific cleverness. The second half introduces max flow—a problem whose study has driven decades of algorithmic progress in graph theory, LP solvers, and data structures.

30-second story

1. The multiplicative weights theorem bounds cumulative expected cost against the best expert, with regret $O(\sqrt{T \ln n})$.
2. LP feasibility with m constraints reduces to repeatedly solving a single-constraint sub-problem via a *row-bounded oracle*.
3. Averaging the oracle's outputs over $O(\gamma^2 \log m/\epsilon^2)$ rounds yields a point that violates every constraint by at most ϵ .
4. For fractional set cover, a greedy oracle picks the set with the highest weighted coverage, giving $\gamma \leq m$.
5. Max flow is an LP, but combinatorial algorithms (Ford–Fulkerson) exploit graph structure to avoid generic LP solvers.
6. Ford–Fulkerson repeatedly augments along residual-graph paths; correctness follows from the max-flow min-cut theorem.

19.2. Multiplicative weights for LP feasibility

19.2.1. The multiplicative weights guarantee (recap)

Recall the theorem from Lecture 19. There are n experts and T rounds. In round t , the algorithm outputs a probability vector $p_t \in \Delta_n$ and then observes a cost vector $m_t \in [-1, 1]^n$. The multiplicative weights update rule guarantees:

$$\sum_{t=1}^T m_t \cdot p_t \leq \min_{i \in [n]} \sum_{t=1}^T m_{t,i} + \varepsilon \sum_{t=1}^T |m_{t,i}| + \frac{\ln n}{\varepsilon} \quad (19.1)$$

for any $\varepsilon \in (0, 1/2)$. Since every $|m_{t,i}| \leq 1$, the absolute-value sum is at most T , so the regret (the excess beyond the best expert) is at most $\varepsilon T + (\ln n)/\varepsilon$. Choosing $\varepsilon = \sqrt{(\ln n)/T}$ balances these terms, giving average regret $O(\sqrt{(\ln n)/T})$ that vanishes as T grows.

19.2.2. The LP feasibility problem and row-bounded oracles

Consider the following feasibility problem:

$$\text{Find } x \in P \text{ such that } Ax \geq b \quad (19.2)$$

where P is a convex set and A is an $m \times n$ matrix. There are m constraints (the rows of A). The key insight is to *softify* the m constraints into a single constraint by taking a weighted combination.

Sub-problem (★). Given a probability vector p over the m constraints, find $x \in P$ such that $p^\top Ax \geq p^\top b$.

i Why is (★) easier?

The original problem has m constraints (plus the membership in P). Sub-problem (★) has *one* linear constraint (plus membership in P). If the original problem is feasible, then (★) is always feasible: any x satisfying $Ax \geq b$ satisfies every non-negative combination of those inequalities.

A γ -bounded oracle is an algorithm that, given (★):

- If feasible: returns $x \in P$ such that $|a_i \cdot x - b_i| \leq \gamma$ for every row i .
- If infeasible: reports “infeasible” (which implies the original problem is infeasible).

The parameter γ measures how far the oracle’s solution can violate individual constraints in absolute value. The oracle need not return a point feasible for the original system—it only solves the easier one-constraint problem while bounding row-wise error.

19.2.3. The theorem: MW + oracle $\Rightarrow$ approximate LP solver

! MW for LP feasibility

Given a γ -bounded oracle with $\gamma \geq \varepsilon/2$, multiplicative weights finds $\tilde{x} \in P$ satisfying $a_i \cdot \tilde{x} \geq b_i - \varepsilon$ for all i in

$$T = O\left(\frac{\gamma^2 \ln m}{\varepsilon^2}\right) \quad (19.3)$$

iterations.

19.2.4. Proof sketch

Map the LP to the experts framework:

- Experts = the m constraints.
- Day t : Given the current probability vector p_t , query the oracle. If it reports infeasible, halt (the original LP is infeasible). Otherwise, receive $x_t \in P$.
- Cost vector: Define $m_t = \frac{1}{\gamma}(Ax_t - b)$. Dividing by γ ensures each entry lies in $[-1, 1]$ (the oracle's guarantee).

The oracle guarantees $p_t^\top Ax_t \geq p_t^\top b$, so $m_t \cdot p_t = \frac{1}{\gamma}(p_t^\top Ax_t - p_t^\top b) \geq 0$.

Summing over all T rounds, the left-hand side of Equation 19.1 is non-negative. The right-hand side, for every constraint i , gives:

$$0 \leq \frac{1}{\gamma} \sum_{t=1}^T (a_i \cdot x_t - b_i) + \eta T + \frac{\ln m}{\eta}$$

where η is the MW error parameter (called η to avoid confusion with the final ε). Multiply through by γ/T and define $\tilde{x} = \frac{1}{T} \sum_t x_t$ (which lies in P by convexity):

$$0 \leq a_i \cdot \tilde{x} - b_i + \gamma\eta + \frac{\gamma \ln m}{\eta T}$$

Set $\eta = \varepsilon/(2\gamma)$ to make $\gamma\eta = \varepsilon/2$. Set T so that $\gamma \ln m/(\eta T) \leq \varepsilon/2$, giving $T = 4\gamma^2 \ln m/\varepsilon^2$. Both error terms are at most $\varepsilon/2$, so:

$$a_i \cdot \tilde{x} \geq b_i - \varepsilon \quad \text{for all } i$$

The output $\tilde{x}$ is the *average* of all oracle outputs—feasible for every constraint up to an additive ε .

19.3. Application: fractional set cover

Recall: given sets $S_1, \dots, S_m$ covering a universe $\{1, \dots, n\}$, the fractional set cover LP asks to minimise $\sum_s x_s$ subject to $\sum_{s \ni e} x_s \geq 1$ for every element e , with $x \geq 0$.

To use the MW framework, separate the *hard* covering constraints (which become the experts) from the *easy* budget constraint:

$$P = \{x \geq 0 : \sum_s x_s \leq \beta\}$$

Binary-search on β (the optimal cost lies between 1 and m) to turn optimisation into feasibility.

19.3.1. Building the oracle

Given probability vector p over elements, the oracle maximises $p^\top Ax$ subject to $x \in P$. Write out the objective:

$$p^\top Ax = \sum_e p_e \sum_{s \ni e} x_s = \sum_s x_s \cdot p(s), \quad \text{where } p(s) = \sum_{e \in s} p_e$$

This is a linear function of x with non-negative coefficients $p(s)$. Maximising over P (a scaled simplex) is trivial: put all mass on the set with the largest $p(s)$.

$$\text{Oracle: set } x = \beta \cdot e_{s^*} \quad \text{where } s^* = \arg \max_s p(s)$$

If $\beta \cdot p(s^*) < 1$, the sub-problem is infeasible (the maximum weighted coverage is less than 1 even using full budget), so report infeasible.

19.3.2. Bounding γ

For any element e , the constraint violation $|a_e \cdot x - b_e|$ has two cases:

- $e \notin s^*$: the dot product is 0, so the violation is $|0 - 1| = 1$.
- $e \in s^*$: the dot product is β , so the violation is $|\beta - 1|$.

Since $\beta \in [1, m]$ (from the binary search), the worst case is $\gamma = \max(1, \beta - 1) \leq \beta \leq m$.

! Important

Plugging $\gamma = m$ into Equation 19.3 gives $O(m^2 \log m / \varepsilon^2)$ iterations of multiplicative weights. Each iteration calls the oracle (linear-time scan over sets) and updates the probability vector. Rescaling the ε -violated solution yields a $(1 + \varepsilon)$ -approximate fractional set cover.

i Aside (mine, not the lecturer's)

The MW approach to LP feasibility originates in Plotkin, Shmoys, and Tardos (1995) for packing/covering LPs. The general oracle-based framework appears in the Arora, Hazan, and Kale survey (2012), which unifies multiplicative weights, boosting, and online learning under a single umbrella.

19.4. Network flows: definitions

19.4.1. Setup

A flow network is a directed graph $G = (V, E)$ with:

- Non-negative integer capacities $c_e \geq 0$ for every edge $e \in E$.
- A distinguished source s and sink t .

Write $n = |V|$ and $m = |E|$. Assume G is connected (otherwise, run DFS first and work only with the component containing both s and t). Since the graph is connected, $m \geq n - 1$; all bounds will be stated in terms of m .

19.4.2. Flows

An s - t flow is a vector $f \in \mathbb{R}^E$ satisfying:

1. Non-negativity: $f_e \geq 0$ for all e .
2. Conservation: for every vertex $v \neq s, t$, the total flow entering v equals the total flow leaving v :

$$\sum_{(u,v) \in E} f_{(u,v)} = \sum_{(v,w) \in E} f_{(v,w)} \quad (19.4)$$

3. Source–sink balance: the total flow leaving s equals the total flow entering t .

A flow is feasible if $f_e \leq c_e$ for every edge e . The value of f , written $\text{val}(f)$, is the net flow leaving s .

19.4.3. A worked example

Consider a graph with four vertices $\{s, a, b, t\}$ and five unit-capacity edges: $s \rightarrow a$, $s \rightarrow b$, $a \rightarrow b$, $a \rightarrow t$, $b \rightarrow t$.

- The zero flow ($f_e = 0$ everywhere) is feasible with value 0.
- Sending 1 unit along $s \rightarrow a \rightarrow b \rightarrow t$ gives a feasible flow of value 1.
- Sending 1 unit along $s \rightarrow a \rightarrow t$ and 1 unit along $s \rightarrow b \rightarrow t$ gives a feasible flow of value 2.

The second option is better. The goal of max flow is to find the best possible.



Tip

Think of flow as water shipping through pipes. Each pipe (edge) has a maximum throughput (capacity). Water enters at s and exits at t ; at every intermediate junction, what flows in must flow out. The goal is to ship as much water as possible.

19.4.4. Max flow as a linear program

Max flow is a linear program:

$$\max \sum_{(s,u) \in E} f_{(s,u)} \quad \text{s.t.} \quad 0 \leq f_e \leq c_e \quad \forall e, \quad \text{conservation at every } v \neq s, t \quad (19.5)$$

Since it is an LP, one could solve it via:

- Simplex: no known polynomial-time pivot rule.
- Ellipsoid: polynomial, but depends on $\log(\max c_e)$; impractical.
- Interior point: $O(\sqrt{m})$ iterations, each solving a linear system.

For a general LP with m constraints, each Newton step costs $O(m^3)$ or $O(m^\omega)$ using fast matrix multiplication, giving $O(m^{3.5})$ overall (times precision factors).

Faster via Laplacian solvers

The linear systems arising from flow LPs have special structure: they are *Laplacian systems*. Daich and Spielman (2008) showed these can be solved in $\tilde{O}(m)$ time, yielding an $\tilde{O}(m^{3/2})$ interior-point algorithm for max flow. Lee and Sidford (2014) further improved the interior-point iteration count from $O(\sqrt{m})$ to $O(\sqrt{n})$ by reweighting the log-barrier, giving $\tilde{O}(m\sqrt{n})$ for max flow.

19.5. The Ford–Fulkerson method

19.5.1. Residual graphs

Given a flow f in G , the residual graph G_f is defined on the same vertex set with modified capacities:

$$c_f(u, v) = c(u, v) - f(u, v) + f(v, u) \quad (19.6)$$

The first term is the *leftover* capacity on edge (u, v) . The second term accounts for the possibility of *cancelling* flow on the reverse edge.

Convention: for every edge $(u, v) \in E$, include both (u, v) and (v, u) in the edge set (adding a zero-capacity reverse edge if it does not already exist). Among any edge and its reverse, at most one carries positive flow—if both did, subtract the smaller from both without changing any constraint. This at most doubles the edge count, which only changes big- O bounds by a constant.

Worked example. Suppose we have four vertices $\{s, a, b, t\}$ with edges $s \rightarrow a$ (capacity 7), $a \rightarrow b$ (capacity 5), $b \rightarrow t$ (capacity 2), plus edges $s \rightarrow b$ (capacity 3) and $a \rightarrow t$ (capacity 4). Push 2 units of flow along $s \rightarrow a \rightarrow b \rightarrow t$. The residual graph has:

- $c_f(s, a) = 7 - 2 = 5$, and reverse $c_f(a, s) = 0 + 2 = 2$.
- $c_f(a, b) = 5 - 2 = 3$, and reverse $c_f(b, a) = 0 + 2 = 2$.
- $c_f(b, t) = 2 - 2 = 0$ (saturated), and reverse $c_f(t, b) = 0 + 2 = 2$.
- Edges $s \rightarrow b$ and $a \rightarrow t$ are unchanged (no flow on them).

The reverse edges (capacity 2) represent the option to *cancel* flow we already pushed. Augmenting along a path that uses a reverse edge effectively reroutes existing flow.

19.5.2. The algorithm

Ford and Fulkerson (1956) proposed:

1. Initialise $f = 0$, so $G_f = G$.
2. While there exists an s - t path in G_f :
 - a. Find such a path P (e.g., by DFS).
 - b. Let $\lambda = \min_{e \in P} c_f(e)$ (the bottleneck capacity).
 - c. Augment: add λ units of flow along P .
 - d. Update G_f .
3. Output f .

Finding a path takes $O(m)$ time (DFS or BFS). Each augmentation pushes at least 1 unit of flow (assuming integer capacities), so the algorithm terminates in at most f^* iterations, where $f^* = \text{val}(f^*)$ is the max flow value.

$$\text{Running time: } O(m \cdot f^*) \quad (19.7)$$

Warning

The $O(m \cdot f^*)$ bound is pseudo-polynomial: f^* can be exponential in the input size. The max flow can be as large as $n \cdot U$ where U is the largest capacity, but writing U in binary takes only $\log U$ bits. A graph with two vertices and one edge of capacity 10^9 takes a few bytes to describe, but Ford–Fulkerson could need a billion iterations. On unit-capacity graphs, $f^* \leq n$, so the bound becomes a more reasonable $O(mn)$.

 Tip

The choice of *which* augmenting path to use matters. DFS (depth-first search) can lead to the worst case. BFS (breadth-first search) always finds a *shortest* augmenting path and yields the Edmonds–Karp bound of $O(m^2n)$. Lecture 21 explores this direction via *blocking flows*.

19.5.3. Correctness: the max-flow min-cut theorem

Ford–Fulkerson terminates when s and t are disconnected in G_f (no augmenting path exists).

Theorem (Ford and Fulkerson, 1956). f is a max flow if and only if G_f has no s - t path.

Equivalently, the maximum flow value equals the minimum s - t cut capacity. The proof appears in any introductory algorithms textbook (e.g., CLRS Chapter 26); the key idea is that the set of vertices reachable from s in G_f defines a minimum cut.

 Tip

The max-flow min-cut theorem is the LP duality theorem applied to Equation 19.5. The dual variables correspond to the cut: each vertex is assigned to the s -side or the t -side, and the cut capacity equals the total capacity of edges crossing from s -side to t -side.

19.6. Cheat sheet

! Lecture 20 — cheat sheet

MW for LP feasibility

Component	Detail
Experts	The m constraints of $Ax \geq b$
Day- t cost	$m_t = \frac{1}{\gamma}(Ax_t - b)$ after querying oracle with p_t
Oracle	Solves single-constraint sub-problem $p^\top Ax \geq p^\top b, x \in P$
γ	Max row-wise violation $\ a_i \cdot x - b_i\ $ across all i
Iterations	$T = O(\gamma^2 \ln m/\varepsilon^2)$
Output	$\tilde{x} = \frac{1}{T} \sum_t x_t$; satisfies $a_i \tilde{x} \geq b_i - \varepsilon$ for all i

Set cover oracle: pick the set s^* with largest $p(s)$; place $x_{s^*} = \beta$. Gives $\gamma \leq m$, so $T = O(m^2 \log m / \epsilon^2)$.

Network flow definitions

Term	Meaning
Flow f	Non-negative edge vector obeying conservation and capacity
$\text{val}(f)$	Net flow leaving s
Residual graph G_f	$c_f(u, v) = c(u, v) - f(u, v) + f(v, u)$
Augmenting path	s - t path in G_f
Ford–Fulkerson	Repeatedly augment along any s - t path; $O(m \cdot f^*)$ time
Max-flow min-cut	$\max \text{val}(f) = \min_{S \text{ cut}} c(S, \bar{S})$

19.7. FAQ — questions students actually asked

Q: Does the MW–LP theorem require the constraint matrix A to have non-negative entries (packing/covering)?

No. The proof never uses non-negativity of A . The original Plotkin–Shmoys–Tardos paper focused on packing and covering LPs, but the oracle-based framework works for general LPs. Non-negativity can *help* in designing a good oracle (small γ), but it is not required by the theorem.

Q: The oracle returns an x with $|a_i \cdot x - b_i| \leq \gamma$. That means x might violate the *original* constraints. How is that useful?

The absolute value means the oracle’s x_t can violate individual constraints by up to γ in either direction. The magic is in the *averaging*: multiplicative weights ensures that the *average* $\bar{x}$ violates constraints by at most ϵ , much less than γ . The γ only affects the *number of iterations* needed.

Q: What’s the difference between P (capital) and p (lowercase) in the LP framework?

P (capital) is the convex feasibility set (e.g., $\{x \geq 0 : \sum x_s \leq \beta\}$). p (lowercase) is the probability vector over experts (constraints) produced by multiplicative weights in each round. They serve completely different roles.

Q: For the flow problem, can both (u, v) and (v, u) carry positive flow simultaneously?

You can always assume at most one direction carries positive flow. If $f(u, v) = 5$ and $f(v, u) = 3$, replace with $f(u, v) = 2$, $f(v, u) = 0$. The conservation and capacity constraints still hold, and the flow value is unchanged.

Q: Why not just solve max flow with interior point?

You can. Generic interior point gives $O(m^{3.5})$. Exploiting Laplacian structure brings it down to $\tilde{O}(m^{3/2})$ (Daitch–Spielman) or $\tilde{O}(m\sqrt{n})$ (Lee–Sidford). Combinatorial algorithms like Ford–Fulkerson are important because they (a) reveal structural insights (augmenting paths, cuts) and (b) lead to strongly polynomial algorithms that don’t depend on capacities at all.

19.8. Further reading

- Arora, Hazan, Kale. “The multiplicative weights update method: a meta-algorithm and applications.” *Theory of Computing*, 8(1):121–164, 2012. The definitive survey unifying MW across learning, optimization, and game theory.
- Plotkin, Shmoys, Tardos. “Fast approximation algorithms for fractional packing and covering problems.” *Mathematics of Operations Research*, 20(2):257–301, 1995. The original MW-for-LPs paper, focused on packing/covering.
- Ford, Fulkerson. “Maximal flow through a network.” *Canadian Journal of Mathematics*, 8:399–404, 1956. The founding paper of network flow theory.

- Daitch, Spielman. “Faster approximate lossy generalized flow via interior point algorithms.” *STOC 2008*. Laplacian solvers meet interior point for $\tilde{O}(m^{3/2})$ flow.
- Lee, Sidford. “Path finding methods for linear programming: solving linear programs in $\tilde{O}(\sqrt{\text{rank}})$ iterations and faster algorithms for maximum flow.” *FOCS 2014*. The $\tilde{O}(m\sqrt{n})$ max flow result via reweighted barriers.

19.9. Try it yourself

1. Warm-up. Verify that setting $\eta = \sqrt{(\ln m)/T}$ in the MW bound gives average regret $O(\sqrt{(\ln m)/T})$. At what T does the average regret drop below 0.01 for $m = 100$ constraints?
2. Set cover oracle. Write a function that, given a probability vector p over elements and a list of sets, returns the oracle's output $x = \beta \cdot e_{s^*}$ and the resulting γ .

```
def set_cover_oracle(p, sets, beta):
    """Return (x, gamma) for the MW set cover oracle.

    Parameters
    -----
    p : array of shape (n,), probability over elements
    sets : list of lists, sets[j] contains element indices in set j
    beta : float, budget parameter

    Returns
    -----
    x : array of shape (m,), oracle's fractional solution
    gamma : float, max row-wise violation
    """
    import numpy as np
    m = len(sets)
    ps = np.array([p[s].sum() for s in sets]) # p(s) for each set
    s_star = np.argmax(ps)
    if beta * ps[s_star] < 1.0:
        raise ValueError("Sub-problem infeasible")
    x = np.zeros(m)
    x[s_star] = beta
    gamma = max(1.0, beta - 1.0)
    return x, gamma
```

3. Residual graph. Given the graph $s \rightarrow a, s \rightarrow b, a \rightarrow b, a \rightarrow t, b \rightarrow t$ with capacities (10, 5, 3, 8, 7) and a flow of 3 along $s \rightarrow a \rightarrow b \rightarrow t$, draw the residual graph G_f and compute all residual capacities.
4. Ford–Fulkerson by hand. Run Ford–Fulkerson on the graph from exercise 3. Find augmenting paths until no more exist. What is the max flow? Identify the min cut.
5. Scaling thought experiment. Construct a graph with $n = 4$ vertices where Ford–Fulkerson with adversarial DFS path choices takes $\Omega(f^*)$ iterations. (*Hint:* use a “diamond” graph with one low-capacity edge in the middle.)

20. Faster Max Flow: Capacity Scaling and Blocking Flows

COMPSCI 224 Lecture 21 — Fall 2014

Ford–Fulkerson’s pseudo-polynomial $O(mf^*)$ running time is unsatisfying: it can be exponential in the input size. Two ideas fix this. *Capacity scaling* processes capacities bit-by-bit from MSB to LSB, reducing general capacities to $O(\log U)$ near-unit-capacity phases and achieving $O(m^2 \log U)$ time. *Blocking flows* (Dinic, 1970) augment along *all* shortest paths at once, guaranteeing that the shortest-path distance increases after each phase; at most n phases suffice. Combined, these yield $O(mn)$ for unit-capacity graphs, $O(mn^2)$ for general capacities, and $O(mn \log n)$ via link-cut trees—close to the state-of-the-art $O(mn)$ that took until 2013 to achieve.

20.1. Why this lecture matters

Ford–Fulkerson is conceptually clean but computationally naïve. This lecture introduces the two main levers for speeding it up—*scaling* (control the magnitudes) and *blocking flows* (exploit shortest-path structure)—and shows how they compose. Every modern combinatorial max-flow algorithm builds on one or both of these ideas. The analysis also demonstrates a recurring theme: combining a *phase count* bound with a *per-phase cost* bound, optimising their product.

30-second story

1. Ford–Fulkerson is pseudo-polynomial ($O(mf^*)$): the running time depends on capacity magnitudes, not just the graph size.
2. Capacity scaling shifts in capacity bits MSB-first, solving $O(\log U)$ near-unit-capacity flow problems; total time $O(m^2 \log U)$.
3. Blocking flows saturate at least one edge on *every* admissible s - t path, forcing the shortest-path distance to strictly increase.
4. At most n blocking-flow phases suffice; finding one takes $O(m)$ in unit-capacity graphs and $O(mn)$ in general.
5. In unit-capacity graphs, a tighter analysis gives $O(\sqrt{m})$ phases, yielding $O(m^{3/2})$ max flow.
6. For bipartite matching, the same framework gives $O(m\sqrt{n})$ —the Hopcroft–Karp bound.

20.2. Polynomial-time classifications

Before diving in, clarify the three flavours of “polynomial”:

Class	Running time	Example
Strongly polynomial	$\text{poly}(m, n)$	Does not depend on capacities at all

Class	Running time	Example
Weakly polynomial	$\text{poly}(m, n, \log U)$	Polynomial in the <i>bit length</i> of the input
Pseudo-polynomial	$\text{poly}(m, n, U)$	Polynomial in the <i>numeric value</i> of capacities

Ford–Fulkerson is pseudo-polynomial. The knapsack DP ($O(nW)$) is another classic example. The distinction matters: U can be exponentially larger than $\log U$.

Warning

“Pseudo-polynomial” sounds almost respectable, but it means the algorithm can take exponentially long on inputs with large numeric values. A capacity of 2^{30} takes only 30 bits to write down, but Ford–Fulkerson might need 2^{30} augmenting-path iterations.

20.3. State of the art (as of 2014)

For context, here is where flow algorithms stood at the time of this lecture:

Strongly polynomial:

- $O(mn)$ — Orlin (STOC 2013). Removed the last $\log n$ factor from the 1983 bound of $O(mn \log n)$ by Sleator and Tarjan.

Weakly polynomial:

- $O(m \cdot \min(m^{1/2}, n^{2/3}) \cdot \log(n^2/m) \cdot \log U)$ — Goldberg and Rao (1998).
- $\tilde{O}(m\sqrt{n} \cdot \log^2 U)$ — Lee and Sidford (FOCS 2014), via interior point with reweighted barriers.

Unit-capacity graphs:

- $O(m \cdot \min(m^{1/2}, n^{2/3}))$ — Even and Tarjan (1975) / Karzanov (1973).
- $O(m^{10/7})$ — Madry (FOCS 2013).

Aside (mine, not the lecturer’s)

Since 2014, there has been dramatic progress. Chen, Kyng, Liu, Peng, Probst Gutenberg, and Sach (FOCS 2022) achieved $m^{1+o(1)}$ time for max flow—almost linear!—using interior point methods with dynamic data structures. The combinatorial algorithms in this lecture remain important for understanding the structural theory.

20.4. Capacity scaling

20.4.1. The idea

Represent each capacity c_e in binary using $\lceil \log_2 U \rceil$ bits. Process the bits from the most significant to the least significant. In each of the $O(\log U)$ phases:

1. Double all current capacities c'_e and flows f_e .
2. Shift in the next bit: add the k -th bit of c_e to c'_e for each edge e .
3. Augment f until it is a max flow for the current capacities c' .

20.4.2. Pseudocode

```
def capacity_scaling(G, c, s, t):
    """Capacity scaling for max flow.

    G : directed graph (adjacency list)
    c : dict mapping edge -> integer capacity
    s, t : source and sink
    """
    f = {e: 0 for e in G.edges}      # zero flow
    c_prime = {e: 0 for e in G.edges} # working capacities
    L = max(c.values()).bit_length()  # number of bits

    for k in range(L - 1, -1, -1):    # MSB to LSB
        # Step 1: double
        for e in G.edges:
            f[e] *= 2
            c_prime[e] *= 2
        # Step 2: shift in bit k
        for e in G.edges:
            c_prime[e] += (c[e] >> k) & 1
        # Step 3: augment to max flow
        ford_fulkerson(G, c_prime, f, s, t)

    return f
```

20.4.3. Correctness

Invariant: at the start of each phase, f is a max flow for the current capacities c' .

- Base case: $f = 0$ is a max flow when all $c' = 0$.
- After doubling: f was a max flow, so some min cut C was saturated (zero residual capacity across C). After doubling both capacities and flows, C still has zero residual capacity. So f is still a max flow.
- After shifting in a bit: f remains *feasible* (capacities only increased), but may no longer be *maximum*. Ford–Fulkerson augments it to the new max flow.

20.4.4. Running time

The key question: how many augmenting paths does Ford–Fulkerson need in Step 3?

Before shifting in the new bit, some cut C had zero residual capacity. The new bit adds at most 1 capacity per edge to C . Since $|C| \leq m$, the new min-cut capacity is at most m . Therefore, the new max flow exceeds the old max flow by at most m , so Ford–Fulkerson needs at most m augmentations.

Each augmentation takes $O(m)$ time (find a path via BFS/DFS, push flow). There are $O(\log U)$ phases.

$$\text{Total time} = O(m^2 \log U) \quad (20.1)$$

! Important

Capacity scaling is weakly polynomial: the running time depends on $\log U$ (the bit length of capacities), not U itself. This is an exponential improvement over Ford–Fulkerson’s $O(mf^*) = O(m \cdot nU)$ in the worst case.

20.5. Blocking flows

20.5.1. Level graphs and admissible edges

Given a residual graph G_f , define:

- Level of vertex v : the shortest-path distance $\ell(v)$ from s to v in G_f (using BFS).
- Admissible edge: an edge (u, v) with $\ell(v) = \ell(u) + 1$. It lies on *some* shortest s - t path.
- Level graph L : the subgraph of G_f containing only admissible edges.

The level graph L is a DAG (directed acyclic graph): every edge goes from level ℓ to level $\ell + 1$, so no cycles are possible.

💡 Tip

Think of the level graph as a “layered cake” with s on the left and t on the right. Every edge points one layer to the right. Shortest s - t paths in G_f correspond to s - t paths in L .

20.5.2. Definition of a blocking flow

A blocking flow f' in the level graph L is an admissible flow (i.e., it decomposes into admissible s - t paths) such that *every* admissible s - t path has at least one edge saturated by f' .

A blocking flow is *not* necessarily a max flow. It just blocks all *shortest* augmenting paths.

🔥 Caution

A max flow is always a blocking flow, but a blocking flow may not be a max flow. Consider the unit-capacity graph $s \rightarrow a, s \rightarrow b, a \rightarrow t, b \rightarrow t, a \rightarrow b$ where all edges have capacity 1. The flow sending 1 unit along $s \rightarrow a \rightarrow b \rightarrow t$ blocks both admissible paths ($s \rightarrow a \rightarrow t$ loses edge a , $s \rightarrow b \rightarrow t$ loses b) and is a blocking flow of value 1, but the max flow is 2.

20.5.3. The key claim: distance strictly increases

! Blocking flow distance lemma

After augmenting along a blocking flow and forming the new residual graph, the shortest-path distance from s to t strictly increases.

Proof. Let $d = d_L(s, t)$ be the shortest-path distance before augmentation. In the new residual graph $G_{f'}$, the possible edges are:

1. Forward edges from L (surviving admissible edges with leftover capacity).
2. Reverse edges from L (edges of L that carried flow, now reversed).

3. Non-admissible edges from G_f that were not in L —these go backward (decreasing level) or stay within the same level.

None of types 2 or 3 help decrease the distance from s to t : reverse edges go from level $\ell + 1$ back to level ℓ , and non-admissible edges do not advance toward t . So any s - t path in the new residual graph that uses only forward L -edges has length $\geq d$.

But can the distance stay *exactly* d ? A length- d path would use only forward L -edges, meaning it is an admissible path in L . By definition of blocking flow, at least one edge on this path is saturated—so that edge has zero residual capacity and does not appear in $G_{f'}$. Contradiction.

Therefore $d_{L'}(s, t) > d_L(s, t)$. $\square$

20.5.4. Dinic's algorithm

Dinic (1970) builds on the blocking flow lemma:

1. Compute the level graph L of the current residual graph G_f (via BFS from s).
2. Find a blocking flow f' in L .
3. Augment: $f \leftarrow f + f'$.
4. Repeat until t is unreachable from s .

Since the shortest-path distance increases by at least 1 each phase and cannot exceed $n - 1$ (beyond which s and t are disconnected), there are at most $n - 1$ phases.

$$\text{Total time} = O(n \times \text{time per blocking flow}) \quad (20.2)$$

20.6. Finding a blocking flow

20.6.1. Unit-capacity case: modified DFS

The algorithm maintains a “finger” on a current vertex v , starting at s . Three operations:

- Advance: Move from v to some neighbour u along an admissible edge (v, u) in L .
- Retreat: If v has no outgoing admissible edge (dead end), step back to the predecessor w and *delete* edge (w, v) from L . If $v = s$, terminate.
- Augment: If $v = t$, add the s - t path to the blocking flow f' (with the minimum capacity along the path), delete all zero-capacity edges, and return the finger to s .

```
def find_blocking_flow_unit(L, s, t):
    """Find a blocking flow in a unit-capacity level graph.

    L : level graph (adjacency list, all capacities 1)
    s, t : source and sink
    Returns a list of augmenting paths.
    """
    paths = []
    path = [s]

    while True:
        v = path[-1]
        if v == t:
```

```

    # Augment: record path, delete its edges
    paths.append(list(path))
    for i in range(len(path) - 1):
        L.remove_edge(path[i], path[i + 1])
    path = [s]
    elif L.out_degree(v) > 0:
        # Advance: pick any outgoing admissible edge
        u = next(L.neighbors(v))
        path.append(u)
    elif v == t:
        break # no more paths
    else:
        # Retreat: delete incoming edge, step back
        w = path[-2]
        L.remove_edge(w, v)
        path.pop()

return paths

```

Running time: Each edge is examined at most a constant number of times—once when advanced on, once when either augmented through (and deleted) or retreated from (and deleted). Total: $O(m)$.

20.6.2. General-capacity case

The same advance/retreat/augment structure works. The analysis changes because augmenting saturates only the bottleneck edge (minimum capacity), not all edges.

Operation	Count	Cost per operation	Total
Augment	$\leq m$	$O(n)$ (path length)	$O(mn)$
Retreat	$\leq m$	$O(1)$	$O(m)$
Advance	$\leq mn$	$O(1)$	$O(mn)$

Why at most m augments? Each augment saturates (and deletes) at least one edge. Why at most mn advances? After every n consecutive advances, the finger either reaches t (triggering an augment) or encounters a dead end (triggering a retreat). Either way, an edge is deleted. So the number of advance “epochs” of length n is at most m .

$$\text{Blocking flow in general capacities: } O(mn) \quad (20.3)$$

i Note

The bottleneck is advances. Each advance is cheap ($O(1)$), but there can be many of them because the path can be as long as n before an augmentation clears it. Link-cut trees (Sleator–Tarjan, 1983) compress path operations to $O(\log n)$ amortised, reducing the blocking-flow time to $O(m \log n)$.

20.7. Putting it all together

20.7.1. Dinic's algorithm: general capacities

n phases $\times O(mn)$ per blocking flow:

$$O(mn^2)$$

20.7.2. Dinic's algorithm: unit capacities

$O(m)$ per blocking flow, but the number of phases can be bounded more tightly.

After d phases, the shortest-path distance from s to t is at least d . Any max flow decomposes into augmenting paths, each of length $\geq d$. Since paths are edge-disjoint in unit-capacity graphs, there are at most m/d paths, so the residual max-flow value is at most m/d . Ford–Fulkerson would find the remaining flow in m/d more iterations.

The total number of phases is at most $d + m/d$, minimised at $d = \sqrt{m}$:

$$\text{Unit capacity: } O(\sqrt{m}) \text{ phases} \times O(m) \text{ per phase} = O(m^{3/2}) \quad (20.4)$$

More precisely, the bound is $O(m \cdot \min(\sqrt{m}, n))$ since the phase count is also at most n .

20.7.3. Bipartite matching

A bipartite graph $G = (L \cup R, E)$ with $|L| = |R| = n/2$ reduces to max flow: add s connected to all of L , add t connected from all of R , all edges unit capacity.

For this graph, a tighter analysis gives at most $d + n/d$ phases (since the max flow is at most $n/2$, and paths have length $\geq d$). Setting $d = \sqrt{n}$:

$$\text{Bipartite matching: } O(\sqrt{n}) \text{ phases} \times O(m) \text{ per phase} = O(m\sqrt{n}) \quad (20.5)$$

This is the Hopcroft–Karp (1973) bound. Ford–Fulkerson alone gives $O(mn)$; blocking flows shave a $\sqrt{n}$ factor.

20.7.4. With link-cut trees

Sleator and Tarjan's link-cut trees (1983) reduce each blocking-flow computation from $O(mn)$ to $O(m \log n)$, yielding:

$$O(mn \log n) \quad (\text{general capacities}) \quad (20.6)$$

Orlin (STOC 2013) removed the $\log n$ to achieve the current best strongly-polynomial bound of $O(mn)$.

20.7.5. Combined with scaling

Replace Ford–Fulkerson's augmentation step inside capacity scaling with Dinic's blocking-flow algorithm. Each scaling phase needs at most m additional flow, and Dinic finds that in $O(\min(\sqrt{m}, n))$ blocking-flow phases (each $O(m)$ in unit capacity, or $O(m \log n)$ with link-cut trees). This can further improve bounds in certain parameter regimes.

20.8. Cheat sheet

! Lecture 21 — cheat sheet

Capacity scaling

- Process capacity bits MSB $\rightarrow$ LSB.
- Each of $O(\log U)$ phases: double capacities and flows, shift in next bit, augment.
- After shifting, min-cut increases by at most m at most m augmentations.
- Total: $O(m^2 \log U)$ (weakly polynomial).

Blocking flows (Dinic's algorithm)

- Level graph L : admissible edges only (shortest-path edges).
- Blocking flow: saturates ≥ 1 edge on every admissible s - t path.
- Key lemma: after augmenting by a blocking flow, $d_L(s, t)$ strictly increases.
- $\leq n$ phases, since distance bounded by n .

Time per blocking flow:

Setting	Time	Total max flow
Unit capacity	$O(m)$	$O(m \cdot \min(\sqrt{m}, n))$
General, naïve	$O(mn)$	$O(mn^2)$
General, link-cut trees	$O(m \log n)$	$O(mn \log n)$

Special cases:

- Unit capacity: $O(m^{3/2})$ via $\sqrt{m}$ phases.
- Bipartite matching: $O(m\sqrt{n})$ via $\sqrt{n}$ phases (Hopcroft–Karp).

Polynomial-time taxonomy:

Type	Bound	Example
Pseudo-polynomial	$\text{poly}(m, U)$	Ford–Fulkerson: $O(mf^*) = O(mnU)$
Weakly polynomial	$\text{poly}(m, \log U)$	Scaling: $O(m^2 \log U)$
Strongly polynomial	$\text{poly}(m)$	Dinic: $O(mn^2)$

20.9. FAQ — questions students actually asked

Q: In capacity scaling, are we really in the “unit capacity” case after shifting in a bit?

Not exactly. After doubling, existing edges may have residual capacities larger than 1. The point is that the *increase* in the max flow is at most m (because the min cut gained at most m capacity from the new bits). So Ford–Fulkerson needs at most m augmentations—same as if you had a unit-capacity graph with m edges.

Q: Do capacities have to be integers for scaling to work?

Yes, as presented. With rational capacities, clear all denominators first (multiply by the LCM). Without integrality, “log U ” doesn’t have a clean meaning—you would need to normalize by the ratio of the largest to smallest capacity.

Q: In the blocking-flow DFS, can we visit a vertex we already used in a previous augmenting path?

Yes. The level graph is a DAG (edges go strictly from level ℓ to $\ell + 1$), so cycles are impossible. But a vertex can appear in multiple augmenting paths. After each augment, we return the finger to s and start a fresh DFS. Deleted edges are gone forever, but surviving edges remain available.

Q: Why can't a blocking flow be a max flow? Doesn't it block everything?

It blocks all *admissible* (shortest) paths, not all paths. Non-admissible edges—edges that skip levels or go backward—may still form longer augmenting paths in the residual graph. That is why Dinic's algorithm needs multiple phases: each blocking flow eliminates all shortest-path augmentations, but longer paths may open up.

Q: How does the $d + m/d$ analysis work for bounding the number of phases?

After d phases, shortest-path distance $\geq d$. Decompose any max flow into edge-disjoint paths (in unit-capacity graphs). Each path has length $\geq d$, and there are at most m edges total, so at most m/d paths. The residual flow is at most m/d , handled in at most m/d more single-path augmentations. Total phases $\leq d + m/d$, minimised at $d = \sqrt{m}$.

20.10. Further reading

- Dinic, E. A. "Algorithm for solution of a problem of maximum flow in networks with power estimation." *Soviet Mathematics Doklady*, 11:1277–1280, 1970. The original blocking-flow paper.
- Sleator, D. D. and Tarjan, R. E. "A data structure for dynamic trees." *JCSS*, 26(3):362–391, 1983. Link-cut trees, reducing blocking-flow time to $O(m \log n)$.
- Even, S. and Tarjan, R. E. "Network flow and testing graph connectivity." *SIAM Journal on Computing*, 4(4):507–518, 1975. The $O(m \cdot \min(m^{1/2}, n^{2/3}))$ bound for unit-capacity graphs.
- Orlin, J. B. "Max flows in $O(nm)$ time, or better." *STOC 2013*. The current best strongly-polynomial bound.
- Goldberg, A. V. and Rao, S. "Beyond the flow decomposition barrier." *JACM*, 45(5):783–797, 1998. The binary-length blocking-flow approach.
- Chen, L., Kyng, R., Liu, Y. P., Peng, R., Probst Gutenberg, M., and Sach, M. "Maximum flow and minimum-cost flow in almost-linear time." *FOCS 2022*. The near-linear-time breakthrough.

20.11. Try it yourself

1. Scaling trace. Run capacity scaling by hand on a triangle graph $s \rightarrow a$ (cap 5), $s \rightarrow b$ (cap 3), $a \rightarrow b$ (cap 2), $a \rightarrow t$ (cap 4), $b \rightarrow t$ (cap 6). Show the state of c' , f , and G_f after each bit is shifted in.
2. Blocking flow vs. max flow. Construct a graph where the blocking flow has value k but the max flow has value $2k$. What is the smallest such graph (fewest vertices)?
3. Phase-count bound. Prove that in a graph where all edges have capacity at most C , the number of blocking-flow phases needed is at most $\min(n, mC/f^*)$. (*Hint*: the residual max flow drops by at least 1 per phase.)
4. Implement Dinic's algorithm. Code Dinic's algorithm (BFS for level graph + DFS for blocking flow) and test it on a random graph with 100 vertices and edge capacities uniform in $[1, 100]$.

```
from collections import deque

def bfs_level_graph(graph, cap, s, t, level):
    """BFS from s, fill level[v] = shortest distance from s.
    Returns True if t is reachable."""
    for v in range(len(graph)):
        level[v] = -1
    level[s] = 0
```

```

queue = deque([s])
while queue:
    u = queue.popleft()
    for v in graph[u]:
        if level[v] == -1 and cap[u][v] > 0:
            level[v] = level[u] + 1
            queue.append(v)
return level[t] != -1

def dfs_blocking(graph, cap, level, u, t, pushed):
    """DFS to find augmenting path and push flow."""
    if u == t:
        return pushed
    for v in graph[u]:
        if level[v] == level[u] + 1 and cap[u][v] > 0:
            d = dfs_blocking(graph, cap, level, v, t, min(pushed, cap[u][v]))
            if d > 0:
                cap[u][v] -= d
                cap[v][u] += d
                return d
    level[u] = -1 # mark dead end
    return 0

def dinic(n, edges, s, t):
    """Dinic's max flow algorithm.

    n : number of vertices
    edges : list of (u, v, capacity) tuples
    s, t : source and sink
    Returns max flow value.
    """
    graph = [[] for _ in range(n)]
    cap = [[0] * n for _ in range(n)]
    for u, v, c in edges:
        graph[u].append(v)
        graph[v].append(u)
        cap[u][v] += c

    flow = 0
    level = [0] * n
    while bfs_level_graph(graph, cap, s, t, level):
        while True:
            pushed = dfs_blocking(graph, cap, level, s, t, float('inf'))
            if pushed == 0:
                break
            flow += pushed
    return flow

```

5. Hopcroft–Karp. Adapt your Dinic implementation to find a maximum matching in a bipartite graph. Verify on a random bipartite graph with 50 vertices on each side that the number of BFS phases is $O(\sqrt{n})$.

21. Link-Cut Trees

COMPSCI 224 Lecture 22 — Fall 2014

Link-cut trees store a dynamic forest of rooted trees and support path operations—find-root, path-minimum, path-update—in $O(\log n)$ amortised time. Plugging them into the blocking-flow subroutine drops the cost of finding a blocking flow from $O(mn)$ to $O(m \log n)$, yielding one of the fastest known max-flow algorithms. The data structure rests on two ideas: decomposing each tree into *preferred paths* and representing each path with an auxiliary splay tree keyed by depth.

21.1. Why this lecture matters

The blocking-flow algorithm from Lecture 21 already achieved $O(mn)$ per phase, but every DFS step touched one edge—wasteful when long fragments of previously explored, still-live edges could be skipped. Link-cut trees are the data structure that makes that skipping possible: they remember which path fragments survive after augmentation and let you jump to the end of a fragment in $O(\log n)$ time. The ideas here—preferred-path decomposition, auxiliary trees, the access primitive—reappear in dynamic connectivity, top-tree algorithms, and the fastest known flow algorithms.

30-second story

1. Blocking flow via DFS costs $O(mn)$ because each advance/retreat step touches one edge.
2. After augmentation, surviving edges form *tree fragments*; link-cut trees remember them.
3. A link-cut tree stores a forest and supports make-tree, find-root, link, cut, find-min, and path-subtract, each in $O(\log n)$ amortised time.
4. The key internal structure is a *preferred-path decomposition*: each tree is partitioned into vertex-disjoint paths, each stored in an auxiliary splay tree keyed by depth.
5. The workhorse subroutine $\text{access}(v)$ makes the root-to- v path preferred by splicing auxiliary trees upward.
6. With link-cut trees, blocking flow runs in $O(m \log n)$, giving $O(mn \log n)$ max flow overall.

21.2. Blocking flow revisited

Recall the blocking-flow framework from Lecture 21. Given a level graph L (edges go from level i to level $i + 1$ only):

1. Repeat: find a blocking flow F in L ; augment the overall flow by F ; rebuild the residual graph.
2. Stop when no s - t path exists in the residual graph.

The number of iterations is at most $n - 1$ (the s - t distance in the residual graph increases by at least one per phase). Last time we found each blocking flow via DFS in $O(m)$ advances and retreats, giving $O(mn)$ per blocking flow and $O(mn^2)$ total.

21.2.1. Why DFS wastes work in general capacities

In the unit-capacity case every augmentation saturates the entire path, so each edge is touched at most twice (once forward, once retreat). In general capacities, only the bottleneck edge dies; the rest survive as a *tree fragment*. The next DFS must re-traverse these surviving edges one by one.

21.2.2. A concrete example

Consider a level graph with four levels (s at level 0, t at level 3). DFS finds an s - t path and augments. The bottleneck edge saturates, but the other edges survive. Two disconnected fragments remain—a length-1 path and a length-2 path.

On the next DFS from s , the algorithm advances to some vertex, discovers a live edge, and enters the length-2 fragment. In the naive approach it must walk through both edges of that fragment to reach its end. If it *remembered* the fragment, it could skip directly from the entry point to the end, saving two steps. Over many augmentations, these savings compound: fragments can be long, and each re-traversal costs proportional to the fragment's length.

After multiple augmentations, the surviving fragments are not necessarily simple paths. When an augmenting path branches through a fragment—entering at one vertex and leaving at another—the leftover may be a *tree*, not a path. In general, the collection of surviving fragments at any point during the algorithm is a *forest*: a collection of rooted trees, where each tree is a subset of the level graph's edges.

💡 Intuition: skip, don't re-walk

Imagine you are walking a hallway and have already mapped every room. When a door closes (edge saturated), you should teleport to the end of the mapped section, not retrace every step. Link-cut trees are the teleporter.

21.3. The link-cut tree interface

A link-cut tree maintains a collection of vertex-disjoint rooted trees and supports these operations. Every operation will run in $O(\log n)$ amortised time (analysis in Lecture 23).

Operation	Effect
<code>make_tree()</code>	Create a new singleton tree (one vertex, no edges).
<code>find_root(v)</code>	Return the root of the tree containing v .
<code>link(v, w, x)</code>	Make v (which must be a root) a child of w with edge capacity x . Requires v and w in different trees.
<code>cut(v)</code>	Destroy the edge from v to its parent, splitting one tree into two.
<code>find_min(v)</code>	Return the edge of minimum capacity on the v -to-root path (ties broken closest to root).
<code>subtract(v, x)</code>	Subtract x from the capacity of every edge on the v -to-root path.

i Why not a DAG?

A student asked why the maintained structure is always a forest, never a DAG. The answer is operational: the algorithm always follows existing fragments first (`find_root` on s), so it never creates a second parent for any

vertex. The level graph itself is a DAG, but the subgraph H we build inside the link-cut tree is always a forest.

21.4. Blocking flow with link-cut trees

Let L be the level graph and H the forest maintained by the link-cut tree (initially n isolated vertices).

```
# Blocking flow using link-cut trees
for each vertex u in L:
    make_tree(u)

while True:
    v = find_root(s)

    if v == t:
        # --- augment ---
        while True:
            (z, x) = find_min(s)      # z = bottleneck vertex, x = capacity
            subtract(s, x)           # reduce capacities along s-to-t path
            if x == 0:                # if the bottleneck edge was already zero...
                break                # ...we need to check again
            cut(z)                    # remove saturated edge z -> parent(z)
            delete edge (z, parent(z)) from L
            # loop: keep removing zero-capacity edges until the min is positive
            (z2, x2) = find_min(s)
            if x2 > 0:
                break
            # else continue loop to remove next zero-capacity edge

        elif v has an outgoing edge to some w in L:
            # --- advance ---
            link(v, w, capacity(v, w) in L)

    else:
        # --- retreat ---
        if v == s:
            break                    # no more s-t paths; done
        for each child y of v in H:
            cut(y)
            delete edge (y, v) from L
```

21.4.1. Counting operations

Each iteration of the outer loop does one of three things:

- Link (advance): explores a new edge. Happens at most m times.
- Cut (retreat or clean-up): deletes an edge. Also at most m times in total.
- Find-min / subtract (augment): the augment branch runs `find_min` repeatedly, but each call except possibly the last deletes an edge. So aggregate `find_min` calls are also $O(m)$.

Total link-cut operations across the entire blocking-flow computation: $O(m)$.

! Important

If every operation costs $O(\log n)$, blocking flow costs $O(m \log n)$. Combined with $O(n)$ phases, max flow is $O(mn \log n)$.

21.5. Preferred-path decomposition

The internal representation of a link-cut tree rests on partitioning each represented tree into vertex-disjoint *preferred paths*.

21.5.1. Preferred children

Every non-leaf vertex v has at most one preferred child: the child whose subtree contains the most recently accessed node. If v itself was the last node accessed in its subtree, it has no preferred child.

A preferred edge leads from a vertex to its preferred child. A preferred path is a maximal chain of preferred edges.

21.5.2. Worked example

Consider a rooted tree where the root r has children a and b ; vertex a has children c and d ; vertex b has child e ; and e has children f and g . Suppose the last node accessed was f .

- At e : the last accessed node in e 's subtree is f , which is in f 's subtree (the left child). So e 's preferred child is f .
- At b : the last accessed node in b 's subtree is f , which is in e 's subtree. So b 's preferred child is e .
- At r : the last accessed node in r 's subtree is f , which is in b 's subtree. So r 's preferred child is b .
- At a : if the last accessed node in a 's subtree was a itself, then a has no preferred child.

The preferred edges are $r \rightarrow b$, $b \rightarrow e$, $e \rightarrow f$. These form one preferred path: (r, b, e, f) . Vertices a , c , d , g are each isolated preferred paths (singletons).

21.5.3. The tree of paths

Collapse each preferred path into a single super-node. The result is a *tree of paths*: each super-node is a preferred path, connected to its parent path by a non-preferred edge. In the example above, collapsing the path (r, b, e, f) into one super-node gives a star: the super-node at the centre, with singletons a , c , d , g hanging off it via non-preferred edges.

This decomposition changes dynamically as we access different vertices. If we next access d , the preferred child of r switches from b to a , the preferred child of a becomes d , and the old preferred path (r, b, e, f) breaks apart.

💡 Mental model

Think of a road network where each road is a preferred path. Intersections (non-preferred edges) connect roads. When you access a vertex, you repave the highway from the root to that vertex, possibly rerouting traffic from old highways.

21.6. Auxiliary trees: splay trees keyed by depth

Each preferred path is stored in an auxiliary tree—a splay tree whose keys are depth in the represented tree. A node closer to the root has a smaller key (is further left in the splay tree).

21.6.1. Path-parent pointers

The root of each auxiliary tree stores a path-parent pointer to the parent of the top node of that path in the represented tree. This pointer records the non-preferred edge connecting this path to its parent path. Crucially, the parent path's auxiliary tree does *not* store a downward pointer to the child path. The asymmetry is intentional: the parent path doesn't need to know about its children paths because the access operation works bottom-up.

21.6.2. Concrete layout

Returning to the earlier example with preferred path (r, b, e, f) : this path is stored in one splay tree with four nodes, keyed by depth. Node r has depth 0 (smallest key, leftmost), b has depth 1, e has depth 2, f has depth 3 (largest key, rightmost). The splay tree might arrange these with b or e at the root after rotations.

Vertex g is a singleton auxiliary tree. Its root stores a path-parent pointer to e (because e is g 's parent in the represented tree and g is not a preferred child). Similarly, a is a singleton with a path-parent pointer to r .

Terminology

The represented tree T is the actual rooted tree we are maintaining (a subgraph of the level graph). The auxiliary tree is the splay tree used to store one preferred path. These are different objects—don't confuse them.

21.7. The access operation

`access( $v$ )` is the workhorse subroutine. Every other operation begins with an access. Its job:

Make the root-to- v path in the represented tree a single preferred path, and make v the root of the root auxiliary tree.

21.7.1. Step-by-step

1. Splay v to the root of its auxiliary tree.
2. Cut the right subtree: v has no preferred child after being accessed, so everything deeper than v on its old preferred path must be detached. In the auxiliary splay tree, “deeper” means “larger key” which means “right subtree.” Set the right child R 's path-parent to v (recording the now non-preferred edge), and set v 's right child to null. The subtree R becomes its own auxiliary tree.
3. Splice upward: follow v 's path-parent pointer to some vertex w in a different auxiliary tree. Splay w to the root of its auxiliary tree. Now w 's right subtree contains all vertices deeper than w on w 's old preferred path—these are w 's old preferred descendants. Detach this right subtree as a new auxiliary tree with path-parent w . Then make v 's entire auxiliary tree the new right subtree of w . Update pointers: v no longer has a path-parent (it is now a real child of w in the auxiliary tree); the detached subtree's root gets w as its path-parent.
4. Repeat step 3 until v is in the root auxiliary tree (i.e., there is no more path-parent to follow).
5. Splay v one final time so it is the root of the (now merged) root auxiliary tree.

The number of splice iterations equals the number of non-preferred edges on the root-to- v path in the represented tree—equivalently, the number of preferred-child changes that happen.

⚠ Pointer bookkeeping

Each splice changes three things: (1) a path-parent becomes a real parent, (2) an old real child becomes a path-parent, and (3) the subtree structure of w 's auxiliary tree changes. Getting these updates wrong is the most common implementation bug.

21.7.2. Cost of access

The number of splices in `access( $v$ )` equals the number of preferred-child changes along the root-to- v path. The full amortised analysis (Lecture 23) shows this is $O(\log n)$ amortised using the splay tree's access lemma.

21.8. Other operations via access

Once `access( $v$ )` is in hand, every other operation is short.

21.8.1. `find_root( $v$ )`

```
def find_root(v):
    access(v)
    # v is now the root of the root aux tree.
    # The root of the represented tree is the leftmost node.
    r = v
    while r.left is not None:
        r = r.left
    splay(r)          # amortised O(1) extra after the walk
    return r
```

After the access, v 's auxiliary tree contains the entire root-to- v path. The root of the represented tree has the smallest depth, so it is the leftmost node.

21.8.2. `find_min( $v$ )`

```
def find_min(v):
    access(v)
    # return the minimum-capacity edge in v's aux tree
    # (augmented splay tree stores subtree min)
    return v.subtree_min
```

The auxiliary splay tree is augmented so each node stores the minimum edge capacity in its subtree. After access, the root's augmented value gives the answer for the entire root-to- v path.

21.8.3. cut(*v*)

```
def cut(v):
    access(v)
    # v has no right child (access guarantees this).
    # The left subtree contains ancestors in the represented tree.
    v.left.parent = None      # detach ancestors
    v.left = None
```

21.8.4. link(*v*, *w*, *x*)

```
def link(v, w, x):
    # v must be the root of its represented tree
    access(v)
    access(w)
    # w becomes v's parent; w has higher depth, so w goes to v's left
    v.left = w
    w.parent = v
    # store capacity x on the edge v -> w
```

After the two accesses, *v* is the root of its auxiliary tree (with no left child, since *v* is the root of its represented tree—no one is shallower). *w* is the root of a different auxiliary tree. Making *w* the left child of *v* is valid because *w* is at a shallower depth (smaller key) than *v* in the combined tree. The edge between them carries capacity *x*.

Preconditions matter

link assumes *v* is a root and *v*, *w* are in different trees. Violating either precondition corrupts the data structure silently—there is no built-in check. In the blocking-flow algorithm, these preconditions are always met because we only link the result of `find_root(s)` (which is a root by definition) to a vertex in the level graph that we haven't yet connected.

Aside (mine, not the lecturer's)

The original Sleator–Tarjan paper presents link-cut trees with worst-case $O(\log n)$ per operation using *biased search trees* instead of splay trees. The splay-tree version gives the same amortised bound with simpler code. Both appear in practice; the splay variant is more common in competitive programming.

21.9. Putting it all together

Component	Cost
Blocking-flow operations	$O(m)$ total across one phase
Cost per link-cut operation	$O(\log n)$ amortised
Blocking flow per phase	$O(m \log n)$
Number of phases	$O(n)$
Total max flow	$O(mn \log n)$

! Historical note

Link-cut trees are due to Sleator and Tarjan (*J. Comput. System Sci.*, 1983). The same paper introduced splay trees. The $O(mn \log n)$ max-flow bound remained the fastest for dense graphs for over a decade.

21.10. Cheat sheet

! Cheat sheet — Link-Cut Trees

What they store: a dynamic forest of rooted trees with weighted edges.

Operations (all $O(\log n)$ amortised):

- `make_tree()` — create isolated vertex
- `find_root(v)` — root of v 's tree
- `link(v, w, x)` — make root v a child of w with edge weight x
- `cut(v)` — detach v from parent
- `find_min(v)` — min-weight edge on v -to-root path (closest to root on tie)
- `subtract(v, x)` — subtract x from all edges on v -to-root path

Internal structure:

- *Preferred-path decomposition*: each tree split into preferred paths
- *Auxiliary trees*: each preferred path stored in a splay tree, keyed by depth
- *Path-parent pointers*: link auxiliary trees across non-preferred edges

Key subroutine — **access(v)**:

1. Splay v ; detach right subtree (no preferred child)
2. Splice upward: splay path-parent, swap preferred descendants, merge auxiliary trees
3. Repeat until v is root of root auxiliary tree

Application to blocking flow:

- $O(m)$ link-cut operations per phase $\times O(\log n)$ each = $O(m \log n)$ per phase
- $O(n)$ phases $\Rightarrow O(mn \log n)$ total for max flow

21.11. FAQ — questions students actually asked

Q: Why is the maintained structure always a forest and never a DAG? The level graph L is a DAG, but the subgraph H maintained in the link-cut tree is always a forest because the algorithm always follows existing fragments first. `find_root(s)` jumps to the end of the current fragment before exploring new edges. You never create a second parent for any vertex.

Q: After augmentation, when multiple edges have zero capacity, don't we need to delete all of them? Yes. After calling `subtract`, the minimum-capacity edge is saturated, but there may be others at zero too. The algorithm loops on `find_min(s)` until the returned capacity is positive, deleting each zero-capacity edge it finds. Each such deletion costs $O(\log n)$ but also permanently removes an edge, so the total extra work is $O(m \log n)$.

Q: When we retreat from a dead-end vertex v , why delete edges into v rather than just the edge we arrived on? If v has no outgoing edges in L , it is permanently useless—no future path can go through it. Aggressively cutting all

incoming edges avoids revisiting v . The descendants of v 's children may still be reachable through other routes, so we do not touch edges deeper in the tree.

Q: Could a vertex have two heavy children? No—that question belongs to Lecture 23's analysis. But the short answer: a heavy child has subtree size $> n/2$, so there can be at most one.

Q: Why do we splay v at the end of **find_root**? After walking to the leftmost node in the auxiliary tree, that node may be deep. Splaying it restores the $O(\log n)$ amortised bound for subsequent operations. Without this final splay, a sequence of **find_root** calls could degrade to linear time.

21.12. Further reading

- Sleator and Tarjan, “A data structure for dynamic trees,” *J. Comput. System Sci.* 26(3):362–391, 1983. The original paper defining link-cut trees and splay trees.
- Sleator and Tarjan, “Self-adjusting binary search trees,” *J. ACM* 32(3):652–686, 1985. Full analysis of splay trees and the access lemma.
- Goldberg and Tarjan, “A new approach to the maximum-flow problem,” *J. ACM* 35(4):921–940, 1988. Uses link-cut trees in the push-relabel framework.
- Tarjan, “Amortized computational complexity,” *SIAM J. Alg. Disc. Meth.* 6(2):306–318, 1985. The potential-function framework that underpins the analysis.

21.13. Try it yourself

1. Warm-up. Draw a rooted tree on 7 vertices with an arbitrary preferred-path decomposition. Identify the auxiliary trees and path-parent pointers. Then perform **access**(v) for a leaf v and redraw the decomposition.
2. Subtract via augmentation. Explain how to implement **subtract**(v , x) using an augmented splay tree. What value does each node store? How do lazy propagation tags work during splay rotations?
3. Edge deletion order. In the blocking-flow pseudocode, the augment branch loops on **find_min** to delete zero-capacity edges. Prove that the total number of **find_min** calls across the entire blocking-flow computation is $O(m)$.
4. Coding challenge. Implement a link-cut tree supporting **make_tree**, **find_root**, **link**, **cut**, and **path_min**. Use splay trees as auxiliary trees. Test it by computing a blocking flow on a small level graph (e.g., 6 vertices, 8 edges) and verify the result against a brute-force BFS/DFS implementation.

22. Link-Cut Tree Analysis and Min-Cost Max Flow

COMPSCI 224 Lecture 23 — Fall 2014

The amortised cost of link-cut tree operations is $O(\log^2 n)$ via a heavy-light decomposition argument that bounds preferred-child changes. With a tighter analysis using splay tree potentials, the bound improves to $O(\log n)$. The lecture then pivots to min-cost max flow: every edge carries both a capacity and a cost, and the goal is to find a max flow of minimum total cost. The key structural result—optimality is equivalent to the absence of negative-cost residual cycles—yields a negative-cycle cancelling algorithm and, for unit-capacity graphs with non-negative costs, a faster shortest-augmenting-path algorithm running in $O(mn + n^2 \log n)$.

22.1. Why this lecture matters

Lecture 22 built the link-cut tree data structure; this lecture proves it works fast. The heavy-light decomposition that drives the proof is itself a widely reused analysis technique—it appears in competitive programming, in Euler-tour trees, and in the analysis of centroid decomposition. The second half introduces min-cost max flow, the natural generalisation of max flow that handles economics: shipping goods through a network where each route has a per-unit price. The optimality characterisation via residual-graph prices is the foundation of every practical min-cost flow solver.

30-second story

1. The runtime of $\text{access}(v)$ is proportional to the number of preferred-child changes along the root-to- v path.
2. Heavy-light decomposition classifies each edge as heavy or light; any root-to-leaf path has at most $\log n$ light edges.
3. Total preferred-child changes across m operations is $O(m \log n)$, giving $O(m \log^2 n)$ with balanced BSTs or $O(m \log n)$ with splay trees.
4. Min-cost max flow asks for a maximum flow minimising $\sum_e f(e) c(e)$; it reduces to min-cost circulation and vice versa.
5. A circulation is optimal iff the residual graph has no negative-cost cycle, iff there exists a price function making all reduced costs non-negative.
6. For unit-capacity, non-negative-cost graphs, repeatedly augmenting along shortest s - t paths (Dijkstra) solves min-cost max flow in $O(mn + n^2 \log n)$.

22.2. Amortised analysis of link-cut trees

22.2.1. Runtime in terms of preferred-child changes

Recall from Lecture 22 that $\text{access}(v)$ repeatedly splays and splices upward. Each splice corresponds to one preferred-child change (PCC): some vertex w along the root-to- v path switches its preferred child.

Let PCC_i be the number of preferred-child changes in the i -th operation. Then:

$$\text{total time for } m \text{ operations} = O\left((\text{cost per splay}) \times m + \sum_{i=1}^m PCC_i\right) \quad (22.1)$$

With balanced BSTs, each splay costs $O(\log n)$ worst case. With actual splay trees, the amortised cost per splay is $O(\log n)$, but the crucial insight is that the splay cost can be folded into the PCC bound—see the problem set for the $O(\log n)$ amortised proof.

i Two levels of amortisation

The $O(\log^2 n)$ bound uses worst-case $O(\log n)$ per splay $\times O(\log n)$ preferred-child changes per operation. The sharper $O(\log n)$ bound replaces the first factor with an amortised argument using the splay tree's access lemma and a carefully chosen weight function.

22.3. Heavy-light decomposition

This decomposition is purely an analysis tool—the data structure does not maintain it.

22.3.1. Definitions

For a vertex v in the represented tree, define $\text{size}(v)$ as the number of vertices in v 's subtree. For a child u of v :

- The edge $u \rightarrow v$ is heavy if $\text{size}(u) > \frac{1}{2} \text{size}(v)$.
- The edge $u \rightarrow v$ is light if $\text{size}(u) \leq \frac{1}{2} \text{size}(v)$.

! Important

A vertex can have at most one heavy child. Proof: if two children u_1, u_2 both had $\text{size}(u_i) > \frac{1}{2} \text{size}(v)$, then $\text{size}(u_1) + \text{size}(u_2) > \text{size}(v)$, contradicting the fact that both subtrees are disjoint subsets of v 's subtree.

22.3.2. Key property: few light edges on any root-to-leaf path

Every time you follow a light edge downward, the subtree size at least halves. Since the root has subtree size n :

$$\text{number of light edges on any root-to-leaf path} \leq \log_2 n \quad (22.2)$$

Heavy edges impose no such bound—a root-to-leaf path could be entirely heavy. But we only need to count *light* edges.

22.4. Bounding preferred-child changes

Each preferred-child change creates a new preferred edge and (possibly) destroys an old one. The new edge is either heavy or light. We track light preferred-child creations (LPCC) separately.

22.4.1. The decomposition

$$\text{PCC} \leq 2 \cdot \text{LPCC} + m + n \quad (22.3)$$

Why? Consider a preferred-child change at vertex v : child u was preferred, now child w is preferred.

- If the new edge $w \rightarrow v$ is light: counted directly as an LPCC.
- If the new edge $w \rightarrow v$ is heavy and u was null (no previous preferred child): either this is the first access in v 's subtree (at most n such events total) or the last access in v 's subtree was to v itself. In the latter case, charge this to that prior access—each access is charged at most once, contributing at most m .
- If the new edge $w \rightarrow v$ is heavy and u was not null: since v can have at most one heavy child, $u \rightarrow v$ must have been *light*. That light preferred edge was created at some earlier time; charge this heavy creation to that earlier light creation. This at most doubles the LPCC count.

Charging argument intuition

Heavy preferred-child creations are either free (first access) or can be blamed on an earlier light creation. So the total PCC is at most twice the light creations plus bookkeeping.

22.4.2. LPCC across the three operations

Access. Does not change the represented tree, so what's heavy/light is fixed. Walking from v to the root, we change preferred children along the path. The number of light edges on this path is at most $\log n$ by Equation 22.2. So:

$$\text{LPCC per access} \leq \log n$$

Link(v, w). First does access(v) and access(w) (already counted). Then adds the edge $v \rightarrow w$, which changes subtree sizes: vertices along the path from w to the root get heavier. Some off-path edges that were heavy may become light, but those are not preferred, so they don't create LPCCs. The new preferred edges along the w -to- v path may include light ones, but these were already created by the preceding access(w). Net new LPCCs beyond the two accesses: zero.

Cut(v). First does access(v). Then removes the edge from v to its parent, decreasing subtree sizes along the path above. Edges along the old root-to- v preferred path may become lighter. The number of edges that flip from heavy to light (and are preferred) is at most $\log n$, since it's a root-to-leaf path.

22.4.3. Totalling up

$$\text{total LPCC across } m \text{ operations} = O(m \log n)$$

Plugging into Equation 22.3:

$$\text{total PCC} = O(m \log n + m + n) = O(m \log n)$$

(assuming $m \geq n$, which holds in the blocking-flow application since m is the number of edges).

22.5. The $O(m \log^2 n)$ bound

Each operation triggers $O(\log n)$ preferred-child changes (from the PCC bound). Each PCC involves a splay in an auxiliary tree of size at most n , costing $O(\log n)$ worst-case with balanced BSTs.

$$m \text{ operations cost } O(m \log^2 n) \quad (22.4)$$

i Aside (mine, not the lecturer's)

For the blocking-flow application with m edges and n phases, the total max-flow time becomes $O(mn \log^2 n)$ with balanced BSTs and $O(mn \log n)$ with splay trees. The improvement from $\log^2 n$ to $\log n$ is on the problem set: define vertex weights as subtree sizes, plug into the splay access lemma's potential function, and the splay costs telescope with the PCC costs.

22.6. Augmented BSTs for path operations

A student asked how `subtract` and `find_min` are actually implemented inside the auxiliary trees. The answer is BST augmentation: each node stores aggregate information about its subtree.

For `find_min`: each node stores the minimum edge capacity in its subtree. After `access(v)`, the root's augmented value gives the minimum on the entire v -to-root path.

For `subtract(v, x)`: use lazy propagation. Store a pending subtraction tag at the root; push it down during splays. This is the same technique used in segment trees and is standard for splay tree augmentations.

The key-value analogy: think of the auxiliary tree as storing items keyed by depth, with values being edge capacities. Operations like “sum of values with key $\leq k$ ” or “min of all values” are standard augmented-BST queries.

22.7. Min-cost max flow

22.7.1. Problem definition

Each edge e now has two attributes:

- Capacity $u(e) \in \{1, 2, \dots, U\}$
- Cost $c(e) \in \{-C, \dots, C\}$ (possibly negative)

The cost of a flow f is:

$$\text{cost}(f) = \sum_e f(e) \cdot c(e) \quad (22.5)$$

Goal: among all maximum s - t flows, find one minimising $\text{cost}(f)$.

⚠ Residual costs

In the residual graph, the reverse of edge e has cost $-c(e)$. Pushing flow back along e recovers the cost you paid. This is analogous to how the residual capacity represents “undoing” flow.

22.7.2. Min-cost circulation

A circulation is a flow with no source or sink: $\text{inflow}(v) = \text{outflow}(v)$ for all v . A min-cost circulation (MCC) minimises $\sum_e f(e) c(e)$ subject to capacity constraints.

! Important

Min-cost max flow and min-cost circulation efficiently reduce to each other.

MCC $\iff$ min-cost max flow: Add a disconnected source s and sink t (no edges to the rest of the graph). The max flow is 0, and any s - t flow is a circulation. Calling the min-cost max flow algorithm finds the min-cost circulation.

Min-cost max flow $\iff$ MCC (two approaches):

1. Negative-cost trick: Add a $t \rightarrow s$ edge with very negative cost and infinite capacity. The MCC algorithm will saturate this edge as much as possible—which means pushing maximum flow from s to t —because doing so decreases cost.
2. Residual approach: Find *any* max flow f (ignoring costs). Then find a min-cost circulation f^* in the residual graph G_f . The flow $f + f^*$ is a min-cost max flow. This works because the difference of any two flows of the same value is a circulation.

22.8. Negative-cycle cancelling

22.8.1. The algorithm

```
# Min-cost circulation via negative-cycle cancelling
f = zero circulation
while residual graph G_f contains a negative-cost cycle C:
    augment f along C    # push flow around C up to bottleneck capacity
```

22.8.2. Correctness

Suppose the current circulation f is suboptimal and the optimal circulation is f^* . Then $f^* - f$ is a negative-cost circulation feasible in the residual graph G_f . Any circulation decomposes into at most m cycles (repeatedly extract a cycle by following positive-flow edges). Since the total cost is negative, at least one cycle must be negative. The algorithm will find and augment along it.

💡 Cycle decomposition

To decompose a circulation into cycles: pick any edge with positive flow, follow outgoing edges until you revisit a vertex (guaranteed since it's a circulation), extract the cycle with the minimum flow on it, subtract, repeat. Each step removes at least one edge, so at most m cycles are extracted.

22.8.3. Detecting negative cycles

Bellman–Ford finds negative cycles in $O(mn)$ time. A faster alternative due to Goldberg (1995) runs in $O(m\sqrt{n} \log C)$ using a scaling approach.

22.8.4. Runtime: pseudo-polynomial

Each augmentation decreases cost by at least 1 (integer costs). The initial cost is at most mUC in absolute value. So the number of iterations is $O(mUC)$, giving total time $O(m^2nUC)$ with Bellman–Ford—pseudo-polynomial, analogous to Ford–Fulkerson for basic max flow.

22.9. Price functions and reduced costs

22.9.1. Definition

A price function $p : V \rightarrow \mathbb{R}$ induces reduced costs:

$$c_p(u, v) = c(u, v) + p(u) - p(v) \quad (22.6)$$

The gold-trading analogy

Imagine buying gold at vertex u (pay $p(u)$), shipping it along edge (u, v) (pay $c(u, v)$), and selling at v (earn $p(v)$). The reduced cost $c_p(u, v)$ is your net expense. If it's negative, the trade is profitable.

22.9.2. Key property: cycle costs are invariant

For any cycle, the prices cancel:

$$\sum_{(u,v) \in \text{cycle}} c_p(u, v) = \sum_{(u,v) \in \text{cycle}} c(u, v)$$

This is because each vertex appears exactly once as a head and once as a tail, so the $p(u)$ and $p(v)$ terms telescope.

22.9.3. Optimality characterisation

Important

A circulation f is optimal (min-cost) if and only if there exists a price function p such that all reduced costs in the residual graph G_f are non-negative.

Forward direction ($\Leftarrow$): If all reduced costs are non-negative, every cycle in G_f has non-negative reduced cost, hence non-negative original cost (by the invariance above). No negative-cost cycle exists, so f is optimal.

Reverse direction ($\Rightarrow$): If G_f has no negative cycles, add a dummy vertex s with zero-cost edges to all vertices. Define $p(v) = \text{dist}(s, v)$ under the cost function (well-defined since no negative cycles). The triangle inequality gives $c_p(u, v) \geq 0$ for every residual edge.

22.10. Faster min-cost flow for unit capacities

22.10.1. Setup

Assume all edge capacities are 1 and all edge costs are non-negative (we handle the general case via a reduction sketched below).

22.10.2. The algorithm

```
# Min-cost max flow for unit-capacity, non-negative cost graphs
f = zero flow
while there exists an s-t path in residual graph G_f:
    P = shortest s-t path in G_f (by cost, using Dijkstra)
    augment f along P
```

22.10.3. Why Dijkstra stays valid

The critical question: augmenting introduces reverse edges with cost $-c(e)$. Why don't we get negative edges?

Set prices $p(v) = \text{dist}(s, v)$ in the current residual graph (shortest-path distances under costs). Every edge (u, v) on a shortest path satisfies:

$$\text{dist}(s, v) = \text{dist}(s, u) + c(u, v)$$

which means its reduced cost $c_p(u, v) = 0$. When we augment along this path in a unit-capacity graph, each edge is fully saturated, removed from the residual graph, and replaced by its reverse. The reverse edge has reduced cost $-c_p(u, v) = 0$.

So every new edge added to the residual graph has reduced cost zero—never negative. Dijkstra remains valid in every iteration.

i Aside (mine, not the lecturer's)

After each augmentation, update prices as $p'(v) = p(v) + \text{dist}_p(s, v)$ where dist_p uses reduced costs. This is *Johnson's reweighting* applied dynamically. It keeps all reduced costs non-negative across iterations without re-running Bellman-Ford.

22.10.4. Runtime

- Iterations: at most n (each augmenting path in a unit-capacity graph carries one unit of flow; max flow value $\leq n$).
- Per iteration: Dijkstra with Fibonacci heaps costs $O(m + n \log n)$.
- Total: $O(mn + n^2 \log n)$.

22.10.5. Handling negative costs or general capacities

If the original graph has negative-cost edges:

1. Find any max flow f (ignoring costs).
2. In the residual graph, saturate every negative-cost edge to eliminate it. This creates excess/deficit at some vertices.
3. Add a super-source s' connected to deficit vertices and a super-sink t' connected to excess vertices, with capacities matching the imbalances.
4. Solve min-cost max flow on this modified graph—which now has all non-negative costs.

For general (non-unit) capacities, a capacity-scaling technique (see Problem Set 8) introduces a $\log U$ factor.

22.11. Cheat sheet

! Cheat sheet — LCT Analysis & Min-Cost Max Flow

Link-cut tree analysis:

- Each edge in the represented tree is heavy (child size $> n/2$) or light ($\leq n/2$)
- At most one heavy child per vertex; at most $\log n$ light edges on any root-to-leaf path
- Total preferred-child changes across m ops: $O(m \log n)$
- With balanced BSTs: $O(m \log^2 n)$ total; with splay trees: $O(m \log n)$

Min-cost max flow:

- Each edge has capacity $u(e)$ and cost $c(e)$; goal: max flow minimising $\sum f(e) c(e)$
- Reverse edges in residual graph have cost $-c(e)$
- Equivalent to min-cost circulation (reductions in both directions)
- Optimal iff no negative-cost cycle in residual graph
- Optimal iff $\exists$ price function p with all reduced costs $c_p(u, v) = c(u, v) + p(u) - p(v) \geq 0$

Algorithms:

Algorithm	Setting	Time
Negative-cycle cancelling	General	$O(m^2 n UC)$ — pseudo-polynomial
Shortest augmenting path (Dijkstra) + capacity scaling	Unit capacity, $c \geq 0$ General capacities	$O(mn + n^2 \log n)$ adds $O(\log U)$ factor
Bellman-Ford (negative-cycle finding)	General	$O(mn)$ per iteration
Goldberg scaling	General	$O(m\sqrt{n} \log C)$ per iteration

22.12. FAQ — questions students actually asked

Q: Why is there an m multiplying the splay cost in the runtime formula? The formula sums over all m operations. Each operation does at least one splay (the initial access), contributing $O(\log n)$. That baseline cost is $O(m \log n)$. On top of that, each preferred-child change triggers an additional splay, and the total PCCs add $O(m \log n)$ more.

Q: Can a vertex have two heavy children? No. If child u has $\text{size}(u) > \frac{1}{2}\text{size}(v)$, then u 's subtree contains more than half of v 's descendants. Two such children would account for more than all of v 's descendants—a contradiction.

Q: In the link analysis, why don't the new preferred edges along the path from w to v cost extra LPCCs? Those preferred edges were created by $\text{access}(w)$, which is already counted separately. The final step of link—adding the actual edge—changes subtree sizes but doesn't create new preferred edges.

Q: For min-cost circulation, what if the graph has negative-cost edges initially? Won't the algorithm diverge? No. Negative-cost edges are fine—the issue is negative-cost cycles. Negative-cycle cancelling terminates because each augmentation strictly decreases total cost, and costs are bounded below. If you want to avoid Bellman–Ford, the reduction to non-negative costs (saturate negative edges, fix excess/deficit with a super-source/sink) converts the problem to one solvable by Dijkstra.

Q: In the unit-capacity shortest-path algorithm, why are there at most n iterations? Each augmentation sends one unit of flow (unit capacity, so the bottleneck is always 1). The maximum flow value is at most $n - 1$ (at most $n - 1$ edge-disjoint s - t paths in a graph with n vertices).

Q: Why does the difference of two max flows of the same value form a circulation? Both flows satisfy flow conservation at every vertex (inflow = outflow + excess, with the same excess at s and t). Subtracting: the differences satisfy conservation with zero excess everywhere—that's a circulation.

22.13. Further reading

- Sleator and Tarjan, “A data structure for dynamic trees,” *J. Comput. System Sci.* 26(3):362–391, 1983. Proves the $O(\log n)$ amortised bound for link-cut trees.
- Goldberg and Tarjan, “Finding minimum-cost circulations by canceling negative cycles,” *J. ACM* 36(4):873–886, 1989. Formalises the negative-cycle cancelling algorithm and proves polynomial bounds with careful cycle selection.
- Goldberg, “Scaling algorithms for the shortest paths problem,” *SIAM J. Comput.* 24(3):494–504, 1995. The $O(m\sqrt{n} \log C)$ algorithm for shortest paths with negative weights.
- Orlin, “A faster strongly polynomial minimum cost flow algorithm,” *Oper. Res.* 41(2):338–350, 1993. Strongly polynomial min-cost flow in $O(m \log n(m + n \log n))$.
- Ahuja, Magnanti, and Orlin, *Network Flows: Theory, Algorithms, and Applications*, Prentice Hall, 1993. The standard reference for all things network flow, including min-cost flow.

22.14. Try it yourself

1. Heavy-light by hand. Draw a rooted tree on 10 vertices. Label each vertex with its subtree size. Classify every edge as heavy or light. Verify that every root-to-leaf path has at most $\lfloor \log_2 10 \rfloor = 3$ light edges.
2. PCC accounting. Consider a path graph $v_1 - v_2 - \dots - v_8$ rooted at v_1 . Perform the following access sequence: v_8, v_4, v_6, v_2, v_7 . For each access, list the preferred-child changes and classify each new preferred edge as heavy or light. Verify that your total LPCC count is consistent with the $O(\log n)$ per-access bound.
3. Reduced costs and Dijkstra. Given a directed graph with costs $c(u, v)$ and a price function p , prove that Dijkstra's algorithm on reduced costs c_p finds the same shortest paths as on c . Then show that after augmenting along a shortest path and updating prices as $p'(v) = p(v) + \text{dist}_p(s, v)$, all reduced costs remain non-negative.
4. Negative-cycle cancelling. Consider a graph with 4 vertices $\{s, a, b, t\}$, edges $s \rightarrow a$ (cap 3, cost 1), $s \rightarrow b$ (cap 2, cost 4), $a \rightarrow b$ (cap 2, cost 2), $a \rightarrow t$ (cap 2, cost 3), $b \rightarrow t$ (cap 3, cost 1). Find the min-cost max flow by first computing any max flow, then cancelling negative cycles in the residual graph.

5. Coding challenge. Implement the successive shortest path algorithm for min-cost max flow using Dijkstra with Johnson's reweighting. Test it on a bipartite matching instance (unit capacities, unit costs on matching edges, zero costs on source/sink edges) and verify the result is a minimum-weight perfect matching.

Part V.

Streaming and Sketching

23. Inclusion-Exclusion, Möbius Inversion, and the Streaming Model

COMPSCI 224 Lecture 25 — Fall 2014

Möbius inversion over the subset lattice turns the inclusion-exclusion principle into a computational tool: given a function and its Zeta transform, you can recover the original in $O^*(2^n)$ time via Yates' algorithm. Applied to graph coloring, this yields a k -colorability test in $2^n \text{poly}(n)$ time and polynomial space—matching the dynamic-programming bound in time while slashing exponential space. The lecture then pivots to the streaming model, proves that deterministic exact algorithms for distinct-element counting require linear space, and sketches the Flajolet–Martin algorithm that breaks the barrier with randomization and approximation.

23.1. Why this lecture matters

Two seemingly unrelated ideas—combinatorial transforms on the Boolean lattice and sublinear-space data structures—share a common thread: both trade exact knowledge for computational efficiency. The Zeta transform / Möbius inversion machinery gives you a principled way to derive and compute formulas like the k -colorability criterion; Yates' algorithm then evaluates them in optimal $O^*(2^n)$ time. The streaming lower bound, proved by a clean encoding argument, shows that randomization and approximation are not luxuries but necessities for any sublinear-space algorithm over massive data.

30-second story

1. The Zeta transform $\hat{f}(S) = \sum_{R \subseteq S} f(R)$ sums a function over all subsets; Möbius inversion $\tilde{f}(S) = \sum_{R \subseteq S} (-1)^{|S|-|R|} f(R)$ undoes it.
2. Composing the two in either order returns the original function—this is the inclusion-exclusion principle in operator form.
3. The number of ways to k -colour a graph equals a Möbius inversion of the k -th power of $\hat{f}$, where f indicates non-empty independent sets.
4. Yates' algorithm computes all 2^n values of $\hat{f}$ in $O(n \cdot 2^n)$ time and $O(n \cdot 2^n)$ space via a simple recurrence.
5. Any deterministic streaming algorithm for distinct-element counting needs $\Omega(n)$ bits—proved by turning it into a lossless compression scheme.
6. The Flajolet–Martin algorithm sidesteps this barrier: hash items into $[0, 1]$, track the minimum, and output $1/z - 1$. Averaging $O(1/\epsilon^2)$ copies gives a $(1 + \epsilon)$ -approximation.

23.2. Möbius inversion: undoing the Zeta transform

23.2.1. The two transforms

Fix a ground set $[n] = \{1, \dots, n\}$ and a function $f : 2^{[n]} \rightarrow \mathcal{R}$ mapping subsets to elements of some ring $\mathcal{R}$. The ring structure is all you need—addition and multiplication, not division.

Zeta transform. The *Zeta transform* of f is

$$\hat{f}(S) = \sum_{R \subseteq S} f(R). \quad (23.1)$$

For each set S , sum f over every subset of S .

Möbius inversion. The *Möbius inversion* of f is

$$\tilde{f}(S) = \sum_{R \subseteq S} (-1)^{|S|-|R|} f(R). \quad (23.2)$$

Same summation range, but each term carries an alternating sign determined by how many elements of S are missing from R .

These two operations are inverses of each other.

💡 Intuition

Think of $\hat{f}$ as a “prefix sum” over the subset lattice—it accumulates f over all subsets of S . Möbius inversion is the lattice analogue of differencing: it peels off the contributions of proper subsets to recover the original value. The alternating signs play the same role as inclusion-exclusion in counting.

23.2.2. Worked example

Let $n = 2$ and $f(\emptyset) = 0$, $f(\{1\}) = 3$, $f(\{2\}) = 5$, $f(\{1, 2\}) = 1$.

S	$\hat{f}(S)$	computation
$\emptyset$	$f(\emptyset) = 0$	one term
$\{1\}$	$f(\emptyset) + f(\{1\}) = 3$	two terms
$\{2\}$	$f(\emptyset) + f(\{2\}) = 5$	two terms
$\{1, 2\}$	$f(\emptyset) + f(\{1\}) + f(\{2\}) + f(\{1, 2\}) = 9$	four terms

Now apply Möbius inversion to $\hat{f}$:

$$\tilde{f}(\{1, 2\}) = \hat{f}(\{1, 2\}) - \hat{f}(\{1\}) - \hat{f}(\{2\}) + \hat{f}(\emptyset) = 9 - 3 - 5 + 0 = 1 = f(\{1, 2\}). \checkmark$$

The signs follow the pattern $(-1)^{|S|-|R|}$: subsets of size 2 get +, size 1 get −, size 0 get +.

23.2.3. The inversion theorem

Theorem. $\tilde{\tilde{f}} = \hat{\hat{f}} = f$. That is, applying the Zeta transform then Möbius inversion (or vice versa) recovers the original function.

Proof sketch (one direction). Evaluate $\tilde{\hat{f}}(T)$:

$$\tilde{\hat{f}}(T) = \sum_{S \subseteq T} (-1)^{|T|-|S|} \hat{f}(S) = \sum_{S \subseteq T} (-1)^{|T|-|S|} \sum_{R \subseteq S} f(R) = \sum_{R \subseteq T} f(R) \underbrace{\sum_{S: R \subseteq S \subseteq T} (-1)^{|T|-|S|}}_{(\star)}. \quad (23.3)$$

The inner sum ($\star$) equals $[R = T]$ by the inclusion-exclusion identity: summing $(-1)^{|T|-|S|}$ over all S sandwiched between R and T yields 1 when $R = T$ and 0 otherwise. Therefore $\widetilde{f}(T) = f(T)$.

i Why the inner sum vanishes

When $R \subsetneq T$, the set $T \setminus R$ is non-empty with $k = |T| - |R| \geq 1$ elements. The sum factors as $\sum_{j=0}^k \binom{k}{j} (-1)^j = (1 - 1)^k = 0$ by the binomial theorem.

23.3. Application: k -colourability

23.3.1. From colourings to independent sets

A graph $G = (V, E)$ is k -colourable if and only if V can be written as the union of k non-empty independent sets (the colour classes). The sets need not be disjoint—any overlap can be resolved greedily by assigning each vertex to the first colour class that contains it.

Why is overlap harmless? Suppose vertex v appears in colour classes C_1 and C_3 . Just keep v in C_1 and remove it from C_3 . After processing all vertices this way, the classes are disjoint and still independent (removing a vertex from an independent set keeps it independent). The k -colouring is valid.

Define:

- $f(S) = [S \text{ is a non-empty independent set in } G]$, the indicator function.
- $g(S) =$ number of ordered k -tuples of non-empty independent sets whose union is S .

Then G is k -colourable if and only if $g(V) > 0$.

23.3.2. The key identity

Claim. $\hat{g}(S) = \hat{f}(S)^k$.

Both sides count the same thing: the number of ways to choose an ordered k -tuple of non-empty independent sets, all contained in S .

- Left-hand side. By definition, $\hat{g}(S) = \sum_{R \subseteq S} g(R)$. The function $g(R)$ counts ordered tuples whose union is *exactly* R . Summing over all $R \subseteq S$ counts all tuples whose union is *contained in* S .
- Right-hand side. $\hat{f}(S)$ counts the number of non-empty independent sets contained in S . Raising to the k -th power counts ordered k -tuples of such sets—choosing each of the k slots independently.

Both sides count the same objects, so they are equal.

By Möbius inversion, the quantity we actually need is:

$$g(V) = \widetilde{f}^k(V) = \sum_{S \subseteq V} (-1)^{|V|-|S|} \hat{f}(S)^k. \quad (23.4)$$

Check whether $g(V) > 0$ to decide k -colourability.

💡 Where did this formula come from?

The derivation is mechanical once you know the Zeta/Möbius framework. Start with the thing you want (g), express its Zeta transform in a computable form ($\hat{f}^k$), then Möbius-invert to recover g itself. The framework

does the bookkeeping; you just need to identify the right f .

! Finding a colouring, not just deciding

Björklund and Husfeldt (FOCS '06) give a short black-box reduction: if you can compute the chromatic number in $O^*(2^n)$ time, you can also *find* a valid k -colouring in roughly the same time. The reduction is a few paragraphs and appears in their arXiv preprint. Crucially, it works for *any* algorithm that decides k -colourability—nothing specific to the inclusion-exclusion approach.

23.4. Computing the transforms efficiently

23.4.1. Brute force: $O^*(3^n)$ time, polynomial space

Evaluate Equation 23.4 directly. For each of the 2^n subsets S , compute $\hat{f}(S)$ by summing over all $R \subseteq S$ —that costs $2^{|S|}$ time per set. The total running time is:

$$\sum_{S \subseteq [n]} 2^{|S|} = \sum_{i=0}^n \binom{n}{i} 2^i = (1 + 2)^n = 3^n.$$

The key step uses the binomial theorem: group subsets by size i , count $\binom{n}{i}$ of them, each contributing 2^i work.

Since each $\hat{f}(S)$ feeds a running sum for $g(V)$ and can then be discarded, the space is polynomial—you only need to store the current $\hat{f}(S)$ value and the accumulator. This already improves on the standard dynamic-programming algorithm for k -colourability, which uses $O^*(3^n)$ time but also $O^*(2^n)$ space to tabulate subproblem solutions.

23.4.2. Yates' algorithm: $O(n \cdot 2^n)$ time, $O(n \cdot 2^n)$ space

Define a family of partial Zeta transforms:

$$G(i, S) = \sum_{\substack{R \subseteq S \\ S_{>i} = R_{>i}}} f(R), \quad (23.5)$$

where $S_{>i} = S \cap \{i + 1, \dots, n\}$ denotes the “upper” portion of S . The constraint $S_{>i} = R_{>i}$ means R must agree with S on coordinates above i but is free on coordinates $1, \dots, i$.

Base case. $G(0, S) = f(S)$, because $S_{>0} = S$ forces $R = S$.

Goal. $\hat{f}(S) = G(n, S)$, because $S_{>n} = \emptyset$ imposes no constraint.

Recurrence.

$$G(i, S) = \begin{cases} G(i-1, S) & \text{if } i \notin S, \\ G(i-1, S) + G(i-1, S \setminus \{i\}) & \text{if } i \in S. \end{cases} \quad (23.6)$$

Proof of correctness (case $i \in S$). Split the sum defining $G(i, S)$ according to whether $i \in R$:

- $i \in R$: Both R and S contain i , so $S_{>i-1} = R_{>i-1}$ iff $S_{>i} = R_{>i}$. This sub-sum equals $G(i-1, S)$.
- $i \notin R$: Now R agrees with S from $i+1$ onward but disagrees at i . That means R agrees with $S \setminus \{i\}$ from i onward. This sub-sum equals $G(i-1, S \setminus \{i\})$.

Complexity. There are $n \cdot 2^n$ states (i, S) . Each state takes $O(1)$ time: look up at most two previously computed values, add them, and store the result. Total: $O(n \cdot 2^n)$ time and $O(n \cdot 2^n)$ space (or $O(2^n)$ space if you overwrite in place, sweeping through layers $i = 1, \dots, n$).

Möbius inversion via Yates. A nearly identical recurrence computes $\tilde{f}$. Change the $+$ to $-$ in the $i \in S$ case:

$$G^-(i, S) = \begin{cases} G^-(i-1, S) & \text{if } i \notin S, \\ G^-(i-1, S) - G^-(i-1, S \setminus \{i\}) & \text{if } i \in S. \end{cases}$$

Everything else—base case, goal, complexity—is unchanged.

Connection to the fast Fourier transform

Yates' algorithm has the same “butterfly” structure as the FFT. Both decompose a sum over a group (here Z_2^n) by peeling off one coordinate at a time. The Zeta transform over the subset lattice is precisely the Fourier transform over the Boolean group. If you have seen the Cooley–Tukey FFT, the recurrence above will feel familiar: double the “resolved” dimensions at each step, combining two half-size sub-problems with an additive operation.

23.4.3. Summary of running times for k -colourability

Table 23.2.: Running times for deciding k -colourability.

Approach	Time	Space	Reference
Dynamic programming	$O^*(3^n)$	$O^*(2^n)$	folklore
Inclusion-exclusion (brute force)	$O^*(3^n)$	$\text{poly}(n)$	—
Inclusion-exclusion + Yates	$O^*(2^n)$	$O^*(2^n)$	Björklund–Husfeldt; Koivisto (FOCS '06)
$O^*(2^{0.2461n})$ time, poly space	$O^*(2^{0.2461n})$	$\text{poly}(n)$	Björklund–Husfeldt (FOCS '06)

Open problem

Is there an algorithm achieving $O^*(2^n)$ time with only $\text{poly}(n)$ space? The best known polynomial-space algorithm runs in $O^*(2^{0.2461n})$, still far from 2^n .

23.5. The streaming model

23.5.1. Setup

You have a high-dimensional vector $x \in \mathbb{R}^n$ that starts as the zero vector. A stream of updates arrives one at a time; each update is a pair (i, v) that increments x_i by v . Between updates, you must answer queries about x —but you cannot afford to store all of x or remember the entire stream.

This is the turnstile model (Muthukrishnan, ~2005): updates can be positive or negative, modelling both insertions and deletions.

Motivating example. Let $n = 2^{128}$ (IPv6 addresses). A router sees packets and wants to track statistics about the destination distribution x . Storing x explicitly requires 2^{128} counters—impossible. Remembering every packet requires space proportional to the stream length m —also impractical when m is billions.

The trivial solutions always exist: store all of x in $O(n)$ space, or record the entire stream in $O(m \log n)$ space (each stream element is an index taking $\log n$ bits). The goal of streaming algorithms is to beat both bounds drastically.

💡 Intuition

A streaming algorithm is a data structure with a brutal space constraint: its memory must be far smaller than either the vector dimension n or the stream length m . Updates arrive one at a time, and there is no going back—you process each update, possibly discard some information, and move on. The surprise is that useful queries can still be answered accurately despite this irreversible compression.

23.5.2. The distinct-elements problem (F_0)

Restrict to the *insertion-only* model: every update has $v = 1$. The query is $F_0(x) = |\{i : x_i > 0\}|$ —the number of distinct elements seen so far.

The trivial solutions use n bits (a bitmap) or $m \log n$ bits (store every stream element). Can you do better?

23.5.3. A linear lower bound for deterministic algorithms

Theorem (Alon–Matias–Szegedy, 1996/99). Any deterministic algorithm for F_0 requires $\Omega(\min(n, m))$ bits of space.

Proof. Via an encoding argument. The strategy: if a space- s streaming algorithm existed, you could use it to losslessly compress *any* n -bit string into s bits. But lossless compression of n -bit strings into fewer than n bits is impossible by pigeonhole (there are 2^n strings but only $2^s < 2^n$ possible compressed forms). So $s \geq n$.

Encoding (Alice). Given $x \in \{0, 1\}^n$, Alice creates a stream containing exactly the set $\{i : x_i = 1\}$, runs $\mathcal{A}$ on this stream, and transmits the s -bit memory state of $\mathcal{A}$ to Bob. This is the “compression” of x .

Decoding (Bob). Bob receives the s -bit memory state and reconstructs x bit by bit. He initialises $z = \mathcal{A}.\text{query}()$ —the current support size. Then, for each $i = 1, \dots, n$:

1. Feed update $(i, 1)$ into $\mathcal{A}$ (inserting element i).
2. Query $\mathcal{A}$ for the new support size z' .
3. If $z' = z$: element i was already present, so $x_i = 1$.
4. If $z' = z + 1$: element i is new, so $x_i = 0$.
5. Set $z \leftarrow z'$ and continue.

After n steps, Bob has recovered all bits of x .

Why this is an injection. Different n -bit strings $x \neq x'$ produce different streams, hence different memory states (since Bob can uniquely decode from the state). So the map $x \mapsto$ memory state is an injection from $\{0, 1\}^n$ to $\{0, 1\}^s$, forcing $s \geq n$. In this construction $m \leq n$ (the stream has at most n elements), so $s = \Omega(\min(n, m))$.

❗ What’s actually impossible?

The encoding argument also rules out deterministic approximate and randomized exact algorithms. Both can still be turned into compression schemes. Only the combination of randomization *and* approximation breaks the barrier.

23.5.4. The Flajolet–Martin algorithm (idealized)

Goal. A randomized algorithm $\mathcal{A}$ such that for all streams S :

$$\Pr[|\mathcal{A}(S) - F_0| > \varepsilon \cdot F_0] \leq \frac{1}{3}.$$

Idealized version. Pick a totally random hash function $h : [n] \rightarrow [0, 1]$ (each element gets an independent, uniformly distributed value in the unit interval). As the stream arrives, maintain a single number:

$$z = \min_{i \in \text{stream}} h(i).$$

When asked for F_0 , output $1/z - 1$.

The algorithm uses just *one register* of memory (ignoring the hash function itself). Every time a stream element i arrives, compute $h(i)$ and update $z \leftarrow \min(z, h(i))$. That is the entire algorithm.

Why this works. Imagine scattering F_0 points uniformly at random onto the interval $[0, 1]$. In expectation, they divide the interval into $F_0 + 1$ roughly equal segments, each of length $\approx 1/(F_0 + 1)$. The smallest point—the minimum hash value—sits at the left end of the first segment, so $z \approx 1/(F_0 + 1)$, and $1/z - 1 \approx F_0$.

Formally. Use the tail-sum formula for non-negative random variables:

$$\mathbb{E}[z] = \int_0^1 \Pr[z > t] dt. \quad (23.7)$$

The event $z > t$ means every hash value exceeds t . Since the F_0 distinct items have independent hash values, each exceeding t with probability $1 - t$:

$$\Pr[z > t] = (1 - t)^{F_0}.$$

Integrating:

$$\mathbb{E}[z] = \int_0^1 (1 - t)^{F_0} dt = \frac{1}{F_0 + 1}. \quad (23.8)$$

Similarly, $\mathbb{E}[z^2] = \int_0^1 2t(1 - t)^{F_0} dt = 2/((F_0 + 1)(F_0 + 2))$, giving:

$$\text{Var}(z) = \mathbb{E}[z^2] - (\mathbb{E}[z])^2 = \frac{F_0}{(F_0 + 1)^2(F_0 + 2)} = O\left(\frac{1}{F_0^2}\right).$$

By Chebyshev's inequality, z concentrates around $1/(F_0 + 1)$, so $1/z - 1$ gives a constant-factor estimate of F_0 with constant probability. But a single copy has high variance—the estimate might be off by a factor of 2 or more. To reduce variance to $(1 + \varepsilon)$ accuracy:

Averaged version. Pick $R = \Theta(1/\varepsilon^2)$ independent hash functions $h_1, \dots, h_R$. Maintain $z_j = \min_{i \in \text{stream}} h_j(i)$ for each j . Output $1/\bar{z} - 1$ where $\bar{z} = \frac{1}{R} \sum_j z_j$.

Each z_j has the same expectation $1/(F_0 + 1)$, so $\bar{z}$ has the same expectation. But the variance of $\bar{z}$ is $\text{Var}(z)/R$ —averaging R independent copies reduces variance by a factor of R . Setting $R = \Theta(1/\varepsilon^2)$ makes the relative standard deviation of $\bar{z}$ drop to $O(\varepsilon)$, which is just enough for a $(1 + \varepsilon)$ -approximation via the Chebyshev bound:

$$\Pr[|z - E[z]| > \varepsilon \cdot E[z]] \leq \frac{\text{Var}(z)}{\varepsilon^2(E[z])^2}.$$

The ratio $\text{Var}(z)/(E[z])^2$ is $O(1/F_0)$ for a single copy—already small when F_0 is large—but the R -copy average makes it $O(1/(R \cdot F_0))$, controllable for any desired ε .

⚠ Two idealizations to fix

1. Storing the hash function. A truly random h requires n random values— $\Omega(n)$ space. Fix: use a k -wise independent hash family (covered earlier in the course), storable in $O(k \log n)$ bits.
2. Continuous range. Real numbers have infinite precision. Fix: discretize $[0, 1]$ into fractions with denominators $\text{poly}(n)$; $O(\log n)$ bits per hash value suffice.

23.5.5. What's possible and what isn't

Table 23.3.: Landscape of distinct-elements algorithms. All “impossible” results are due to Alon–Matias–Szegedy.

	Exact	Approximate
Deterministic	Impossible ($\Omega(n)$ bits)	Impossible
Randomized	Impossible	Possible (Flajolet–Martin, JCSS '85)

Failure probability δ can be driven arbitrarily small at the cost of a $\log(1/\delta)$ factor in space.

i Las Vegas vs. Monte Carlo

Las Vegas: always correct, running time is expected (like randomised quicksort). Monte Carlo: always efficient, correctness is probabilistic (like Flajolet–Martin). The streaming lower bounds kill even Las Vegas algorithms for F_0 —you need Monte Carlo.

23.6. Cheat sheet

! Cheat sheet — Lecture 25

Zeta transform / Möbius inversion

- $\hat{f}(S) = \sum_{R \subseteq S} f(R)$ and $\tilde{f}(S) = \sum_{R \subseteq S} (-1)^{|S|-|R|} f(R)$.
- $\tilde{\tilde{f}} = \hat{\hat{f}} = f$ (the two are mutual inverses).

k -colourability via inclusion-exclusion

- $f(S) = [S \text{ is a non-empty independent set}]$.
- G is k -colourable iff $\sum_{S \subseteq V} (-1)^{|V|-|S|} \hat{f}(S)^k > 0$.

Yates' algorithm

- Computes all 2^n values of $\hat{f}$ (or $\tilde{f}$) in $O(n \cdot 2^n)$ time and space.
- Recurrence: $G(i, S) = G(i-1, S) + [i \in S] \cdot G(i-1, S \setminus \{i\})$.

Streaming / distinct elements

- Turnstile model: updates (i, v) ; goal is sublinear space.
- Deterministic exact F_0 : $\Omega(n)$ bits (encoding argument).
- Flajolet–Martin: hash to $[0, 1]$, track $z = \min h(i)$, output $1/z - 1$.
- Space for $(1 + \varepsilon)$ -approximation: $O(1/\varepsilon^2)$ hash functions $\times O(\log n)$ bits each.

23.7. FAQ — questions students actually asked

Q: In the k -colourability formula, the independent sets are ordered tuples. Doesn't that over-count? A: The formula counts ordered tuples whose union covers V . If *any* such tuple exists, $g(V) > 0$, which is all we need for a decision problem. Over-counting ordered tuples doesn't affect the yes/no answer.

Q: Can you also *find* a k -colouring, not just decide if one exists? A: Yes. Björklund and Husfeldt give a black-box reduction: if you can compute the chromatic number in $O^*(2^n)$ time, you can find the colouring in roughly the same time. It's a short argument (a few paragraphs in their paper).

Q: In the streaming lower bound, why can Bob decode? Doesn't inserting element i when $x_i = 1$ change nothing? A: Exactly—that's the point. If $x_i = 1$, element i was already in the stream, so re-inserting it doesn't change F_0 . If $x_i = 0$, element i is new and F_0 increases by one. Bob detects the difference.

Q: What if I want to query a specific x_i instead of the support size? Does the same lower bound apply? A: Deterministic exact and randomized exact are still impossible. But a deterministic *approximate* algorithm exists for point queries: you can output $x_i \pm \varepsilon \|x\|_1$ in space $O(1/\varepsilon^2)$. Randomisation can improve this to $O(1/\varepsilon)$.

Q: Can you make the Flajolet–Martin failure probability δ instead of $1/3$? A: Yes. Run $O(\log(1/\delta))$ independent copies and take the median. The probability that the median fails decays exponentially in the number of copies, by a Chernoff bound.

23.8. Further reading

1. A. Björklund, T. Husfeldt. *Inclusion-exclusion algorithms for counting set partitions*. FOCS 2006. The source for $O^*(2^n)$ -time k -colourability and the search-to-decision reduction.
2. A. Koivisto. *An $O^*(2^n)$ algorithm for graph coloring and other partitioning problems via inclusion-exclusion*. FOCS 2006. Independent work arriving at the same result.
3. T. Husfeldt. *Invitation to algorithmic uses of inclusion-exclusion*. ICALP 2011 (invited paper). A survey of further applications: Steiner tree, Hamiltonian path, chromatic polynomial, and more.
4. P. Flajolet, G.N. Martin. *Probabilistic counting algorithms for data base applications*. JCSS 31(2), 1985. The original distinct-elements algorithm using hash-based minimum tracking.
5. N. Alon, Y. Matias, M. Szegedy. *The space complexity of approximating the frequency moments*. JCSS 58(1), 1999 (conference version STOC '96). Lower bounds for F_0 and the AMS sketch for higher frequency moments.

23.9. Try it yourself

1. Möbius inversion by hand. Let $n = 3$ and $f(S) = |S|$. Compute $\hat{f}(S)$ and $\tilde{f}(S)$ for all $S \subseteq \{1, 2, 3\}$. Verify that $\tilde{\tilde{f}} = f$.
2. Yates for Möbius inversion. Modify the recurrence in Equation 23.6 to compute Möbius inversion instead of the Zeta transform. (*Hint*: change one sign.)

- Encoding argument for approximate F_0 . Extend the lower-bound proof in Section 23.5.3 to show that any *deterministic* $(1 + \varepsilon)$ -approximate algorithm for F_0 also requires $\Omega(n)$ bits, for any constant $\varepsilon < 1$.
- Implement Flajolet–Martin. Write a Python function that estimates the number of distinct elements in a stream using the averaged Flajolet–Martin algorithm. Use a k -wise independent hash family (or Python’s random module as a stand-in). Test on a stream of 10^5 elements drawn from a universe of size 10^4 with many repeats.

```
import random
import math

def flajolet_martin(stream, R=100):
    """Estimate the number of distinct elements in stream.

    Args:
        stream: iterable of hashable elements
        R: number of independent hash functions (controls accuracy)

    Returns:
        Estimated number of distinct elements
    """
    # Use R independent random seeds to simulate independent hash functions
    seeds = [random.randint(0, 2**64) for _ in range(R)]
    z = [1.0] * R # track minimum hash value for each function

    for item in stream:
        for j in range(R):
            # Hash item to [0, 1) using seed j
            random.seed(hash((item, seeds[j])))
            h = random.random()
            z[j] = min(z[j], h)

    z_bar = sum(z) / R
    if z_bar == 0:
        return float('inf')
    return 1.0 / z_bar - 1

# Test
random.seed(42)
universe = range(10_000)
true_distinct = 3_000
elements = random.choices(list(universe)[:true_distinct], k=100_000)
estimate = flajolet_martin(elements, R=200)
print(f"True distinct: {true_distinct}")
print(f"Estimated:      {estimate:.0f}")
print(f"Relative error: {abs(estimate - true_distinct) / true_distinct:.2%}")
```

- Chromatic number search. Using Equation 23.4, write a brute-force Python program that computes the chromatic number of a graph on $n \leq 15$ vertices. Represent subsets as bitmasks. Compare with a backtracking colouring algorithm.

24. The Power of Random Signs

COMPSCI 224 Lecture 26 — Fall 2014

A single idea—multiplying by a matrix of random ± 1 entries—solves four different problems: estimating ℓ_2 -norms in a stream, accelerating least-squares regression, reducing dimensionality with the Johnson–Lindenstrauss guarantee, and answering point queries over a streaming vector. The AMS sketch (Alon–Matias–Szegedy) achieves $O(1/\varepsilon^2 \cdot \log(1/\delta))$ space for ℓ_2 -norm estimation; the Thorup–Zhang sparse variant cuts update time from $O(1/\varepsilon^2)$ to $O(1)$ per stream element. Applied to regression, random-sign sketching replaces the nd^2 bottleneck of forming $A^\top A$ with md^2 where $m = O(d^2/\varepsilon^2)$, yielding input-sparsity-time algorithms.

24.1. Why this lecture matters

Random sign matrices are the Swiss army knife of modern algorithm design. Pick an $m \times n$ matrix Π with independent $\pm 1/\sqrt{m}$ entries, multiply your data by Π , and the result preserves geometric structure—norms, dot products, angles—up to $(1 + \varepsilon)$ error. This lecture deploys that one idea four times, each application building on the last: streaming norm estimation, fast regression, dimensionality reduction, and point queries. The progression reveals a unifying principle: compressing high-dimensional data by random projection loses almost nothing that matters.

30-second story

1. AMS sketch: store $y = \Pi x$ where Π has random $\pm 1/\sqrt{m}$ entries. Then $\|y\|_2^2$ is an unbiased estimator of $\|x\|_2^2$ with variance $O(\|x\|_2^4/m)$.
2. Set $m = O(1/\varepsilon^2)$ rows, repeat $O(\log(1/\delta))$ times, take the median $\approx (1 \pm \varepsilon)$ -approximation with probability $1 - \delta$.
3. Thorup–Zhang sparse sketch: one non-zero per column $\approx O(\log(1/\delta))$ update time instead of $O(1/\varepsilon^2 \cdot \log(1/\delta))$.
4. Subspace embedding for regression: Π preserves all norms in a d -dimensional subspace simultaneously when $m = O(d^2/\varepsilon^2)$, reducing regression from nd^2 to $\text{nnz}(A) + \text{poly}(d/\varepsilon)$ time.
5. Johnson–Lindenstrauss: $m = O(\varepsilon^{-2} \log N)$ rows suffice to preserve all $\binom{N}{2}$ pairwise distances among N points.
6. Point query: JL applied to the n standard basis vectors gives a streaming point-query sketch with $O(\varepsilon^{-2} \log n)$ space.

24.2. ℓ_2 -norm estimation via linear sketching

24.2.1. The problem

You maintain a vector $x \in \mathbb{R}^n$ under turnstile updates: each update (i, v) sets $x_i \leftarrow x_i + v$, where v can be positive or negative. The query is $\|x\|_2^2 = \sum_i x_i^2$, and you want a $(1 + \varepsilon)$ -approximation using space much smaller than n .

 Why the distinct-elements approach fails here

The Flajolet–Martin algorithm from Lecture 25 tracked $\min_i h(i)$ over inserted elements. Deletions ($v < 0$) break this: if the element with the minimum hash value gets deleted, the stored minimum becomes stale, and the true second-minimum was never recorded. Linear sketching handles deletions natively.

24.2.2. Linear sketches

A linear sketch is a compressed representation of x defined by a fixed $m \times n$ matrix Π (with $m \ll n$). You store $y = \Pi x \in \mathbb{R}^m$ —a vector of m numbers instead of n . Write the columns of Π as $\pi_1, \dots, \pi_n$.

When an update (i, v) arrives, the sketch updates as $y \leftarrow y + v \cdot \pi_i$ —a single vector addition. This works because $\Pi(x + v e_i) = \Pi x + v \pi_i$. The linearity is the key property: the sketch of the updated vector equals the update applied to the sketch. Deletions are just updates with negative v , and they work identically.

You never store x itself. You never store Π explicitly either (you generate its entries on the fly from a hash function). You only store y — m numbers—and that is your entire memory.

 Aside (mine, not the lecturer's)

Li, Nguyen, and Woodruff showed that for any streaming query that must support deletions, any algorithm can be converted into a linear sketch with at most an $O(\log n)$ blow-up in space. So linear sketching is essentially *the* framework for turnstile streaming.

24.2.3. The AMS sketch

Construction. Set Π to be an $m \times n$ matrix with independent entries $\pi_{ij} = \sigma_{ij}/\sqrt{m}$, where each σ_{ij} is a uniform random sign (+1 or −1 with equal probability). Store $y = \Pi x$ and estimate $\|x\|_2^2$ as $\|y\|_2^2$.

24.2.4. Analysis: expectation

Each coordinate of y is $y_r = \frac{1}{\sqrt{m}} \sum_{i=1}^n \sigma_{ri} x_i$. Squaring:

$$y_r^2 = \frac{1}{m} \left(\sum_i \sigma_{ri}^2 x_i^2 + \sum_{i \neq j} \sigma_{ri} \sigma_{rj} x_i x_j \right). \quad (24.1)$$

Since $\sigma_{ri}^2 = 1$ and $E[\sigma_{ri} \sigma_{rj}] = E[\sigma_{ri}] E[\sigma_{rj}] = 0$ for $i \neq j$ (by independence):

$$E[y_r^2] = \frac{1}{m} \|x\|_2^2. \quad (24.2)$$

Summing over $r = 1, \dots, m$:

$$E[\|y\|_2^2] = \|x\|_2^2. \quad (24.3)$$

The sketch gives an unbiased estimator.

24.2.5. Analysis: variance

By independence of the rows of Π , the variance of $\|y\|_2^2 = \sum_r y_r^2$ is $m \cdot \text{Var}(y_r^2)$. To bound $\text{Var}(y_r^2) = \mathbb{E}[y_r^4] - (\mathbb{E}[y_r^2])^2$, expand y_r^4 :

$$y_r^4 = \frac{1}{m^2} \left(\sum_i \sigma_{ri} x_i \right)^4.$$

When you expand the fourth power, you get n^4 monomials, each a product of four random signs times four x values. Push the expectation inside by linearity. Any monomial where some σ_{ri} appears to an odd power has zero expectation (since $\mathbb{E}[\sigma^{\text{odd}}] = 0$ for a symmetric ± 1 variable). This kills most terms. The surviving terms fall into two categories:

- Four equal indices ($i = j = k = l$): $\sigma_{ri}^4 x_i^4$. Since $\sigma^4 = 1$, these contribute $\sum_i x_i^4$.
- Two pairs ($i = j \neq k = l$): $\sigma_{ri}^2 \sigma_{rk}^2 x_i^2 x_k^2$. Since $\sigma^2 = 1$, these contribute the cross terms $\sum_{i \neq k} x_i^2 x_k^2$, with a combinatorial factor (the number of ways to pair four slots into two equal pairs yields a coefficient of $\binom{4}{2} = 6$, but accounting for the ordering gives a factor of 3 for the $i \neq j$ terms).

All terms with three or four distinct indices vanish because at least one index appears to an odd power.

The bottom line:

$$\mathbb{E}[y_r^4] = \frac{1}{m^2} \left(\sum_i x_i^4 + 3 \sum_{i \neq j} x_i^2 x_j^2 \right) \leq \frac{3}{m^2} \left(\sum_i x_i^2 \right)^2 = \frac{3}{m^2} \|x\|_2^4.$$

Subtracting $(\mathbb{E}[y_r^2])^2 = \|x\|_2^4/m^2$:

$$\text{Var}(y_r^2) = \mathbb{E}[y_r^4] - (\mathbb{E}[y_r^2])^2 \leq \frac{3}{m^2} \|x\|_2^4 - \frac{1}{m^2} \|x\|_2^4 = \frac{2}{m^2} \|x\|_2^4,$$

which gives

$$\text{Var}(\|y\|_2^2) = m \cdot \text{Var}(y_r^2) \leq \frac{2}{m} \|x\|_2^4. \quad (24.4)$$

24.2.6. Putting it together: Chebyshev

By Chebyshev's inequality:

$$\Pr[|\|y\|_2^2 - \|x\|_2^2| > \varepsilon \|x\|_2^2] \leq \frac{\text{Var}(\|y\|_2^2)}{\varepsilon^2 \|x\|_2^4} \leq \frac{2}{m \varepsilon^2}. \quad (24.5)$$

Setting $m = 6/\varepsilon^2$ makes the failure probability at most $1/3$.

24.2.7. Amplification via the median trick

Run $k = O(\log(1/\delta))$ independent copies of the sketch, producing estimates $\hat{F}_1, \dots, \hat{F}_k$. Output the median.

Each copy succeeds with probability $\geq 2/3$. For the median to fail, more than half the copies must fail. The expected number of failures is $\leq k/3$, but a majority failure requires $\geq k/2$ —a large deviation. By a Chernoff bound, this probability is $\exp(-\Omega(k)) \leq \delta$.

Total space: $O\left(\frac{1}{\varepsilon^2} \log \frac{1}{\delta}\right)$ words, each holding a partial sum. This is optimal (Jayaram–Woodruff, TALG).

💡 Why median, not average?

Averaging would also reduce variance, but it requires the estimator to have bounded variance in every direction. The median is more robust: even if a few copies produce wildly wrong answers, the median ignores them. This makes the analysis cleaner and the bound tighter.

24.3. The Thorup–Zhang sparse sketch

24.3.1. Motivation: update time

The dense AMS sketch has m non-zeros in every column of Π , so each stream update costs $O(m) = O(1/\varepsilon^2 \cdot \log(1/\delta))$ time. For applications like network monitoring, this overhead matters.

24.3.2. Construction

Replace the dense random-sign matrix with a sparse one: each column has exactly *one* non-zero entry, a random sign ± 1 placed in a random row.

Formally, for each column j , pick $r_j \in [m]$ uniformly at random and $\sigma_j \in \{+1, -1\}$ uniformly. Set $\pi_{r_j, j} = \sigma_j$ and all other entries in column j to zero.

The sketch is still $y = \Pi x$. The estimate is still $\|y\|_2^2$. The expectation and variance analysis go through with the same asymptotic bounds—you can verify that $\mathbb{E}[\|y\|_2^2] = \|x\|_2^2$ and $\text{Var}(\|y\|_2^2) = O(\|x\|_2^4/m)$. The computation is similar but not identical: instead of independence between entries in the same row, you use the fact that each entry of y is now a sum over a random subset of coordinates of x (those mapped to that row).

Update time: When update (i, v) arrives, column i of Π has a single non-zero in row r_i . So y changes in exactly one coordinate: $y_{r_i} \leftarrow y_{r_i} + v \cdot \sigma_i$. That is $O(1)$ time. With $O(\log(1/\delta))$ independent copies for the median amplification, the total update time is $O(\log(1/\delta))$ —independent of ε .

! Strictly better

The sparse sketch matches the dense one in space and accuracy while improving update time from $O(1/\varepsilon^2 \cdot \log(1/\delta))$ to $O(\log(1/\delta))$. There is no reason to use the dense version.

i Open problem

Can ℓ_2 -norm estimation achieve $O(1)$ update time (or even $O(\sqrt{\log(1/\delta)})$) with optimal space? It is known that any *linear sketch* requires update time $\Omega(\sqrt{\log(1/\delta)})$, but there is a gap to the $O(\log(1/\delta))$ upper bound.

24.4. Application: cosine similarity

Random-sign sketches preserve dot products, not just norms. This unlocks a classic application: comparing documents without storing them in full.

Setup. Represent each document as a bag-of-words vector $x \in \mathbb{R}^n$, where x_i is the number of occurrences of word i . Normalise to unit length: $x \leftarrow x/\|x\|_2$. The cosine similarity of documents x and y is $\langle x, y \rangle = \cos \theta$, where θ is the angle between them. Similar documents have $\cos \theta$ close to 1; orthogonal (unrelated) documents have $\cos \theta \approx 0$.

The vocabulary size n can be hundreds of thousands. Storing the full vector for every document is expensive. Instead, store the sketch $\Pi x \in \mathbb{R}^m$ with $m \ll n$.

Why sketches preserve dot products. Use the polarisation identity:

$$\langle a, b \rangle = \frac{\|a\|_2^2 + \|b\|_2^2 - \|a - b\|_2^2}{2}. \quad (24.6)$$

The right-hand side involves three ℓ_2 -norms. The sketch Π preserves each up to $(1 \pm \epsilon)$, so the reconstructed dot product $\langle \Pi x, \Pi y \rangle$ is within additive $O(\epsilon)$ of $\langle x, y \rangle$. You can estimate cosine similarity from the compressed representations alone, without ever decompressing.

24.5. Fast regression via subspace embeddings

24.5.1. The least-squares bottleneck

In linear regression, you have n observations and d parameters. There is some unknown parameter vector $x \in \mathbb{R}^d$, and each observation gives you a row $a_i \in \mathbb{R}^d$ of predictor values and a response b_i . In the ideal noiseless case, $b_i = a_i^\top x$. In reality, observations are corrupted by noise: $b_i = a_i^\top x + \text{noise}$.

Stack the observations into an $n \times d$ matrix A (with $n \gg d$) and a vector $b \in \mathbb{R}^n$. The least-squares solution minimises the sum of squared residuals:

$$x^* = \operatorname{argmin}_x \|Ax - b\|_2 = (A^\top A)^+ A^\top b, \quad (24.7)$$

where $(A^\top A)^+$ is the pseudoinverse (which equals the ordinary inverse when A has full column rank). The dominant cost is forming $A^\top A$: a $d \times n$ matrix times an $n \times d$ matrix. Each of the d^2 entries requires a dot product of length n , giving $O(nd^2)$ total time. The subsequent solve—inverting the $d \times d$ matrix $(A^\top A)$ —costs only $O(d^3)$, which is negligible when $n \gg d$.

24.5.2. Sketched regression

Instead of solving the original problem, solve the *sketched* version:

$$\tilde{x}^* = \operatorname{argmin}_x \|\Pi A x - \Pi b\|_2, \quad (24.8)$$

where Π is an $m \times n$ sketch matrix with $m \ll n$. The bottleneck becomes $(\Pi A)^\top (\Pi A)$, which is $d \times m$ times $m \times d$ —costing $O(md^2)$.

24.5.3. Subspace embeddings

Definition (Sarlós, 2006). An $m \times n$ matrix Π is an ϵ -subspace embedding for a linear subspace $V \subseteq \mathbb{R}^n$ if for all $v \in V$:

$$(1 - \epsilon) \|v\|_2^2 \leq \|\Pi v\|_2^2 \leq (1 + \epsilon) \|v\|_2^2. \quad (24.9)$$

💡 Intuition

A subspace embedding is a random-sign sketch that works *simultaneously* for every vector in a d -dimensional subspace—not just one vector at a time. It is a uniform guarantee over an infinite set of directions.

Claim. If Π is an ε -subspace embedding for the column space of $[A \mid b]$ (the $(d + 1)$ -dimensional subspace spanned by b and the columns of A), then:

$$\|A\tilde{x}^* - b\|_2 \leq \frac{1 + \varepsilon}{1 - \varepsilon} \|Ax^* - b\|_2.$$

Proof. Since $\tilde{x}^*$ minimises the sketched objective: $\|\Pi A\tilde{x}^* - \Pi b\| \leq \|\Pi Ax^* - \Pi b\|$. Both $A\tilde{x}^* - b$ and $Ax^* - b$ lie in the subspace, so Π preserves their norms up to $(1 \pm \varepsilon)$. Combining these two facts:

$$(1 - \varepsilon)\|A\tilde{x}^* - b\|^2 \leq \|\Pi(A\tilde{x}^* - b)\|^2 \leq \|\Pi(Ax^* - b)\|^2 \leq (1 + \varepsilon)\|Ax^* - b\|^2.$$

Rearranging gives the result.

24.5.4. How many rows does Π need?

Let U be an $n \times d$ matrix whose columns form an orthonormal basis for V (so $U^\top U = I_d$). The subspace embedding condition is equivalent to:

$$\|\Pi U^\top \Pi U - I_d\|_{\text{op}} < \varepsilon, \tag{24.10}$$

where $\|\cdot\|_{\text{op}}$ denotes the operator norm (largest singular value). The analysis mirrors the vector case but uses a matrix Chebyshev argument:

$$\Pr[\|\Pi U^\top \Pi U - I\|_{\text{op}} > \varepsilon] \leq \frac{\mathbb{E}[\|\Pi U^\top \Pi U - I\|_F^2]}{\varepsilon^2},$$

where $\|\cdot\|_F$ is the Frobenius norm (square root of the sum of squared eigenvalues). The key inequality: $\|M\|_{\text{op}}^2 \leq \|M\|_F^2 = \text{tr}(M^2)$ for symmetric M .

The computation shows this is $O(d^2/m)$, so setting $m = O(d^2/\varepsilon^2)$ suffices.

i References

The subspace-embedding approach to regression was introduced by Sarlós (FOCS 2006). The analysis via Frobenius norm appears in Clarkson–Woodruff (STOC 2013) and, in the clean form shown here, in Meng–Mahoney (STOC 2013) and Nelson–Nguyen (FOCS 2013).

24.5.5. Time to compute ΠA

With the Thorup–Zhang sparse sketch (one non-zero per column), multiplying $\Pi \cdot A$ costs $O(\text{nnz}(A))$ —proportional to the number of non-zero entries in A . Each non-zero A_{ij} contributes to a single entry of ΠA (because column j of Π has one non-zero), so the total work is one addition per non-zero.

Total regression time: $O(\text{nnz}(A)) + O(\text{poly}(d/\varepsilon))$, down from $O(nd^2)$.

24.6. Johnson–Lindenstrauss dimensionality reduction

24.6.1. Statement

Lemma (Johnson–Lindenstrauss, 1984). For any set $X \subset \mathbb{R}^n$ of N points and any $\varepsilon > 0$, there exists a linear map $\Pi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with $m = O(\varepsilon^{-2} \log N)$ such that for all $x, y \in X$:

$$(1 - \varepsilon) \|x - y\|_2^2 \leq \|\Pi x - \Pi y\|_2^2 \leq (1 + \varepsilon) \|x - y\|_2^2. \quad (24.11)$$

This is optimal: $m = \Omega(\min(n, \varepsilon^{-2} \log N))$ rows are necessary (the min with n because the identity matrix trivially works).

24.6.2. Proof idea

Design a distribution $\mathcal{D}$ over $m \times n$ matrices such that for any fixed unit vector z :

$$\Pr_{\Pi \sim \mathcal{D}} [|\|\Pi z\|_2^2 - 1| > \varepsilon] \leq \delta.$$

Take $\mathcal{D}$ to be the uniform distribution over random sign matrices $(\pm 1/\sqrt{m})^{m \times n}$, and set $m = O(\varepsilon^{-2} \log(1/\delta))$.

For a single vector, Chebyshev (second moment) gives failure probability $O(1/m\varepsilon^2)$ —enough for constant probability but not small δ . To get δ , use higher moments: the p -th moment of $\|\Pi z\|_2^2 - 1$ involves monomials in the random signs, and only terms where every sign appears to an even power survive. Optimising $p \approx \log(1/\delta)$ yields the exponential tail bound.

Apply a union bound over all $\binom{N}{2}$ pairs with $\delta = 1/\binom{N}{2}$: $\log(1/\delta) = O(\log N)$, giving $m = O(\varepsilon^{-2} \log N)$.

i Aside (mine, not the lecturer’s)

JL is often presented as a “surprising” result, but the mechanism is simple once you see it through the lens of this lecture. Random projection into m dimensions preserves the norm of any *single* vector with high probability (just the AMS analysis). JL is the observation that if you want this to hold *simultaneously* for $\binom{N}{2}$ direction vectors $x - y$, you pay a union bound— $\log \binom{N}{2} = O(\log N)$ extra dimensions—and that is all.

💡 Connection to Chernoff bounds

The moment-method proof is exactly the same technique used to prove Chernoff bounds: bound the moment-generating function (or equivalently, optimise over which moment to use in Markov’s inequality). The Chernoff bound itself is the special case for sums of independent Bernoulli variables; JL is the generalisation to quadratic forms in random signs.

24.7. Point query via JL

24.7.1. The problem

A different streaming query: instead of asking for a global statistic like $\|x\|_2^2$, you want to recover *individual entries* of x . Support turnstile updates to $x \in \mathbb{R}^n$, and given a query index i , output $\tilde{x}_i$ satisfying $|\tilde{x}_i - x_i| \leq \varepsilon \|x\|_1$. The error is relative to the total “mass” $\|x\|_1 = \sum_j |x_j|$, so entries much larger than the average are recovered accurately.

24.7.2. Construction

Store $y = \Pi x$ for a suitable Π . Estimate x_i as $\tilde{x}_i = (\Pi^\top y)_i = (\Pi^\top \Pi x)_i$.

Expanding: $\tilde{x}_i = x_i + \sum_{j \neq i} x_j \langle \pi_i, \pi_j \rangle$. If the columns of Π are *nearly orthonormal*— $\|\pi_i\|_2 \approx 1$ and $|\langle \pi_i, \pi_j \rangle| \leq \varepsilon$ for $i \neq j$ —then the error is at most $\varepsilon \sum_j |x_j| = \varepsilon \|x\|_1$.

24.7.3. Getting the right Π

Apply the JL lemma to $X = \{0, e_1, \dots, e_n\}$ (the origin plus the n standard basis vectors). JL preserves all $\binom{n+1}{2}$ pairwise distances, which implies:

- $\|\Pi e_i\|_2 \approx 1$ (distances to the origin are preserved).
- $\|\Pi e_i - \Pi e_j\|_2 \approx \sqrt{2}$ (distances between basis vectors are preserved).

By the polarisation identity, $|\langle \Pi e_i, \Pi e_j \rangle| \leq O(\varepsilon)$ for $i \neq j$.

Space: $m = O(\varepsilon^{-2} \log n)$.

i Aside (mine, not the lecturer's)

This construction is closely related to the Count-Sketch data structure (Charikar–Chen–Farach-Colton, 2002), which achieves point-query error $\varepsilon \|x\|_2$ (an ℓ_2 guarantee, stronger than ℓ_1) with space $O(1/\varepsilon^2)$ and $O(1)$ update time using the Thorup–Zhang sparse structure plus separate sign and bucket hash functions.

24.8. Four-wise independence suffices

A crucial practical point: the dense AMS analysis only uses expectations of degree-four polynomials in the random signs. Independence beyond degree four is never needed. A four-wise independent family of random signs—specifiable with $O(\log n)$ bits per row—produces the same expectation and variance bounds. So the entire matrix Π (with k independent stacked copies for the median trick) is described by $O(k \log n)$ bits—far less than the mn entries of Π itself.

24.9. Cheat sheet

! Cheat sheet — Lecture 26

AMS / random-sign sketch

- Π : $m \times n$ matrix, entries $\pm 1/\sqrt{m}$ i.i.d. Store $y = \Pi x$.
- $E[\|y\|_2^2] = \|x\|_2^2$; $\text{Var}(\|y\|_2^2) \leq 2\|x\|_2^4/m$.
- Space for $(1 \pm \varepsilon)$ with prob $\geq 1 - \delta$: $O(\varepsilon^{-2} \log(1/\delta))$.
- Update time (dense): $O(\varepsilon^{-2} \log(1/\delta))$.

Thorup–Zhang sparse sketch

- One random sign per column; everything else zero.
- Same space/accuracy; update time drops to $O(\log(1/\delta))$.
- Open: can update time reach $O(1)$?

Subspace embedding (regression)

- Π is ε -subspace embedding for V if $\|\Pi v\|^2 = (1 \pm \varepsilon)\|v\|^2$ for all $v \in V$.
- For d -dimensional subspace: $m = O(d^2/\varepsilon^2)$ rows suffice.
- Sketched regression: $\tilde{x}^* = \operatorname{argmin} \|\Pi A x - \Pi b\|$ achieves $(1 + O(\varepsilon))$ -approximate residual.
- Time: $O(\operatorname{nnz}(A) + \operatorname{poly}(d/\varepsilon))$.

Johnson–Lindenstrauss

- $m = O(\varepsilon^{-2} \log N)$ rows preserve all pairwise distances among N points.
- Optimal up to constants.
- Proof: moment method (Markov with $p \approx \log(1/\delta)$ moment).

Point query

- JL on the standard basis $\Rightarrow$ near-orthonormal columns $\Rightarrow |\tilde{x}_i - x_i| \leq \varepsilon \|x\|_1$.
- Space: $O(\varepsilon^{-2} \log n)$.

24.10. FAQ — questions students actually asked

Q: Is there any reason to ever use the dense AMS sketch instead of the Thorup–Zhang sparse one? A: No. The sparse sketch matches the dense one in space and accuracy while strictly improving update time. Use the sparse version.

Q: In the variance calculation, you said terms with odd powers of σ vanish. Why? A: Each σ_{ri} is ± 1 with equal probability, so $E[\sigma_{ri}^{2k+1}] = 0$ for any non-negative integer k . When you expand y_r^4 and push the expectation inside, any monomial where some σ_{ri} appears to an odd power contributes zero. Only the “even-power” monomials survive—specifically, terms like $\sigma_{ri}^4 x_i^4$ and $\sigma_{ri}^2 \sigma_{rj}^2 x_i^2 x_j^2$.

Q: For the regression application, don’t you have to spend time computing ΠA ? That’s still $O(n)$ per column. A: With the Thorup–Zhang sparse sketch, each column of Π has one non-zero entry. So multiplying $\Pi \times A$ costs $O(\operatorname{nnz}(A))$ —you touch each non-zero of A exactly once. For sparse matrices, this can be much less than nd .

Q: Why is preserving ℓ_2 -norms useful for detecting denial-of-service attacks? A: A DoS attack concentrates traffic on a few destinations, creating spikes in the vector x of packet counts. The ℓ_2 -norm squared $\sum x_i^2$ is sensitive to such spikes (unlike the ℓ_1 -norm, which only measures total traffic). Thorup and Zhang at AT&T Research used this to detect skewness in destination distributions—ideally you’d use ℓ_∞ , but no space-efficient streaming algorithm exists for it, and ℓ_2 is the best tractable proxy.

Q: You said four-wise independence suffices for the AMS analysis. Does that mean we don’t need truly random Π ? A: Correct. The expectation calculation uses independence of pairs of signs; the variance uses independence of quadruples. A four-wise independent family (storable in $O(\log n)$ bits) provides both. The entire sketch, including $\log(1/\delta)$ independent copies, is specified by $O(\log(1/\delta) \cdot \log n)$ bits.

Q: For the JL proof, why do you need higher moments instead of Chebyshev? A: Chebyshev (second moment) gives failure probability $O(1/(m\varepsilon^2))$ for a single vector. Setting this to $1/N^2$ (for a union bound over $\binom{N}{2}$ pairs) requires $m = O(N^2/\varepsilon^2)$ —far too large. The p -th moment of $\|\Pi z\|_2^2 - 1$ decays as $(C/m)^{p/2}$ (by the same even-power argument). Optimising $p \approx \log(1/\delta)$ yields $\exp(-\Omega(m\varepsilon^2))$, so $m = O(\varepsilon^{-2} \log N)$ suffices.

24.11. Further reading

1. N. Alon, Y. Matias, M. Szegedy. *The space complexity of approximating the frequency moments*. JCSS 58(1), 1999. Introduced the AMS sketch for ℓ_2 -norm estimation and proved streaming lower bounds.

2. M. Thorup, Y. Zhang. *Tabulation-based 5-independent hashing with applications to linear probing and second moment estimation*. SODA 2004. The sparse one-nonzero-per-column sketch for faster updates.
3. T. Sarlós. *Improved approximation algorithms for large matrices via random projections*. FOCS 2006. Introduced subspace embeddings for fast regression and low-rank approximation.
4. K.L. Clarkson, D.P. Woodruff. *Low-rank approximation and regression in input sparsity time*. STOC 2013. Input-sparsity-time algorithms using the Thorup–Zhang sketch as a subspace embedding.
5. W.B. Johnson, J. Lindenstrauss. *Extensions of Lipschitz mappings into a Hilbert space*. Contemporary Mathematics 26, 1984. The original dimensionality-reduction lemma.
6. R. Jayaram, D.P. Woodruff. *Optimal streaming and tracking distinct elements with high probability*. TALG. Proves the space lower bound $\Omega(\epsilon^{-2} \log(1/\delta))$ for ℓ_2 -norm estimation, matching the AMS upper bound.

24.12. Try it yourself

1. AMS by hand. Let $x = (3, -1, 4)$ and $\Pi = \frac{1}{\sqrt{2}} \begin{pmatrix} +1 & -1 & +1 \\ -1 & -1 & +1 \end{pmatrix}$. Compute $y = \Pi x$ and $\|y\|_2^2$. Compare with $\|x\|_2^2 = 26$. How large is the relative error?
2. Variance bound. Prove that $\text{Var}(y_r^2) \leq \frac{2}{m^2} \|x\|_2^4$ by carefully expanding $E[y_r^4]$ and using $\sigma_{ri}^{2k} = 1$.
3. Sketched regression. Implement the Thorup–Zhang sketched least-squares solver. Generate a random 1000×5 matrix A , a true parameter x^* , set $b = Ax^* + \text{noise}$. Compare the sketched solution ($m = 200$) against the exact least-squares solution.

```
import numpy as np

def sparse_sketch(n, m, rng):
    """Build a Thorup-Zhang sparse sketch matrix (m x n)."""
    Pi = np.zeros((m, n))
    rows = rng.integers(0, m, size=n)
    signs = rng.choice([-1, 1], size=n)
    Pi[rows, np.arange(n)] = signs
    return Pi

rng = np.random.default_rng(42)
n, d = 1000, 5
A = rng.standard_normal((n, d))
x_true = rng.standard_normal(d)
b = A @ x_true + 0.1 * rng.standard_normal(n)

# Exact least squares
x_exact, _, _, _ = np.linalg.lstsq(A, b, rcond=None)

# Sketched least squares
m = 200
Pi = sparse_sketch(n, m, rng)
PA = Pi @ A
Pb = Pi @ b
x_sketch, _, _, _ = np.linalg.lstsq(PA, Pb, rcond=None)

print(f"Exact solution:      {x_exact.round(4)}")
```

```
print(f"Sketched solution: {x_sketch.round(4)}")
print(f"Exact residual:    {np.linalg.norm(A @ x_exact - b):.4f}")
print(f"Sketched residual: {np.linalg.norm(A @ x_sketch - b):.4f}")
```

4. JL experiment. Generate $N = 500$ random points in $\mathbb{R}^{1000}$. Project to $\mathbb{R}^m$ using a random sign matrix for several values of m . Plot the maximum distortion $\max_{x,y} |\|\Pi x - \Pi y\| / \|x - y\| - 1|$ vs. m . At what m does the distortion drop below 0.1? Compare with the JL prediction $m \approx \epsilon^{-2} \log N$.
5. Cosine similarity. Create two “document” vectors (bag-of-words over a vocabulary of $n = 10,000$ words), normalise them to unit norm, and compare the true cosine similarity with the estimate from $\langle \Pi x, \Pi y \rangle$ for $m = 50, 100, 500$. How does the error scale with m ?

